RISE Toolbox Documentation

Release 20240624

Junior Maih

Jun 24, 2024

CONTENTS:

1 Getting started **1**
- 1.1 RISE at a glance . . . 1
- 1.2 License and Legal Mumbo-Jumbo . . . 2
- 1.3 Setting up Your RISE environment . . . 2
  - 1.3.1 Software requirements . . . 2
  - 1.3.2 Installing RISE . . . 2
    - 1.3.2.1 The zip file option . . . 3
    - 1.3.2.2 Github for the bleeding-edge installation (highly recommended) . . . 3
    - 1.3.2.3 The git option (never tested!!!) . . . 3
    - 1.3.2.4 Testing your installation . . . 3
  - 1.3.3 Loading and starting RISE . . . 3
  - 1.3.4 Updating RISE . . . 4
  - 1.3.5 The work environement . . . 4
- 1.4 Documentation of the +rise package . . . 4
  - 1.4.1 Opening the PDF documentation . . . 4
  - 1.4.2 Leaving/Quitting RISE . . . 4
  - 1.4.3 Obtaining the release information . . . 5
  - 1.4.4 Starting RISE . . . 5
  - 1.4.5 Obtaining the toolbox information . . . 5
  - 1.4.6 Obtaining the version of the Toolbox . . . 6
  - 1.4.7 Why : Answers to random question rhyming with RISE . . . 6
  - 1.4.8 Whatsup : Random sentences rhyming with RISE . . . 7
- 1.5 Finding help . . . 7
- 1.6 Citing RISE in your research . . . 7
- 1.7 Contributing to RISE . . . 7
- 1.8 Acknowledgements . . . 8

2 Hit the ground running : An introductory example **9**
- 2.1 The model economy . . . 9
- 2.2 Baseline calibration . . . 10
- 2.3 Preparing the model for RISE . . . 11
- 2.4 Creating the model object (rising the model) . . . 12
- 2.5 Parameterizing the model . . . 12
- 2.6 Setting some options for the solution . . . 13
- 2.7 Stochastic simulation in a Dynare-like run . . . 13
- 2.8 Solving the model and printing the solution . . . 14
- 2.9 Solving a 5th-order perturbation . . . 14
- 2.10 Checking the accuracy of derivatives . . . 14
- 2.11 Impulse response functions . . . 14
- 2.12 Collecting Data from FRED . . . 15

2.13 Filtration . . . 15
2.14 Priors for Bayesian estimation . . . 16
2.15 Visualizing the priors . . . 16
2.16 Posterior maximization . . . 16
2.17 Posterior simulation . . . 17
2.18 Computation of the marginal data density using the Bridge approach . . . 17

3 DSGE Modeling **19**
3.1 Description . . . 19
3.2 An example . . . 20
3.3 The model(rise/dsge) file language . . . 20
3.3.1 Conventions . . . 20
3.3.1.1 Comments and block comments . . . 20
3.3.1.2 Time and steady state . . . 20
3.3.1.3 Auxiliary variables . . . 20
3.3.1.4 Functions . . . 21
3.3.1.5 Discountinous functions . . . 22
3.3.1.6 Shock distributions . . . 23
3.3.2 Ways to declare a model . . . 24
3.3.2.1 Declaring a model in a file . . . 24
3.3.2.2 Declaring a model online . . . 24
3.3.3 Handling multiple multiple model files . . . 24
3.3.3.1 Method 1: Simple Model File . . . 24
3.3.3.2 Method 2: Multiple Model Files . . . 24
3.3.3.3 Method 3: Simple Online Model (Using Cell Array) . . . 25
3.3.3.4 Method 4: Multiple Online Model Codes (Using Cell Array of Cell Arrays) . . . 25
3.3.3.5 Method 5: Combination of Model Files and Online Model Codes . . . 25
3.3.4 Atoms declarations . . . 26
3.3.4.1 @endogenous . . . 26
3.3.4.2 @exogenous . . . 26
3.3.4.3 @parameters . . . 26
3.3.4.4 @observables . . . 28
3.3.4.5 @log_variables . . . 28
3.3.4.6 @level_variables . . . 28
3.3.4.7 On-the-fly atoms declarations . . . 29
3.3.5 The @functions block . . . 29
3.3.6 The @model block . . . 29
3.3.6.1 Regular equations . . . 29
3.3.6.2 Equations broken across multiple lines . . . 30
3.3.6.3 The @E operator for expectations . . . 30
3.3.6.4 Equations with auxiliary expressions :"…$[]$…" . . . 30
3.3.6.5 Regular equations with equation simplification :"…#…" . . . 31
3.3.6.6 Definitions : # Equation . . . 31
3.3.6.7 Time-varying transition probabilities : ! Equation . . . 32
3.3.7 The @optimization_problem block . . . 33
3.3.7.1 Ramsey Policy . . . 33
3.3.7.2 Discretionary Policy . . . 33
3.3.7.3 Loose commitment . . . 34
3.3.7.4 Stochastic Replanning . . . 35
3.3.7.5 Optimal Policy with no u-turn . . . 35
3.3.8 The @optimization_constraints block (optional) . . . 35
3.3.9 The @steady_state_model block (optional) . . . 37
3.3.10 The @parameterization block (optional) . . . 38
3.3.11 The @parameter_restrictions block (optional) . . . 39

3.3.12 Tools for bounded rationality (Non-rational expectations) . . . 39
3.3.12.1 Hybrid Expectations . . . 39
3.3.12.2 Cognitive discounting . . . 39
3.3.13 Macro Language for preparsing . . . 40
3.3.13.1 Inclusion of files . . . 40
3.3.13.2 Pseudo functions . . . 40
3.3.13.3 Control flow : for loops . . . 41
3.3.13.4 control flows : zipped for loops . . . 42
3.3.13.5 control flows : enumerated for loops . . . 42
3.3.13.6 Evaluations . . . 42
3.3.13.7 Control flow : @if … @elseif … @else … @end . . . 42
3.3.13.8 Control flow : @switch … @case … @otherwise … @end . . . 43
3.3.13.9 lower and upper cases . . . 43
3.4 The RISE Editor . . . 43
3.4.1 System Requirements . . . 44
3.4.2 Starting the App . . . 44
3.4.3 dark vs light modes . . . 44
3.4.4 Syntax highlighting . . . 45
3.4.5 Indentations and nested indentations . . . 45
3.4.6 Editing files . . . 46
3.4.7 Control flow . . . 46
3.5 Setting up calibration and priors outside the model file . . . 46
3.6 Deterministic and quasi-deterministic solutions . . . 47
3.6.1 Perfect foresight . . . 47
3.6.2 Extended path . . . 47
3.7 Stochastic solution via perturbation . . . 47
3.7.1 Zero-th order approximation : The steady state and the balanced-growth path . . . 47
3.7.1.1 Incomplete analytical solution . . . 49
3.7.1.2 Steady state loops . . . 49
3.7.1.3 Facilitators . . . 49
3.7.1.4 The case of multiple regimes . . . 50
3.7.2 First-order perturbation . . . 50
3.7.2.1 The state vector . . . 50
3.7.2.2 Solvers for the quadratic matrix polynomial . . . 50
3.7.3 Higher-order perturbation . . . 52
3.7.3.1 Choosing the derivatives . . . 52
3.7.3.2 Solving the system of generalized Sylvester equations . . . 52
3.8 Optimal/Optimized Simple Rules . . . 53
3.8.1 The Loss function . . . 53
3.8.2 (Approximate) Analytical Loss . . . 53
3.8.3 Simulated Loss . . . 53
3.8.4 OSR and Indirect Inference . . . 53
3.8.5 OSR and user-defined objective function . . . 53
3.8.6 Policy frontiers . . . 53
3.9 Stochastic simulations . . . 53
3.9.1 Artificial data . . . 53
3.9.2 Pruned simulations . . . 53
3.9.3 Theoretical and Simulated moments . . . 54
3.9.4 Variance decomposition . . . 54
3.9.5 Impulse responses . . . 54
3.9.5.1 Simple Impulse Responses . . . 54
3.9.5.2 Generalized Impulse Responses . . . 54
3.10 Generic simulations with plans . . . 54
3.10.1 The properties . . . 54

3.10.1.1 endo_list . . . 54
3.10.1.2 exo_list . . . 54
3.10.1.3 start_date . . . 54
3.10.1.4 end_date . . . 54
3.10.1.5 InitialRegime . . . 54
3.10.1.6 Conditions . . . 55
3.10.1.7 simul_periods . . . 55
3.10.1.8 variable_list . . . 55
3.10.2 The methods . . . 55
3.10.2.1 append . . . 55
3.10.2.2 endval . . . 56
3.10.2.3 export . . . 56
3.10.2.4 histval . . . 57
3.10.2.5 initval . . . 58
3.10.2.6 simplan . . . 59
3.11 Forecasting . . . 59
3.11.1 Conditional forecasting in linear models . . . 60
3.11.2 Conditional forecasting in nonlinear models . . . 60
3.12 DSGE Filtering . . . 60
3.12.1 Filtering and decompositions . . . 60
3.12.1.1 Linear filtration . . . 60
3.12.1.2 Historical Decomposition . . . 60
3.12.1.3 Observables decomposition . . . 60
3.12.1.4 Conditional forecasting via filtering . . . 60
3.12.1.5 User-defined filter . . . 61
3.12.2 Linear filtration with real-time information . . . 61
3.12.2.1 Data setup . . . 61
3.12.2.2 routine . . . 61
3.12.3 Nonlinear filtering . . . 61
3.12.3.1 Switching Unscented Kalman filter . . . 61
3.12.3.2 Switching Divided difference filter . . . 62
3.12.3.3 Switching Cubature Kalman filter . . . 63
3.13 Occasionally-Binding Constraints . . . 64
3.13.1 Modeling Occasionally-Binding Constraints using Anticipated Shocks . . . 64
3.13.1.1 The principle . . . 64
3.13.1.2 A quick example . . . 64
3.13.1.3 The implementation . . . 64
3.13.1.4 Estimation . . . 64
3.13.2 Modeling Occasionally-Binding Constraints using Piecewise-Linear approximations . . . 64
3.13.2.1 principle and the routine . . . 64
3.13.2.2 A quick example . . . 64
3.13.2.3 The model file . . . 64
3.13.2.4 The basic setup . . . 64
3.13.2.5 Generalization . . . 64
3.13.2.6 Estimation . . . 64
3.13.3 Modeling Occasionally-Binding Constraints using Regime Switching . . . 64
3.13.3.1 The principle . . . 64
3.13.3.2 A quick example . . . 64
3.13.3.3 Under Optimal policy . . . 64
3.13.3.4 The implementation . . . 65
3.13.3.5 Estimation . . . 65
3.14 Understanding a rise or dsge model object . . . 65
3.14.1 Ordering and information about atoms . . . 65
3.14.1.1 Endogenous variables . . . 65

3.14.1.2 Exogenous variables . . . 65
3.14.1.3 Parameters . . . 65
3.14.1.4 Observables . . . 65
3.14.2 Information on Model equations . . . 66
3.14.3 Information on Model results . . . 66
3.14.3.1 First-order structural form . . . 66
3.14.3.2 Solution results . . . 66
3.14.3.3 Posterior maximization results . . . 66
3.15 Extending the DSGE functionalities . . . 66
3.15.1 Extracting the structural form . . . 66
3.15.2 Extracting the solution . . . 66
3.15.3 Your own solver . . . 66
3.15.4 Your own filtering algorithm . . . 66
3.15.5 Your own optimizer . . . 66
3.15.6 Your own Likelihood function . . . 66
3.15.6.1 Relation to Indirect Inference . . . 66
3.15.7 Your own posterior sampler . . . 66
3.16 Automatic Translation of files . . . 66
3.16.1 Dynare files . . . 67
3.16.2 Dynare steady state files . . . 68
3.16.3 AIM/YADA files . . . 68
3.17 Technical documentation for rise/dsge objects . . . 69
3.17.1 The properties . . . 69
3.17.1.1 user_data . . . 69
3.17.1.2 nsols . . . 69
3.17.1.3 definitions . . . 69
3.17.1.4 equations . . . 69
3.17.1.5 filename . . . 69
3.17.1.6 model_description . . . 69
3.17.1.7 legend . . . 69
3.17.1.8 endogenous . . . 70
3.17.1.9 exogenous . . . 70
3.17.1.10 observables . . . 70
3.17.1.11 options . . . 70
3.17.1.12 parameters . . . 70
3.17.1.13 markov_chains . . . 70
3.17.1.14 estimation . . . 70
3.17.2 The methods . . . 70
3.17.2.1 abcd . . . 70
3.17.2.2 accuracy . . . 71
3.17.2.3 calculate_loss . . . 72
3.17.2.4 calibrate . . . 72
3.17.2.5 check_derivatives . . . 73
3.17.2.6 condition_draws_on_model . . . 73
3.17.2.7 counterfactual . . . 73
3.17.2.8 dbminusdb . . . 74
3.17.2.9 draw_parameter . . . 75
3.17.2.10 drop . . . 75
3.17.2.11 estimate . . . 75
3.17.2.12 extract_first_order_structure . . . 78
3.17.2.13 filter . . . 79
3.17.2.14 fisher . . . 81
3.17.2.15 fold_solution . . . 81
3.17.2.16 forecast . . . 82

3.17.2.17 forecast_real_time . . . 83
3.17.2.18 frontier . . . 83
3.17.2.19 get . . . 84
3.17.2.20 growth_database . . . 87
3.17.2.21 hessian . . . 87
3.17.2.22 historical_decomposition . . . 87
3.17.2.23 historical_decomposition_switch . . . 89
3.17.2.24 indirect_inference . . . 90
3.17.2.25 initial_conditions . . . 90
3.17.2.26 initial_sstate . . . 90
3.17.2.27 irf . . . 91
3.17.2.28 is_forward_guidance_puzzle . . . 92
3.17.2.29 is_stable_system . . . 93
3.17.2.30 is_stationary_system . . . 94
3.17.2.31 isnan . . . 94
3.17.2.32 itranslate . . . 94
3.17.2.33 link_parameters . . . 95
3.17.2.34 loadObj . . . 95
3.17.2.35 load_parameters . . . 95
3.17.2.36 log_posterior_kernel . . . 96
3.17.2.37 log_prior_density . . . 96
3.17.2.38 max_discrepancy . . . 97
3.17.2.39 mode_curvature . . . 97
3.17.2.40 model_information . . . 98
3.17.2.41 observables_decomposition . . . 98
3.17.2.42 optimal_simple_rule . . . 99
3.17.2.43 parameters_to_file . . . 99
3.17.2.44 perfect_foresight . . . 100
3.17.2.45 plan2database . . . 101
3.17.2.46 posterior_sample . . . 102
3.17.2.47 predictive_analysis . . . 103
3.17.2.48 print_estimation_results . . . 104
3.17.2.49 print_solution . . . 104
3.17.2.50 pull_objective . . . 105
3.17.2.51 randsample . . . 106
3.17.2.52 refresh . . . 106
3.17.2.53 report . . . 106
3.17.2.54 resid . . . 107
3.17.2.55 resimulate . . . 108
3.17.2.56 rise . . . 108
3.17.2.57 saveObj . . . 110
3.17.2.58 set . . . 110
3.17.2.59 set_z_eplus_horizon . . . 111
3.17.2.60 simulate . . . 111
3.17.2.61 simulate_nonlinear . . . 113
3.17.2.62 solve . . . 113
3.17.2.63 solve_alternatives . . . 117
3.17.2.64 srf . . . 117
3.17.2.65 sstate . . . 118
3.17.2.66 state_var_list . . . 121
3.17.2.67 stoch_simul . . . 121
3.17.2.68 theoretical_autocorrelations . . . 122
3.17.2.69 theoretical_autocovariances . . . 122
3.17.2.70 translate . . . 123

3.17.2.71 unlink_parameters . . . 124
3.17.2.72 update_file . . . 124
3.17.2.73 variance_decomposition . . . 125
3.17.2.74 view_linked_parameters . . . 126

4 DSGE-VAR Modeling **127**
4.1 Description . . . 127
4.2 A quick-start example . . . 127
4.2.1 A simple New Keynesian DSGE model . . . 127
4.2.2 Setting up the BVAR-DSGE model . . . 128
4.2.3 Fixed parameters . . . 128
4.2.4 Setting up and visualing the priors . . . 128
4.2.5 Collecting and transforming the data . . . 128
4.2.6 Maximizing the posterior . . . 129
4.2.7 IRFs of the BVAR-DSGE at the maximized posterior . . . 129
4.2.8 IRFs of the DSGE model at the maximized posterior . . . 129
4.2.9 IRF comparison . . . 129
4.3 Technical documentation for dsge_var objects . . . 129
4.3.1 The properties . . . 129
4.3.1.1 nlags . . . 129
4.3.1.2 constant . . . 129
4.3.1.3 endogenous . . . 129
4.3.1.4 options . . . 130
4.3.1.5 shocks . . . 130
4.3.1.6 parameters . . . 130
4.3.1.7 markov_chains . . . 130
4.3.1.8 estimation . . . 130
4.3.2 The methods . . . 130
4.3.2.1 autocov . . . 130
4.3.2.2 bvar_dsge . . . 131
4.3.2.3 draw_parameter . . . 131
4.3.2.4 estimate . . . 132
4.3.2.5 fisher . . . 132
4.3.2.6 forecast . . . 132
4.3.2.7 get . . . 132
4.3.2.8 hessian . . . 133
4.3.2.9 historical_decomposition . . . 134
4.3.2.10 irf . . . 134
4.3.2.11 load_parameters . . . 135
4.3.2.12 log_posterior_kernel . . . 135
4.3.2.13 log_prior_density . . . 136
4.3.2.14 mode_curvature . . . 136
4.3.2.15 posterior_sample . . . 137
4.3.2.16 print_estimation_results . . . 138
4.3.2.17 pull_objective . . . 138
4.3.2.18 residuals . . . 139
4.3.2.19 set . . . 139
4.3.2.20 solve . . . 139
4.3.2.21 variance_decomposition . . . 140

5 Reduced-form VAR Modeling **141**
5.1 Description . . . 141
5.2 Quick-start examples . . . 141
5.2.1 A constant-parameter VAR . . . 141

5.2.1.1 Collecting and transforming data . . . 141
5.2.1.2 Setting up the Reduced.form VAR . . . 142
5.2.1.3 Estimating the VAR using classical techniques . . . 142
5.2.1.4 Restrictions on the VAR : Domestic variables do not affect oil prices . . . 142
5.2.1.5 Estimate the restricted VAR . . . 143
5.2.1.6 Identification . . . 143
5.2.1.7 Compare structural shocks . . . 144
5.2.1.8 Compare CHOLESKI irfs . . . 145
5.2.1.9 Compare irfs based on the identification scheme . . . 146
5.2.1.10 Variance decomposition . . . 146
5.2.1.11 Historical decomposition . . . 147
5.2.1.12 Bootstrap . . . 148
5.2.1.13 Variance decomposition distribution . . . 148
5.2.1.14 Historical decomposition distribution . . . 149
5.2.1.15 IRF distribution . . . 150
5.2.1.16 Bayesian estimation . . . 151
5.2.1.17 Posterior sampling of parameters . . . 151
5.2.1.18 Bayesian forecasting . . . 151
5.2.1.19 Conditional forecasting . . . 152
5.2.2 Adding regime switching with constant transition probabilities . . . 153
5.2.3 Adding regime switching with time~varying transition probabilities . . . 154
5.3 Technical documentation for rfvar objects . . . 155
5.3.1 The properties . . . 155
5.3.1.1 endogenous . . . 155
5.3.1.2 exogenous . . . 155
5.3.1.3 parameters . . . 155
5.3.1.4 nonvar_parameters . . . 155
5.3.1.5 constant . . . 155
5.3.1.6 debug . . . 155
5.3.1.7 markov_chains . . . 155
5.3.1.8 is_time_varying_trans_prob . . . 156
5.3.1.9 is_switching . . . 156
5.3.1.10 nlags . . . 156
5.3.1.11 nx . . . 156
5.3.1.12 nvars . . . 156
5.3.1.13 nparams . . . 156
5.3.1.14 nregs . . . 156
5.3.2 The methods . . . 156
5.3.2.1 autocov . . . 156
5.3.2.2 bootstrap . . . 157
5.3.2.3 codify . . . 157
5.3.2.4 estimate . . . 158
5.3.2.5 filter . . . 160
5.3.2.6 forecast . . . 160
5.3.2.7 get . . . 160
5.3.2.8 historical_decomposition . . . 160
5.3.2.9 identification . . . 161
5.3.2.10 irf . . . 161
5.3.2.11 posterior_mode . . . 162
5.3.2.12 print_solution . . . 162
5.3.2.13 prior_template . . . 163
5.3.2.14 pull_objective . . . 164
5.3.2.15 residuals . . . 164
5.3.2.16 rfvar . . . 165

5.3.2.17 set . . . 166
5.3.2.18 solve . . . 166
5.3.2.19 structural_shocks . . . 167
5.3.2.20 transform_var_into_dsge . . . 167
5.3.2.21 variance_decomposition . . . 167

6 Panel Reduced-form VAR Modeling **169**
6.1 Description . . . 169
6.2 Quick-start examples . . . 169
6.2.1 constant . . . 169
6.2.2 constant transition probabilities . . . 169
6.2.3 time~varying transition probabilities . . . 169
6.3 Estimation . . . 169
6.4 Simulation . . . 169
6.5 Forecasting and conditional forecasting . . . 169
6.6 Technical documentation for pvar objects . . . 169
6.6.1 The properties . . . 169
6.6.1.1 members . . . 169
6.6.1.2 homogeneity . . . 170
6.6.1.3 endogenous . . . 170
6.6.1.4 exogenous . . . 170
6.6.1.5 parameters . . . 170
6.6.1.6 nonvar_parameters . . . 170
6.6.1.7 constant . . . 170
6.6.1.8 debug . . . 170
6.6.1.9 markov_chains . . . 170
6.6.1.10 is_time_varying_trans_prob . . . 171
6.6.1.11 is_switching . . . 171
6.6.1.12 nlags . . . 171
6.6.1.13 nx . . . 171
6.6.1.14 nvars . . . 171
6.6.1.15 nparams . . . 171
6.6.1.16 nregs . . . 171
6.6.2 The methods . . . 171
6.6.2.1 autocov . . . 171
6.6.2.2 bootstrap . . . 172
6.6.2.3 codify . . . 172
6.6.2.4 estimate . . . 173
6.6.2.5 filter . . . 175
6.6.2.6 forecast . . . 175
6.6.2.7 get . . . 175
6.6.2.8 historical_decomposition . . . 175
6.6.2.9 identification . . . 176
6.6.2.10 irf . . . 176
6.6.2.11 posterior_mode . . . 177
6.6.2.12 prfvar . . . 178
6.6.2.13 print_solution . . . 179
6.6.2.14 prior_template . . . 179
6.6.2.15 pull_objective . . . 180
6.6.2.16 residuals . . . 181
6.6.2.17 set . . . 181
6.6.2.18 solve . . . 181
6.6.2.19 structural_shocks . . . 181
6.6.2.20 transform_var_into_dsge . . . 181

6.6.2.21 variance_decomposition . . . 182

7 Structural VAR Modeling **183**
7.1 Description . . . 183
7.2 Quick-start examples . . . 183
7.2.1 constant . . . 183
7.2.2 constant transition probabilities . . . 183
7.2.3 time-varying transition probabilities . . . 183
7.3 Estimation . . . 183
7.4 Simulation . . . 183
7.5 Forecasting and conditional forecasting . . . 183
7.6 Technical documentation for svar objects . . . 183
7.6.1 The properties . . . 183
7.6.1.1 endogenous . . . 183
7.6.1.2 exogenous . . . 184
7.6.1.3 parameters . . . 184
7.6.1.4 nonvar_parameters . . . 184
7.6.1.5 constant . . . 184
7.6.1.6 debug . . . 184
7.6.1.7 markov_chains . . . 184
7.6.1.8 is_time_varying_trans_prob . . . 184
7.6.1.9 is_switching . . . 184
7.6.1.10 nlags . . . 185
7.6.1.11 nx . . . 185
7.6.1.12 nvars . . . 185
7.6.1.13 nparams . . . 185
7.6.1.14 nregs . . . 185
7.6.2 The methods . . . 185
7.6.2.1 autocov . . . 185
7.6.2.2 codify . . . 186
7.6.2.3 estimate . . . 187
7.6.2.4 filter . . . 188
7.6.2.5 forecast . . . 188
7.6.2.6 get . . . 189
7.6.2.7 historical_decomposition . . . 189
7.6.2.8 identification . . . 189
7.6.2.9 irf . . . 189
7.6.2.10 posterior_mode . . . 190
7.6.2.11 print_solution . . . 190
7.6.2.12 print_structural_form . . . 190
7.6.2.13 prior_template . . . 190
7.6.2.14 pull_objective . . . 191
7.6.2.15 residuals . . . 192
7.6.2.16 set . . . 192
7.6.2.17 solve . . . 193
7.6.2.18 svar . . . 193
7.6.2.19 transform_var_into_dsge . . . 194
7.6.2.20 variance_decomposition . . . 195

8 Proxy Structural VAR Modeling **197**
8.1 Description . . . 197
8.2 Quick-start examples . . . 197
8.2.1 constant . . . 197
8.2.2 constant transition probabilities . . . 197

8.2.3 time~varying transition probabilities . . . 197
8.3 Estimation . . . 197
8.4 Simulation . . . 197
8.5 Forecasting and conditional forecasting . . . 197
8.6 Technical documentation for proxy_var objects . . . 197
8.6.1 The properties . . . 197
8.6.1.1 proxies . . . 197
8.6.1.2 endogenous . . . 198
8.6.1.3 exogenous . . . 198
8.6.1.4 parameters . . . 198
8.6.1.5 nonvar_parameters . . . 198
8.6.1.6 constant . . . 198
8.6.1.7 debug . . . 198
8.6.1.8 markov_chains . . . 198
8.6.1.9 is_time_varying_trans_prob . . . 198
8.6.1.10 is_switching . . . 199
8.6.1.11 nlags . . . 199
8.6.1.12 nx . . . 199
8.6.1.13 nvars . . . 199
8.6.1.14 nparams . . . 199
8.6.1.15 nregs . . . 199
8.6.2 The methods . . . 199
8.6.2.1 autocov . . . 199
8.6.2.2 codify . . . 200
8.6.2.3 estimate . . . 201
8.6.2.4 filter . . . 202
8.6.2.5 forecast . . . 202
8.6.2.6 get . . . 203
8.6.2.7 historical_decomposition . . . 203
8.6.2.8 identification . . . 203
8.6.2.9 irf . . . 203
8.6.2.10 posterior_mode . . . 203
8.6.2.11 print_solution . . . 203
8.6.2.12 print_structural_form . . . 204
8.6.2.13 prior_template . . . 204
8.6.2.14 proxy_svar . . . 205
8.6.2.15 pull_objective . . . 206
8.6.2.16 residuals . . . 206
8.6.2.17 set . . . 207
8.6.2.18 solve . . . 207
8.6.2.19 transform_var_into_dsge . . . 208
8.6.2.20 variance_decomposition . . . 208

9 Bayesian Estimation **211**
9.1 Restrictions on parameters during estimation . . . 211
9.2 Prior distributions and implementation . . . 211
9.2.1 Bayesian priors . . . 211
9.2.1.1 Setting Uniform Priors . . . 212
9.2.1.2 Setting priors using Means and Standard deviations . . . 212
9.2.1.3 Setting priors using quantiles . . . 212
9.2.1.4 Setting priors using hyperparameters . . . 213
9.2.1.5 About the t-student prior . . . 213
9.2.1.6 The dirichlet prior . . . 213
9.2.2 Endogenous Priors . . . 214

9.2.3 User-defined Priors . . . 214
9.2.4 Visualizing priors and posteriors . . . 214
9.2.5 List of prior distributions . . . 215
9.3 Posterior maximization . . . 216
9.3.1 Optimizers . . . 216
9.3.2 User-defined Optimizers . . . 216
9.4 Posterior sampling . . . 216
9.4.1 The sampling function . . . 216
9.4.2 Algorithm : Random-walk Metropolis Hastings . . . 217
9.4.3 Algorithm : Independent Metropolis Hastings . . . 218
9.4.4 Algorithm : Adaptive parallel tempering . . . 219
9.4.5 Algorithm : Slice Sampler . . . 220
9.4.6 User-defined Algorithms . . . 220
9.4.6.1 Obtaining the DRAM inputs . . . 222
9.4.6.2 Running DRAM . . . 222
9.4.6.3 Transforming the DRAM output . . . 222
9.5 Processing Posterior draws : The mcmc class . . . 222
9.5.1 The properties . . . 222
9.5.1.1 pnames . . . 222
9.5.1.2 nchains . . . 223
9.5.1.3 chain_names . . . 223
9.5.1.4 npop . . . 223
9.5.1.5 nparams . . . 223
9.5.1.6 draws . . . 223
9.5.1.7 psrf . . . 223
9.5.1.8 best . . . 223
9.5.1.9 log_post . . . 223
9.5.2 The methods . . . 223
9.5.2.1 autocorrplot . . . 223
9.5.2.2 brooks_gelman . . . 224
9.5.2.3 densplot . . . 225
9.5.2.4 geweke . . . 225
9.5.2.5 inefficiency_factors . . . 225
9.5.2.6 mcmc . . . 226
9.5.2.7 meanplot . . . 227
9.5.2.8 plot_brooks_gelman . . . 227
9.5.2.9 posterior_parameter_statistics . . . 227
9.5.2.10 process_draws . . . 228
9.5.2.11 psrf_plot . . . 228
9.5.2.12 raftery_lewis . . . 229
9.5.2.13 reload_draws . . . 229
9.5.2.14 scatterplot . . . 229
9.5.2.15 statistics . . . 229
9.5.2.16 summary . . . 230
9.5.2.17 traceplot . . . 230
9.5.3 Visualizing posteriors (and priors) . . . 231
9.6 Marginal Data Density computation : the mdd class . . . 231
9.6.1 The properties . . . 231
9.6.1.1 log_post_kern . . . 231
9.6.1.2 theta_draws . . . 231
9.6.1.3 lb . . . 231
9.6.1.4 ub . . . 231
9.6.1.5 maximization . . . 231
9.6.1.6 H . . . 231

9.6.1.7 subset . . . 231
9.6.1.8 thecoef . . . 231
9.6.2 The methods . . . 232
9.6.2.1 bridge . . . 232
9.6.2.2 cj . . . 232
9.6.2.3 global_options . . . 233
9.6.2.4 is . . . 233
9.6.2.5 laplace . . . 233
9.6.2.6 laplace_mcmc . . . 234
9.6.2.7 laplace_mdd . . . 234
9.6.2.8 mdd . . . 234
9.6.2.9 mhm . . . 235
9.6.2.10 mueller . . . 236
9.6.2.11 normal_weighting . . . 236
9.6.2.12 ris . . . 236
9.6.2.13 swz . . . 237
9.7 Indirect Inference . . . 237
9.7.1 The principle of Indirect inference . . . 237
9.7.2 Generalized Method of Moments . . . 237
9.7.3 Simulated Method of Moments . . . 237
9.7.4 Impulse Response Matching . . . 237

10 Time Series and Data Management 239
10.1 Dates . . . 239
10.1.1 Base Class Methods . . . 239
10.1.2 Help Text for Derived Classes . . . 240
10.1.2.1 Daily dates : rd . . . 240
10.1.2.2 Weely dates : rw . . . 241
10.1.2.3 Monthly dates : rm . . . 241
10.1.2.4 Quarterly dates : rq . . . 242
10.1.2.5 Half-yearly dates : rh . . . 242
10.1.2.6 Yearly dates : ry . . . 243
10.1.2.7 Annual dates : ra . . . 243
10.1.3 Technical documentation for rise_dates.dates objects . . . 244
10.1.3.1 The properties . . . 244
10.1.3.2 The methods . . . 244
10.2 Time Series . . . 251
10.2.1 Data Cleaning . . . 252
10.2.2 Time Series Transformations . . . 252
10.2.3 Statistical Analysis . . . 252
10.2.4 Plotting tools for time series . . . 252
10.2.5 Other Automations . . . 253
10.2.6 Technical documentation for ts objects . . . 253
10.2.6.1 The properties . . . 253
10.2.6.2 The methods . . . 254

11 Global Sensitivity Analysis and Uncertainty Quantification 293
11.1 Monte Carlo Filtering . . . 293
11.1.1 Description . . . 293
11.1.2 Quick start examples . . . 293
11.1.2.1 A very simple example . . . 293
11.1.2.2 A simple DSGE example . . . 294
11.1.3 Details on the mcf class . . . 295
11.1.3.1 The properties . . . 295

11.1.3.2 The methods . . . 295
11.1.4 Support facility for filtration : the mcfprop class . . . 298
11.1.4.1 The properties . . . 298
11.1.4.2 The methods . . . 298
11.1.4.3 An example . . . 300
11.2 High Dimensional Model Representation . . . 301
11.2.1 Description . . . 301
11.2.2 Quick-start examples . . . 302
11.2.2.1 A known Polynomial function . . . 302
11.2.3 Details . . . 302
11.2.3.1 The properties . . . 302
11.2.3.2 The methods . . . 304
11.3 Uncertainty Quantification . . . 305
11.3.1 Description . . . 305
11.3.2 Quick start examples . . . 305
11.3.3 Details . . . 305

12 Conditional forecasting using Relative Entropy 307

13 RISE's reporting system 309
13.1 Description . . . 309
13.2 Quick start examples . . . 309
13.3 Details . . . 310
13.3.1 The properties . . . 310
13.3.1.1 Title . . . 310
13.3.1.2 Author . . . 311
13.3.1.3 Date . . . 311
13.3.2 The methods . . . 311
13.3.2.1 cleardoublepage . . . 311
13.3.2.2 clearpage . . . 311
13.3.2.3 code_formatter . . . 311
13.3.2.4 create_figure . . . 312
13.3.2.5 description . . . 313
13.3.2.6 enumerate . . . 313
13.3.2.7 equation . . . 314
13.3.2.8 escape . . . 314
13.3.2.9 figure . . . 314
13.3.2.10 footnote . . . 315
13.3.2.11 itemize . . . 315
13.3.2.12 newpage . . . 315
13.3.2.13 pagebreak . . . 316
13.3.2.14 paragraph . . . 316
13.3.2.15 plain . . . 316
13.3.2.16 publish . . . 317
13.3.2.17 quotation . . . 317
13.3.2.18 quote . . . 318
13.3.2.19 rise_file . . . 318
13.3.2.20 rise_model_parameterization . . . 319
13.3.2.21 rprt . . . 319
13.3.2.22 section . . . 320
13.3.2.23 set_options . . . 320
13.3.2.24 subparagraph . . . 320
13.3.2.25 subsection . . . 320
13.3.2.26 subsubsection . . . 321

13.3.2.27 table . . . 321
13.3.2.28 text . . . 322
13.3.2.29 url . . . 322
13.3.2.30 verbatim . . . 323

14 Stochastic Global Optimization **325**

15 Troubleshooting **327**
15.1 deciphering return codes . . . 327
15.2 debugging tools . . . 327
15.3 The steady state does not solve . . . 327
15.4 The model does not solve . . . 327
15.5 Speed issues : known bottlenecks . . . 328
15.6 A sneak peek into the future . . . 328

16 Plotting tools **329**

17 References **331**

18 Things to do **333**

19 Indices and tables **335**

Bibliography **337**

Index **341**

CHAPTER

ONE

GETTING STARTED

1.1 RISE at a glance

RISE is the acronym for **R**ationality **I**n **S**witching **E**nvironments.

It is an object-oriented Matlab toolbox primarily designed for solving and estimating nonlinear dynamic stochastic general equilibirium (**DSGE**) or more generally Rational Expectations(**RE**) models with **regime switches** .

The switching process and can be endogenous. RISE uses perturbation to approximate the nonlinear Regime Switching DSGE (RS-DSGE) model and solves it using efficient algorithms.

In the paper "Efficient Perturbation Methods for Solving Regime-Switching DSGE Models", [Maih, 2015] describes the basic problem and the solution approach in RISE. Other good references in the field include various papers by Dan Waggoner and Tao Zha as well as papers by Eric Leeper, among others.

RISE also implements special cases of the general RS-DSGE model, including

- The DSGE model with constant parameters
- The DSGE-VAR model
- **VAR**s with and without switching parameters
- **Panel VAR**s with and without switching parameters
- **Proxy VAR**s with and without switching parameters
- **SVAR**s with and without switching paramters
- etc.

Within the DSGE framework, RISE also includes the solution and estimation of

- Loose commitment (or optimal policy) models
- hybrid expectations models
- models in which agents have information about future events

1.2 License and Legal Mumbo-Jumbo


```
Copyright (c) 2009-2023, Junior Maih
All rights reserved.

Redistribution and use in source and binary forms, with or without modification,
are permitted provided that the following conditions are met:

        1. Redistributions of source code must retain the above copyright notice,
           this list of conditions and the following disclaimer.

        2. Redistributions in binary form must reproduce the above copyright notice,
           this list of conditions and the following disclaimer in the documentation
           and/or other materials provided with the distribution.

        3. Neither the name of the RISE Toolbox nor the names of its contributors
           may be used to endorse or promote products derived from this software
           without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.
```


1.3 Setting up Your RISE environment

1.3.1 Software requirements

I order to use RISE, the following software will need to be installed:

- Matlab version ? or higher
- MikTex (Windows users) MacTex (mac users)

1.3.2 Installing RISE

There are (at least) two ways to acquire RISE:

1.3.2.1 The zip file option

1. Go online to https://github.com/jmaih/RISE_toolbox
2. download the zip file and unzip it in some directory on your computer.

This option is not recommended but is convenient for people who are not allowed to install new software on their machines/laptop.

1.3.2.2 Github for the bleeding-edge installation (highly recommended)

1. Go to http://windows.github.com if you are a windows user or to http://mac.github.com if you are a mac user
2. Create an account online through the website and download the Github program
3. Sign in both online and on the github on your machine. It is obvious online, but on your machine, just go to Github>Preference>Account
4. Go online to https://github.com/jmaih/RISE_toolbox
5. Look for an icon with title 'Clone in Desktop' (or possibly clone in mac). There are options to locate where the repository will reside

The reason why this option is recommended is that you don't need to re-download the whole toolbox every time a marginal update is made. With one click and within seconds you can have the version of the toolbox on your computer updated.

1.3.2.3 The git option (never tested!!!)

The following has never been tested and so the syntax might be wrong:

```
git clone https://github.com/jmaih/RISE_toolbox.git
```

1.3.2.4 Testing your installation

More on this later…

1.3.3 Loading and starting RISE

1. Locate the RISE_toolbox directory and add its path to matlab in the command window as

   ```
   addpath('C:/Users/JMaih/GithubRepositories/RISE_toolbox')
   ```

2. You will need to adapt this path to conform with the location of the toolbox on your machine.
3. run rise_startup()

1.3.4 Updating RISE

New features are constantly added, efficiency is improved, and users sometimes report bugs that are corrected. All this makes it necessary to update RISE every now and then in order to keep up with the latest changes and developments.

However, updating RISE depends on precisely how you installed it in the first place:

- If you downloaded a zip file, you will have to redownload a zip file even if the recent change was just an added comma.
- if instead you invested in opening a github account, with one click you will be able to update just the changes you don't have.
- with git, you would just execute the command

```
git pull
```

1.3.5 The work environement

Anylyzing RISE's DSGE models is done by writing a model file whose file name can have any of the following extensions `.rs`, `.rz` or `.dsge`. All other actions are run directly from within Matlab.

Running other classes of models is done directly from within Matlab.

1.4 Documentation of the +rise package

1.4.1 Opening the PDF documentation

doc: Opens the RISE documentation

Syntax:

```
rise.doc

rise.doc()
```

Args:

none

Returns:

none

1.4.2 Leaving/Quitting RISE

exit: exit from RISE

Syntax:

```
rise.exit

rise.exit()
```

Args:

none

Returns:

none

1.4.3 Obtaining the release information

release : obtain release information

Syntax:

```
r=rise.release()
```

Inputs: None

Outputs:

- **r** [char]: release information i.e. compatible Matlab versions

1.4.4 Starting RISE

STARTUP - Start RISE Toolbox.

Syntax:
rise.startup() rise_startup()

Args:

- None

Returns:

- None

Note:

- This function initializes the RISE Toolbox.

Example:

- rise.startup()

See also:

- *rise.exit*
- *rise_exit*

1.4.5 Obtaining the toolbox information

toolbox : provides toolbox information

Syntax:

```
 v=rise.toolbox()

Inputs: None

Outputs:
```

(continues on next page)

(continued from previous page)

```
- **v** [struct]: structure with information on

    #. The name of the toolbox
    #. Its version
    #. The Release i.e. the compatible version of Matlab
    #. The Date of the release
```

1.4.6 Obtaining the version of the Toolbox

version : current version of RISE

Syntax:

```
r=rise.version()
```

Inputs: None

Outputs:

- **r** [char]: version tag

1.4.7 Why : Answers to random question rhyming with RISE

why(debug)

This function generates answers to the question "Why do we [verb]?" by providing random or specific responses related to different actions.

Parameters:

- **debug: A logical value or a word character (preferably ending in "rize").**
  - If set to true, the function will display all the possible responses.
  - If set to false (default), the function will randomly select and display one response.
  - If a word character is provided, the function will search for that specific response and display it.

Usage:

- why() : Displays a random response to the question "Why do we [verb]?".
- why(true) : Displays all possible responses to the question "Why do we [verb]?".
- **why('specific_word')**
  [Displays the response related to the specific word, if found.] (e.g., why('polarize') will display the response for polarize, if available.)

Examples:

- why() Output: "Why do we accessorize? To add flair and personality to our outfits, transforming them from ordinary to extraordinary."
- why(true) Output:

"Why do we accessorize? To add flair and personality to our outfits, transforming them from ordinary to extraordinary."

"Why do we catheterize? To ensure proper urinary function and provide necessary medical support in challenging situations." … (Displays all possible responses)

- why('polarize') Output: "Why do we polarize? To create divisions or differences in opinion, beliefs, or attitudes, often leading to opposing or conflicting viewpoints."

Note: The function includes a collection of pre-defined responses related to different actions. The responses can be modified or extended within the function code.

1.4.8 Whatsup : Random sentences rhyming with RISE

1.5 Finding help

Basic principle

1. create an empty RISE object e.g.

```
r=rise.empty(0);
```

2. run methods(rise) or methods(tao) to see the functions/methods that can be applied to a RISE object
3. run those methods on r". e.g. "irf(r)", simulate(r)", solve(r)", etc. this will give you the default options of each method and tell you how you can modify the behavior of the method

1.6 Citing RISE in your research

Junior Maih (2015): Efficient perturbation methods for solving regime-switching DSGE models. Working Paper 2015/01, Norges Bank, January 2015.

1.7 Contributing to RISE

There are various ways you can contribute to RISE

- Asking questions in the forum
- Answering questions in the forum
- Reporting bugs
- Making suggestions
- Adding a star on Github
- Teaching RISE
- Contributing code for unsolved or novel problems
- Joining the team as a benevolent programmer

1.8 Acknowledgements

Many people have, oftentimes unknowingly, provided help in the form of reporting bugs, making suggestions, asking challenging questions, etc. I would like to single out a few of them, but the list is far from exhaustive:

- Stephane Adjemian
- Sehyoun Ahn
- Andrew Binning
- Michel Juillard
- Jesper Linde
- Francesca Loria
- Yasin Mimir
- Marco Ratto
- Dan Waggoner
- Pablo Winnant
- Raf Wouters

CHAPTER

TWO

HIT THE GROUND RUNNING : AN INTRODUCTORY EXAMPLE

2.1 The model economy

Consider the following 3-equation new Keynesian model (the derivations are available upon request)

The first equation is an (Euler equation) IS curve relating output to the real interest rate.

$$\beta E_t \frac{r_t}{\pi_{t+1}} \left[\frac{1 - \frac{\kappa}{2}\left(\pi_t - \pi_{t-1}^{\alpha}\pi^{1-\alpha}\right)^2}{1 - \frac{\kappa}{2}\left(\pi_{t+1} - \pi_t^{\alpha}\pi^{1-\alpha}\right)^2} \right] \frac{Y_t}{Y_{t+1}} = 1 \tag{2.1}$$

The second equation is a nonlinear Phillips curve with price adjustment costs.

$$\begin{aligned} 0 \quad = \quad & \chi_0 \eta_t \left[1 - \frac{\kappa}{2}\left(\pi_t - \pi_{t-1}^{\alpha}\pi^{1-\alpha}\right)^2\right] \left(\frac{Y_t}{Z_t}\right)^{1+\chi} \\ & + (1-\eta_t) - \kappa\left(\pi_t - \pi_{t-1}^{\alpha}\pi^{1-\alpha}\right)\pi_t \\ & + \beta\kappa E_t\left(\pi_{t+1} - \pi_t^{\alpha}\pi^{1-\alpha}\right)\pi_{t+1}\left[\frac{1-\frac{\kappa}{2}\left(\pi_t - \pi_{t-1}^{\alpha}\pi^{1-\alpha}\right)^2}{1-\frac{\kappa}{2}\left(\pi_{t+1} - \pi_t^{\alpha}\pi^{1-\alpha}\right)^2}\right] \end{aligned} \tag{2.2}$$

The third equation is a monetary policy reaction function given by a Taylor-type of rule.

$$\frac{r_t}{r} = \left(\frac{r_{t-1}}{r}\right)^{\rho} \left[\left(\frac{\pi_t}{\pi_*}\right)^{\psi_\pi(s_t)}\right]^{(1-\rho)} \exp\left(\sigma_r \varepsilon_{r,t}\right) \tag{2.3}$$

The fourth equation describe the evolution of the cost-push shock process

$$\log \eta_t = (1-\rho_\eta)\log\eta + \rho_\eta \log \eta_{t-1} + \sigma_\eta \varepsilon_{\eta,t} \tag{2.4}$$

The fifth equation describes the evolution of the technology shock process.

$$\log\left(\frac{Z_t}{Z_{t-1}}\right) = (1-\rho_z)\,\mu_z(s_t) + \rho_z \log\left(\frac{Z_{t-1}}{Z_{t-2}}\right) + \sigma_z \varepsilon_{z,t} \tag{2.5}$$

Where, Y_t denotes output, r_t the gross nominal interest rate (and r its steady state value), π_t inflation (and π_* the inflation rate targeted by the central bank and π the steady state inflation). Z_t is the technology level. These variables will be referred to as **endogenous** variables.

In the notation above, we use the convention that uppercase letters denote nonstationary variables and lower-case letters denote stationary variables. This is simply stress the fact that the model is nonstationary and will not be stationarized.

The vector of shocks, known in RISE as **exogenous** variables, is distributed as

$$\begin{bmatrix} \varepsilon_{z,t} \\ \varepsilon_{r,t} \\ \varepsilon_{\eta,t} \end{bmatrix} \sim N(0, I_3)$$

The other elements of the model are **parameters**, whose description and baseline values are given in the table below. Later on we will demonstrate how to update the values of those parameters via estimation.

We introduce regime switching in the model by assuming — following an example by [Foerster *et al.*, 2016] — that μ_z, the growth rate of the economy, and ψ_π, the central bank's reaction to deviation of inflation from its target, are switching in locksteps (This constraint is made simply to keep the example as simple as possible. But it is very easy to relax such an assumption).

We would like to :

1. Apply a baseline parameterization and solve the model without stationarizing it.
2. Analyze its dynamics
3. Estimate the parameters of the model on US data using Bayesian techniques

Such an exercise is straightforward to implement and run in RISE. We will need to

1. Write down the model equations in a text file with extension rs, rz or dsge
2. Collect and process the data
3. Write Matlab instructions for the different tasks.

2.2 Baseline calibration

We consider the following calibration for illustration purposes.

Parameter	value	Interpretation
β	0.99	discount factor
κ	161	Adjustment cost coefficient
π^*	$1.02^{0.25}$	Inflation target
α	0.5	Indexation to past inflation
η	6	Steady state elasticity of subst. intermediate goods
χ	0.7	Labor supply elasticity
χ_0	1	scale of disutility of labor
$\psi_\pi(1)$	2.5	Reaction to inflation in state 1
$\psi_\pi(2)$	0.9	Reaction to inflation in state 2
ρ	0.7	Interest rate smoothing
ρ_z	0.75	Persistence of technology shocks
ρ_η	0.75	Persistence of cost-push shocks
$\mu_z(1)$	$1.04^{0.25}$	Growth rate of technology in hawkish regime
$\mu_z(2)$	$1.01^{0.25}$	Growth rate of technology in dovish regime
σ_z	0.05	Standard deviation technology shocks
σ_η	0.05	Standard deviation cost-push shocks
σ_r	0.05	Standard deviation monetary policy shocks
p_{11}	0.95	Probability of staying in state 1 (hawkish state)
p_{22}	0.9	Probability of staying in state 2 (dovish state)

2.3 Preparing the model for RISE

The equations above alongside their variables and parameters can be written in the most natural way in a text file with extension **.rs**, **.rz**, **.dsge** . In this case, we created a file with name "model.rs".

In the file, although the order is not important, we first declare the various atoms using keywords to announce the type. In particular,

- endogenous variables are listed under @endogenous
- exogenous variables are listed under @exogenous
- parameters are listed under @parameters

We can optionally give a description of each atom using pairs of quote marks. e.g. R is the "interest rate".

Successive atoms can be separated by a blank spaces, a comma or both.

```
1   %% Simple New Keynesian model
2
3   @endogenous Y "Output", R "interest rate", PAI "Inflation"
4   Z "Technology", ETA "Cost-push shock process"
5
6   @log_variables @all_but
7
8   @exogenous EZ "Technology shock", ER "Monetary policy shock", EETA "Cost-push shock"
9
10  @parameters beta "discount factor", kappa, alpha, eta, chi0, chi,
11  rhoz, rhoeta, sigr, sigz, sigeta, paitarg
12  snk_tp_1_2, snk_tp_2_1
13
14  @parameters(snk,2) muz, psi_pai
15
16  @observables PAI, R
17
```

Note that there are two types of parameters. The constant parameters and the switching parameters are declared in different lists.

For the switching parameters, the user must give a name for each Markov process. In the example at hand, we called the process **snk** and assigned it two states (**parameters(snk,2)**), and then listed the parameters controlled by the Markov process. Obviously, a parameter cannot belong to more than one list.

Markov processes give rise to transition probabilities, which in the current example are constant over time.

It should be clear by now that in RISE a block starts with a keyword and ends when another keyword appears or when the file ends.

There are two more declarations that deserve our attention. The one at line 16 simply lists the variables that will be observed in case we conduct a filtration exercise.

The declaration at line 6 is more subtle. *@log_variables* is the keyword for listing the endogenous variables for which we are going to take a log approximation. In this particular case we would like to take a log approximation of all the variables. Therefore the natural way to write this would have been **@log_variables Y R Z PAI ETA**. Instead of doing that we chose to list the variables for which we are not going to take a log approximation **@log_variables @all_but**. That list is obviously empty.

The second part of the code lists the model equations. We declare the model equations by issuing keyword **@model** and there can be many such declarations for the same model. Each equation can be optionally prepended with an equation tag describing the equation. Just like the description of the atoms, the description of the equation tags appears in pairs of quote marks.

```
@model

    "IS curve"
    beta*R{t}/PAI{t+1}*[(1-kappa/2*(PAI{t}-PAI{t-1}^alpha*PAI{stst}^(1-alpha))^2)/...
        (1-kappa/2*(PAI{t+1}-PAI{t}^alpha*PAI{stst}^(1-alpha))^2)]*Y{t}/(Y{t+1}*PAIZ{t+1}) = 1;

    "Phillips curve"
    0 = chi0*ETA*[1-kappa/2*(PAI{t}-PAI{t-1}^alpha*PAI{stst}^(1-alpha))^2]*Y^(1+chi)+...
        (1-ETA{t})-kappa*(PAI{t}-PAI{t-1}^alpha*PAI{stst}^(1-alpha))*PAI{t}+...
        beta*kappa*(PAI{t+1}-PAI{t}^alpha*PAI{stst}^(1-alpha))*PAI{t+1}*...
        [(1-kappa/2*(PAI{t}-PAI{t-1}^alpha*PAI{stst}^(1-alpha))^2)/...
        (1-kappa/2*(PAI{t+1}-PAI{t}^alpha*PAI{stst}^(1-alpha))^2)];

    "Monetary policy reaction function"
    R{t}/R{stst} = (R{t-1}/R{stst})^rho*[(PAI{t}/paitarg)^psi_pai]^(1-rho)*exp(sigr*ER{t})
    # PAI{t}=paitarg;

    "Cost-push shock process"
    log(ETA{t}) = (1-rhoeta)*log(eta) + rhoeta*log(ETA{t-1}) + sigeta*EETA{t};

    "Technology shock process"
    log(PAIZ{t}) = (1-rhoz)*log(muz) + rhoz*log(PAIZ{t-1}) + sigz*EZ{t};

    "Output growth"
    DY = Y{t}/Y{t-1};
```

In this example, we choose to denote time using curly braces. RISE would also accept parentheses although in that case it becomes difficult to know whether we are referring to a variable or a function. In any case, **stst** denotes the steady state.

2.4 Creating the model object (rising the model)

Once the model file is written, we can go ahead an create a rise model object

```
m=rise('model');
```

or alternatively a dsge model object

```
m=dsge('model');
```

Both classes are isomorphic except that a dsge object is not a rise object.

Without further options, RISE will compute and store in memory, first-order symbolic derivatives suitable for solving a first-order perturbation.

2.5 Parameterizing the model

There are different ways of parameterizing a model. In the code block below, we use a Matlab structure.

```
p=struct();
p.beta=0.99;
p.kappa=161;
p.paitarg=1.02^0.25;
p.alpha = 0.5;
p.eta = 6;
```

(continues on next page)

(continued from previous page)

```
p.chi = 0.7;
p.chi0 = 1;
p.psi_pai_snk_1=2.5;
p.psi_pai_snk_2=0.9;
p.rho = 0.7;
p.rhoz=0.75;
p.rhoeta=0.75;
p.muz_snk_1=1.04^.25;
p.muz_snk_2=1.01^.25;
p.sigz=0.05;
p.sigeta=0.05;
p.sigr=0.05;
p.snk_tp_1_2=1-0.95;
p.snk_tp_2_1=1-.9;

m=set(m,'parameters',p);
```

Note how we write the switching parameters : the name of the parameter is followed by an underscore, then the name of the controlling Markov process, then a second underscore, then the state of the Markov process.

2.6 Setting some options for the solution

We could have stationarized the model but we didn't do so. Here we let RISE know that

1. the model is nonstationary : solve_bgp =true
2. We would like to use the Maih-Waggoner perturbation : solve_perturbation_type = 'mw'
3. Because we are using the Maih-Waggoner perturbation, we can apply an efficient solver : solver='dsge_udc'

```
m=set(m,'solve_perturbation_type','mw',...
    'solve_bgp',true,...
    'solver','dsge_udc');
```

2.7 Stochastic simulation in a Dynare-like run

Dynare's keyword and function "stoch_simul" has been overloaded to work with dsge/rise objects. But unlike Dynare's stoch_simul, RISE's stoch_simul will not create graphic files and store them. It is the responsibility of the user to collect whatever information he needs in the stochastic simulation structure, called **info** in the code below, for further processing.

```
info =stoch_simul(m);

disp(info)
```

The output structure will contain among other things

- simulated data
- moments of endogenous variables
- impulse responses

- autocorrelation of endogenous variables
- skewness and kurtosis of endogenous variables

2.8 Solving the model and printing the solution

An alternative way of running rise is to do one thing at a time. In this section we are simply solving the model and printing the policy functions.

By default, the model is solved at first-order of perturbation.

```
m = solve(m);

print_solution(m)
```

2.9 Solving a 5th-order perturbation

```
m5=solve(m,'solve_order',5,...
    'solve_derivatives_type','automatic');

print_solution(m5)
```

Because we have not instructed RISE to compute symbolic derivatives of order 5, we have two choices:

- we can go back and run $m = rise('model','max_deriv_order',5)$ so as to have symbolic derivatives ready or
- change the type of derivatives

In the statement above, we opt for automatic (or algorithmic) derivatives. Another choice could have been (finite-difference) numerical derivatives, but they are very imprecise.

2.10 Checking the accuracy of derivatives

```
check_derivatives(m)
```

Here we check wether symbolic and numerical derivatives give similar results as automatic derivatives taken to be the most accurate.

2.11 Impulse response functions

```
myirfs=irf(m);

% plot irfs for the original variables only (exclude auxiliary ones)

quick_irfs(m,myirfs,get(m,'endo_list(original)'))
```

Here we compute impulse responses. RISE will return the result as a structure of time series

- The first level will be the names of the shocks (exogenous variables)

- The second level will be the names of the endogenous variables

The irf function just as virtually all functions in the dsge/rise class can be called with options.

2.12 Collecting Data from FRED

Now we want to estimate unobservable variables through filtration and eventually confront the model to the data. In both cases we need observable variables. One way to obtain those variables is to fetch them from the FRED database.

Once the variables we want are fetched, we can transform them so as to relate them to model variables.

Below here, we are selecting the sample to use under filtration, but we do not have to do that at this stage. It can be done during filtration or during estimation.

```
xrange='1960Q1:2022Q3';

rawdb=fetch_fred({'BPCCRO1Q156NBEA','BOGZ1FL072052006Q','NAEXKP01USQ657S'});

db=struct();

db.PAI=1+rawdb(1).series(xrange)/100;

db.R=1+rawdb(2).series(xrange)/100;

db.GROWTH=1+rawdb(3).series(xrange)/100;
```

2.13 Filtration

Where we apply the Kalman filter to estimate unobservable variables at the current calibration of the model.

```
m=set(m,'data',db,...
    'kf_init_variance',1,...
        'kf_presample',10);

f=filter(m);
```

Before doing that, we set some options. The 'data' option does not need any further explanation. However the two other options are there for a purpose :

- kf_init_variance = 1 : we set the initial variance for all endogenous variables to 1. This is because the model is nonstationary and therefore does not have a finite variance for all endogenous variables.
- kf_presample = 10 : we are burning 10 initial observations that will not be taken into account when computing the likelihood. This is an attempt to amend the fact that we have imposed a finite variance to nonstationary variables.

2.14 Priors for Bayesian estimation

We set priors for Bayesian estimation. In this example we do not set priors in terms of means and standard deviations but rather in terms of quantiles. Of course we could have a mixture in which some priors are set in terms of means and standard deviations and some other set in terms of quantiles. See the next chapter for details.

```
priors=struct();
priors.kappa={p.kappa,5,20,'gamma(.9)'};
priors.alpha={p.alpha, 0.05,0.948,'beta(.9)'};
priors.chi={p.chi,1.5,3,'gamma(.9)'};

priors.eta={p.eta,3,8,1,12,'beta(.9)'};
priors.muz_snk_1={p.muz_snk_1,1.02^.25,1.05^.25,1,1.07^.25,'beta(.9)'};
priors.muz_snk_2={p.muz_snk_2,1.01^.25,1.03^.25,1,1.04^.25,'beta(.9)'};

priors.rho={p.rho, 0.05,0.948,'beta(.9)'};
priors.rhoz={p.rhoz, 0.05,0.948,'beta(.9)'};
priors.rhoeta={p.rhoeta, 0.05,0.948,'beta(.9)'};

priors.sigz={p.sigz, 0.0005,1.0,'sichisq(.9)',0,3};
priors.sigeta={p.sigeta, 0.0005,1.0,'sichisq(.9)',0,3};
priors.sigr={p.sigr, 0.0005,1.0,'sichisq(.9)',0,3};

priors.snk_tp_1_2={p.snk_tp_1_2, 0.01,0.411,'beta(.9)'};
priors.snk_tp_2_1={p.snk_tp_2_1, 0.01,0.411,'beta(.9)'};

priors.psi_pai_snk_1={p.psi_pai_snk_1, 1.1,2.5,1,3,'beta(.9)'};
priors.psi_pai_snk_2={p.psi_pai_snk_2, 0.5,1.1,0.1,1.5,'beta(.9)'};
```

2.15 Visualizing the priors

```
plotOpts=struct();
plotOpts.prior_trunc=2.6e-3;

rdist.plot(priors,plotOpts,[],struct('linewidth',2))
```

2.16 Posterior maximization

The data were already pushed into the model object while we were doing filtration. All we need now in order to maximize the posterior distribution is prior information and possibly some additional options for tailoring the estimation process.

```
mest=estimate(m,'estim_priors',priors);
```

2.17 Posterior simulation

We don't have to maximize the posterior distribution and use it as a starting point for posterior simulation. However, in this class of models, it is a good practice to do so if we do not have a good posterior sampling procedure.

The posterior samplers in RISE require a starting point and a positive-definite covariance matrix. Again, let's stress that the starting point need not be the mode of the posterior distribution.

```
[objective,lb,ub,x0,SIG]=pull_objective(mest);

scale=0.0794;
myOpts=struct();
myOpts.tunedCov=scale*SIG;
myOpts.N=20000;

% Use a maximization objective for sampling
energy=@(varargin)-objective(varargin{:});

results=sample(rsamplers.rwmh(energy,x0,lb,ub,myOpts));
```

Here we illustrate the basic call to the **Random-Walk Metropolis-Hastings** algorithm. More options can be added.

Note that **objective** is a function to be minimized while **energy** is a function to be maximized.

This is just one out of many possibilities for sampling the posterior distribution in RISE.

2.18 Computation of the marginal data density using the Bridge approach

RISE has 9 ways of computing the marginal data density. Here we illustrate the computation of the marginal likelihood or marginal data density by the method of Meng and Wong (1996).

```
mddobj=mdd(results,energy,lb,ub,...
        [],[],true);

bridge(mddobj,true,mdd.global_options)
```

CHAPTER

THREE

DSGE MODELING

3.1 Description

DSGE (Dynamic Stochastic General Equilibrium) models are a type of macroeconomic model that are widely used in modern macroeconomics. Some of the strengths of these models include:

1. **Microfoundations**: DSGE models are based on microeconomic principles and are therefore able to provide a consistent and coherent framework for analyzing macroeconomic phenomena.
2. **Flexibility**: DSGE models can be used to analyze a wide range of macroeconomic phenomena, including business cycles, inflation, and growth.
3. **Quantitative**: DSGE models are able to generate quantitative predictions about the economy, and forecasts that compare well with other classes of models such as Bayesian VARs.
4. **Policy Analysis**: DSGE models can be used to analyze the effects of different policy changes, such as changes in interest rates, government spending or exchange rate regimes.
5. **International aspects**: Some DSGE models are able to incorporate international aspects such as trade, currency and capital flows.

RISE extends the constant-parameter DSGE model to allow for regime switching. The regime-switching DSGE model is given by:

$$E_t \sum_{r_{t+1}=1}^{h} p_{r_t,r_{t+1}}\left(\mathcal{I}_t\right) f_{r_t}\left(x_{t+1}\left(r_{t+1}\right), x_t\left(r_t\right), x_{t-1}, \theta_{r_t}, \theta_{r_{t+1}}, \varepsilon_t\right) = 0$$

- $p_{r_t,r_{t+1}}\left(I_t\right)$: probability of going from state r_t in the current period to state r_{t+1} in the next one
- f_{r_t}: (potentially) nonlinear function of its arguments
- $x_t\left(r_t\right)$: vector of all endogenous variables in the current regime r_t
- θ_{r_t}: parameters in the current regime
- $\varepsilon_t \sim N\left(0, I\right)$: vector of stochastic shocks

3.2 An example

See the previous chapter.

3.3 The model(rise/dsge) file language

3.3.1 Conventions

3.3.1.1 Comments and block comments

RISE supports single line comments %, or //:

```
% this is a comment

// this is also a comment
```

RISE supports block comments of the Matlab style %, or //:

```
%{
This is a block comment
That RISE will ignore
%}
```

3.3.1.2 Time and steady state

Our preferred way to denote time uses curly braces. But RISE would also accept parentheses although in that case it becomes difficult to know whether we are referring to a variable or a function. That said,

- For contemporaneous variables we have : X = X{0} = X{t+0} = X{t} = X(0) = X(t+0) = X(t)
- For lagged variables we have : X{-1} = X{t-1} = X(-1) = X(t-1)
- For lead variables we have : X{1} = X{+1} = X{t+1} = X(1) = X(+1) = X(t+1)
- For the steady state we have : X{stst} = steady_state(X) = X(stst)

In brief, RISE is flexible. Use the notation that you prefer!

3.3.1.3 Auxiliary variables

RISE automatically creates auxiliary variables so that the maximum lag and the maximum lead for any model is always 1. The user should not worry about these variables.

For the moment, RISE does not handle expectations that appear inside an equation. E.g.

$$Y_t = 1/E_t\left(X_{t+1}\right)$$

RISE will instead compute

$$Y_t = E_t\left(1/X_{t+1}\right)$$

It is the responsibility of the user to create the appropriate auxiliary variable, say, $At = E_t\left(X_{t+1}\right)$ and then rewrite the equation as

$$Yt = 1/At$$

Or to write the equation differently

$$E_t\left(Y_t X_{t+1}\right) = 1$$

Note: RISE automatically creates auxiliary variables to deal with

- leads and lags greater than 1 and
- switching parameters with a different dating than the current period

3.3.1.4 Functions

Known functions

RISE supports four basic arithmetical operations (+, -, *, /) as well as powers (^). RISE relies on Matlab for these operations and so one has to be careful with the power and rdivide (/) operators :

- $a^{b^c} = (a^b)^c$ for a^b^c
- $a/b^c = a/(b^c)$ for a/b^c

Naturally, the precedence of arithmetical operators can be changed using parentheses.

The list of functions that RISE knows how to differentiate includes : *abs, acos, acosh, and, asin, asinh, atan, atanh, betacdf, betainv*, *betapdf, char, cos, cosh, cot, eq, erf, exp, ge, get, gt*, *kron, le, log, log10, lt, max, min, minus, mpower, mrdivide, mtimes, ne, normcdf, norminv*, *normpdf, or, plus, power, rdivide, sign, sin, sinh, sqrt, tan, tanh, times, uminus, uplus*.

External functions

Sometimes, the user may have a function that is unknown to RISE. In that case, the user should write the function such that RISE can evaluate it as well as its derivatives.

The function should be written so that it is possible to call it in two different ways:

```
x=f(a1,a2,...,an)

x=f(a1,a2,...,an,'diff',der)
```

where :

- x is the output of the function
- a1, a2, …, an : are the main inputs of the function
- 'diff' is an input that signals that triggers the computation of derivatives
- der is a scalar of or a 1 x k vector specifying the derivatives requested. The number of elements, k, in der is the order of differentiation. The numbers in the vector are the location of the elements to differentiate. E.g:
  - x=f(a1,a2,…,an,'diff',1) : first-order derivative wrt first argument

- x=f(a1,a2,...,an,'diff',[1,2]) : second-order derivative wrt arguments 1 and then 2
- x=f(a1,a2,...,an,'diff',[1,2,3]) : third-order derivative wrt arguments 1, 2, and 3
- x=f(a1,a2,...,an,'diff',[i,j,k,l,m]) : fifth-order derivative wrt arguments i, k, k, l, and m

It is expected that

- f(a,'diff',[i,j])=f(a,'diff',[j,i])
- f(a,'diff',[i,j,k]) = f(a,'diff',[i,k,j]) = f(a,'diff',[j,i,k]) = f(a,'diff',[j,k,i]) = f(a,'diff',[k,i,j]) = f(a,'diff',[k,j,i])
- etc

3.3.1.5 Discountinous functions

On local approximations

RISE approximates discontinuous functions in the same way it does with continuous functions. As a result, inequality constraints such as max or min will not hold when solving a model using perturbation.

One way to assuage the problem is to replace discontinuous functions with smooth approximations and solve the model using higher-order perturbations. Even so, the inequalities will not necessarily be satisfied.

Another way to deal with the problem is to use algorithms designed for occasionally-binding constraints. In that respect, RISE implements four classes of algorithms :

1. perfect foresight (and extended path)
2. piecewise linear approximations
3. anticipated shocks
4. regime switching

Danger: There is a widespread belief that projection methods can successfully deal with problems of with discontinuous functions. This is untrue !

Smooth approximations

RISE implements smooth approximations of abs, min and max functions. They can be accessed respectively as :

- sabs
- smin
- smax

3.3.1.6 Shock distributions

In RISE, all shocks have zero mean and standard deviation 1. By default, they are normally distributed. Skewed shocks or more generally shocks having non-normal distributions are easily accommodated. However, even they should have mean zero and variance 1. Therefore, we will not have variance-covariance matrices of shocks.

The way to handle correlated shocks in RISE is to create auxiliary endogenous variables that are combinations of uncorrelated shocks.

First and second-order moments of shocks

All shocks have mean 0 and standard deviation 1

Higher-order moments of shocks

Skewed shocks are declared in a structure. Each element of the structure represents a particular shock and is given in a 1-by-2 cell array of functions.

- The first element of the cell array computes moments and takes two inputs : the model object and the moment order
- The second element of the cell array computes draws from the distribution. It should take as inputs the same inputs as matlab's rand or randn functions.

Eventually, the structure is passed to the model object through the option **solve_user_defined_shocks** e.g. m=set(m,'solve_user_defined_shocks',myshocks)

Here is an example on how to write a function of skewed shocks

```
function myshocks=online_skewed_shocks(dof)
if nargin==0
    dof=1;
end
myshocks=struct();
momfunE=@(obj,order)moments.tailored_chisquare(order,dof);
tmpfun=@(varargin)if_then_else(length(varargin)>1,[varargin{:}],[1,varargin{1}]);
drawfunE=@(varargin)reshape(...
    (sum(randn(dof,prod([varargin{:}])).^2,1)-dof)/sqrt(2*dof),...
    tmpfun(varargin{:}));
myshocks.E={momfunE,drawfunE};

norm_moments=[0,1,0,3,0,15,0,105,0,945];
momfunOMEGA=@(obj,order)norm_moments(1:order);
drawfunOMEGA=@(varargin)reshape(randn(1,prod([varargin{:}])),tmpfun(varargin{:}));
myshocks.OMEGA={momfunOMEGA,drawfunOMEGA};
end
```

3.3.2 Ways to declare a model

A model can be declared either inside a model file or directly within Matlab.

3.3.2.1 Declaring a model in a file

A model can be declared in text file with extensions ".rs", ".rz" or ".dsge".

Putting double comments "%%" at the (top) beginning of the file, followed by a comment will result in RISE using that comment as a description of the model.

3.3.2.2 Declaring a model online

Declaring a model online simply amounts to writing whatever we would write in a model file as cell array of strings directly in Matlab. Below is an example

```
simpleModel={
    '%% model without parameters and shocks'
    '@endogenous x'
    '@model'
    'x=1.05*x{-1};'
    };
```

3.3.3 Handling multiple multiple model files

This section outlines various methods to create models in RISE, defining endogenous variables, exogenous variables, parameters, and equations.

3.3.3.1 Method 1: Simple Model File

To create a RISE model from a single model file

```
m = rise('modelFile')

m = rise('modelFile.rs')
```

3.3.3.2 Method 2: Multiple Model Files

To create a RISE model from multiple model files

```
m = rise({'modelFile1.rs', 'modelFile2.rs', ..., 'modelFilen.rs'})

m = rise({'modelFile1', 'modelFile2', ..., 'modelFilen'})
```

3.3.3.3 Method 3: Simple Online Model (Using Cell Array)

To declare a RISE model online using a cell array

```
m = rise({
    '@endogenous x y z',
    '@parameters alpha beta',
    '@model',
    'x = alpha * y',
    'y = beta * z'
})
```

3.3.3.4 Method 4: Multiple Online Model Codes (Using Cell Array of Cell Arrays)

To declare multiple RISE models online using a cell array of cell arrays

```
onlineCellstr1={
                '@endogenous e'
                '@exogenous x'
                '@model '
                ' x = e; '
                }

onlineCellstr2 = ...

onlineCellstrj={
                '@log_variables @all_but'
                }

m = rise({onlineCellstr1,onlineCellstr2,...,onlineCellstrn})
```

3.3.3.5 Method 5: Combination of Model Files and Online Model Codes

To create a RISE model by combining model files and online model codes

```
info = struct();
info.disk_files = {'modelFile1.rs', 'modelFile2.rs', ..., 'modelFilen.rs'};
info.online_data = {'online_info1',...,'online_infom'};
or even using cell array of cell arrays as
info.online_data = {online_info1,...,online_infom};
m = rise(info);
```

3.3.4 Atoms declarations

Successive atoms can be separated by a blank spaces, a comma or both.

Each variable and parameter name begins with a letter. RISE is case sensitive and supports Latin alphabet only. Digits and underscores are allowed in names provided they do not come first.

3.3.4.1 @endogenous

@endogenous is the keyword for declaring the endogenous variables. Each variable name is optionally followed by its description in double quote marks. Example

```
@endogenous C, I, R "Interest rate", PAI
```

This block is compulsory!.

The keyword **@endogenous(log)** can be used to declare variables for which a log-transform will be applied. Example

```
@endogenous(log) C, I
```

There can be multiple blocks of **@endogenous(log)**.

While **@endogenous(log)** is the preferred syntax, one can also use **@endogenous(log_vars)** or **@endogenous(log_variables)**.

@endogenous(log) (or its variants) cannot be used in combination with the blocks **@log_vars**, **@log_variables**, **@level_variables** with can also be used to declare the variables for which a log-transform is to be taken or not.

3.3.4.2 @exogenous

@exogenous is the keyword for declaring the exogenous variables or shocks. Each variable name is optionally followed by its description in double quote marks. Example

```
@exogenous EA, EZ "Technology shock", ER "Interest rate", EG
```

This is only necessary if the model has some exogenous variables.

3.3.4.3 @parameters

@parameters is the keyword for listing the parameters. Each parameter name is optionally followed by its description in double quote marks. Example

```
@parameters alpha, beta "discount factor", rho
```

This is only necessary if there are parameters whose values can change

In RISE, every parameter is controlled by a named Markov process. The name of the process controlling the constant parameters does not appear explicitly. That name cannot be changed, it is **const**.

In the presence of a regime switching model, *@parameters* is followed by the name of a markov process and the number of states of the markov process and then the list of the parameters controlled by the markov process. Example

```
@parameters(volatility,2) sige, siga "volatility of pref. shock", sigz
```

In the RISE "religion", we use

- "state" to denote the modes of a particular markov process. e.g. "volatility" in the code snippet above can be in state 1 or in state 2.
- "regime" to denote a combination of states from different independent Markov processes. e.g. consider a model with two independent Markov processes : one controlling the behavior of policy (hawkish or dovish) and one controlling the heteroskedasticity of shocks (low volatility or high volatility). We can have
  - a regime of dovish stance of monetary policy and high volatility
  - a regime of hawkish stance of monetary policy and high volatility
  - a regime of dovish stance of monetary policy and low volatility
  - a regime of hawkish stance of monetary policy and low volatility

Parameters for Transition probabilities

Markov processes give rise to transition probabilities. Transition probabilities have a specific syntax. The following rules have to be followed :

- each transition probability starts with the name of the markov process it caracterizes.
- each transition probability name must have exactly 3 underscores.
- It is forbidden to declare diagonal elements of a transition matrix.
- When transition probabilities are constant, they can be declared as parameters.
- the general syntax is mcName_tp_i_j, {i,j}=1,2,..., where
  - mcName is the name of Markov process
  - tp stands for transition probability
  - i is the state today
  - j is the state tomorrow

Parameters for Measurement Errors

Just like regular shocks, measurement errors are mean-zero and variance-one processes. However, by default, measurement errors are scaled by zero when not declared. The variability of measurement errors is changed by declaring parameters with a specific syntax. The syntax for declaring measurement error parameters and possibly their correlation is as follows

- stderr_OBSNAME : scale on measurement error for observed endogenous variable OBSNAME
- corr_OBSNAME1_OBSNAME2 : correlation of measurement errors on observed endogenous variables OBSNAME1 and OBSNAME2.

Warning: for any observed variable X declared as log variable, the measurement errored variable is $X_{obs} = X \exp\left(stderr_X \times E_X\right)$ so that in log terms $x_{obs} = x + stderr_X \times E_X$.

Parameters for Bounded Rationality

RISE implements bounded rationality through the declaration of special parameters. See the section on tools for bounded rationality for details.

3.3.4.4 @observables

@**observables** is the keyword for declaring the variables to be observed during filtration. Only declared endogenous or exogenous variables can be observed. Example

```
@observables PAI, R, Y
```

Three remarks are in order :

- Only observed endogenous variables can give rise to measurement error parameters
- An exogenous variable that is observed is considered deterministic. For instance one can observed a time trend that affect some measurement equation. This gives us the flexibility to accommodate all kinds of exogenous observed variables.
- time-varying probabilities end up being endogenous variables and so they can be observed.

This block is optional. Necessary only if there is filtration.

3.3.4.5 @log_variables

@*log_variables* is the keyword for listing the endogenous variables for which we are going to take a log approximation. It is directly followed either by the list of the variables of interest or by @*all_but* followed by the list of the variables we DO NOT want to take a log approximation of.

In general, we don't have to take a log approximation and there are some rules one needs to be aware of when taking log approximations.

- we cannot take log approximation of variables that are either zero or negative in steady state
- we can optionally take log approximations for stationary variables whose steady state is positive.
- For nonstationary variables, we must take a log approximation for variables whose growth rate is computed as X_t/X_{t-1}.

More on @log_variables in the section for the computation of the steady state.

3.3.4.6 @level_variables

@*level_variables* is the keyword for listing the endogenous variables for which we are NOT going to take a log approximation. It is directly followed either by the list of the variables of interest or by @*all_but* followed by the list of the variables we DO want to take a log approximation of. In that sense, @*level_variables* is the complement of @*log_variables*.

Caution: Only one type (@log_variables or @level_variables) can appear in a model file at a time, but there can be several of the same type.

3.3.4.7 On-the-fly atoms declarations

We can also declare certain types of variables or parameters on the fly, i.e. directly in the model block. Each atom is followed by an appostrophe, followed by its type.

- endogenous : X'e | X'e{t+1} | X'e{t-1} | X'e(t+1) | X'e(t-1) ...
- exogenous : E'x | E'e{t+1} | E'e{t-1} | E'e(t+1) | E'e(t-1) ...
- parameters : theta'p | theta'e{t+1} | theta'e{t-1} | theta'e(t+1) | theta'e(t-1) ...

Note: If an atom is dated, the date comes after they type is declared.

3.3.5 The @functions block

This block serves to define carry-around functions that appear in the other blocks such as @model, @steady_state_model @optimization_problem, @parameter_restrictions.

Example

```
@functions

    Uc(a,b,xc) = 1/sigma*xc - 1/sigma*a + k{t};

    Uh(a,b,xh) = log(psi)-nu*xh+nu*b;
```

The requirements are as follows:

- The function should hold in one line and end with a semicolon.
- The input arguments should be non-dated.
- the right-hand side can have dates
- preferably the function should not be called with arguments containing complicated comma structures. For instance in the expression below, if a=minus(plus(e,b),rdivide(c,d)) plus and rdivide will be correctly parsed but minus will not
- The block can have comments but not equation tags

3.3.6 The @model block

The **@model** block is compulsory. This is where we declare the model equations. There are three types of equations that can enter the @model block.

3.3.6.1 Regular equations

E.g.

```
log(X{t}) = (1-rho)*log(xss) + rho*log(X{t-1}) + sig*E{t};
```

3.3.6.2 Equations broken across multiple lines

An Equation can be broken across multiple lines

E.g.

```
x{t} =
        rho1*x{t-1} +
        rho2*x{t-2} +
        rho3*x{t-3} +
stde*e{t};
```

As in Matlab, one can add "..." to denote the breaking of lines

E.g.

```
x{t} = ...
        rho1*x{t-1} + ...
        rho2*x{t-2} + ...
        rho3*x{t-3} + ...
stde*e{t};
```

As in Matlab, the "..." can be optionally followed by comments with no incidence on the subsequent computations.

E.g.

```
x{t} = ...
        rho1*x{t-1} + ... lag 1
        rho2*x{t-2} + ... lag 2
        rho3*x{t-3} + ... lag 3
stde*e{t};
```

3.3.6.3 The @E operator for expectations

Expectations are not always taken at time t. They can be taken earlier or after. The syntax is simple : @E[s](expression) or @E[t+s](expression) for expectations taken s period ahead (earlier if s is negative). Note that @E[0](expression) = expression, @E[s](y{t}) = y{t} if s>0.

E.g.

```
c{t}*theta*h{t}^(1+psi)=(@E[1]((1-alpha)*y{t})+@E[-2]((1-alpha)*y{t}))/2;
```

3.3.6.4 Equations with auxiliary expressions :"...$[]$..."

E.g.

```
exp(lb{t})=beta*exp(lb{t+1})*(alpha*exp($s{t+1}+y{t+1}$-k{t})+1-delta);
```

RISE will treat the term $[]$ as a forward-looking term and create an auxiliary variable to it. The auxiliary variable will have prefix **DISCRDOLL**. In the example above, the equation above becomes

```
exp(lb{t})=beta*exp(lb{t+1})*(alpha*exp(DISCRDOLL1{t+1}-k{t})+1-delta);
```

And the definition of the auxiliary variable is

DISCRDOLL1{t} = s{t} + y{t};

There is no requirement that all variables between $ signs be forward-looking. RISE will always lag the terms correctly when creating the corresponding auxiliary variable.

Hint: Relevance : When solving discretionary or stochastic replanning problems, the derivatives of forward-looking terms enter the equations of the model. To reduced the costs of computation, to the extent that this is feasible, we can gather together several forward-looking variables instead of implicitly creating the derivatives of each one of them separately.

3.3.6.5 Regular equations with equation simplification :"...#..."

Regular structural equations can optionally be followed by a # sign and then a simplified version of the same equation to be used when computing he steady state or the balanced growth path. E.g.

```
log(X{t}) = (1-rho)*log(xss) + rho*log(X{t-1}) + sig*E{t} # ...
        X{t}=xss;
```

3.3.6.6 Definitions : # Equation

RISE uses the pound sign at the beginning of an equation to declare a definition. This is an atom that is not declared elsewhere. Examples

```
# beta = alpha + gamma;

# OMG = Y{t}/X{t} + delta;
```

The rules are as follows :

- definitions can be functions of variables and parameters
- if a definition is a function of variables, as is the case for the second definition above, then it has to be systematically substituded.
- if a definition is a function of parameters only, it does not have to be substituted.
- If one definition is a function of variables, all definitions have to be substituted
- If definitions are functions of parameters, they have to be written in a recursive fashion such that one definition is either independent of the previous one or is a function of it. e.g

  ```
  # alpha = delta + omicron;
  # varrho = cos(alpha);
  ```

- The substitution of definitions is done by setting the option *definitions_inserted* to true while "rising" the model

  ```
  m=rise(...,'definitions_inserted',true)
  ```

3.3.6.7 Time-varying transition probabilities : ! Equation

When the transition probabilities are time varying, they can no longer be declared as parameters. They have to be declared in the model block. In order to differentiate them from other equations, we prefix them with an exclamation mark. E.g.

```
! volatility_tp_1_2 = 1/(1+exp(alpha*X{t}));

! pol_tp_1_2 = R>1;
```

Caution: time-varying probabilities should always evaluate in [0,1].

Hint: time-varying probabilities end up being endogenous variables and so they can

- be observed
- have measurement errors, e.g. stderr_mpeff_tp_1_2, stderr_mpeff_tp_2_1 for some markov process mpeff with transition probabilities mpeff_tp_1_2 and mpeff_tp_2_1
- be dynamic and used in other equations

Useful cumulative density functions that can be used for transition probablities

1. arc sine: $u = \frac{2}{\pi}\sin^{-1}\left(\sqrt{\frac{x-a}{b-a}}\right)$
2. Exponential: $u = 1 - \exp\left(-g\left(x-c\right)\right)$
3. Gumbel: $u = \exp\left(-\exp\left(-\frac{x-\mu}{b}\right)\right)$
4. Inverse Gaussian: $u = normcdf\left(\sqrt{\frac{\lambda}{x}}\left(\frac{x}{\mu}-1\right)\right) + \exp\left(\frac{2\lambda}{\mu}\right) normcdf\left(-\sqrt{\frac{\lambda}{x}}\left(\frac{x}{\mu}+1\right)\right)$
5. Kumaraswamy: $u = 1 - \left(1 - \left(\frac{x-c}{d-c}\right)^a\right)^b$ for $x \in [c, d]$
6. Logistic: $u = \frac{1}{1+\exp(-g(x-c))}$ for $x \in (-\infty, +\infty)$
7. Log Normal : $u = 0.5\left[1 + \text{erf}\left(\frac{\log(x)-a}{b\sqrt{2}}\right)\right]$ for $x \in (-\infty, +\infty)$
8. Normal: $u = 0.5\left[1 + \text{erf}\left(\frac{x-a}{b\sqrt{2}}\right)\right]$ for $x \in (-\infty, +\infty)$
9. Pareto: $u = 1 - \left(\frac{a}{x}\right)^b$ for $x \in [a, +\infty)$
10. Raised cosine: $u = 0.5\left[1 + \frac{x-\mu}{s} + \frac{1}{\pi}\sin\left(\frac{(x-\mu)}{s}\pi\right)\right]$ for $x \in [\mu - s, \mu + s]$ and $s > 0$
11. Weibull: $u = 1 - \exp\left(-\left(\frac{x}{b}\right)^a\right)$ for $x \in [0, +\infty)$

Hint: Note that those functions are monotonically increasing. One way to get them decreasing is to use the complement i.e. 1-u or 1-cdf(x,a,b)

3.3.7 The @optimization_problem block

The **@optimization_problem** block formerly known as **@planner_objective** is necessary only in the presence of an optimal policy problem. It is necessary for solving :

1. Commitment : with or without regime switching
2. Discretion (linear-quadratic and nonlinear): with or without regime switching
3. Loose Commitment : switch between discretion and commitment with constant probability in a linear-quadratic framework
4. stochastic replanning : switch between discretion and commitment with possibly time-varying probabilities in a nonlinear framework.

In that case the number of equations should be strictly less than the number of endogenous variables.

3.3.7.1 Ramsey Policy

```
@optimization_problem {commitment=1,discount=beta}  pi_c^2 + y^2;
```

or preferably

```
@optimization_problem {discount=beta}  pi_c^2 + y^2;
```

and then

```
m = set(m,'solve_policy_type','ramsey');
```

where "m" is the constructed model (dsge or rise) object

3.3.7.2 Discretionary Policy

```
@optimization_problem {commitment=0,discount=beta}  pi_c^2 + y^2;
```

or preferably

```
@optimization_problem {discount=beta}  pi_c^2 + y^2;
```

and then

```
m = set(m,'solve_policy_type','discretion');
```

where "m" is the constructed model (dsge or rise) object

Hint: by default, the "solver" option is always empty if not set

- RISE will check whether the system can be solved by eigenvalue methods. If this is the case, the option will be set to the default RISE solver for constant-parameter models : "rise_1"
- else RISE will use the default solver for regime-switching models : "mfi"
- Given the way RISE is organized, discretionary policy can be solved using "rise_1". This is because the problem can be solved conditional on the knowledge of the solution of the forward-looking variables. In this way, one can in principle, study the indeterminacy properties of a discretionary model.

- But we believe that insisting that the solution should be determinate is a waste of time. One thing is to say that for a particular parameterization we have a unique solution. Yet another thing is to say that for a solution to be interesting or useful, it has to be determinate. There is no justification for such a stance other than the fact that people sometimes say : "If I have many solutions I don't know which one to choose." Note that the only people who would make such complains are the ones solving linear or linearized DSGE models.
- With that said, the user is now free to choose which solver he wants to use. But I am just stressing that the behavior of "rise_1", which insists on determinacy is different from that of "mfi", which applies functional iteration to find a good solution. Both of those solvers are different from the Newton solvers "mn" and "mnk", which are locally stable around an equilibrium and try to find the solution that is the closest to the starting point. Other solvers are "dsge_groebner", which tries to find all possible solutions.
- Should you want to solve for all possible solutions, there are also dsge_schur and dsge_udc that can do the job but they are not appropriate for this context as they can be computationally taxing.

3.3.7.3 Loose commitment

```
@optimization_problem {discount=beta}  pi_c^2 + y^2;

@parameters(looseCommit,2) commitment

@parameters looseCommit_tp_1_2, looseCommit_tp_2_1
```

with the restriction

```
looseCommit_tp_1_2 = 1 - looseCommit_tp_2_1;
```

Note: The name for the markov process "looseCommit" is arbitrary. However,

- the number of states must be exactly 2.
- "commitment" is the exact name for the state indicator and can only assume values 1 and 0
- It is up to the user to choose the state in which commitment=1 and the state in which commitment = 0

Hint: Under estimation, one way to enforce the restriction that looseCommit_tp_1_2 = 1 - looseCommit_tp_2_1 is to

- declare an auxiliary parameter, say probCom, as non-switching
- set both looseCommit_tp_1_2 and looseCommit_tp_2_1 as endogenous probabilities i.e.:

  ```
  ! looseCommit_tp_1_2 = probCom;
  ! looseCommit_tp_2_1 = 1-probCom;
  ```

- estimate probCom

3.3.7.4 Stochastic Replanning

```
@optimization_problem {discount=beta}  pi_c^2 + y^2;

@parameters(stochrepl,2) commitment
```

The probabilities "stochrepl_tp_1_2" and "stochrepl_tp_2_1" can be endogenous or exogenous.

Note: The name for the markov process "stochrepl" is arbitrary. However, the number of states must be exactly 2.

3.3.7.5 Optimal Policy with no u-turn

RISE gives you the possibility of changing between discretion and commitment seemlessly. This flexibility comes at a cost: some of the derivatives that are expensive to compute are not used in the commitment/Ramsey case.

In order to avoid this inefficiency, it is advisable to set the option "no_u_turn" to true if you plan to do only Ramsey/Commitment policy. The syntax is

```
@optimization_problem {...,no_u_turn=true}  pi_c^2 + y^2;
```

Setting the option to true means that the standard algorithms for computing the solution for discretionary policy or stochastic replanning cannot be used.

Hint: It is still possible to run discretionary policy even when no_u_turn is set to true. However, a few considerations are in order

- The only solver one can use in this case is "loose_commitment"
- This works best in linear-quadratic systems
- There is a loss in accuracy if the original problem is not linear quadratic
- One cannot perform higher-order perturbation

3.3.8 The @optimization_constraints block (optional)

This is where we add inequality constraints to the optimization problem. The correct way to think about the @optimization_constraints block is as another block of equations that could have been written in the model block.

Warning: There can't be an **@optimization_constraints** block without an **@optimization_problem** block

There are two types of syntaxes that depend on whether one plans to solve the inequalities using regime switching or perfect foresight.

The syntax under regime switching is

```
@optimization_constraints
        "Equation tag 1"
        regime_indicator_1:Inequality_constraints_1;
```

(continues on next page)

(continued from previous page)

```
        ...

        "Equation tag m"
        regime_indicator_m:Inequality_constraints_m;
```

In this case, the "regime_indicator" components are switching parameters such that when they assume value 1, the inequality restriction (constraint) becomes binding. Conversely, when it is 0, the constraint does not bind.

Example

```
@optimization_constraints
        "Inequality constraint on the zero-lower bound"
        gam:I>=elb;
```

Tip: In the solving of the steady state, in the absence of an analytical solution it often helps to have a steady state version of the euler equation that gives the solution of the interest rate both in normal times and in abnormal times

```
RHO = 40/41*RHO(+1)+1/41*(I-PAI(+1))
# (1-gam)*(I-0) + gam*(I-elb) = 0
;
```

The syntax under perfect foresight (or potentially other solution techniques) is

```
@optimization_constraints
        "Equation tag 1"
        Inequality_constraints_1;

        ...

        "Equation tag m"
        Inequality_constraints_m;
```

In other words, the only difference in this case is that there are no regime indicators.

Caution: Each constraint must be an inequality "<", ">", "<=" or "<="

Attention: Strict inequalities will be treated as loose inequalities.

Error: The inequalities cannot be represented by functions such as "le", "lt", "gt", "ge".

Error: It is not possible to combine inequalities with regime indicators and inequalities without regime indicators. Either you do perfect foresight or you do regime switching.

Tip: It is possible to define a regime indicator as an endogenous variable instead of a parameter and still use it in the

@optimization_constraints block for the purpose of doing perfect foresight.

3.3.9 The @steady_state_model block (optional)

RISE gives you the flexibility of setting the program for solving the steady state or the balanced-growth path of the model either in the model file or outside the model file. The former strategy, presented below, is appealing when the user does not want to create too many files and that the model is small. The latter strategy gives the most flexibility including the possibility of overriding the information set inside the model file.

The **@steady_state_model** block should compute the steady state (and the balanced-growth path) analytically.In the absence of this block, all variables are assumed to be zero in steady state. Of course, RISE will check whether this is true and try to improve on this if indeed the zero assumption does not hold.

The program will calculate both the variables and the parameters whose values are set in steady state. It is important that the calculation be done in a recursive fashion.

If the model is stationary, a single value is required for each variable.

If the model is nonstationary, each variable should be a 1 x 2 vector in which the first element is the steady state and the second element is the growth rate.

You can also loop around the steady state model conditional on the knowledge of variables not calculated in steady state. This arises typically in optimal policy problems, where for instance the value of, say inflation is not known but the system is such that if we knew the value of inflation we could solve for many other endogenous variables.

Warning: In the steady state model, avoid using temporary variable that correspond to the names of endogenous variables!

Hint: For level nonstationary variables, the growth rate is computed as $X_t - X_{t-1}$ whereas for log nonstationary variables, the growth rate is computes as X_t/X_{t-1}

Hint: The steady state for nonstationary variables is undefined but can be normalized. Suppose, say, we have the following system

$$\begin{aligned} h_t &= \frac{1}{3} \\ Y_t &= Z_t h_t \\ \log\left(\frac{Z_t}{Z_{t-1}}\right) &= \log\left(\mu_z\right) + \sigma_z \varepsilon_{z,t} \end{aligned}$$

where $\varepsilon_{z,t}$, Y_t is output, Z_t is the TFP and h_t is the labor input.

Assuming we do not take a log approximation of h_t, the RISE steady state is given by

$$\begin{aligned} h &= \left[\frac{1}{3}, 0\right] \\ Z &= [zss, \mu_z] \\ Y &= [Z\left(1\right) \times h\left(1\right), Z\left(2\right)] \end{aligned}$$

where zss is the normalization for the steady state of Z or more precisely the point around which the approximation will be taken.

Hint: Sometimes, part of the steady state cannot be computed analytically. You can write a separate function in an m-file that solves the sub-problem using a root-finding procedure. That function can then be called from within the *@steady_state_model* block.

Warning: If you write a separate function that solves part of the steady state and that can fail at times for some parameterizations of the model, there is no way to inform RISE of the failure when using a *@steady_state_model* block. The most flexible way to deal with this is to use a steady-state file.

The alternative to the @steady_state_model block is the steady state file.

3.3.10 The @parameterization block (optional)

RISE gives you the flexibility of setting the parameterization of the model either in the model file or outside the model file. The former strategy, presented below, is appealing when the user does not want to create too many files and that the model is small. The latter strategy gives the most flexibility including the possibility of overriding the information set inside the model file.

Inside the model file, the parameterization is set under the "@parameterization" block. This concerns both the parameters that are fixed and the parameters that are to be estimated. A parameter cannot be repeated and the start value for estimation will also serve as the baseline calibration, should the parameter not be estimated later on.

For fixed parameters, the syntax is

```
paramName, value ;

paramName = value ;
```

For estimated parameters, the syntax is

```
paramName, start_value, arg1, arg2,...,argn ;

paramName = start_value, arg1, arg2,...,argn ;
```

where arg1, arg2,…,argn refers to additional arguments. See the section on priors for detail. Example

```
rho, 0.75, 0.4, 0.9, beta(0.9);

rho = 0.75, 0.4, 0.9, beta(0.9);
```

Switching parameters are written as

```
sigz(vol,1)
sigz(vol,2)
```

or as

```
sigz_vol_1
sigz_vol_2
```

3.3.11 The @parameter_restrictions block (optional)

The **@parameter_restrictions** block sets the restrictions to be enforced during estimation

The syntax is

```
@parameter_restrictions
        f1(theta)>=f2(theta);
        ...
        h1(theta)<=h2(theta);
```

This can also be set outside the model file. See e.g. the chapter on estimation.

3.3.12 Tools for bounded rationality (Non-rational expectations)

3.3.12.1 Hybrid Expectations

The key equations in a hybrid expectations system are the following (see e.g. [Gelain *et al.*, 2013]):

$$F_t X_{t+1} = F_{t-1} X_t + \lambda \left(X_t - F_{t-1} X_t \right)$$
$$E_t^{he} X_{t+1} = \omega F_t X_{t+1} + \left(1 - \omega \right) E_t X_{t+1}$$

where $0 < \lambda \leq 1$ and $0 \leq \omega \leq 1$.

RISE implements **Hybrid expectations** through the declaration of the following two parameters :

- hbe_w : weight on adaptive (or backward-looking) expectations
- hbe_lambda : weight on current relative to past in backward-looking expectations

3.3.12.2 Cognitive discounting

RISE implements cognitive discounting a-la Gabaix in two distinct ways:

1. Through the creation of parameter names of the form "mcd_VARIABLENAME" where VARIABLENAME is the name of an endogenous variable present in the model
2. Through the declaration of list of endogenous variables to which one wants to apply cognitive discounting. e.g. @cognitive_discount VARIABLE1, VARIABLE2,... In this case, RISE will create the corresponding cognitive discounting parameters "mcd_VARIABLE1", "mcd_VARIABLE2", ...

This way of setting cognitive discounting can be applied to nonlinear and regime-switching models and in that case, the degree of discounting can change from one regime to another.

One way to apply regime switching to cognitive discounting parameters is just to add them to the list of parameters controlled by a particular markov process. But another way to do the same using the second approach above is just to add the markov process to the declaration block. e.g

```
@cognitive_discount(MCCHAIN,nstates) VARIABLE1, VARIABLE2,...
```

3.3.13 Macro Language for preparsing

3.3.13.1 Inclusion of files

```
@include(filename)
@include "filename"
@import(filename)
@import "filename"
```

3.3.13.2 Pseudo functions

Without arguments, the function returns the list of model keywords

Moving averages

```
movave(X,n) = movavg(X,n)
movavg(X,n) = (X{t} + X{t-1} + X{t-2} + ... + X{t-n+1})/abs(n)
movave(X) = movavg(X) = movave(X,-4)
```

Moving products

```
movprod(X,n) = prod(X{t},X{t-1},X{t-2},...X{t-n+1})
movprod(X) = movprod(X,-4)
```

Geometric averages

```
movgeom(X,n) = prod(X{t},X{t-1},X{t-2},...X{t-n+1})^(1/abs(n))
movgeom(X) = movgeom(X,-4)
```

Moving sum:

```
movsum(X,n) = X{t} + X{t-1} + X{t-2} + ... + X{t-n+1}
movsum(X) = movsum(X,-4)
```

Differences:

```
diff(X,n) = X{t} - X{t+n}
diff(X) = diff(X,-1)
```

log differences

```
difflog(X,n) = log(X{t}) - log(X{t+n})
difflog(X) = difflog(X-1)
```

dot products

```
dot(X,n) = X{t}/X{t+n}
dot(X) = dot(X,-1)
```

3.3.13.3 Control flow : for loops

```
@for c=[1,4,6]
        Y_@{c}=rho_@{c}*Y_@{c}{-1}+sig_@{c}*E_@{c};
@end

will produce

Y_1=rho_1*Y_1{-1}+sig_1*E_1;
Y_4=rho_4*Y_4{-1}+sig_4*E_4;
Y_6=rho_6*Y_6{-1}+sig_6*E_6;
```

```
@for c=1:3
        Y_@{c}=rho_@{c}*Y_@{c}{-1}+sig_@{c}*E_@{c};
@end

will produce

Y_1=rho_1*Y_1{-1}+sig_1*E_1;
Y_2=rho_2*Y_2{-1}+sig_2*E_2;
Y_3=rho_3*Y_3{-1}+sig_3*E_3;
```

```
@for c={'','_f'}
        Y@{c}=rho@{c}*Y@{c}{-1}+sig@{c}*E@{c};
@end

will produce

Y=rho*Y{-1}+sig*E;
Y_f=rho_f*Y_f{-1}+sig_f*E_f;
```

```
@for c={'ca','fr','de','be'}
   Y_@{c}=rho_@{c}*Y_@{c}{-1}+sig_@{c}*E_@{c};
@end

will produce

Y_ca=rho_ca*Y_ca{-1}+sig_ca*E_ca;
Y_fr=rho_fr*Y_fr{-1}+sig_fr*E_fr;
Y_de=rho_de*Y_de{-1}+sig_de*E_de;
Y_be=rho_be*Y_be{-1}+sig_be*E_be;
```

3.3.13.4 control flows : zipped for loops

```
@for (aa,bb)=zip({'S','I','R'},{'Susceptible','Infected','Recovered'})
        BIG_U@{aa} "welfare @{bb}", C@{aa} "consumption (@{bb})"
@end

will produce

BIG_US "welfare Susceptible", CS "consumption (Susceptible)"
BIG_UI "welfare Infected", CI "consumption (Infected)"
BIG_UR "welfare Recovered", CR "consumption (Recovered)"
```

3.3.13.5 control flows : enumerated for loops

```
@for (aa,bb)=enumerate({'Susceptible','Infected','Recovered'})
   BIG_U@{aa} "welfare @{bb}", C@{aa} "consumption (@{bb})"
@end

will produce

BIG_U1 "welfare Susceptible", C1 "consumption (Susceptible)"
BIG_U2 "welfare Infected", C2 "consumption (Infected)"
BIG_U3 "welfare Recovered", C3 "consumption (Recovered)"
```

3.3.13.6 Evaluations

@{term to be evaluated} where the term to be evaluated is not an index

```
@for c=[1,4,6,8]
        Y_@{c}=rho_@{3*@{c}+1}*Y_@{c}{@{-1+3}}+sig_@{c}*E_@{c};
@end

will produce

Y_1=rho_4*Y_1{2}+sig_1*E_1;
Y_4=rho_13*Y_4{2}+sig_4*E_4;
Y_6=rho_19*Y_6{2}+sig_6*E_6;
Y_8=rho_25*Y_8{2}+sig_8*E_8;
```

3.3.13.7 Control flow : @if ... @elseif ... @else ... @end

```
@if strcmp(Policy,'Taylor rule')
   R{t}=rho*R{t-1}+(1-rho)*[psi_pai*PAI+psi_y*Y]+sigr*ER;
@if strcmp(Policy,'Money growth rule')
   DM{t}=rhom*DM{t-1}+(1-rhom)*[-delta_pai*PAI-delta_y*Y]+sigm*EM;
@else
   omgr*R{t}+omgm*DM{t}=omgr1*R{t-1}+omgm1*DM{t-1}+omg_pai*PAI+omg_y*Y+sigrm*ERM;
@end
```

3.3.13.8 Control flow : @switch ... @case ... @otherwise ... @end

```
@switch Policy
@case 'Taylor rule'
   R{t}=rho*R{t-1}+(1-rho)*[psi_pai*PAI+psi_y*Y]+sigr*ER;
@case 'Money growth rule'
   DM{t}=rhom*DM{t-1}+(1-rhom)*[-delta_pai*PAI-delta_y*Y]+sigm*EM;
@otherwise
   omgr*R{t}+omgm*DM{t}=omgr1*R{t-1}+omgm1*DM{t-1}+omg_pai*PAI+omg_y*Y+sigrm*ERM;
@end
```

3.3.13.9 lower and upper cases

```
@for cc = {'cmr','rca','cng'}
    YSTAR_CEMAC_@{cc} ='
    @for cc2 = {'CMR','RCA','CNG'}
        @if ~strcmp('@{cc2}','@{cc}')
            + w_@:{cc}_@.{cc2}_5*YGAP_@{cc2}
        @end
    @end
    ;
@end

    will produce

    YSTAR_CEMAC_cmr =
    + w_CMR_cmr_5*YGAP_CMR
    + w_CMR_rca_5*YGAP_RCA
    + w_CMR_cng_5*YGAP_CNG
    ;
    YSTAR_CEMAC_rca =
    + w_RCA_cmr_5*YGAP_CMR
    + w_RCA_rca_5*YGAP_RCA
    + w_RCA_cng_5*YGAP_CNG
    ;
    YSTAR_CEMAC_cng =
    + w_CNG_cmr_5*YGAP_CMR
    + w_CNG_rca_5*YGAP_RCA
    + w_CNG_cng_5*YGAP_CNG
    ;
```

3.4 The RISE Editor

Developed in collaboration with the Mathworks, this tool is meant ot facilitate the development of RISE dsge model files by providing language highlighting and smart indentation.

3.4.1 System Requirements

The app itself is platform independent, and it should work in Windows, Linux, and Mac.

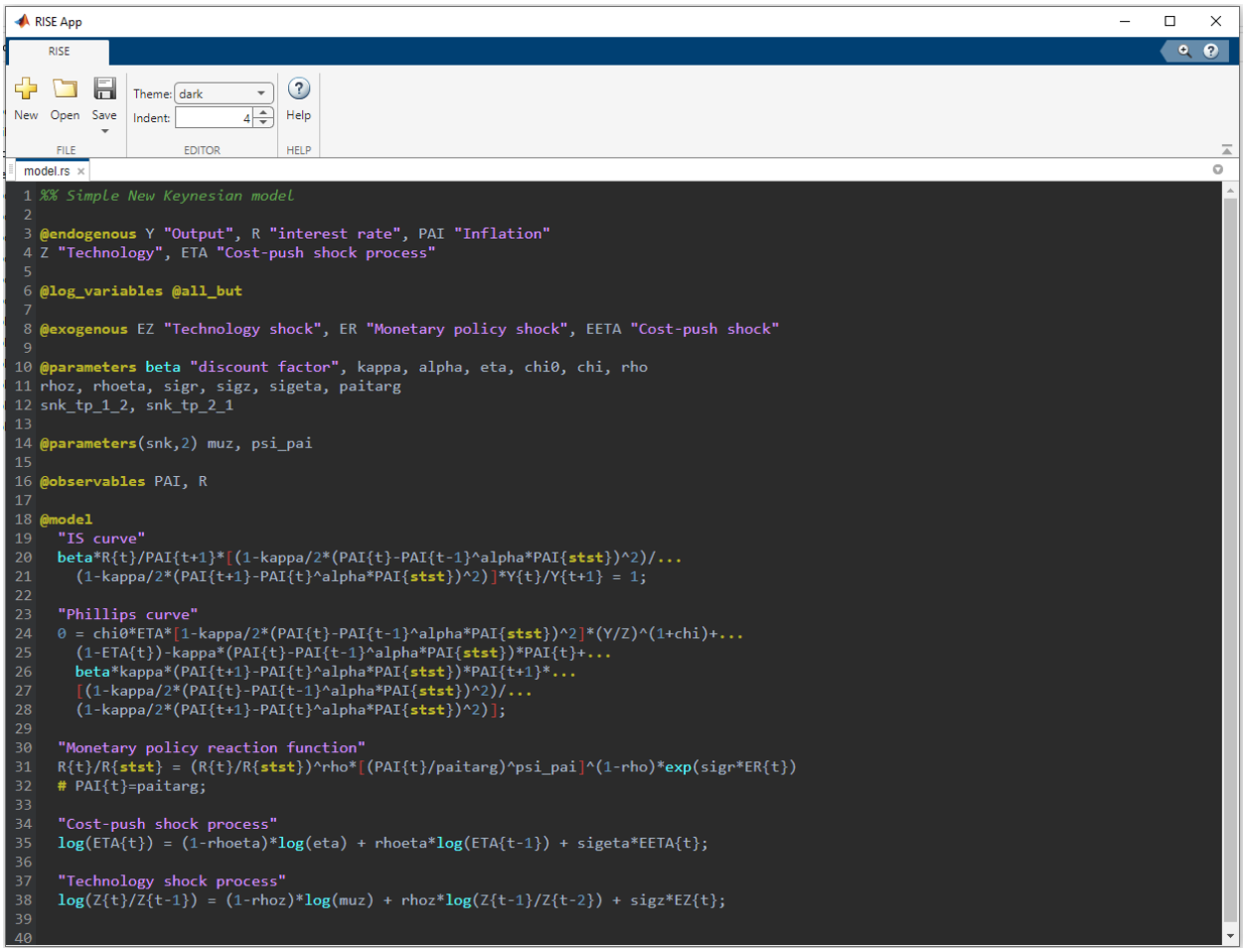
3.4.2 Starting the App

You can run it as follows:

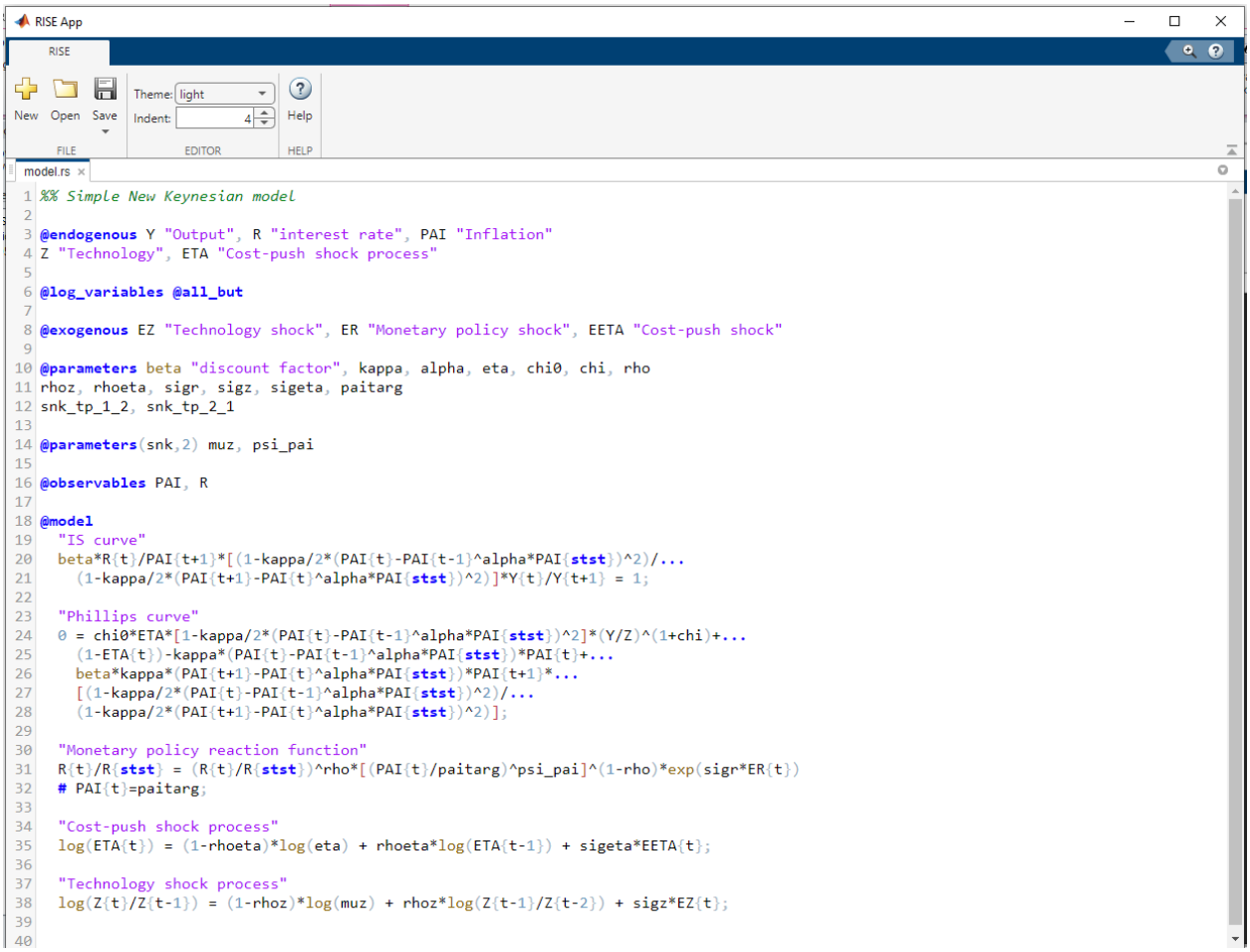
```
launchRISE
```

3.4.3 dark vs light modes

You can choose the theme : Dark mode



Light mode



Or create your own

3.4.4 Syntax highlighting

3.4.5 Indentations and nested indentations

You can get an entire file automatically indented just as in Matlab by :

- CTRL + A : Select all
- CTRL + I : Indent

Otherwise, indentation is done automatically as you are typing.

3.4.6 Editing files

You can either open an existing file or create a new file

3.4.7 Control flow

Control flow following the RISE syntax is well understood by the App :

- @if … @elseif … @else … @end
- @swicth … @case … @otherwise … @end
- @for … @end
- @while … @end

Again, indentation is done automatically

3.5 Setting up calibration and priors outside the model file

Outside the model file, fixed parameters and priors are separated.

Fixed parameters can be given in a cell array

```
params={
        'rho',0.6
        'beta',0.998
        ...
        'kappa', 161
        }
```

Or as a structure

```
params=struct();
params.rho=0.6;
params.beta=0.998;
...
params.kappa=161;
```

Then the parameterization can be pushed into a model object as

```
m=set(m,'parameters',params);
```

Priors for Bayesian (and MLE) estimation are given as a structure. See the section on prior distributions.

3.6 Deterministic and quasi-deterministic solutions

3.6.1 Perfect foresight

3.6.2 Extended path

3.7 Stochastic solution via perturbation

3.7.1 Zero-th order approximation : The steady state and the balanced-growth path

We have already seen how to create a program for solving the steady state inside the model file. But as we have argued, that approach has some limitations. The most flexible way to approach the solving of the steady state and the balanced-growth path is through the writing of a steady-state file.

Below is an example for some function, say, simple_ssfile

```
simple_ssfile--  computes the steady state of the model analytically

Syntax:

  [y,newp,retcode]=simple_ssfile(m,y,p,d,id)

Args:

   - **m** [rise|dsge]: model object (not always needed)

   - **y** [vector|2-column matrix]:

                - endo_nbr x 1 vector of initial steady state : stationary models
                - endo_nbr x 2 matrix of initial steady state and growth rate :␣
↪nonstationary models

   - **pp** [struct]: parameter structure

   - **d** [struct]: definitions

   - **id** [vector]: location of the variables to calculate

Returns:

   - **y** [vector|2-column matrix]:

                - endo_nbr x 1 vector of updated steady state : stationary models
                - endo_nbr x 2 matrix of updated steady state and growth rate :␣
↪nonstationary models

   - **newp** [struct]: structure containing updated parameters if any

   - **retcode** [0|number]: return 0 if there are no problems, else return
     any number different from 0
```

Note: this is new approach has three main advantages :

- The file is valid whether we have many regimes or not
- The user does not need to know what regime is being computed
- It is in sync with the steady state model

RISE will call the function twice. The first call will be with one argument (the first one) only and will be about collecting two types of information :

- The list of variables for which the steady state is solved and the order, freely chosen by the user, in which the calculations will be returned.
- The list of parameters whose value is computed in the steady state

In the second call, RISE will provide all input arguments. It is then expected that the user returns the steady state and the parameters computed in steady state.

Here is a complete listing for a balanced-growth path file for the simple model above but without the comments

```
function [y,newp,retcode]=simple_ssfile(m,y,p,d,id) %#ok<INUSL>

retcode=0;

if nargin==1
    % list of endogenous variables to be calculated and the order in which they will␣
↪appear
    y={'Y','Z','h'};
    newp={};% list of parameters to be computed during steady state calculation
else
    % steady states and new parameter values
    %----------------------------------------
    newp = struct();

    h=[1/3,0];
    Z=[p.zss,p.mu];
    Y=[Z(1)*h(1),Z(2)];

    ys=[Y;Z;h]; % ordering the output according to the list above

    if any(isnan(ys(:)))% check the validity of the calculations
        retcode=1;
    else
        % push the calculations
        %----------------------
        y(id,:)=ys;
    end
end

end
```

3.7.1.1 Incomplete analytical solution

Sometimes the programs written by the user do not solve the steady state or the balanced-growth path completely. This can occur either because the user can only calculate part of the steady state or that RISE created auxiliary variables.

In the case of optimal policy, for instance, RISE creates lagrange multipliers.

While RISE knows how to handle the auxiliary variables it creates, it will try to solve for the other variables numerically, given the incomplete steady state provided by the user.

3.7.1.2 Steady state loops

Sometimes, the user can compute the steady state or part of the steady state conditional on the knowledge of a particular variable. For instance, in an optimal policy problem in which only monetary policy is missing, the knowledge of steady-state inflation may help solve for all other endogenous variables excluding the lagrange multipliers (automatically taken care of).

The user will write a steady state file that takes as given the missing variables he does not know how to compute. RISE gives a proposal for those missing variables in the the second argument of the steady state file. Locating the proposal for the missing variables is not difficult. All the user has to do is to collect the location the missing variables in the list given by:

```
get(m,'endo_list')
```

With the locations in hand, the user will go ahead and collect the proposed values in the y vector (or 2-column matrix) and then do the rest of the computations.

The burden of RISE is lessened because RISE needs only to find the solution for a subset of variables.

Hint: The user needs to set the option **sstate_loop** to true to inform RISE that only a subset of variables is unknown

3.7.1.3 Facilitators

When solving the steady state or balanced-growth path numerically, one can take advantage of some facilitators. And those facilitators can be combined.

Equation simplification

This is the situation where the user provides both a dynamic equation and a steady-state version of the equation. For instance

```
log(X{t}) = (1-rho)*log(xss) + rho*log(X{t-1}) + sig*E{t} # ...
        X{t}=xss;

log(Z{t}/Z{t-1}) = log(muz) +sigz*EZ{t} # ...
        Z{t} = muz*Z{t-1};
```

Declaring Log and level variables

Besides the economic interpretation, there is another benefit of taking log approximations. When the steady state is solved numerically, the search domain for log variables is $(0, +\infty)$. This prevents us from running into singularities such as :

- division by 0
- log of zero
- log of negative values

Setting bounds on variables

RISE gives us the possibility of defining the search space for the steady state and the growth rate of variables when solving numerically. See the **sstate** method of rise objects for more details.

3.7.1.4 The case of multiple regimes

The presence of multiple regimes does not pose any additional difficulties for the computation of the steady state. In this case, the steady state is just computed for :

- the ergodic mean of parameters: see [Foerster *et al.*, 2016]
- different vectors of parameters corresponding to each regime: see e.g. [Maih and Waggoner, 2018] or [Chang *et al.*, 2021].

3.7.2 First-order perturbation

3.7.2.1 The state vector

$$z_t \equiv \left[\begin{array}{ccc} x'_{t-1} & \sigma & \varepsilon'_t \end{array} \right]'$$

where σ is the perturbation parameter.

In some applications, we can assume agents have information on shocks $k \geq 0$ periods ahead into the future. In that case, the state variables under anticipated shocks (see e.g. [Maih, 2010] and [Juillard and Maih, 2010]) is

$$z_t \equiv \left[\begin{array}{cccccc} x'_{t-1} & \sigma & \varepsilon'_t & \varepsilon'_{t+1} & \cdots & \varepsilon'_{t+k} \end{array} \right]'$$

3.7.2.2 Solvers for the quadratic matrix polynomial

The constant-parameter case (and the Maih-Waggoner perturbation)

The algorithms implemented in RISE for solving linear rational expectations models are

- *rise_1* : A Klein-like algorithm (Default).
- *sims* : This is the Gensys algorithm by [Sims, 2002]
- *Klein* : An algorithm by [Klein, 2000]
- *AIM* : See e.g. [Anderson, 2008]
- *cyclic_reduction* : the cyclic reduction algorithm

Note: If there is a unique and stable solution in the Maih-Waggoner perturbation, these algorithms will also work in the regime-switching case.

The regime-switching case

The algorithms implemented in RISE for solving regime-switching linear rational expectations models are

- *mnk* : Newton algorithm by [Maih, 2015], which explicitly forms and store kronecker products.
- *mn* : Newton algorithm by [Maih, 2015], which avoids the creation of kronecker products and is thereby more efficient for solving large systems.
- *fwz* : Newton algorithm by [Farmer *et al.*, 2011]
- *dsge_udc* : Undetermined coefficients method by [Maih and Waggoner, 2018]. This algorithm can only be applied with the Maih-Waggoner perturbation.
- *dsge_schur* : Schur-based method by [Maih and Waggoner, 2018]. This algorithm returns the most stable solution. Therefore it gives the same answer as *rise_1* (and other constant-parameter solvers) when there is a unique stable solution. This algorithm can only be applied with the Maih-Waggoner perturbation.
- *dsge_groebner* : Groebner-basis based solver from Computational algebraic geometry by [Maih and Waggoner, 2018]. When it works, this algorithm finds all possible solutions to the problem.
- *mfi* : Functional iteration algorithm by [Maih, 2015]

Note: All these algorithms for solving regime-switching linear rational expectations can solve constant-parameter models even when there is no unique and stable solution.

User-defined solver

A user-defined solver should be a function with inputs and outputs as follows

```
[Tz_pb,Tsig,Te,retcode]=user_solver(Gplus01,A0,Aminus,B,Q,stst_,vargs{:});
```

where the inputs are :

- *Gplus01* is an $nd \times nd \times h \times h$ array of coefficient matrices on forward-looking terms multiplied by the transition probabilities
- *A0* is an $nd \times nd \times h$ array of coefficient matrices on contemporaneous terms
- *Aminus* is an $nd \times nd \times h$ array of coefficient matrices on backward-looking terms
- *B* is an $nd \times nx \times h$ array of coefficient matrices on shock terms
- *Q* is an $h \times h$ transition matrix with Qij the probability of going from i to j
- *stst_* is an $nd \times h$ matrix whose columns represent the steady state in each regime
- *vargs* represents potential additional arguments entering the user-defined function.

And the outputs are:

- *Tz_pb* is an $nd \times nd \times h$ array of the impact of autoregressive terms
- *Tsig* is an $nd \times nd$ matrix whose columns represent the impact of the perturbation parameter in each regime

- *Te* is an $nd \times nx \times h$ array representing the impact of shock terms
- *retcode* is a flag which takes on value 0 if there is no problem or a number different from 0 if there is a problem finding the solution.

3.7.3 Higher-order perturbation

You can solve for higher-order perturbation if you are in one of the following cases:

1. you are using *symbolic* derivatives and those derivatives have been precomputed while "rising" the model, through option **max_deriv_order**
2. you are using *automatic* derivatives
3. you are using *numerical* derivatives
4. you are using *matlab* derivatives

And you are able to solve the generalized system of Sylvester equations.

3.7.3.1 Choosing the derivatives

To change the type of derivatives, use the option **solve_derivatives_type**. RISE implements the following 4 types of derivatives:

- symbolic
- numerical
- automatic
- matlab

3.7.3.2 Solving the system of generalized Sylvester equations

The complicated linear problem is solved using a solver that can be called using the syntax

```
[X,flag,relres,iter,resvec] = linsolver(A,b,...
TolFun,Maxiter,...
M1,M2,x0,vargs{:});
```

Where the input arguments are given by

- *A* : either a square matrix of a function handle that return the result of the operation $A \times x$ without explicitly forming and storing A.
- *b* : the right-hand side of the operation $A \times x = b$.
- *TolFun* : the tolerance criterion
- *Maxiter* : the maximum number of iterations
- *M1* = [] : not set
- *M2* = [] : not set
- *x0* : initial guess for the solution
- *vargs* : potential additional input arguments for the solver utilized.

And the output arguments are :

- *X* : the final solution
- *flag* : = 0 if the problem solves, =1 if the maximum number of iterations is reached, = some other number otherwise
- *relres* : the relative residual norm(b-A*x)/norm(b)
- *iter* : the iteration number iter at the final solution X was computed
- *resvec* : a vector of the residual norm at each half iteration, including the first residual norm(b-A*x0)

Change the default setting using option **solve_linsyst_user_algo**.

These are some of the algorithms that can be used in Matlab.

- *'tfqmr'* (default)
- *'bicg'*
- *'bicgstab'*
- *'bicgstabl'*
- *'cgs'*
- *'gmres'*

3.8 Optimal/Optimized Simple Rules

3.8.1 The Loss function

Here we define a loss function outside the model file given that the number of equations is equal to the number of endogenous variables.

3.8.2 (Approximate) Analytical Loss

3.8.3 Simulated Loss

3.8.4 OSR and Indirect Inference

3.8.5 OSR and user-defined objective function

3.8.6 Policy frontiers

3.9 Stochastic simulations

3.9.1 Artificial data

3.9.2 Pruned simulations

RISE implements 5-th order approximation of the perturbation solution of regime switching DSGE models.

But the pruning algorithm, which generalizes, [Andreasen *et al.*, 2013] and extends it to regime-switching models can handle any arbitrary order of approximation.

3.9.3 Theoretical and Simulated moments

3.9.4 Variance decomposition

3.9.5 Impulse responses

3.9.5.1 Simple Impulse Responses

- Simple IRFs
- Simple IRF with anticipation
- Simple IRF with delayed reaction
- Simple IRF with Anticipated delayed reaction
- Simple IRF with Unanticipated delayed reaction

3.9.5.2 Generalized Impulse Responses

3.10 Generic simulations with plans

3.10.1 The properties

3.10.1.1 endo_list

endo_list - list of endogenous

3.10.1.2 exo_list

exo_list - list of exogenous

3.10.1.3 start_date

start_date - last date of history

3.10.1.4 end_date

end_date - last date of simulation

3.10.1.5 InitialRegime

InitialRegime - Initial regime for forecasting

3.10.1.6 Conditions

Conditions - Conditioning information cell array {var_names, dates, values}

3.10.1.7 simul_periods

simplan/simul_periods is a property.

3.10.1.8 variable_list

simplan/variable_list is a property.

3.10.2 The methods

3.10.2.1 append

append - Add conditioning information to a pre-existing simulation plan (simplan).

Syntax:
obj = append(obj, vname, date, value) obj = append(obj, {vname, date, value}) obj = append(obj, C)

Inputs:

- *obj*: Existing simplan object.
- *vname*: Name of the endogenous or exogenous variable to condition on.
- *date*: Date object recognizable by RISE for conditioning.
- *value*: Conditioning value.
- {vname, date, value}: Cell array containing variable name, date, and value.
- ***C*: 3-column cell array with potentially multiple rows, each row**
  representing a conditioning entry.

Outputs:

- *obj*: Updated simplan object with added conditioning information.

Description:
The append method adds conditioning information to a pre-existing simulation plan (simplan). You can provide the information either as separate inputs or in a cell array. Multiple conditioning entries can be added simultaneously using a 3-column cell array.

Example:
% Create a simplan object plan = simplan(model, horizon, initialRegime);

% Append a conditioning entry plan = append(plan, 'inflation', someDate, 2.5);

% Append multiple entries using a cell array conditions = {'interest_rate', someOtherDate, 1.8; 'output', anotherDate, -0.5}; plan = append(plan, conditions);

See also:

simplan

3.10.2.2 endval

endval : Overloads/emulates Dynare's endval for the simplan class

Syntax:
obj = endval(obj, steady, C)

Description:
The *endval* method overloads/emulates Dynare's endval for the *simplan* class. It sets initial conditions for simulation periods, considering the presence of steady state calculations and historical values.

Input:

- obj: The simplan object.
- steady: A logical flag indicating whether steady state calculations should be performed. If true, the provided initial conditions (C) will be used to calculate the steady state, resulting in an updated C.
- C: Initial conditions, start values and terminal conditions. For the syntax, see the help for initval.
  - In the absence of any other block, (the possibly updated) C provides initial conditions, terminal conditions and start values for perfect-foresight optimization.
  - In the presence of an initval block and/or a histval block, (the possibly updated) C provides terminal conditions and start values for perfect-foresight optimization.

Output:

- obj: Updated simplan object with initialized conditions.

Example:
sp = simplan(model, date_range, 1); sp = initval(sp, true, init_conditions);

Notes:

- The *endval* method dominates *initval* in that it overrides the terminal conditions and the initial values for perfect-foresight optimization

See also:

- simplan

3.10.2.3 export

export - Compiles the conditioning information in a *simplan* object for simulation or conditional forecasting.

Syntax:
[histdb, np] = export(obj)

Inputs:

- *obj*: simplan object containing conditioning information.

Outputs:

- *histdb*: Time series object with compiled conditioning information.
- *np*: Number of periods of simulation.
- *ss* (empty|struct): start values for all endogenous variables

Description:

The *export* method compiles the conditioning information stored in a *simplan* object into a time series database suitable for simulation or conditional forecasting. The resulting *histdb* contains the conditioning values for endogenous and exogenous variables.

Example:

% Create a *simplan* object plan = simplan(model, horizon, initialRegime);

% Add conditioning entries plan = append(plan, 'inflation', someDate, 2.5); plan = append(plan, 'interest_rate', someOtherDate, 1.8);

% Export the compiled conditioning information [histdb, np] = export(plan);

See also:

simplan, *plan2database*

3.10.2.4 histval

histval : Overloads/emulates Dynare's histval for the simplan class

Syntax:

obj = histval(obj, C)

Description:

The *histval* method overloads/emulates Dynare's histval for the *simplan* class. It sets the historical values (initial conditions) for the perfect-foresight simulations.

Input:

- obj: The simplan object.
- C: historical values to be set. The syntax is as follows:

```
Example 1:

C = {
     'y(0)', 1
     'k(-1)', 12
     'm{0}', 0.5
     'x{-1}', 1
    };

Example 2:

C = {
     'y(0)=1;'
     'k(t-1)=12;'
     'm{0}=0.5;'
     'x{t-1}=1;'
     };

Example 3:

C = 'y(0)=1; k(-1)=12; m{0}=0.5; x{-1}=1;';
```

Output:

- obj: Updated simplan object with initialized conditions.

Example:
sp = simplan(model, date_range, 1); sp = initval(sp, true, init_conditions);

Notes:

- The *histval* method dominates both *initval* and *endval*

See also:

- simplan

3.10.2.5 initval

initval : Overloads/emulates Dynare's initval for the simplan class

Syntax:
obj = initval(obj, steady, C)

Description:
The *initval* method overloads/emulates Dynare's initval for the *simplan* class. It sets initial conditions for simulation periods, considering the presence of steady state calculations and historical values.

Input:

- obj: The simplan object.
- steady: A logical flag indicating whether steady state calculations should be performed. If true, the provided initial conditions (C) will be used to calculate the steady state, resulting in an updated C.
- C: Initial conditions, start values and terminal conditions. It can be provided as a cell array with variable-value pairs, or as a cell array with expressions for each variable:

```
Example 1:
C = {
     'y', 1
     'k', 12
     'm', 0.5
     'x', 1
    };
Example 2:
C = {
     'y=1;'
     'k=12;'
     'm=0.5;'
     'x=1;'
    };
Example 3:
C = 'y=1; k=12; m=0.5; x=1;';
```

  - In the absence of any other block, (the possibly updated) C provides Initial conditions, terminal conditions and start values for perfect-foresight optimization.
  - In the presence of a histval block, (the possibly updated) C provides, terminal conditions and start values for perfect-foresight optimization.
  - In the presence of a endval block, (the possibly updated) C provides only initial conditions.

Output:

- obj: Updated simplan object with initialized conditions.

Example:
sp = simplan(model, date_range, 1); sp = initval(sp, true, init_conditions);

Notes:

- The *initval* method cannot be called twice.

See also:

- simplan

3.10.2.6 simplan

simplan - A class for handling simulation plans.

Syntax:
obj = simplan(m, horizon, []) obj = simplan(m, horizon, initialRegime) obj = simplan(m, horizon, initialRegime, varargin)

Args:

- *m*: dsge model object
- ***horizon* (1 x 2 vector): [start_date, end_date] Time**
  horizon for the simulation.
- *initialRegime* (scalar, default =1): Initial regime for forecasting. This is needed particularly for simulating a regime-switching model as the initial regime selects the row of the transition matrix to draw from.
- *varargin*: Simulation/forecast conditions in 3 inputs : (variable names, dates, values) or in a 3-column cell array {variable names, dates, values} or a time series or structure or time series

Returns:

- *obj*: A *simplan* object.

Documentation for simplan/simplan
doc simplan

3.11 Forecasting

RISE extends the algorithms in [Maih, 2010] for conditional forecasting in linear constant-parameter models to non-linear regime-switching DSGE models.

3.11.1 Conditional forecasting in linear models

3.11.2 Conditional forecasting in nonlinear models

3.12 DSGE Filtering

3.12.1 Filtering and decompositions

3.12.1.1 Linear filtration

Filtering with constant parameters can be done efficiently with algorithms presented in e.g. [De Jong and Chu-Chun-Lin, 2003], [Koopman and Durbin, 2000], [Koopman and Durbin, 2003], [Durbin and Koopman, 2012]. Unfortunately those procedures cannot be used in a regime-switching context.

The main references in the economics literature for the filtration of Markov-switching state-space models are [Kim and Nelson, 1999] and [Kim and Nelson, 2001], which combine the Kalman and Hamilton filters. Similar algorithms can be found in the engineering literature. See e.g. [Bar-Shalom *et al.*, 2001], [bar-shalom *et al.*, 2004].

RISE uses an extension of the algorithms by [Durbin and Koopman, 2012] except for the initialization process, which in the case of nonstationarity, hasn't been extended to regime-switching models. In the case of nonstationarity, RISE simply applies an arbitrary variance for the initialization process.

Also, for efficiency reasons, RISE deviates from [Kim and Nelson, 2001] when it comes to the collapsing of the regimes. While [Kim and Nelson, 2001] collapse the updates, RISE collapse the forecasts. The two procedures yield numerically similar results but the procedure in RISE is faster.

3.12.1.2 Historical Decomposition

Warning: Exact only for constant-parameter linear models.

3.12.1.3 Observables decomposition

Important: Available only for constant-parameter models

3.12.1.4 Conditional forecasting via filtering

Conditional forecasting can be implemented using filtering algorithms.

Suppose we want to condition on a variable X0.

Hard conditions

We simply smooth over an extended sample.

Soft conditions

We relate X0 to some variable X in the model as $X0_t = X_t + \sigma\varepsilon_t$.

And we let σ control the softness of the condition :

- If σ tends to ∞ the condition is irrelevant
- If σ tends to 0 we have **hard conditions**
- If σ is greater than 0 but finite (small enough) we have **soft conditions**.

3.12.1.5 User-defined filter

3.12.2 Linear filtration with real-time information

3.12.2.1 Data setup

3.12.2.2 routine

3.12.3 Nonlinear filtering

RISE implements Sigma point filters for the filtration of nonlinear regime-switching state-space models following [Binning and Maih, 2015].

3.12.3.1 Switching Unscented Kalman filter

switching_unscented_kalman_filter - filter for nonlinear regime-swiching models

```
[loglik,Incr,retcode,Filters]=switching_unscented_kalman_filter(...
 syst,data_y,U,z,options)
```

Args:

- **syst** [struct]: structure containing:
  - **PAI00** [vector]: initial probability distributions of regimes
  - **a** [cell]: initial conditions in each regime
  - **Qfunc** [function handle]: transition matrix generator
  - **ff** [function handle]: ft=ff(rt,xt,et), where rt is the regime, xt is the vector of state variables and et the vector of shocks
  - **P** [cell]: initial covariance matrix of the states in each regime
  - **H** [cell]: Measurement error covariance matrices in each regime
  - **SIGeta** [cell]: Covariance matrix of structural shocks.
- **data_y** [matrix]: ny x T matrix of data
- **U** [[]|matrix]: ndx x T matrix of exogenous data

- **z** [function handle|logical|vector]: linear connection of the observables to the state.
- **include_in_likelihood** [logical]: selector of increments to include in the likelihood calculation (temporarily disabled)
- **options** [struct]: structure with various options

Returns:

- **loglik** [scalar]: log likelihood
- **Incr** [vector]: increments of elements going into the likelihood
- **retcode** [{0}|integer]: flag for problems.
- **Filters** [struct]: Filtered, updated and smoothed variables

Note:

Example:

See also:

switching_cubature_kalman_filter, switching_divided_difference_filter

3.12.3.2 Switching Divided difference filter

switching_divided_difference_filter - filter for nonlinear regime-swiching models

```
[loglik,Incr,retcode,Filters]=switching_divided_difference_filter(...
 syst,y,U,z,options)
```

Args:

- **syst** [struct]: structure containing:
  - **PAI00** [vector]: initial probability distributions of regimes
  - **a** [cell]: initial conditions in each regime
  - **Qfunc** [function handle]: transition matrix generator
  - **ff** [function handle]: ft=ff(rt,xt,et), where rt is the regime, xt is the vector of state variables and et the vector of shocks
  - **P** [cell]: initial covariance matrix of the states in each regime
  - **H** [cell]: Measurement error covariance matrices in each regime
  - **SIGeta** [cell]: Covariance matrix of structural shocks.
- **y** [matrix]: ny x T matrix of data
- **U** [[]|matrix]: ndx x T matrix of exogenous data
- **z** [function handle|logical|vector]: linear connection of the observables to the state.
- **include_in_likelihood** [logical]: selector of increments to include in the likelihood calculation
- **options** [struct]: structure with various options

Returns:

- **loglik** [scalar]: log likelihood
- **Incr** [vector]: increments of elements going into the likelihood

- **retcode** [{0}|integer]: flag for problems.
- **Filters** [struct]: Filtered, updated and smoothed variables

Note:

Example:

See also:

switching_cubature_kalman_filter, switching_unscented_kalman_filter

3.12.3.3 Switching Cubature Kalman filter

switching_unscented_kalman_filter - filter for nonlinear regime-swiching models

```
[loglik,Incr,retcode,Filters]=switching_unscented_kalman_filter(...
 syst,data_y,U,z,options)
```

Args:

- **syst** [struct]: structure containing:
  - **PAI00** [vector]: initial probability distributions of regimes
  - **a** [cell]: initial conditions in each regime
  - **Qfunc** [function handle]: transition matrix generator
  - **ff** [function handle]: ft=ff(rt,xt,et), where rt is the regime, xt is the vector of state variables and et the vector of shocks
  - **P** [cell]: initial covariance matrix of the states in each regime
  - **H** [cell]: Measurement error covariance matrices in each regime
  - **SIGeta** [cell]: Covariance matrix of structural shocks.
- **data_y** [matrix]: ny x T matrix of data
- **U** [[]|matrix]: ndx x T matrix of exogenous data
- **z** [function handle|logical|vector]: linear connection of the observables to the state.
- **include_in_likelihood** [logical]: selector of increments to include in the likelihood calculation (temporarily disabled)
- **options** [struct]: structure with various options

Returns:

- **loglik** [scalar]: log likelihood
- **Incr** [vector]: increments of elements going into the likelihood
- **retcode** [{0}|integer]: flag for problems.
- **Filters** [struct]: Filtered, updated and smoothed variables

Note:

Example:

See also:

switching_divided_difference_filter, switching_unscented_kalman_filter

3.13 Occasionally-Binding Constraints

3.13.1 Modeling Occasionally-Binding Constraints using Anticipated Shocks

3.13.1.1 The principle

3.13.1.2 A quick example

3.13.1.3 The implementation

3.13.1.4 Estimation

3.13.2 Modeling Occasionally-Binding Constraints using Piecewise-Linear approximations

3.13.2.1 principle and the routine

3.13.2.2 A quick example

3.13.2.3 The model file

3.13.2.4 The basic setup

3.13.2.5 Generalization

3.13.2.6 Estimation

Estimation not feasible in RISE! Please see Dynare

3.13.3 Modeling Occasionally-Binding Constraints using Regime Switching

3.13.3.1 The principle

3.13.3.2 A quick example

3.13.3.3 Under Optimal policy

The planner will internalize the constraint. We approximate

$$\lambda_i C_i = 0$$

with

$$\alpha_i \lambda_i + (1 - \alpha_i) C_i = 0$$

Under perfect foresight, we can let α_i be an endogenous variable

$$\alpha_i = 1_{C_i = 0}$$

In a stochastic context, α_i follows a Markov process with endogenous transition probabilities. In this case, note that $\alpha_i = 0$ implies that $C_i = 0$. But then the equation determines C_i but not λ_i. If there is not other place where λ_i

occurs, the system becomes structurally singular. There are two equations potentially that determine C_i but none that determines λ_i. One way to break that singularity is to impose an arbitrary value for λ_i when $\alpha_i = 0$. To put it simply,

Suppose we have an euler equation relating consumption and interest rate

3.13.3.4 The implementation

Tip: always ensure there is a feasible path by adjusting potentially-extreme probabilities

$$p_{ij} = p_{\min} + p_{ij}^{orig} \left(p_{\max} - p_{\min}\right)$$

3.13.3.5 Estimation

3.14 Understanding a rise or dsge model object

3.14.1 Ordering and information about atoms

3.14.1.1 Endogenous variables

Endogenous variables are ordered by groups and within each group, the variables are ordered alphabetically :

1. static variables : variables dated in period t only.
2. purely predetermined variables : variables dated in period t and t-1.
3. predetermined and forward-looking variables : variables dated in period t, t-1 and t+1.
4. purely forward-looking variables : variables dated in period t and t+1.

3.14.1.2 Exogenous variables

Exogenous variables are ordered alphabetically.

3.14.1.3 Parameters

Parameters are ordered alphabetically.

3.14.1.4 Observables

Observable variables are ordered alphabetically.

3.14.2 Information on Model equations

3.14.3 Information on Model results

3.14.3.1 First-order structural form

See extract_first_order_structure

3.14.3.2 Solution results

See print_solution

See m.state_space

3.14.3.3 Posterior maximization results

See m.estimation

3.15 Extending the DSGE functionalities

3.15.1 Extracting the structural form

3.15.2 Extracting the solution

3.15.3 Your own solver

3.15.4 Your own filtering algorithm

Note: more meat is needed here!!!

3.15.5 Your own optimizer

3.15.6 Your own Likelihood function

3.15.6.1 Relation to Indirect Inference

3.15.7 Your own posterior sampler

3.16 Automatic Translation of files

RISE can automatically translate files from other DSGE software such as Dynare (see [Adjemian *et al.*, 2011]) and YADA (visit this link for more details).

3.16.1 Dynare files

dynare2rise – converts a basic dynare file into a RISE one

```
duplicates=dynare2rise(dynFileName)
duplicates=dynare2rise(dynFileName,riseFileName)
duplicates=dynare2rise(dynFileName,riseFileName,stderr_name)
duplicates=dynare2rise(dynFileName,riseFileName,stderr_name,detrend)
[duplicates,listOfFilesCreated]=dynare2rise(...)
```

Args:

- **dynFileName** [char] : name of the dynare model file with or without extension (.dyn or .mod)
- **riseFileName** [char|{'dynFileName.rs'}] : name of the created RISE file
- **stderr_name** [char|{'std'}] : prepended name for the newly created parameters (RISE transforms all the variances and standard deviations into parameters)
- **detrend** [true|{false}]:
  - when true, the model is written in stationary form like dynare would do behind the scenes. For log growth variables, y–>(y*trend). For linear growth variables, y–>(y+trend). The steady state model in dynare then corresponds exactly to the steady state file produced by RISE, except that the trend variables are added as new variables.
  - when false, the model is kept in nonstationary form. For log growth variables the growth rate in an expression of the form f(x) becomes log(f(exp(x)). The growth rate for linear growth variables remains unchanged. The variables detected to have steady state = 0 but that need to be stationarized are assigned a growth rate of 0 and are declared as LEVEL variables. The steady state remains unchanged.
  - The steady state for all trend variables with log growth is normalized to 1, while the steady state for trend variables with linear growth is normalized to 0.

Returns:

- **duplicates** [struct] : structure containing information on the redundancies
- **listOfFilesCreated** [cellstr] : list of files created

Note: the created files are

- riseFileName.rs
- riseFileName_params.m : parameter function file with calibration and priors as separate outputs
- riseFileName_sstate.m (optional): steady state function file

---

See also:

AIM/YADA files

3.16.2 Dynare steady state files

dynare_sstate_file2rise - Translate Dynare steady state files for RISE.

This function serves as a wrapper for translating Dynare steady state files into a format compatible with RISE for solving Dynamic Stochastic General Equilibrium (DSGE) models. The function takes a Dynare steady state file as input and generates a function handle that RISE can utilize for solving the DSGE model.

Syntax:
risefile = dynare_sstate_file2rise(dynfile)

INPUTS:

- *dynfile* [char or function handle]: Dynare steady state file. If provided as a character array, it is converted to a function handle.

OUTPUTS:

- *risefile* [function handle]: RISE-compatible function handle for solving the DSGE model.

Example: - Create a RISE-compatible function handle from a Dynare steady state file:

```
`matlab risefile = dynare_sstate_file2rise('NK_baseline_steadystate');
`
```

3.16.3 AIM/YADA files

aim2rise – converts a basic dynare file into a RISE one

```
duplicates=aim2rise(aimFileName)
duplicates=aim2rise(aimFileName,riseFileName)
duplicates=aim2rise(aimFileName,riseFileName,stderr_name)
```

Args:

- **aimFileName** [char] : name of the dynare model file with or without extension (.aim)
- **riseFileName** [char|{'aimFileName.rs'}] : name of the created RISE file
- **stderr_name** [char|{'std'}] : prepended name for the newly created parameters (RISE transforms all the variances and standard deviations into parameters)

Returns:
:

- **duplicates** [struct] : structure containing information on the redundancies

NB: the created files are

- riseFileName.rs
- riseFileName_params.m : parameter function file with calibration and priors as separate outputs
- riseFileName_sstate.m (optional): steady state function file

See also:

Dynare files

3.17 Technical documentation for rise/dsge objects

3.17.1 The properties

3.17.1.1 user_data

this is so that the user can tailor the information to pass around functions written by him but called by RISE

Help for rise/user_data is inherited from superclass DSGE

3.17.1.2 nsols

number of solutions

Help for rise/nsols is inherited from superclass DSGE

3.17.1.3 definitions

values of auxiliary parameters defined in the model file with a #

Help for rise/definitions is inherited from superclass DSGE

3.17.1.4 equations

equations of the system

Help for rise/equations is inherited from superclass DSGE

3.17.1.5 filename

name of the rs/rz/dsge file read

Help for rise/filename is inherited from superclass DSGE

3.17.1.6 model_description

comments on the model

Help for rise/model_description is inherited from superclass DSGE

3.17.1.7 legend

attribute for giving a tag to a specific version of a model

3.17.1.8 endogenous

information on endogenous variables (names, number, types, etc.)

3.17.1.9 exogenous

information on exogenous variables (names, number, types, etc.)

3.17.1.10 observables

information on observable variables (names, number, types, etc.)

3.17.1.11 options

structure holding information on modifiable settings

3.17.1.12 parameters

information on parameters (names, number, types, etc.)

Help for rise/parameters is inherited from superclass ESTIMABLE

3.17.1.13 markov_chains

information on markov chains, regimes and related items

Help for rise/markov_chains is inherited from superclass ESTIMABLE

3.17.1.14 estimation

information on estimation: posterior maximization and simulation

Help for rise/estimation is inherited from superclass ESTIMABLE

3.17.2 The methods

3.17.2.1 abcd

Compute the ABCD test statistics of Fernandez-Villaverde, Rubio-Ramirez, Sargent, and Watston (2007) Uses the state-space system

and computes the eigenvalues of

If all eigenvalues are smaller than 1, the poor man's invertibility condition is satisfied and the structural shocks can be recovered from the observables

Syntax

```
[test, A, B, C, D] = abcd(m);

[test, A, B, C, D,retcode] = abcd(m);
```

Args:

- m (model object): model object

Returns:

:

- test : check above
- A : check above
- B : check above
- C : check above
- D : check above
- retcode : =0 if no problem

Reference:

Fernandez-Villaverde, Rubio-Ramirez,Sargent, and Watson (2007), "ABCs (and Ds) of Understanding VARs", American Economic Review, 97(3), 1021-1026

Help for rise/abcd is inherited from superclass DSGE

3.17.2.2 accuracy

accuracy : computes the Euler errors of dsge models

```
Euler_errors=accuracy(m)
Euler_errors=accuracy(m,order)
Euler_errors=accuracy(m,order,nxcuts)
Euler_errors=accuracy(m,order,nxcuts,nsims)
```

Args:

m (rise | dsge): scalar or vector of model objects.

order ({solve_order} | integer): Approximation order for which to
compute the accuracy. Always less than or equal to solve_order

nxcuts ({10} | integer): Number of quantiles per discretized
continuous shock

nsims ({1000} | integer| 1x2 cell): Number of simulations to generate
the state of the endogenous variables.

Returns:

:

- **g** [cell | struct]: if the number of models is greater that one, the output is a cell array of structures. Each structure has the following fields: - eqtn1, eqtn2,... : structures containing "mean" (mean of the euler error across all simulations), "max" (maximum of the euler error across all simulations) and "min" (minimum of the euler error across all simulations)

Note:

- The errors are NOT log10(abs(error)). The user can take the log10 or any other transformation as desired.
- The smaller the standard deviation of the shocks the higher the accuracy even if the solution is not very accurate in the first place.

- The accuracy of the solution may critically depend on the tolerance level used for solving the model in the first place. For instance, if an iterative algorithm (e.g. mfi) is used, the solution might not be 100% accurate even in a linear model
- The errors are provided in absolute value for all equations, rather than being normalized as is customarily done in the literature

Help for rise/accuracy is inherited from superclass DSGE

3.17.2.3 calculate_loss

Calculates welfare

```
[welf,retcode,V,d]=calculate_loss(m,lossstr)

[welf,retcode,V,d]=calculate_loss(m,lossstr,shocksdb)

[welf,retcode,V,d]=calculate_loss(m,lossstr,shocksdb,varargin)
```

Args:

- **m** (rise | dsge): scalar or vector of model objects.
- **lossstr** (char): loss function
- **shocksdb** (ts | struct | empty): Shocks database to condition on over simulation. if empty, the unconditional welfare is returned. if not empty the conditional welfare is calculated instead
- **varargin** : optional arguments coming in pairs

Returns:

- **welf** [scalar|vector]: conditional loss (scalar) or unconditional loss (scalar if number of regimes = 1)
- **retcode** [scalar]: return code
- **V** [n x n x h]: array of equilibrium matrices
- **d** [h x 1 vector]: constant in loss

Note:

The loss is such that $v(x_t, r_t) = x_t.'Vrtx_t + drt$

Help for rise/calculate_loss is inherited from superclass DSGE

3.17.2.4 calibrate

calibrate : estimates the parameters of a model without the need of evaluating the likelihood

```
[m]=calibrate(m)

 m=calibrate(m,varargin)
```

Args:

- **m** (rise | dsge): model object or vectors of model objects

Returns:

- **m** (rise | dsge): model object or vectors of model objects with information on the calibrated parameters

---

Note:

- It is assumed that varargin includes information on priors and possibly endogenous priors

---

Help for rise/calibrate is inherited from superclass DSGE

3.17.2.5 check_derivatives

Compares the derivatives and the solutions from various differentiation techniques

```
check_derivatives(obj)
retcode=check_derivatives(obj)
```

Args:
obj (rise | dsge): model object or vectors of model objects

Returns:
:

- **retcode** [numeric]: 0 if no problem is encountered during the comparisons. Else the meaning of recode can be found by running decipher(retcode)

Note:

- The derivatives computed are 'automatic', 'symbolic' or 'numerical'
- The comparisons are done relative to automatic derivatives, which are assumed to be the most accurate.

Help for rise/check_derivatives is inherited from superclass DSGE

3.17.2.6 condition_draws_on_model

condition routines on the model e.g. routines for drawing skewed shocks may depend on the model and its parameterization.

Help for rise.condition_draws_on_model is inherited from superclass DSGE

3.17.2.7 counterfactual

counterfactual Computes counterfactual history of a nonlinear DSGE model

Syntax:

```
counterf=counterfactual(m)
counterf=counterfactual(m,sim_engine)
counterf=counterfactual(m,sim_engine,nsim)
counterf=counterfactual(m,sim_engine,nsim,shock_names)
counterf=counterfactual(m,sim_engine,nsim,shock_names,varargin)
[counterf,actual]=counterfactual(...)
```

Inputs:

- m : [rise|dsge] model(s) for which to compute the counterfactual. m could be a vector of models. The computation of the counterfactual requires a computation of smoothed history. Hence either m should contain all the information needed for that purpose or the information should be passed along through varargin below
- sim_engine : [empty|function handle] function computing forecasts
- nsim : [empty|numeric|{100}] Number of simulation to consider for the integration exercise. nsim is automatically set to 1 if the model is detected to be solved at order 1 and not contain regime switches.
- shock_names : [empty|char|cellstr] list of shocks to consider in the computation of the counterfactual
- varargin : additional information needed for the computation of the smoothed quantities through filtration.

Outputs:

- counterf : [struct|cell array] structure or cell array of structures with the counterfactuals in each model and each solution.
- actual : [struct|cell array] structure or cell array of structures with the actuals (smoothed variables) in each model and each solution.

Remarks:

Note:

- For a nonlinear model (e.g linear but switching), the type of decomposition/counterfactual we do for linear/linearized constant-parameter models is not feasible. RISE uses a monte carlo integration to provide an approximation to the decomposition/counterfactual
- if m is a vector of models, then each model should return a unique solution (implying a unique filtration), else the concatenation of counterfactuals will fail. In that case it is better to run one model at a time.
- When a model has multiple solutions, there is no guarantee that every solution will have succeed at filtration. For this reason, the result of each counterfactual is stored in a separate cell. Empty cells represent the solutions for which the filtration could not be obtained.

Examples:

See also:

dsge/historical_decomposition

Help for rise/counterfactual is inherited from superclass DSGE

3.17.2.8 dbminusdb

INTERNAL FUNCTION

3.17.2.9 draw_parameter

Random parameter draws for RISE model objects.

```
[draw,obj] = draw_parameter(obj,simulation_folder);
```

Args:

- obj (rise | dsge | rfvar | svar): RISE model object
- simulation_folder (char | struct):
  - char(1): simulation folder : the stored elements should be structures with fields:
    - **x** : parameter vectors
    - **f** : value of minus(log posterior kernel)
  - char(2): ['mode'|'prior']: draw from the prior distribution or from a multivariate normal distribution around the mode.

Returns:

- **draw** [cell]: the first entry is the names of the estimated parameters and the second is a vector of drawn parameters. The whole cell can be pushed in to a model as obj=set(obj,'parameters',draw).
- **obj** [rise|dsge|rfvar|svar]: RISE model object in which the drawn parameter has been pushed.

Help for rise/draw_parameter is inherited from superclass ESTIMABLE

3.17.2.10 drop

DROP : drops model parameterizations

```
obj=irf(obj,w)
```

Args:

- obj (rise | dsge): single or vector of RISE models
- w (scalar|vector) : parameterizations to drop

Returns:

- **obj** [dsge|rise]: model(s) with dropped parameterizations

Help for rise/drop is inherited from superclass DSGE

3.17.2.11 estimate

Estimate the parameters of a RISE model

```
obj=estimate(obj)
obj=estimate(obj,varargin)
[obj,filtration]=estimate(...)
[obj,filtration,post_max]=estimate(...)
```

Args:

obj (rise | dsge | rfvar | svar): model object

varargin : additional optional inputs among which the most relevant for estimation are:

- **estim_eval_lik** [false|{true}]: if false, likelihood is not evaluated. This allows for an easy calibration of parameters in the case of instance of endogenous priors.
- **estim_parallel** [integer | {1}]: Number of starting values
- **estim_start_from_mode** [true | false | {[]}]: when empty, the user is prompted to answer the question as to whether to start estimation from a previously found mode or not. If true or false, no question is asked.
- **estim_start_date** [numeric | char | serial date]: date of the first observation to use in the dataset provided for estimation
- **estim_end_date** [numeric | char | serial date]: date of the last observation to use in the dataset provided for estimation
- **estim_max_trials** [integer | {500}]: When the initial value of the log-likelihood is too low, RISE uniformly draws from the prior support in search for a better starting point. It will try this for a maximum number of **estim_max_trials** times before squeaking with an error.
- **estim_general_restrictions** [{[]} | function handle | cell array]: when not empty, the argument can be a function handle or a cell array containing the function handle and additional input arguments. The general syntax for the calling the function handle is viol=myfunc(obj,varargin), with **obj** the parameterized RISE object which will be used in the computation of the restrictions violations. Hence the restrictions are entered either as @myfunc or as {@myfunc,arg2,arg3,...}. Originally, RISE will call the function without any inputs. In that case, RISE expects the output to be a structure with fields :
  - **number_of_restrictions** : number of restrictions
  - **kf_filtering_level** [0 | 1 | 2 | 3]: if 0, no filters are required for the computation of the restrictions. If 1, only the filtered variables are required. If 2, the updated variables are required. If 3, the smoothed variables are required.

  When the function is called with inputs, RISE expects as output the values of the restrictions. The sign of the violations does not matter. All the user has to do is to put a zero where the restrictions are not violated.
- **estim_linear_restrictions** [{[]} | cell]: This is most often used in the estimation of rf-var or svar models either to impose block exogeneity or to impose other forms of linear restrictions. When not empty, **estim_linear_restrictions** must be a 2-column cell:
  - Each row of the first column represents a particular linear combination of the estimated parameters.
  - Each row of the second column holds the value of the linear combination.
- **estim_nonlinear_restrictions** [{[]} | cell]: When not empty, **estim_nonlinear_restrictions** must be a k x 1 cell, with each row representing a particular restriction on the parameters. e.g. for a switching model, one can have alpha(zlb,1)>alpha(zlb,2), which can also be written as alpha_zlb_1>alpha_zlb_2. The restrictions can also be equality restrictions. In this case, however, it is assumed that the parameters entering the lhs of restrictions are not estimated. e.g. alpha(zlb,1)=3*cos(alpha(zlb,2))+1.
- **estim_priors** [{[]}|struct]: This provides an alternative to setting priors inside the rise/dsge model file. Each field of the structure must be the name of an estimated parameter. Each field will hold a cell array whose structure is described in help GENERIC/setup_priors.
- **estim_endogenous_priors** [empty|cell array | function handle]: When not empty, **estim_endogenous_priors** must be either a cell array of cell arrays with each subcell array having the format {item,args2,args3,...} where "item" is the thing to evaluate and args2,

args3, etc. are the usual arguments for setting priors EXCEPT the start value. These are some of the forms that "item" can take

- 'corr{endo,endo,horizon}'
- 'irf{endo,exo,horizon}'
- 'userfun(irf{endo,exo,horizon})'
- 'userfun(a1,a2,...,an)'

In other words there is support for correlations, impulse responses and combinations of parameters and all those items should evaluate to a scalar. This approach works well but is not very flexible.

Alternatively, **estim_endogenous_priors** could be a function handle such that when called without inputs, it returns a struct with fields:

- **priors** : cell array of estimation priors. more explicitly, each entry of the cell array is itself a cell with the same syntax as the priors for estimation, EXCEPT the start value!
- **kf_filtering_level** [0 | 1 | 2 | 3]: if 0, no filters are required for the computation of the endogenous priors. If 1, only the filtered variables are required. If 2, the updated variables are required. If 3, the smoothed variables are required.

When the function handle is called with TWO inputs, what is returned is a vector of values for which RISE will evaluate the endogenous prior. This vector should have the same length as the previous cell array. The two inputs are:

- **obj** : The model object
- **filtration** : a structure containing the filters

- **estim_blocks** [{[]} | cell]: When not empty, this triggers blockwise optimization. For further information on how to set blocks, see help for dsge.create_estimation_blocks
- **estim_penalty_factor** [numeric | {10}]: when general nonlinear restrictions are present, RISE uses an estimation strategy in which the objective function is penalized as f_final=fval+estim_penalty_factor*sum(max(0,g)^2) where g is a vector of the values of the restrictions, which are expected to be of the form g(x)<=0. See **estim_general_restrictions** above.
- **optimizer** [char | function handle | cell|{fmincon}]: This can be the name of a standard matlab optimizer or RISE optimization routine or a user-defined optimization procedure available of the matlab search path. If the optimzer is provided as a cell, then the first element of the cell is the name of the optimizer or its handle and the remaining entries in the cell are additional input arguments to the user-defined optimization routine. e.g. estimate(m,'optimizer',{'fmincon','MaxFunEvals',1000,...}). A user-defined optimization function should have the following syntax

```
[xfinal,ffinal,exitflag,H]=optimizer(fh,x0,lb,ub,options,varargin);
```

That is, it accepts as inputs:

- **fh**: the function to optimize
- **x0**: a vector column of initial values of the parameters
- **lb**: a vector column of lower bounds
- **ub**: a vector column of upper bounds
- **options**: a structure of options whose fields will be similar to matlab's optimset

  - **varargin**: additional arguments to the user-defined optimization procedure

  That is, it provides as outputs:

  - **xfinal**: the vector of final values
  - **ffinal**: the value of **fh** at **xfinal**
  - **exitflag**: a flag similar to the ones provided by matlab's optimization functions.
  - **H**: an estimate of the Hessian

- **estim_barrier** [{false} | true]: never allow constraints to be violated in no circumstances.

Returns:

- **obj** [rise | dsge | rfvar | svar]: model object parameterized with the mode found and holding additional estimation results and statistics that can be found under obj.estimation
- **filtration** [struct]: structure with the filtration information of the model parameterized at the mode
- **post_max** [struct]: structure with prior and posterior maximization information

Note:

- recursive estimation may be done easily by passing a different estim_end_date at the beginning of each estimation run.
- It is also possible to estimate a dsge model using conditional future information on endogenous (**forecast_cond_endo_vars**) as well. as on exogenous (**forecast_cond_exo_vars**). The dataset provided in this case must have several pages. The first page is the actual data, while the subsequent pages are the expectations data. See help dsge.forecast for more information
  - Option **estim_penalty** which is the value of the objective function when a problem occurs (e.g. no solution found, very low likelihood, stochastic singularity, problems computing the initial covariance matrix, non-positive definite covariance matrices, etc.) is deprecated. If needed you should rather set **ObjectiveLimit** as part of the options of the optimizer. e.g. estimate(m,'optimizer',{'fmincon','ObjectiveLimit', -1e+10})

See also:

ESTIMABLE/SETUP_PRIORS

Help for rise/estimate is inherited from superclass DSGE

3.17.2.12 extract_first_order_structure

EXTRACT_FIRST_ORDER_STRUCTURE Extracts the first-order structure of a DSGE model.

Syntax:

[Aplus, A0, Aminus, B, Q, ss, growth] = extract_first_order_structure(obj) [Aplus, A0, Aminus, B, Q, ss, growth] = extract_first_order_structure(obj, sm)

Inputs:

- obj [rise|dsge]: Model object.
- sm [struct]: Derivatives computed by RISE (optional).

Outputs:

- Aplus [h x h cell array]: Coefficients on the forward-looking terms multiplied by the transition matrix.
- A0 [1 x h cell array]: Coefficients on the contemporaneous terms.

- Aminus [1 x h cell array]: Coefficients on the backward-looking terms.
- B [1 x h cell array]: Coefficients on shocks.
- Q [h x h matrix]: Transition matrix (qij) with i=today and j=tomorrow.
- ss [n x h matrix]: Steady states.
- growth [n x h matrix]: Balanced growth.

Notes:

- The steady state is returned instead of the constant term. This assumes that the model is at least conditionally stationary. The constant term can be recovered as b = -(Aplus{i,i}/Q(i,i) + A0{i} + Aminus{i}) * ss(:, i).
- In case of a model with multiple parameterizations, all the outputs are cell arrays, with each cell representing one parameterization.

Description:

The function extracts the first-order structure of a DSGE model, allowing the user to use their own algorithms externally. It returns the coefficients on various terms in the linearized model equations, as well as the transition matrix, steady states, and balanced growth rates.

Example:

% Extract first-order structure from a DSGE model model = rise('my_model_file'); [Aplus, A0, Aminus, B, Q, ss, growth] = extract_first_order_structure(model);

See also:

solve

Help for rise/extract_first_order_structure is inherited from superclass DSGE

3.17.2.13 filter

Filtering of DSGE models

```
[filtration,LogLik,Incr,retcode,obj]=filter(obj,varargin)
```

Args:

- obj (rise | dsge): model object
- varargin (name,value): valid pairwise options with the most relevant being:
  - **kf_ergodic** [{true}|false]: initialization at the ergodic distribution
  - **kf_init_variance** [{[]}|scalar]: initial variance factor (Harvey scale factor). If not empty, the information in T and R is ignored.
  - **kf_presample** [{0}|integer]: Number of observations to discard before computing the likelihood.
  - **kf_filtering_level** [0|1|2|{3}]: 0: Likelihood only, 1: 0+ filtered series 2: 1+ updated series 3: 2+ smoothed series
  - **kf_user_init** [{[]}|cell]: User-defined initialization. When not empty, it can take three forms. {a0}, {a0,cov_a0}, {a0,cov_a0,PAI00} where a0 is the initial state vector with the same order as the rows of T, cov_a0 is the initial covariance of the state vector (same order as a0) and PAI00 is the initial vector of regime probabilities. Any information provided here will override *kf_init_variance*.

    * *a0* (numeric, struct or function_handle): State vector for each regime. Default is the steady state.
      · The struct is such that each field is a numeric e.g p.y=0.3 or a char in which the parameters of the model can appear e.g. p.y='0.3-alpha/beta';
      · The function handle should accept a regime-specific parameter structure and a steady-state vector i.e. *a0(p,stst)* and returns a vector corresponding to the regime-specific state.
    * *cov_a0* (numeric or function_handle): Covariance matrix of state vector for each regime. Default is repmat(eye(m), [1, 1, h]).
      · The function handle should accept a regime-specific parameter structure and a steady-state vector i.e. *cov_a0(p,stst)* and returns a regime-specific covariance matrix.
      · If numeric and scalar, then cov_a0 = repmat(cov_a0*eye(m), [1, 1, h])
      · If numeric and vector, then cov_a0 = repmat(diag(cov_a0), [1, 1, h])
      · If numeric and size(cov_a0,3)=1, then cov_a0 = repmat(cov_a0, [1, 1, h])
    * *PAI00* (numeric, function_handle, or vector): Initial probabilities for each regime. The function handle should accept a regime-specific parameter structure and a steady-state vector i.e. *PAI00(p,stst)* and returns a scalar!!!: the probability of the specific regime. Default is ones(h, 1) / h.

  – **kf_user_algo** [{''}|char|cell|function handle]: User-defined filtering algorithm. The filter should take as inputs (syst,y,U,z,options), with
    * **syst** [struct]: structure or model object both provided by dsge.filter (see notes below)
    * **y** [matrix]: matrix of data or structure both provided by dsge.filter (see notes below)
    * **U** [matrix]: matrix of trends provided by dsge.filter
    * **z** [matrix]: matrix of deterministic terms provided by dsge.filter
    * **options** [struct]: options provided by dsge.filter

    The filter should return [LogLik,Incr,retcode,Filters], with
    * **LogLik** [numeric]: value of the log likelihood
    * **Incr** [vector]: contributions to the likelihood in each period
    * **retcode** [numeric]: flag equal to 0 if there is no problem
    * **Filters** [struct]: structure containing all the filtering information

    In some (rare) circumstances, the information provided by the inputs is not sufficient to run the specific filter of the user. In that case, it is assumed that the user would know how to retrieve the relevant information from the parameterized model object. The user filter should then be written in such a way that when provided with two inputs (input1= model object, input2=structure), it returns a modified structure (input2) containing the information needed. This process is triggered by writing a star in front of the name of the user filter. e.g. 'kf_user_algo','*user_filter', or 'kf_user_algo',{'*user_filter',...}. In this case the function handle option is not available.

  – **kf_householder_chol** [{false}|true]: if true, return the cholesky decomposition when taking the householder transformation. This option is primarily used in the switching divided difference filter.

Returns:

- **filtration** [struct]: structure containing the filters
- **LogLik** [numeric]: log likelihood

- **Incr** [vector]: contributions to the likelihood for each time t
- **retcode** [integer]: 0 if no problem encountered
- **obj** [rise|dsge]: model object possibly parameterized and solved.

See also:

rise.filter_initialization, dsge_tools.filtering_initialization

Help for rise/filter is inherited from superclass DSGE

3.17.2.14 fisher

INTERNAL FUNCTION

Help for rise/fisher is inherited from superclass ESTIMABLE

3.17.2.15 fold_solution

fold_solution : folds higher-order solutions in order to facilitate comparison with other software

Syntax :

[foldsol,zlist_reordered,ylist_reordered]=fold_solution(m)

[foldsol,zlist_reordered,ylist_reordered]=fold_solution(m,order)

[foldsol,zlist_reordered,ylist_reordered]=fold_solution(m,order,user_state_list)

[foldsol,zlist_reordered,ylist_reordered]=fold_solution(m,order,user_state_list,user_endo_list)

Inputs:

- **m** [rise|dsge]: scalar or vector of solved model objects, possibly with multiple parameterizations
- **order** [numeric|empty]: Desired order of perturbation <= order of solution
- **user_state_list** [cellstr|empty]: List of state variables as appearing in the non-RISE solution
- **user_endo_list** [cellstr|empty]: List of endogenous variables in the non-RISE solution

Outputs:

- **foldsol** [cell array]: folded solutions
- **zlist_reordered** [cellstr]: reordered list of state variables
- **ylist_reordered** [cellstr]: reordered list of endogenous variables

Help for rise/fold_solution is inherited from superclass DSGE

3.17.2.16 forecast

Compute forecasts for rise/dsge/svar/rfvar models

```
cond_fkst_db = forecast(obj,varargin)
```

Args:

obj (rise | dsge | svar | rfvar): model object

varargin
[additional inputs coming in pairs. These include but are] not restricted to:

- **forecast_to_time_series** [{true}|false]: sets the output to time series format or not
- **forecast_nsteps** [integer|{12}]: number of forecasting steps
- **forecast_start_date** [char|numeric|serial date]: date when the forecasts start (end of history + 1)
- **forecast_cond_endo_vars** [{''},char|cellstr]: names of conditional endogenous variables to be used either in forecasting or in estimation
- **forecast_cond_exo_vars** [{''},char|cellstr]: names of conditional exogenous variables to be used either in forecasting or in estimation
- **forecast_endo_exo_vars** [{''},char|cellstr]: names of exogenous variables that are potentially modified under conditional forecasting with forward-back shooting
- **forecast_shock_uncertainty** [true|{false}]: draw shocks over the simulation horizon.

Returns:
:

- **cond_fkst_db** [struct|matrix]: depending on the value of **forecast_to_time_series** the returned output is a structure with time series or a cell containing a matrix and the information to reconstruct the time series.

Note:

- the historical information as well as the conditioning information come from the same database. The time series must be organized such that for each series, the first page represents the actual data and all subsequent pages represent conditional information. If a particular condition is "nan", that location is not constrained
- Conditional forecasting for nonlinear models is also supported. However, the solving of the implied nonlinear problem may fail if the model displays instability
- Both HARD CONDITIONS and SOFT CONDITIONS are implemented. In order to do soft conditions, the variables one wishes to condition on must have lower and upper bounds represented by the presence of variables with names "lower_CONDNAME" and "upper_CONDNAME", where CONDNAME is the name of a particular variable we want to condition on.
- The data may also contain time series for a variable with name **regime** in that case, the forecast/simulation paths are computed following the information therein. **regime** must be a member of 1:h, where h is the maximum number of regimes.
- Further options for conditional forecasting can be found in RSCF.FORECAST

See also:

- rise_generic/simulate
- rscf.forecast

3.17.2.17 forecast_real_time

Forecast from each point in time

Syntax

```
[ts_fkst,ts_rmse,rmse,Updates]=forecast_real_time(obj)

[ts_fkst,ts_rmse,rmse,Updates]=forecast_real_time(obj,varargin)
```

Args:

- obj (dsge): model object
- varargin : valid optional inputs coming in pairs. The main inputs of interest for changing the default behavior are:
  - **forecast_rt_nsteps** [integer] : number of periods ahead

Returns:

- **ts_fkst** [struct] : fields are forecasts in the form of ts objects for the different endogenous variables
- **ts_rmse** [ts|struct] : if only one object is processed, the output is a ts. If instead several objects are processed, fields are RMSEs in the form of ts objects for the different endogenous variables
- **rmse** [matrix] : RMSEs for the different endogenous variables
- **Updates** [struct] : fields are the updated (in a filtering sense) in the form of ts objects for the different endogenous variables

See also:

- plot_real_time

Help for rise/forecast_real_time is inherited from superclass DSGE

3.17.2.18 frontier

Compute standard devations of the model for a grid over a given parameter

```
f = frontier(obj,lambda_name,lambda_vals)
```

Args:

- **obj** (rise | dsge): model object
- **lambda_name** (char): name of the parameter to vary
- **lambda_vals** (vector): 1 x 2 or 1 x N vector of values for the parameter. When N=2, a grid of 50 points is constructed between the two values. When N>2, lambda_vals is the grid.
- **simul** (true | {false}): use simulation instead of theoretical moments.
- **seed** (numeric | {1971}): seed for simulations

Returns:

- **f** [struct]: standard deviations of all variables in the model for each value of lambda_name and some further information about the simulation process under a substructure with name **stats__**. The fields of the sub-structure are:
  - **lambda** [vector]: discretized values of lambda

- **ngrid** [scalar]: number of grid points
- **simul_periods** [integer]: number of simulations. If strictly positive, then simulation is used for computing the moments of the process
- **retcode** [vector]: information on how successful each run in the grid is. if retcode=0, there is no problem. If retcode different from zero, then the information can be retrieved by running decipher(retcode)

Note:

- improvements to consider are how to deal with regime switches or nonlinear models in general. One solution is to use simulation

---

Help for rise/frontier is inherited from superclass DSGE

3.17.2.19 get

Fetches information from generic objects

```
[Reply,retcode] = get(obj,PropertyName)
```

Args:

obj (rise | dsge | rfvar | svar): model object

PropertyName (char): name of the property or element desired. This includes:

- **'structure'** [char]: derivatives + transition matrices + other information need to solve the model
- **'equations'** [char]: dynamic model equations in format Ef(x{t+1},x{t},x{t-1},e{t},theta)=0
- **'definitions'** [char]: definitions. This requires the model to be solved. If there are no definitions or if the definitions have been substituted, the result will be an empty structure.
- **'solution'** [char]: solution of the model
- **'bgp'** [char]: balanced growth path. It is also possible to further taylor the output:
  - 'bgp'|'bgp(default)' will give the same result (default) result
  - '...(struct)' will return the BGP in vector of structures, where each structure is a separate regime
  - '...(cell)' will return the BGP in a cell array in which the first column is the list of variables and the subsequent columns are the different regimes.

  N.B. For linear variables the BGP is x_t-x_{t-1}, whereas for log-linear variables the BGP is x_t/x_{t-1}
- **'sstate'** [char]: steady state. It is also possible to further taylor the output:
  - 'sstate' | 'sstate(default)' will give the same result (default) result
  - '...(struct)' will return the sstates in vector of structures, where each structure is a separate regime

  - '…(cell)' will return the sstates in a cell array in which the first column is the list of variables and the subsequent columns are the different regimes.

- **'parameters'|'par_vals'** [char]: parameter values. It is also possible to further taylor the output:
  - 'parameters'|'parameters(default)'|'par_vals'|'par_vals(default)' will give the same result: only the declared parameters will appear and for switching ones, the name will be appended with the chain name and the state. This is the most natural way of getting parameters and setting parameters from this output should be unproblematic.
  - '…(legacy)' will return the parameters without appending the name of the chain and the state of the chain. The parameter values are given for all regimes. It is not advisable to push the parameters "set(m,'parameters',p)" in this form.
  - '…(struct)' will return the parameters in vector of structures, where each structure is a separate regime
  - '…(cell)' will return the parameters in a cell array in which the first column is the list of the parameters and the subsequent columns are the different regimes.
- **'par_list'** [char]: list of parameters. Instead of the full list, a sub-list or its complement (using a "~" sign in from of the attribute) can also be queried:
  - '…(switching)' : list of parameters that are switching
  - '…(trans_prob)' : list of transition probability parameters
  - '…(measurement_error)' : list of measurement-error parameters
  - '…(in_use)' : list of parameters that are in use
- **'const_list'** [char]: list of constant terms ordered as p|d|s|x|g (i.e parameters,definitions,endo_stst,exo_stst,endo_growth)
- **'par_tex'** [char]: description of the parameters
- **'endo_list'** [char]: list of the endogenous variables. Instead of the full list, a sub-list or its complement (using a "~" sign in from of the attribute) can also be queried:
  - '…(lagrange_multiplier)' : Lagrange multipliers for optimal policy models
  - '…(static)': static variables or variables appearing only contemporaneously in the model
  - '…(predetermined)': predetermined variables i.e. variables appearing with lags and not with leads
  - '…(pred_frwrd_looking)': variables appearing with both a lead and a lag
  - '…(state)': endogenous state variables i.e. all variables appearing with a lag
  - '…(frwrd_looking)': variables appearing with leads but not lags
  - '…(log_var)': variables for which a log-linear approximation is declared from within the model file.
  - '…(log_expanded)': variables for which a log-linear approximation is requested after the model object is built.
  - '…(auxiliary)': auxiliary variables automatically created by RISE for support. Leads and lags greater than 1, lags or leads in parameters or exogenous variables.
  - '…(original)': endogenous variables declared in the model file i.e. excluding the auxiliary variables

  - '...(affect_trans_probs)': variables entering the calculation of endogenous probabilities
  - '...(hybrid_expect)': variables for which a hybrid expectation is taken.
  - '...(stationary)': variables that are not trending over time.
- **'endo_tex'** [char]: description of endogenous variables
- **'exo_list'** [char]: list of exogenous variables. Instead of the full list, a sub-list or its complement (using a "~" sign in from of the attribute) can also be queried:
  - '...(observed)' : list of exogenous variables that are observed
  - '...(in_user)' : list of exogenous variables that appear in the model block
- **'exo_tex'** [char]: description of exogenous variables
- **'obs_list'** [char]: list of observable variables. Instead of the full list, a sub-list or its complement (using a "~" sign in from of the attribute) can also be queried:
  - '...(endogenous)' : list of observable variables that are endogenous.
- **'obs_tex'** [char]: description of observable variables
- **'def_list'** [char]: list of definitions
- **'chain_list'** [char]: list of markov chains
- **'chain_tex'** [char]: description of markov chains
- **'regime_list'** [char]: list of regimes (i.e. composites of states from different chains)
- **'regime_tex'** [char]: description of regimes. Few things to keep in mind :
- if you give names to the different states, those names will be used in the description of the regimes
- if you give names to the regimes, the regimes will not be described, i.e. written as regime_1(policy=1 & const=1 & vol=1)
- The description always includes const=1
- **'state_list'** [char]: list of states of all the markov chains
- **'state_tex'** [char]: description of the states of all the markov chains
- **'tex'|'description'** [char]: description for all the atoms in the system.
- **'state_vars'** [char]: variables and their lag structure as required for forecasting.
- **'mode'** [char]: parameters maximizing the posterior distribution
- **'prior_mean'** [char]: prior mean of the parameters
- **'start'** [char]: initial values for estimation (maximization of the posterior)
- **pname** [char]: name of a particular parameter in the models
- **sstate_exogenous** [char]: steady state for exogenous variables, which is not always zero. Very useful for instance in the context of deterministic simulations

Returns:

:

- **Reply** []: value for the queried property/information
- **retcode** [numeric]: 0 if an error is not encounted

Note: It is possible to get the math tex_name for any tex_name type e.g. get(m,'par_tex(math)'). You may want to also try get(m,'par_tex(math)') or get(m,'par_tex(decoy)')

Example:

3.17.2.20 growth_database

INTERNAL FUNCTION

3.17.2.21 hessian

Computes the hessian of the model at a specific point

```
[obj,H,issue] = hessian(obj)

[obj,H,issue] = hessian(obj,x)

[obj,H,issue] = hessian(obj,x,varargin)
```

Args:

- obj (rise | dsge | rfvar | svar): model object
- x ([] | vector): vector at which one wants to compute the hessian
- varargin: additional optional inputs among which the most relevant for estimation is:
  - **hessian_type** [{'fd'}|'opg']: The hessian is either computed by finite differences (fd) or by outer-product-gradient (opg)

Returns:

:

- **obj** [rise|dsge|rfvar|svar]: model object containing the new hessian in case the model was previously estimated.
- **H** [d x d matrix]: hessian matrix
- **issue** [char|'']: any issue encountered during the computation of the hessian

Help for rise/hessian is inherited from superclass ESTIMABLE

3.17.2.22 historical_decomposition

historical_decomposition Computes historical decomposition of a nonlinear DSGE model

Syntax:

```
mycontrib=historical_decomposition(m)

mycontrib=historical_decomposition(m,sim_engine)

mycontrib=historical_decomposition(m,sim_engine,nsim)

mycontrib=historical_decomposition(m,sim_engine,nsim,groups)

mycontrib=historical_decomposition(m,sim_engine,nsim,groups,varargin)
```

Inputs:

- m : [rise|dsge] model(s) for which to compute the historical decomposition. m could be a vector of models. The computation of the historical decomposition requires a computation of smoothed history. Hence either m should contain all the information needed for that purpose or the information should be passed along through varargin below
- sim_engine : [empty|function handle] function computing forecasts
- nsim : [empty|numeric|{100}] Number of simulation to consider for the integration exercise. nsim is automatically set to 1 if the model is detected to be solved at order 1 and not contain regime switches.
- groups : [structure|cell array |{empty}] grouping of shocks in the decomposition. By default, the shocks are not grouped. The syntax is of the form {group1,{v11,v12,...},...,groupn,{vn1,vn2,...}}. The shocks that are not listed are put in a special group called "others". The "others" group does not include the effect of initial conditions. e.g. p=struct(); p.demand={'Ey','Er'}; p.supply={'Ep'}; e.g. p={'demand',{'Ey','Er'},'supply',{'Ep'}};
- varargin : additional information needed for the computation of the smoothed quantities through filtration.

Outputs:

- mycontrib : [struct|cell array] structure or cell array of structures with the contributions in each model and each solution. The decompositions are given in terms of:
  - the exogenous variables
  - **init** : the effect of initial conditions, which includes the steady state!!!

Remarks:

- **N.B** : For a nonlinear model (e.g linear but switching), the type of decomposition/historical_decomposition we do for linear/linearized constant-parameter models is not feasible. RISE uses a monte carlo integration to provide an approximation to the decomposition/historical_decomposition
- **N.B** : if m is a vector of models, then each model should return a unique solution (implying a unique filtration), else the concatenation of decompositions will fail. In that case it is better to run one model at a time.
- **N.B** : When a model has multiple solutions, there is no guarantee that every solution will have succeed at filtration. For this reason, the result of each decomposition is stored in a separate cell. Empty cells represent the solutions for which the filtration could not be obtained.
- **N.B**
  [For the variables declared as "log_variables", the] decomposition is given in terms of transformed variables and not in terms of their level or original units.

Examples:

See also:

rise/historical_decomposition

Help for rise/historical_decomposition is inherited from superclass DSGE

3.17.2.23 historical_decomposition_switch

historical_decomposition Computes historical decomposition of a regime-switching DSGE model

Syntax:

```
mycontrib=historical_decomposition_switch(m)

mycontrib=historical_decomposition_switch(m,groups)

mycontrib=historical_decomposition_switch(m,groups,varargin)
```

Inputs:

- m : [rise|dsge] model(s) for which to compute the historical decomposition. m could be a vector of models. The computation of the historical decomposition requires a computation of smoothed history. Hence either m should contain all the information needed for that purpose or the information should be passed along through varargin below
- groups : [structure|cell array |{empty}] grouping of shocks in the decomposition. By default, the shocks are not grouped. The syntax is of the form {group1,{v11,v12,...},...,groupn,{vn1,vn2,...}}. The shocks that are not listed are put in a special group called "others". The "others" group does not include the effect of initial conditions. e.g. p=struct(); p.demand={'Ey','Er'}; p.supply={'Ep'}; e.g. p={'demand',{'Ey','Er'},'supply',{'Ep'}};
- varargin : additional information needed for the computation of the smoothed quantities through filtration.

Outputs:

- mycontrib : [struct|cell array] structure or cell array of structures with the contributions in each model and each solution. The decompositions are given in terms of:
  - the exogenous variables
  - **init** : the effect of initial conditions, which includes the steady state!!!

Remarks:

- **N.B** : For a model with switching, RISE adds the contribution of the switching process
- **N.B** : if m is a vector of models, then each model should return a unique solution (implying a unique filtration), else the concatenation of decompositions will fail. In that case it is better to run one model at a time.
- **N.B** : When a model has multiple solutions, there is no guarantee that every solution will have succeed at filtration. For this reason, the result of each decomposition is stored in a separate cell. Empty cells represent the solutions for which the filtration could not be obtained.
- **N.B**
  [For the variables declared as "log_variables", the] decomposition is given in terms of transformed variables and not in terms of their level or original units.

Examples:

See also:

rise/historical_decomposition

Help for rise/historical_decomposition_switch is inherited from superclass DSGE

3.17.2.24 indirect_inference

indirect_inference : Estimates coefficients using indirect inference

```
[m]=indirect_inference(m,myobjective)
[m]=indirect_inference(m,myobjective,varargin)
```

Args:

m (rise | dsge): scalar or vector of model objects.

myobjective (function handle): function that takes as input the model object and returns :

1. the criterion to minimize
2. **a flag to inform whether the criterion is successfully computed**
   or not

varargin : usual options for a dsge/rise object

Returns:

:

- **m** [scalar|vector]: scalar or vector of model objects.

Note:

Help for rise/indirect_inference is inherited from superclass DSGE

3.17.2.25 initial_conditions

INTERNAL FUNCTION

growth_type,'zero' growth_type,'steady'

3.17.2.26 initial_sstate

initial_sstate : provides a template for initial values in the calculation of the steady state of a dsge model

Syntax

```
db=initial_sstate(obj)
```

Args:

obj (rise | dsge): scalar or vector of model objects.

Returns:

- **db [struct | cell]: structure or cell array of structures if**
  several models are given as input. the fields of a structure are the names of the endogenous variables of the model. The number of concatenated structures is the number of regimes of the model. each field is a 1 x 2 cell array. e.g bounds.C={sstateInfo,bgpInfo} where sstateInfo and bgpInfo are 1 x 3 vectors organized as [start_value,lower_bound,upper_bound].

Note:

- The information in each structure is given in original units/levels of the variables before any potential log transformation.

See also rise.sstate

Help for rise/initial_sstate is inherited from superclass DSGE

3.17.2.27 irf

Computes impulse responses for a RISE model

```
[myirfs,retcode]=irf(obj)

[myirfs,retcode]=irf(obj,varargin)
```

Args:

- obj (rise | dsge): single or vector of RISE models
- varargin : optional options coming in pairs. The notable ones that will influence the behavior of the impulse responses are:
  - **irf_shock_list** [char | cellstr | {''}]: list of shocks for which we want to compute impulse responses
  - **irf_var_list** [char | cellstr | {''}]: list of the endogenous variables we want to report
  - **irf_periods** [integer | {40}]: length of the irfs
  - **irf_shock_sign** [numeric | -1 | {1}|struct]: sign or scale of the original impulse.
    - If **irf_shock_sign** is numeric and scalar and >0, we get impulse responses to a positive shock. If **irf_shock_sign** <0, the responses are negative. If If **irf_shock_sign** =0, all the responses are 0.
    - If **irf_shock_sign** is a structure, the fields are names of shocks. This is not a proper impulse response as such but a scenario with a cocktail of shocks. Note that in this case, the irfs are only computed for the cocktail of shocks. However, the result is written to all the shocks and therefore, the output is identical for all individual shocks !!!
  - **irf_draws** [integer | {50}]: number of draws used in the simulation impulse responses in a nonlinear model. A nonlinear model is defined as a model that satisfies at least one of the following criteria
    - solved at an order >1
    - has more than one regime and option **irf_regime_specific** below is set to false
  - **irf_type** [{irf} | girf]: type of irfs. If the type is irf, the impulse responses are computed directly exploiting the fact that the model is linear. If the type is girf, the formula for the generalized impulse responses is used: the irf is defined as the expectation of the difference of two simulation paths. In the first path the initial impulse for the shock of interest is nonzero while it is zero for the second path. All other shocks are the same for both paths in a given simulation.
  - **irf_regime_specific** [{true} | false]: In a switching model, we may or may not want to compute impulse responses specific to each regime.
  - **irf_girf_regime_uncertainty** [{false} | true]: If we consider regimes as shocks, then they should be treated as other structural shocks under generalized impulse response functions : the path for the regimes across the two simulations needed for computing an impulse response is the same (uncertainty = false). Otherwise the sequence of regimes may differ in the two simulations (uncertainty =true).
  - **irf_initial_conditions** [{empty} | struct]: if not empty, this is a structure with fields **endogenous** and **regime**

- **irf_order** [{empty} | integer]: order of the solution simulation of the irfs. If the order is greater than **solve_order**, the model is resolved.
- **irf_to_time_series** [{true} | false]: If true, the output is in the form of time series. Else it is in the form of a cell containing the information needed to reconstruct the time series.

Returns:

- **myirfs** [{struct}|cell]: Impulse response data
- **retcode [{0}|numeric]: return code in case different from 0 if** there is a problem

Note:

- for linear models or models solved up to first order, the initial conditions as well as the steady states are set to 0 in the computation of the impulse responses.
- for nonlinear models, the initial conditions is the ergodic mean

Help for rise/irf is inherited from superclass DSGE

3.17.2.28 is_forward_guidance_puzzle

is_forward_guidance_puzzle: Checks whether a DSGE model exhibits a forward-guidance puzzle.

The function can handle multiple models simultaneously, and each model can have multiple parameterizations. Under regime switching, it is not possible to characterize the forward guidance as in constant-parameter models because the impact of shocks differs across regimes.

Syntax:

```
o=is_forward_guidance_puzzle(m)

o=is_forward_guidance_puzzle(m,errIfMultReg)
```

Inputs:

- m: DSGE model or an array of DSGE models to analyze.
- errIfMultReg [{true}|false]: If `True`, raise an error when handling regime switching models. If `False`, assume shock impacts are the same across regimes.

Output:

- o: A logical array indicating whether each model exhibits a forward-guidance puzzle for each parameterization and regime.
- maxEig: maximum eigenvalue criterion

The function checks whether a given DSGE model or a list of DSGE models exhibit a forward-guidance puzzle. The forward-guidance puzzle occurs when the model's dynamics lead to explosive responses to future shocks.

This function returns a logical array where each element corresponds to a specific model parameterization and regime. A value of `True` indicates that a forward-guidance puzzle is present, and `False` indicates no forward-guidance puzzle.

Note:

- When analyzing a single model, the result is a logical array.

- When analyzing multiple models, the result is a cell array of logical arrays, with one element for each model.

> **Warning:** When dealing with regime-switching models (models with multiple regimes), setting `errIfMultReg=True` will raise an error because it's not possible to characterize forward guidance in such models due to differing shock impacts across regimes.

Help for rise/is_forward_guidance_puzzle is inherited from superclass DSGE

3.17.2.29 is_stable_system

Checks the stability of a linear markov switching system.

Syntax

```
flag = is_stable_system(obj)

flag = is_stable_system(obj,varargin)
```

Args:

- obj (dsge|rise): model object
- varargin (name,value): pairwise valid options for RISE. The most relevant in this case are
  - **stability_criterion** (numeric\|{1.000001}): stability criterion. All eigenvalues must be smaller than this criterion for the system to be MSS
  - **stability_algorithm** ('cfm'\|{'gmh'}): CFM stands for Costa-Fragoso-Marques while HMG stands for Hassibi-Murray-Gupta.

Returns:

- **flag** (false\|true|vector|cell array): result of the investigation on whether the system is stable or not.
- **retcode** (0|25|vector|cell array): result of the investigation on whether the system is stable or not.
- **overall_retcode** (0|25|vector): result of the investigation on whether the system is stable or not.
- **cellFlag** (cell array): stability flag for each parameterization

Note:

RISE implements two algorithms from the engineering literature to check for the stability. They are

- Costa-Fragoso-Marques [Costa *et al.*, 2005]
- Hassibi-Murray-Gupta [Gupta *et al.*, 2003]

Refer to the references to see the specific algorithms. However, for most applications, one can just use the default options.

References:

- [Costa *et al.*, 2005]
- [Gupta *et al.*, 2003]

3.17.2.30 is_stationary_system

Checks whether a the model is stationary. i.e., does not containt trending variables

```
flag=is_stationary_system(obj)
```

Args:

obj (rise|dsge): rise/dsge model object

Returns:

:

- **flag** (bool): true if the model is stationary

Note:

- There is a difference between stability and stationarity
  - stability refers to the system as a whole and conditions for stability are often assessed through eigenvalues inside the unit circle
  - stationarity refers to a scalar stochastic process. And such a process will be said stationary if its first and second moment do not vary with time.

See also:

- rise_generic/is_stable_system

Help for rise/is_stationary_system is inherited from superclass DSGE

3.17.2.31 isnan

INTERNAL FUNCTION

3.17.2.32 itranslate

translate – Translates comprehensible atoms (model variables) into RISE codes

```
outList=translate(obj,inList)
outList=translate(obj,inList,order)
```

Args:

obj (rise | dsge): scalar model object.

inList (char | cellstr): List of atoms to translate e.g. C,
X{+1}, lambda_x, EPS_A,

Returns:

:

- **outList** [cellstr]: List of translated atoms

Example:

:

list=get(obj,'endo_list') outList=itranslate(obj,list) outList=itranslate(obj,list,0) % contemporaneous or steady state

list=get(obj,'exo_list') outList=itranslate(obj,list)

list=get(obj,'param_list') outList=itranslate(obj,list)

list=get(obj,'def_list') outList=itranslate(obj,list)

See also:

rise/translate :

Help for rise/itranslate is inherited from superclass DSGE

3.17.2.33 link_parameters

LINK_PARAMETERS Dynamically binds parameters together by creating links between parameters for future evaluation.

m = LINK_PARAMETERS(m, expressions)

This function is responsible for dynamically binding parameters together by creating links between parameters for future evaluation.

- *m*: Scalar or vector of model objects. Each model may have multiple parameterizations.
- *expressions*: Character or cell array of strings representing the expressions to bind. For example, expressions = 'alpha = beta + gamma'.

Returns: - *m*: Updated model object with dynamically bound parameters.

Example:
m = LINK_PARAMETERS(m, expressions)

See also:

rise.view_linked_parameters, rise.unlink_parameters

Help for rise/link_parameters is inherited from superclass DSGE

3.17.2.34 loadObj

loadObj loads a dsge object from a MAT file.

Help for rise.loadObj is inherited from superclass DSGE

3.17.2.35 load_parameters

load_parameters loads the parameters. This allows the user to quickly load the parameters from a file, which may be the output of estimation, and get going with irfs, simulations, etc.

```
model=load_parameters(model,the_mode_file)
```

Args:

- **model** (estimable object): model object
- **the_mode_file** (m-file): file containing the parameters and their values

Returns:

- **model** (estimable object): reparameterized model object

Help for rise/load_parameters is inherited from superclass ESTIMABLE

3.17.2.36 log_posterior_kernel

Computes the log posterior of the dsge model

```
[log_post,log_lik,log_prior,Incr,retcode,obj]=log_posterior_kernel(obj, param)
```

Args:

- obj (estimable object): model object
- param (column vector): parameter values

Returns:

- **log_post** (double): log posterior
- **log_lik** (double): log likelihood
- **log_prior** (double): log prior
- **Incr** (double):
- **retcode** : return code
- obj (estimable object): model passed through

See also:

- *log_prior_density*

Note: In effort to make RISE modular, this function is available so that one can use a different sampler if needed, but most likely, one should just use available stock samplers.

Help for rise/log_posterior_kernel is inherited from superclass ESTIMABLE

3.17.2.37 log_prior_density

Computes the probability density function of the prior corresponding to the parameter values

```
[lnprior, retcode] = log_prior_density(model)
[lnprior, retcode] = log_prior_density(model, param)
[lnprior, retcode] = log_prior_density(model, param,filtration)
```

Args:

- **model** (estimable object): model object
- **param** (column vector): parameter values
- **filtration** (empty|struct): results from model filtration that can potentially be used for endogenizing priors.

Returns:

- **lnprior** (double): log of prior density function
- **retcode**: return code

See also:

- *log_posterior_kernel*

Note: In effort to make RISE modular, this function is available so that one can use a different sampler if needed, but most likely, one should just use available stock samplers.

Help for rise/log_prior_density is inherited from superclass ESTIMABLE

3.17.2.38 max_discrepancy

max_discrepancy – Calculate the maximum absolute difference between the solutions of two DSGE models.

This function calculates the maximum absolute difference between the solutions of two DSGE (Dynamic Stochastic General Equilibrium) models. It can be used to assess the dissimilarity between the state space representations of two models.

INPUTS: - *m1*: The first DSGE model. - *m2*: The second DSGE model.

OUTPUT: - *o*: The maximum absolute discrepancy between the two models.

The function works by comparing various properties of the state space representation of the two models, such as state transition matrices, transition matrices for different orders, and covariances of measurement errors.

EXAMPLE: - Calculate the maximum discrepancy between two DSGE models *model1* and *model2*:

```
` max_discrepancy(model1, model2); `
```

Help for rise/max_discrepancy is inherited from superclass DSGE

3.17.2.39 mode_curvature

Checks the curvature at the posterior mode

```
db = mode_curvature(obj)

db = mode_curvature(obj,varlist)

db = mode_curvature(obj,varlist,N)

db = mode_curvature(obj,varlist,N,type)
```

Args:

- **obj** (rise | dsge | rfvar | svar): model object
- **varlist** (char | cellstr | empty): list of parameters for which we want to check the curvature
- **N** ({20} | integer): Number of grid points
- **type** ({'max'} | 'min' | 'range'): normalization of the log-posterior and the log-likelihood.
- **dbin** (struct|empty): structure containing the information to plot the curvature. Each field is the name of a particular parameter. This is to avoid a costly recomputation of db

Returns:

- **db** [struct|cell array|vector]: structure containing the information to plot the curvature. Each field is the name of a particular parameter. Alternatively, when dbin is not empty, db is a handle to the plots.

Note:

- when no output is requested, plots are made but not saved.
- one way to plot the curvatures from the output is to use the function utils.plot.curvature

See also:

- utils.plot.curvature

Help for rise/mode_curvature is inherited from superclass ESTIMABLE

3.17.2.40 model_information

rise/model_information is a function.
out = model_information(obj, info)

3.17.2.41 observables_decomposition

Decomposes all variables of a DSGE model in terms of observables.

Syntax

```
weights=observables_decomposition(obj,select,xrange,db)

[weights,dec1,...,decn]=observables_decomposition(obj,select,xrange,db1,...,
↪dbn)
```

Args:

- obj (rise | dsge): scalar model object
- select ({[]} | 'a' | 'att' | 'alpha' | 'v' | 'r' | 'epsilon' | 'eta'): type of decomposition to perform
  - **a** : filter
  - **att** : update
  - **alpha** : smooth
  - **v** : forecast errors
  - **r** : variables calculated during the smoothing process
  - **epsilon** : measurement errors
  - **eta** : structural shocks
  - **[]** : all the above
- xrange ({[]} | vector | cell array): start date and end date for the decomposition
- db (ts): database with the data to be used in the decomposition

Returns:

- **weights** [struct]: weights for the different elements requested from the variable **select**
- **dec1** [struct]: hyper structure containing the time series for the various decomposition types and variables

Note:

- if **select** is empty, all the decompositions are performed

- After doing the decomposition of different variables, one can take the differences in the decomposition to perform, e.g., the decomposition of forecast errors in terms of observables.

See also:

- historical_decomposition

Help for rise/observables_decomposition is inherited from superclass DSGE

3.17.2.42 optimal_simple_rule

optimal_simple_rule : Estimates optimal simple rules coefficients

```
[m]=optimal_simple_rule(m,lossstr)
[m]=optimal_simple_rule(m,lossstr,shocks)
[m]=optimal_simple_rule(m,lossstr,[],varargin)
[m]=optimal_simple_rule(m,lossstr,shocks,varargin)
```

Args:

m (rise | dsge): scalar or vector of model objects.

lossstr (string|cell array): loss function two possibilities

- string : lossString. In this case discount = 0.99
- cell array : {discount,lossString}

priors (struct): parameters to estimate

shocks (ts | struct | double| function handle| empty): if empty, the theoretical (unconditional) welfare will be calculated. Otherwise the conditional welfare will be computed.

Returns:

:

- **m** [scalar|vector]: scalar or vector of model objects.

Note:

Help for rise/optimal_simple_rule is inherited from superclass DSGE

3.17.2.43 parameters_to_file

PARAMETERS_TO_FILE Writes the parameters of a model object to a file.

PARAMETERS_TO_FILE(model, fname) writes the parameters of the model object (typically a DSGE model) to a file with the specified filename.

Inputs: - *model*: Model object containing the parameters. - *fname*: File name to write the parameters to. The extension should be ".m".

Example:

parameters_to_file(model, 'parameters_file.m')

See also:

get

Help for rise/parameters_to_file is inherited from superclass DSGE

3.17.2.44 perfect_foresight

perfect_foresight : Perfect foresight and extended path simulations

```
[db,states,retcode] = perfect_foresight(obj,varargin)
```

Args:

- **obj** [dsge|rise]: model object
- **varargin** : additional arguments including but not restricted to
  - **simul_exogenous_func** [function handle|{empty}]: function that generates exogenous values for exogenous variables
  - **simul_stack_solve_algo** [{'sparse'}|'fsolve'|'lsqnonlin'| user defined]: alorithm for solving the problem. If the algorithm is user defined, it should accept the same inputs as lsqnonlin and return the same outputs as fsolve. [y1,fval,exitflag,output]=user_algo(objfun,y0,lb,ub,options); output is a structure whose only relevant field is "iterations"
  - **simul_recursive** [true|{false}]: if true, extended path is triggered. It is assumed that agents no longer anticipate the whole sequence of shocks. Rather, in each period they expect zero shocks in the future but are suprised that the realizations of the shocks are nonzero. If false, perfect foresight is computed.
  - **simul_enforce_hybrid** [true|{false}]: if true, all subproblems are solved assuming that there are both backward and forward-looking terms. Use this option to avoid a long sequence of for-loops and simple iterations in problems that can otherwise be solved one period at a time. This happens for instance when the simulation horizon is very long.
  - **simul_pf_bounds** [struct|{empty}]: sets the search space/bounds for endogenous variables during perfect foresight and extended path simulations. Those constraints can be implemented in similar fashion to the way the constraints for the steady state are implemented but using a distinct option rather than 'sstate_bounds'. e.g. Assume we want to impose S>0, R<1 and I<1. This is implemented as bounds=struct(); bounds.S=[nan,0,inf]; bounds.R=[nan,-inf,1]; bounds.I=[nan,-inf,1]; sims=perfect_foresight(...,'simul_pf_bounds',bounds)

Returns:

- **sims** [struct]: simulated series
- **fval** [scalar|vector]: maximum error
- **retcode** [integer]: if 0, the simulation went fine. Else something got wrong. In that case one can understand the problem by running decipher(retcode)

Note:

- the algorithm allows for regime switching : the economy then jumps from one regime to the other and the economic agents expect and are aware of those jumps in advance. In this case the user should specify the whole history of regimes
- the algorithm allows for the use of symbolic, automatic or numerical derivatives for the computation of the jacobian.

The first element in each vector is supposed to represent the start value but this is not used in this context. In a future update, those start values will override whatever start values (or more precisely the initial conditions) specified in the historical database

- the algorithm uses block decomposition to efficiently solve the simulation problem recursively. Besides the obvious advantage that solving many small problems is more efficient than solving one large one, it turns out that the decomposition will also isolate blocks that

are either purely backward-looking, purely forward-looking or purely static. In those three cases further computational gains can be obtained because then the problem can be solved period by period.

- Perfect foresight is kind of nice in the sense that it illustrates the fact that although the true solution of the model is not analytic, it is potentially of the form x{t}=T(x{t-1},e{t},e{t+1},...,e{n}), where n is the last observation. It is interesting to note that going forward, we have x{t+1}=T(x{t},e{t+1},...,e{n}) ... so that in the end x{n}=T(x{n-1},e{n})
- Another insight from perfect foresight relates to the equivalence between a stochastic model and a deterministic model when both of them are linear : we get certainty equivalence, which can be verified in the fact that the perfect foresight and the stochastic solutions yield the same impulse response functions. Of course this happens only when there is a unique solution. Under multiple stable solutions, there is still certainty equivalence but the solution found using the perfect foresight algorithm might depend on the vagaries of the optimization routine.

Example:

See also:

rise.simulate

Help for rise/perfect_foresight is inherited from superclass DSGE

3.17.2.45 plan2database

plan2database creates a database suitable for conditional forecasting

```
db=plan2database(obj,plan,start_date,end_date)
```

Args:

obj (rise | dsge): model file

plan
[n x 3 cell array where n is the number of conditions]

- the first column includes the names of the variables in strings or in cell array
- the second column includes the dates for conditioning
- the third column includes the values scalar or vectors

***start_date*: start date of the database as recognizable by the ts**
class. this is typically the end of history

***end_date*: end date of the database as recognizable by the ts**
class.

Returns:
:

- **db** [ts]: time series object with the various variables in and conditioning information

N.B:
:

There cannot be multiple variables and multiple dates simultaneously in one row

See also ts.fold, rise.growth_database, rise.initial_conditions

Help for rise/plan2database is inherited from superclass DSGE

3.17.2.46 posterior_sample

Computes a sample of any quantity of interest using parameter draws from a population e.g. a posterior simulation

```
[result]=posterior_sample(m,pop,dowhat)

[result]=posterior_sample(m,pop,dowhat,howmany)

[result]=posterior_sample(m,pop,dowhat,howmany,ouf)

[result]=posterior_sample(m,pop,dowhat,howmany,ouf,varargin)

[result,is_failed,time_it_took]=posterior_sample(...)
```

Args:

- **m** [rise|dsge|svar|rfvar|valid rise object]: model object
- **pop** [m x n struct]: parameter draws, with "m" the number of chains and "n" the number of draws in each chain. Each element of "pop" is a structure with fields "f" (not used), the value of the posterior and "x" the parameter vector
- **dowhat** [fhandle]: function (handle) to apply to each parameterized model object. e.g. dowhat=@irf, dowhat=@simulate, dowhat=@forecast, etc. "dowhat" need not be a method of "m": it represents the quantity of interest.
- **howmany** [integer|{m x n}]: number of draws to use in the calculation
- **ouf** [fhandle|{[]}]: output update function. Function that updates the output before storing it. e.g. if dowhat=@filter, one may be interested in the filters only and in that case ouf=@(x)x.filtering.
- **varargin** [pairwise args]: valid pairwise arguments for the model object

Returns:

- **result** [1 x howmany cell]: container of the various applications of the "dowhat" handle
- **time_it_took** [numeric]: number of seconds needed to run all the simulations.

See also:

draw_parameter

Note:

- the function will exploit parallel computation if there are workers idle.
- Because the solving of the model is sometimes iterative, a change of solver or of the settings of the solver can result in the model not being solved or more generally simulations failures. The algorithm will loop until the requested number of simulations is obtained. But it will not point to the parameter vectors that fail.

Help for rise/posterior_sample is inherited from superclass ESTIMABLE

3.17.2.47 predictive_analysis

PREDICTIVE_ANALYSIS: Perform predictive analysis for a dsge model

[draws, ok, retcode, timing, varargout] = predictive_analysis(m, priors, userfun)

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,myDrawsMatrix)

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,'halton')

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,{'halton',N})

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,'sobol')

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,{'sobol',N})

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,'latin_hypercube')

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,{'latin_hypercube',N})

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,'prior')

[draws, ok, retcode, timing, varargout] = predictive_analysis(…,{'prior',N})

Inputs:

- *m* (rise/dsge model object): RISE/DSGE model object.
- *priors* (structure): Structure with parameter names and their distributions.
- *userfun* (function_handle): Function that checks whether a particular parameterization yields the desired outcome. The function should take as input the parameterized model (m) and return at least two outputs in which the first is a boolean (true or false) and the second is the retcode. The function can return additional outputs that will be captured in varargout (see below).
- *DrawsInfo*: could be
  - Matrix : this represents the draws computed/obtained elsewhere
  - char : procedure to use for the draws ({'prior'}|'latin_hypercube'|'sobol'|'halton'):
  - cell array : {procedure,N} where N is the number of draws with default value 2^12.

Outputs:

- *draws* (numeric): Draws from the sampling.
- *ok* (logical): 1 x n vector of booleans, with true where a particular parameter vector checks the desired behavior.
- *retcode* (numeric): 1 x n vector of return codes, where 0 means there are no problems.
- *timing* (structure): Structure with the timing of various sampling.
- *varargout* (cell): User desired additional outputs beyond the two first outputs of *userfun*

Notes:

- The function performs prior predictive analysis by drawing parameter values from specified distributions and checking their effects using a user-defined function.
- The function can exploit parallelization if workers are fired up.

Example:

```
` [draws, ok, retcode, timing, o3,o4,...,on] = predictive_analysis(m,
priors, userfun); `
```

See also: *QUASI_MONTE_CARLO*

Help for rise/predictive_analysis is inherited from superclass DSGE

3.17.2.48 print_estimation_results

PRINT_ESTIMATION_RESULTS: Display the results of estimation.

retcode = print_estimation_results(obj) retcode = print_estimation_results(obj, detail)

Args:

- *obj* (estimable): Model object.
- *detail* (true|{false}): If true, the description of parameters is given alongside the code name. If they are the same, i.e., the description has not been provided, then the code name is given only once.

Returns:

- *retcode* (numeric): 0 if there is no problem.

Notes:

- If there are multiple objects in the array, each object's results will be displayed.
- The function displays information such as log-posterior, log-likelihood, log-prior, etc.
- The displayed results include information about the estimation sample, solution algorithm, estimation algorithm, number of estimated parameters, number of function evaluations, and the time taken for estimation.
- If there are any issues, a list of issues is displayed.

Example:
print_estimation_results(obj) print_estimation_results(obj, true)

See also: *TABLE_DISPLAYER*

Help for rise/print_estimation_results is inherited from superclass ESTIMABLE

3.17.2.49 print_solution

Print the solution of a model or vector of models

```
print_solution(obj)
print_solution(obj,varlist)
print_solution(obj,varlist,orders)
print_solution(obj,varlist,orders,compact_form)
print_solution(obj,varlist,orders,compact_form,chop)
print_solution(obj,varlist,orders,compact_form,chop,capture)
```

Args:

- **obj** (rise | dsge): model object or vector of model objects
- **varlist** (char | cellstr | {[]}): list of variables of interest
- **orders** (numeric) : orders for which we want to see the solution (default: [1:solve_order])
- **compact_form** ({true} | false): if true, only the solution of unique tuples (i,j,k) such that i<=j<=k is presented. If false, the solution of all combinations is presented. i.e. (i,j,k)(i,k,j)(j,i,k)(j,k,i)(k,i,j)(k,j,i)

- **chop** ({1e-9} | numeric >0 && <1e-4) : any number, which in absolute value is less than chop is set to 0.
- **capture** ({''} | char) : generic name for files to be written to disk in capturing the solution for each model.

Returns:

- **outcell** [cellstr] : dummy output not important, kept for legacy compatibility. Might be removed in future versions

---

Note:

- In the state vector (first column), the variables appearing should be understood as deviations from the steady state.
- @sig denotes the impact of the perturbation parameter or more technically, the impact of future uncertainty.
- for nonstationary models:
  - the real part of @sig denotes the impact of future uncertainty.
  - the imaginary part of @sig denotes the growth rate
- In constant-parameter models, the impact of future uncertainty is zero for odd orders of perturbation i.e. 1, 3, 5, 7, etc. For a first-order approximation, this result is known as certainty equivalence. In regime switching, sometimes the first-order approximation is not certainty equivalent
- When the solution is printed in the presence of log variables, the policy functions appear as follows :
  - Each log variable X appears as log(X) in the row giving the solution for each variable (first row)
  - In the state vector (first column) the log variables appear with an asterisk (*). This is to ease the writing of higher-order perturbation solutions as they involve cross products of the state variables. in other words X*{-1} = log(X{t-1})

Help for rise/print_solution is inherited from superclass DSGE

3.17.2.50 pull_objective

INTERNAL FUNCTION: pulls the objective function to optimize for a DSGE or RISE model

Note:

- pull_objective is the same as generic/pull_objective except for the kf_filtering_level

See also:

- rise_generic/pull_objective

Help for rise/pull_objective is inherited from superclass DSGE

3.17.2.51 randsample

Generate random time series from a model returns K samples of length N

```
- db=randsample(obj,N,K)
- db=randsample(obj,N,K,varargin)
- [db,states,retcode]=randsample(...)
```

Args:

- **obj** (rise | dsge | svar | rfvar): model object
- **N** (integer): length of the time series
- **K** (integer): number of time series
- **varargin** : additional options for the model object

Returns:

- **db** [struct]: time series
- **states** [matrix]: NxK matrix of visited states
- **retcode** [numeric]: 0 if no problem

3.17.2.52 refresh

INTERNAL FUNCTION: - refresh the options of an old object with a newer version of the software

Note:

- REFRESH is the same as RISE_GENERIC.REFRESH except that it also refreshes parts of the system that are specific to DSGE or RISE objects.

See also:

RISE_GENERIC.REFRESH

Help for rise/refresh is inherited from superclass DSGE

3.17.2.53 report

Assigns the elements of interest to a rise_report.report object

- display the default inputs:

```
report(rise.empty(0))
```

- assign the reported elements in rep_items to destination_root:

```
report(obj,destination_root,rep_items)
```

- assign varargin to obj before doing the rest:

```
report(obj,destination_root,rep_items,varargin)
```

Args:

- obj (rise | dsge):

- destination_root (rise_report.report): handle for the actual report
- rep_items (char | cellstr): list of desired items to report. This list can only include
  - endogenous
  - exogenous
  - observables
  - parameters
  - solution
  - estimation
  - estimation_statistics
  - equations
  - code

Returns:
:

- none (Update the report in place)

3.17.2.54 resid

Compute the residuals from the steady state

```
r=resid(obj)
r=resid(obj,trim)
```

Args:

- obj (rise | dsge): model object
- trim (true|{false}|1x1 cell|1x2 cell): * if false, print all residuals as is and Do not print equations * if true, set to zero residuals that are less than a tolerance level whose default is 1e-13. Do not print equations * If it is a cell array, then it must be either {tol,false} or {tol,true}. Either element can be empty, the default is {1e-13,false}
  - the first element is the tolerance level below which all residuals are set to 0
  - the second element is either "true" or "false" and decides whether to additionally print the equation

Returns:
:

- **r** [vector|matrix]: residuals

Note:

- if no output is requested, the residuals are printed on screen
- if the model has not been solved, resid will call sstate to try to solve for the steady state, with option imposed set to true in order to avoid reoptimization.
- The trim option is for display only. If the function is called with an output argument, trim is not used.

Help for rise/resid is inherited from superclass DSGE

3.17.2.55 resimulate

Uses the intial conditions given in x0h and shocks information in structure fx to resimulate the data using model m. The variables in m not appearing in fx are initialized at zero if x0h is empty

Syntax:

```
mysimul=resimulate(m,fx)

mysimul=resimulate(m,fx,x0h)

[mysimul,retcode]=resimulate(...)
```

Inputs:

- m : [rise|dsge] model(s) to simulate
- fx : (possibly modified output of) filtration.
- x0h : (n_x x 1 x h): initial conditions in all the regimes. N.B: this has to be in log-form. If the model has variables declared as log_variables, those variables should be entered here in log. RISE will not take the log of x0h.
- fxlogvars : vector of booleans specifying which endogenous variables in fx are log variables

Outputs:

- mysimul : [struct|cell array] structure or cell array of structures with the simulations
- retcode : [double] return code. 0 if there is no issue

Note: all the simulations are returned in their log-form i.e. they are not re-exponentiated !!!

See also:

rise/historical_decomposition

Help for rise/resimulate is inherited from superclass DSGE

3.17.2.56 rise

rise – constructor for dsge models

```
obj=rise(model_filename)

obj=rise(model_filename,varargin)
```

Args:

- **model_filename** [char | cellstr]:

  if "char",name of the model file. The file should have extensions rs, rz or dsge.

  If "cellstr"

  - each cell contains the name of a separate model file. The files are then meant to be combined into one single model.
  - the entire model is given as a cell with an example as follows

```
simpleModel={
    'model:someModel'
    '%% model without parameters and shocks'
    '@endogenous x'
    '@model'
    'x=1.05\*x{-1};'
    };

m=rise(simpleModel);
```

- **varargin** []: pairwise arguments with possiblities as follows:
  - **parameter_differentiation** [true|{false}]: compute or not parameter derivatives
  - **definitions_inserted** [true|{false}]: substitute definitions given in the model block. Necessary if the definitions contain variables.
  - **definitions_in_param_differentiation** [true|{false}]: insert or not definitions in equations before differentiating with respect to parameters
  - **saveas** [true|false|char|{''}]: save the possibly expanded model file. If "true", the name of the main original file is used appended with "_expanded.dsge". Alternatively, the user can provide a name under which he wants the file to be saved.
  - **max_deriv_order** [integer|{1}]: order for symbolic differentiation. It is recommended to set to 1, especially for large models in case one does not intend to solve higher-order approximations. It is also possible to set it 0 in case one intends to use numerical or automatic derivatives or just do perfect foresight-type of simulations.
  - **parse_debug** [true|{false}]: debugging in the parser
  - **add_welfare** [true|{false}]: add the welfare equation when doing optimal policy. N.B: The welfare variable, WELF is the true welfare multiplied by (1-discount). The within-period utility variable, UTIL is automatically added. The reason why welfare is not automatically added is that oftentimes models with that equation do not solve.
  - **rise_flags** [struct|cell]: instructions for the partial parsing of the rise file. In case of a cell, the cell should be a k x 2 cell, where the first column collects the conditional parsing names and the second column the values.

Returns:

- **obj** [rise|dsge]: model object

Note:

- The pairwise options listed above are the ones that will be processed in the parser. Additional options related to specific methods can also be passed at this stage, but will only be applied or used when the specific method dealing with them is called.
- In RISE it is possible to declare exogenous and make them observable at the same time. The exogenous that are observed are determisitic. This is the way to introduce e.g. time trends. This strategy also opens the door for estimating partial equilibrium models.

Documentation for rise/rise
doc rise

3.17.2.57 saveObj

saveObj saves the dsge object to a MAT file.

Help for rise/saveObj is inherited from superclass DSGE

3.17.2.58 set

Sets options for dsge|rise models

Syntax

```
obj=set(obj,varargin)
```

Args:

obj(rise | dsge): model object

varargin

[valid input arguments coming in pairs. Notable fields to] that can be set include and are not restricted to:

- **parameters** [struct|cell] : e.g. set(obj,'parameters',p), where p is a struct or a cell
- **parameters(ignore)** [struct|cell] same as above but issues a warning instead of an error if a parameter is not found
- **parameters(vector)** [struct|cell] : syntax for multiple parameterization. e.g.
  - set(obj,'parameters(2)',p):
  - set(obj,'parameters(2:3)',p): As usual, p is either a strucutre or a cell array. In case of a cell array, the first columns gathers the names of the parameters to update and the second column the values to add. **dsge_tools.cell2struct** is responsible for the dispatch.
  - set(obj,'parameters([2:3,5,7])',p): The holes i.e. 4 and 6 are filled with the last parameterization in the model object, which is 1 in the present example
- **parameters(A,B)**: Combination of the two above i.e. ignore and vector. e.g.
  - set(obj,'parameters(ignore,2:3)',p)
  - set(obj,'parameters(2:5,ignore)',p)
- **solve_shock_horizon** [integer|struct|cell]
  - for the integer case, all shocks are set to the same integer
  - for the struct case, the input must be a structure with shock names as fields. Only the shock names whose value is to change have to be listed. In this case, different shocks can have different horizons k. The default is k=0 i.e. agents don't see into the future. The value of the fields is one of the following:
    * k
    * {k,chainName_state}
    * {chainName_state,k}
  - for the cell case, the cell should have two columns. The first column includes the names of the shocks whose horizon is to change. The second column is one of the following:

    * the horizon for the shock
    * {horizon,chainName_state}
    * {chainName_state,horizon}
  - **solve_function_mode** [{explicit/amateur}|vectorized/professional|disk]
    - in the **amateur** or **explicit** mode the functions are kept in cell arrays of anonymous functions and evaluated using for loops
    - in the **vectorized** or **professional** mode the functions are compacted into one long and unreadable function.
    - in the **disk** mode the functions are written to disk in a subdirectory called routines.
  - **sstate_exogenous** [struct|cell|scalar]: values to push. There are three possibilities:
    - struct: the fields are the exogenous names and each name contains a exogenous value.
    - cell: the cell has two columns. The first column holds the names of the exogenous to change and the second column their values.
    - scalar: the scalar value is pushed as the steady state for all exogenous variables. This is useful for instance when resetting all the steady state values for the shocks.

Returns:

:

- **obj** [rise|dsge]: model object

Example:

obj=set(obj,'solve_shock_horizon',struct('shock1',2,'shock3',4))

obj=set(obj,'solve_shock_horizon',5)

See also generic/set

Help for rise/set is inherited from superclass DSGE

3.17.2.59 set_z_eplus_horizon

INTERNAL FUNCTION

Help for rise.set_z_eplus_horizon is inherited from superclass DSGE

3.17.2.60 simulate

simulate - simulates a RISE model

```
[db,states,retcode] = simulate(obj,varargin)
```

Args:

- **obj** [dsge|rise]: model object
- **varargin** : additional arguments including but not restricted to
  - **simul_sig** [numeric|{1}]: value of the perturbation parameter
  - **simul_tol** [numeric|{sqrt(eps)}]: tolerance used for complementarity constraints. e.g. R>=1 becomes tol+R>=1 and A<=b becomes A<=b+tol

- **simul_pruned** [true|'one_step_pruning'| 'one_step_pruning_automatic'|function handle|{false}]: if true, pruning is applied using engine "one_step_pruning_automatic" as default.
- **simul_initial_regime** [integer|{1}]: Historical regime for the simulations.
- **simul_order** [integer|{[]}]: Suppose the model has been solved with an n-order perturbation. Then one may still want to simulate the model for an order m<n, without having to resolve the model. The default, of course, is to apply the same order as the solution.
- **simul_with_growth** [{true}|false]: if true, the growth rate is added to simulations for nonstationary models. x_t-xss = g + Tz*zhat
- **simulator [empty|{'regime_switch'}|'occbin'|'anticip_shocks'|...** 'cforecast'|cell array|char]: Simulation procedure
- **simul_constraints** [{empty}|cell array]: cell array of simulation constraints used to solve occasionally binding constraints. The constraints are of the form restr = { 'f1(v,c)<g1(v,c)','x1=h1(c)',shkList1 ... 'fn(v,c)<=gn(v,c)','xn=hn(c)',shkListn }

where the third column is optional. "shkListi" : list of shocks for fixing restriction i. Shocks need to be unique in each least but can appear in other lists as well. The second column, 'xi=hi(c)', is needed only in the case of anticipated shocks but is compulsory anyway even in regime switching (including piecewise linear approximation)

Returns:

- **db** [struct|cell array]: if **simul_to_time_series** is true, the output is a time series, else a cell array with a matrix and information on elements that help reconstruct the time series.
- **states** [vector]: history of the regimes over the forecast horizon
- **retcode** [integer]: if 0, the simulation went fine. Else something got wrong. In that case one can understand the problem by running decipher(retcode)

Note:

- **simul_historical_data** contains the historical data as well as conditional information over the forecast horizon. It may also include as an alternative to **simul_regime**, a time series with name **regime**, which indicates the regimes over the forecast horizon.
- For a **model with measurement errors**, the simulations are stored as complex numbers. The real part is the data without measurement errors and the imaginary part is the measurement error component. The two can be added for some variable x as **x = real(x)+imag(x)**.

Example:

See also:

generic.simulate

Help for rise/simulate is inherited from superclass DSGE

3.17.2.61 simulate_nonlinear

For help see : dsge.perfect_foresight

Help for rise/simulate_nonlinear is inherited from superclass DSGE

3.17.2.62 solve

Solves dsge models

```
[obj,retcode,structural_matrices]=solve(obj)

[obj,retcode,structural_matrices]=solve(obj,varargin)
```

Args:

- obj (rise | dsge): scalar or vector of model objects. The optional options below come in pairs.
- solve_initial_guess ({empty} | cell array): Initial guess for the first-order dynamics. When not empty, it should be the equivalent of obj.state_space{1}.Tz
- solve_accelerate ({false} | true): Accelerate or do not accelerate the solving
- solve_check_stability ({true} | false): check stability of Markov switching models while solving. The stability of constant-parameter models is always checked whether that of markov-switching models is optional. This is optional because (1) the procedure is computationally intensive and (2) there is no define stability criterion under endogenous switching
- solve_derivatives_type (numeric | automatic | matlab| {symbolic}): choice of derivatives
- solve_order (integer|{1}): order of approximation
- solve_shock_horizon (integer|{0}|struct|cell): anticipation horizon of shocks beyond the current period. When the input is :
  - **integer** : all the shocks receive the same anticipation horizon
  - **struct** : the fields are the names of the shocks whose horizon is to be modified. e.g. struct('ea',4,'eb',3) means shock ea has horizon 4 while shock eb has horizon 3
  - **cell** : the cell must have two colums. Each row in the first column holds the name of a particular shock and each row in the second column holds the horizon of the shock. e.g. {'ea',4;'eb',3}
  - NOTE : Under regime switching, it is possible to set different horizons for the same shock in different regimes. in that case, the values are no longer scalars. They are 2-columns cell arrays in which one of the columns is the state and the other column the value. e.g. shks.EPS_R={'dan_1',0;'dan_4',0} column the value. e.g. shks={'EPS_R',{'dan_1',0;'dan_2',4}}
- solve_alternatives_nsim (integer | {100}): Number of initial guesses for the solution sampled randomly, when attempting to find all possible solutions.
- solve_function_mode ({explicit/amateur} | vectorized/professional | disk)
  - in the **amateur** or **explicit** mode the functions are kept in cell arrays of anonymous functions and evaluated using for loops
  - in the **vectorized** or **professional** mode the functions are compacted into one long and unreadable function.
  - in the **disk** mode the functions are written to disk in a subdirectory called routines.

- solve_initialization ({'backward'} | 'zeros' | 'random'): Type of initialization of the solution of switching models:
  - **backward** : the initial guess is the solution of the model without forward-looking terms
  - **zeros** : the initial guess is zero
  - **random** : the initial guess is random
- solve_linsyst_user_algo ({''} | cell | char): user-defined solver for linear systems. It should be possible to call the function as:

```
[X,FLAG,RELRES,ITER,RESVEC] = bicgstabl(A,B,TOL,MAXIT,varargin)
```

  Hence, the function should accept at least 4 inputs where:
  - **A** : matrix or function handle
  - **B** : right-hand side of system to solve
  - **TOL** : tolerance level
  - **MAXIT** : maximum number of iterations

  The function must return 5 outputs as in Matlab's tfqmr, bicgstab, bicgstabl, gmres, etc. :
  - **X** : the solution
  - **FLAG** : convergence flag
  - **RELRES** : relative residual NORM(B-A*X)/NORM(B)
  - **ITER** : iteration number
  - **RESVEC** : residual vector for various iterations

  As far as the outputs are concerned, only the two first are relevant for RISE and so the three last can be empty. But they must be returned by the function all the same.
- solver ({[]} | char | user_defined): solver for the dsge model. The following are possible:
  - **loose_commitment** : RISE automatically uses this when it detects an optimal policy model to be solved under commitment, discretion or loose commitment
  - **rise_1** : default solver for constant-parameter dsge models. Similar to the dynare solver.
  - **cr/cyclic_reduction** : solves the constant-parameter dsge model using cyclic reduction
  - **klein** : solves the constant-parameter dsge model using the Klein method
  - **AIM** : solves the constant-parameter dsge model using the AIM solver
  - **mfi** : functional interation solver: default for switching dsge models
  - **mnk** : newton solver with kronecker products
  - **mn** : newton solver without kronecker products
  - **mfi_full** : full version of mfi, that does not exploit sparsity
  - **mnk_full** : full version of mnk, that does not exploit sparsity
  - **mn_full** : full version of mn, that does not exploit sparsity
  - **fwz** : Farmer, Waggoner and Zha (2011)'s Newton solver
  - **dsge_groebner** : Attempts to find all solutions using Groebner basis, a technique from computational algebraic geometry

- **dsge_udc** : Solves the regime-switching dsge model using a version of the undetermined coefficient method
- **dsge_schur** : Solves the regime-switching dsge model using a schur decomposition (works only for the Maih-Waggoner perturbation)
- **user_defined** : In this case the function must take as inputs:
  - **Gplus01** [n x n x h x h array]: where the 3rd dimension represent the current regime and the 4th dimension is the future regime.
  - **A0** [n x n x h array]: matrix of contemporaneous variables
  - **Aminus** [n x n x h array]: matrix of backward-looking terms
  - **Q** [square matrix]: transition matrix, in which the rows are the current period and the columns the next period
  - **T0** [n x h x h array]: initial guess of the solution
  - **tol** [scalar]: A tolerance criterion
  - **maxiter** [scalar]: the maximum number of iterations

  The function should return as outputs
  - **Tz_pb** [n x h x h array]: solution of the problem given the inputs
  - **eigenvalues** [empty|vector]: optional vector of eigenvalues of the problem
  - **retcode** : 0 if no problem found when solving the problem

  The function can be passed along as a string, a function handle or a cell array in which case the first element of the cell contains the function itself and the remaining cells contain further arguments required by the function but unknown to RISE.

  N.B:: Oftentimes, RISE checks the type of the model in order to determine what the most appropriate solver should be. This may not always be the behavior that the user wants. It is possible to override that behavior by forcing RISE to use your intended solver. The way to do this is simply to add a "+" sign in front of the name of the solver. e.g. m=set(m,'solver','+mySolver')

- solve_automatic_differentiator (function_handle | @aplanar_.diff | @aplanar.diff | {@adolm.diff}): automatic differentiator engine
- solve_occbin (integer | {empty}): Solves the occasionally binding constraints model ala [Cagliarini and Kulish, 2013], [Guerrieri and Iacoviello, 2015] or [Guerrieri and Iacoviello, 2017]. It is then assumed that the transition matrix is diagonal. if not empty, the value entered is the reference regime.
- solve_bgp (true | {false}): Solves the model as non-stationary.
- solve_linear (true | {false}): set this to true in order to use more efficient algorithms for solving the steady state when the original model is truly linear.
- solve_derivatives_only (true | {false}): if true, the derivatives are computed but the model is not solved. The derivatives can be collected in the third output argument of the function.
- solve_user_defined_shocks (struct|{empty}): When empty, an independent normal distribution is assumed for each shock. Hence the shocks are not skewed! When not empty, the argument is a structure whose fields are the names of the shocks for which the user wants to change the distribution. Each field/shock is 1 x 2 cell array such that:
  - The first entry is a function handle that admits two inputs: the model object (in case it is needed) and the maximum order for the moments of the shocks. The function should then return in a row vector the moments from 1 to order. e.g. f(obj,5)=[0,1,0,3,0,15] for a standard normal distribution.

  - the second entry is also a function handle with one input argument. This function should return in a row vector random variates for the underlying distribution. e.g. we could have f(3)=[-0.6685 1.0613 -0.5339] for a normal distribution.

- solve_perturbation_type (char | cell array | {'maih'}): pertubation type which should be one of the following
  - 'maih' or 'm': pertubation strategy of Maih(2014): regime-specific steady states
  - 'maih_waggoner' or 'mw': pertubation strategy of Maih and Waggoner(2018): regime-specific steady states + perturbed transition probabilities for ALL Markov chains.
  - {'maih_waggoner',mkv_list} or {'mw',mkv_list}: where mkv_list is a cellstr with the list of the partitioned markov processes.
  - 'frwz': pertubation strategy of Foerster, Rubio-Ramirez, Waggoner and Zha(2016): unique steady state solved at the ergodic mean of the parameters, imposing a perturbation on ALL the switching parameters.
  - {'frwz',part_list}: where part_list is a cellstr with the list of the partitioned parameters (perturbed parameters).
  - mkv_list and part_list : could also be {'*none'}, {'*all'}, {'*all_but'}, {'*all_but','v1','v2',...'vn'}
- solve_kill_g (false | {true}): kill off the growth element in first-order solution
- solve_policy_type ('discretion' | {'ramsey'}): choice of the type of policy in the absence of loose commitment or more generally stochastic replanning. Under stochastic replanning the option is set automatically as 'stochastic_replanning', which cannot be set directly by the user. The option solve_policy_type is case insensitive.
- solve_qz_criterium (numeric | {sqrt(eps)}): cutoff criterium for eigenvalues in the solving of constant-parameter DSGE models.

Returns:

- **obj** [rise | dsge]: scalar or vector of model objects
- **retcode** [integer]: if 0, no problem was found when solving the model. If positive, the reason for the problem can be obtained by running decipher(retcode)
- **structural_matrices** [struct]: Structure holding various types of derivatives that are used in the computation of the solution

Note:

- constant-parameter models can also be solved using the procedures for switching parameter models. There is no guarantee, however, any obtained solution will be determinate (unique and stable)
- determinacy is only checked for constant-parameter models. For determinacy computations for switching dsge models with constant transition matrices, see [Cho, 2014]
- In RISE we consider that it is in the nature of nonlinear models to have many solutions. Obtaining even one of those solution is usually difficult and a concept such as determinacy is NEVER used in the solving of models with, say, global methods.

See also rise.sstate

Help for rise/solve is inherited from superclass DSGE

3.17.2.63 solve_alternatives

Attempts to find all solutions of a MSDSGE model

```
[allobj,mean_square_stable] = solve_alternatives(obj)
[allobj,mean_square_stable] = solve_alternatives(obj,varargin)
```

Args:

- obj (rise | dsge): RISE model object
- solve_alternatives_nsim (numeric | {100}): Number of random starting values generated.

Returns:

:

- **allobj** [rise|dsge]: vector of RISE objects with the solutions found

Help for rise/solve_alternatives is inherited from superclass DSGE

3.17.2.64 srf

SRF : shock response function, a flexible way to implement irfs.

Syntax :

myirfs=srf(m)

myirfs=srf(m,x0)

myirfs=srf(m,x0,r0)

myirfs=srf(m,x0,r0,shkCombo)

myirfs=srf(m,x0,r0,shkCombo,irf_periods)

myirfs=srf(m,x0,r0,shkCombo,irf_periods,order)

myirfs=srf(m,x0,r0,shkCombo,irf_periods,order,girf_runs)

Inputs:

- **m** [rise|dsge]: model object(s)
- **x0** [empty|struct]: initial conditions for endogenous variables
- **r0** [empty|integer]: initial/historical regime
- **shkCombo** [numeric|struct|empty]: shock information such that
  - if numeric : shock sign and or magnitude
  - if struct : structure with the names of the shocks and their values for the initial impulse : the impulse response is computed for a **cocktail of shocks (scenario)** rather than for one specific shock. Note that in this case, the outputwill be the same for all individual shocks!!!
- **irf_periods** [empty|numeric]: period length for the irfs
- **order** [empty|integer]: order of the solution used in the computation of the irfs
- **girf_runs** [empty|numeric]: if girf_runs>1, this triggers the computation of generalized impulse responses. It automatically sets **irf_regime_specific** and **irf_girf_regime_uncertainty** to false

Outputs:

- **myirfs** [struct]: structure in which the fields are the names of the shocks of the model. This is true whether we have a cocktail of shocks or not. The only difference is that for convenience, under a cocktail of shocks, the responses are the same for all the shocks in the model.

3.17.2.65 sstate

Computes the steady state of a dsge model

```
[obj,structural_matrices,retcode]=sstate(obj,varargin)
```

Args:

- obj (rise | dsge): model file
- varargin : usual optional arguments in pairs
- sstate_blocks (true|{false}): blockwise solution of the steady state.
- sstate_file (char | function_handle | {''}): name of the steady state file
- sstate_use_file (false | {true}): use the steady state file or program(in the model file) to solve the steady state
- sstate_solver (char | function_handle | {'lsqnonlin'}): Other solvers known to RISE include "fsolve", "fminunc", "fminsearch". If you want to supply your own solver, then it should be of the format [x,f,exitflag]=yourSolver(Objective,x0,lb,ub,options,varargin) i.e. it accepts the same inputs as lsqnonlin and returns the same 3 first outputs of fsolve
- sstate_bgp_shift (numeric | {5}): shift/lead of the static model for solving the balanced growth path.
- sstate_unique (true | {false}): In a regime switching model, there are potentially multiple steady states if some of the switching parameters affect the steady state of the model. When this is the case, the option "sstate_unique" is used for forcing the steady state to be unique nevertheless. In that case, the unique steady state is computed at the ergodic mean of the parameters.
- sstate_imposed (true | {false}): is typically used when
  - one wants to approximate the model around a point that is not the steady state of the model. For instance, in the piecewise linear approach, there is a reference regime (normal times) which assumes that no alternative regime exists and therefore the steady state of the model is computed as the steady state of that particular regime. That steady state is then imposed to be the steady state even in the alternative regime (e.g. the zero-lower bound regime).
  - one wants to prevent RISE from treating the steady state file or model as initial condition. For instance, under estimation, with sstate_imposed set to false, RISE will always try to compute the steady state numerically if the program computing the steady state fails. But this is wasteful since the numerical solution can never be better than the analytical solution. In other words, if the analytical solution fails, the numerical solution cannot succeed.
  - one wants to prevent RISE from trying to solve the steady state if the steady state equations do not hold. This useful for inspecting the offending equations by means of the "resid" method.
- sstate_loop (true | {false}): This option is used to inform RISE that the steady state file (or model) would accurately compute the steady state of a selected number of variables if it is given the steady state values for some other variables. Therefore, RISE will loop over (iterate) over values of the unsolved variables until the steady state is found. This is typically the case for instance in optimal policy, where, say, we do not know the steady state value of the interest rate. Note that when sstate_loop is triggered, block decomposition (sstate_blocks) becomes "inactive" It is important to note that when the option is used in combination with sstate_bounds, only the variables looped over are constrained.

- sstate_use_jacobian (true | {false}): In linear models, the jacobian is always used. In the nonlinear case, however, the true jacobian tends not to work as well as its finite differences approximation. Hence by default, we do not invoke the use of the true jacobian.
- sstate_endo_param_swap (cell array | {}): When not empty, it is a cell array with n rows and 3 columns, where n is the number of restrictions, the first column gathers the names of the endogenous variables whose values are given in the second column, the third column includes the parameters names that are endogenized.
- sstate_impose_commitment (true|{false}): impose the steady state of commitment to the solving of the steady state for discretion, loose commitment and stochastic replanning.
- sstate_bounds (struct | {}): When not empty, it is a structure such that each field is the name of an endogenous variable for which we have constraints. Each field itself is either
  - a scalar e.g bounds.C=2, which becomes bounds.C=[2,-inf,inf]
  - or a 3-element vector bounds.C=[2,-inf,inf], where the first element represents the start value (initial guess), the second element the lower bound and the third element the upper bound.

  It is also possible to simultaneously specify the growth rate of a variable in case the model is nonstationary. In that case the syntax above generalizes to a 1 x 2 cell array bounds.C={sstateInfo,bgpInfo} where sstateInfo and bgpInfo at scalars of 1 x 3 vectors as above.

  It is also possible to specify multiple regimes in case of a regime-switching DSGE model. In that case the user as to specify as many dimensions to the structure as the number of regimes. e.g. bounds(2).C=bounds(1).C

  If no information is provided beyond the first regime, RISE automatically extrapolates the first regime information to the others.

  The default bounds for the steady states are -inf and inf The default bounds for the growth rates are -log(2) and log(2), i.e.:

```
-log(2) <= C{t}-C{t-1} <= log(2) or
-log(2) <= log(C{t}/C{t-1}) <= log(2)
```

  Irrespective of the initial conditions chosen through sstate_bounds, in the presence of a steady state file or a steady state model, the procedure will go through such program and distort the initial conditions

  Setting "sstate_bounds" to be initial_sstate(m), where m has been previously solved, returns good initial conditions for both the steady state and growth but poor bounds, which doesn't matter if the guess is exact or good

  Setting "sstate_bounds" to be get(m,'sstate') returns the steady state only, which is fine if the model is stationary, internally those values are going to be imposed so that x0=lb=ub

  There is probably a need for a sstate_bgp or sstate_growth call that will also return the growth rate simultaneously but this is not implemented at this point

  In the presence of good or exact initial conditions, it is the responsibility of the user to remove a defectuous or inexact steady state program. This is done by simply reinitializing the steady state m=set(m,'sstate_file','') This also suggests that having a steady state model written inside the model file is not a good idea.

Returns:

- **obj** [rise|dsge]: model file
- **structural_matrices** [struct]: structure containing various important elements for the solution of the system

- **retcode** [numeric]: 0 if there was problem computing the steady state.

Note:

- There are 2 cases to consider:
  - The user does not provide any steady state equations: RISE will attempt to solve the steady state using a vector of zeros as initial guess. It makes life easy if the user provides the status of the variables in the system i.e. whether they grow linear or log-linearly.
  - The user provide some equations for solving the steady state. This is done in two ways:
    1. the steady_state_model block: the variables that do not appear in the block will be initialized at zero. Some parameters can also be computed inside the block. The user can define an optimization to solve for a subset of steady state values inside the block. The block has three attributes:
       a. imposed(default=false): RISE computes the solution at the specified point without checking that the point solves for the steady state
       b. unique (default=false): RISE computes the steady state at the ergodic distribution of the parameters. In case the probabilities are endogenous, the ergodic distribution of the parameters is itself a function of the steady state of the variables.
       c. loop(default=false): RISE considers the equations calculating the steady state as true and just solves for the missing variables by looping over the steady state program. The user can then use the values pushed into the steady state program to calculate the steady state for the included variables.
    2. the steady state file: The user writes a function which can be called in two possible ways:
       (i) [vnames,info]=ssfile();

       In this case the first output argument is the list of variables for which the user computes the steady state; the second output is a structure with fields unique, imposed and loop just as in the case of the steady state model.

       (ii) The other call to the function is

       ```
       [y,newp,retcode]=ssfile(y,p,d,id,obj)
       ```

       In this case, the first input (y) is the vector of steady states, which is updated and returned as the first output. The locations of the modifications are indicated by the fourth input (id), which is computed based on the list of the variables in vnames above. As for the other outputs, p is a structure with parameters, d is a structure with definitions, obj is the model object in case the user needs some further information for computing the steady state. In case some parameters are computed in the steady state file, they should be returned in the structure "newp". The last output "retcode" indicates whether no problem was encoutered in the computation of the steady state (retcode=0) or the opposite (retcode = any number different from 0).
- Writing the steady state file in this ways makes it possible to use the same file whether there is regime switching or not.

See also rise.initial_sstate, dsge_tools.sstate.set_bounds

Help for rise/sstate is inherited from superclass DSGE

3.17.2.66 state_var_list

INTERNAL FUNCTION: state_var_list creates the list of the state variables in the solution

```
final_list=state_var_list(m)
final_list=state_var_list(m,orders)
final_list=state_var_list(m,orders,compact_form)
```

Args:

- m (dsge | rise): model object
- orders (integer array | {1:m.options.solve_order}): approximation orders
- compact_form (true | {false}): if true, only unique combinations will be returned. Else, all combinations will be returned.

Returns:

:

- **final_list** [cellstr] : list of the state variables
- **kept** [vector] : location of kept state variables (computed only if compact_form is set to true)

Help for rise/state_var_list is inherited from superclass DSGE

3.17.2.67 stoch_simul

stoch_simul – attempts to emulate dynare's stoch_simul

```
oo_=stoch_simul(obj)

oo_=stoch_simul(obj,var_list)

oo_=stoch_simul(obj,var_list,varargin)
```

Args:

- obj (dsge | rise): model object
- var_list (empty | char | cellstr): List of variables for which to run stoch_simul
- varargin : Pairwise list of extra arguments

Returns:

:

- **oo_** [struct]: structure containing
  - **irfs** [struct]: structure containing impulse responses in time series format
  - **simulations** [struct]: structure containing impulse responses in time series format
  - **vcov** [matrix]: (simulated) variance-covariance
  - **skewness** [vector]: skewness measure
  - **kurtosis** [vector]: kurtosis measure
  - **variance** [vector]: (simulated) variances for the endogenous variables
  - **stdev** [vector]: (simulated) standard deviations for the endogenous variables
  - **corrcoef** [matrix]: (simulated) correlation coefficients for the endogenous variables

3.17.2.68 theoretical_autocorrelations

theoretical_autocorrelations computes (mostly) regime-specific partial autocorrelations

Args:

- obj : [rise|dsge] : model object
- varargin : pairwise arguments with the most relevant for this function being :
  - autocorr_ar : [numeric>=0|{5}] : order of autocorrelation. If you want to compute the contemporaneous correlation only set to order to 0
  - autocov_aggregate : [true|{false}] : when true and under regime switching, the autocovariances are aggregated across regimes with the weights being the ergodic distribution of the regimes. Then the autocorrelations are computed out of the aggregated autocovariances. Otherwise, by default, the theoretical autocorrelations are regime-specific
  - autocov_model_resolve : [{true}|false] : if true, the function tries to solve/resolve the model first

Returns:

:

- **Acorr** [cell array]: cell array of regime-specific correlations, when autocov_aggregate is false. When there is only one regime or when autocov_aggregate is true, the cell array contains only one element. Inside each element is Acorr is a possibly 3-dimensional array where the 3rd dimension represents the order of autocorrelation. In order words, Acorr{regime=i}(:,:,1) is the contemporaneous correlation while Acorr{regime=i}(:,:,2) is the autocorrelation of order 1, so that Acorr{regime=i}(:,:,k+1) is the autocorrelation of order k.
- **Acov** [cell array]: cell array of regime-specific autocovariances, following the same logic as the autocorrelations described above
- **retcode** [scalar]: return code

Note:

- The sucessful computation of the autocorrelations depends on the solving of the discrete Lyapunov or Stein equation.

3.17.2.69 theoretical_autocovariances

THEORETICAL_AUTOCOVARIANCES - Compute regime-specific autocovariances

Syntax:

[V,retcode] = theoretical_autocovariances(obj) [V,retcode] = theoretical_autocovariances(obj, autocov_ar) [V,retcode] = theoretical_autocovariances(obj, autocov_ar, vList) [V,retcode] = theoretical_autocovariances(obj, autocov_ar, vList, varargin)

Args:

- *obj* : [rise|dsge] : Model object
- *autocov_ar* : [numeric>=0|{5}] : Order of autocovariance. If you want to compute the variance only, set the order to 0.
- *vList* : [empty|cellstr|vector] : Names of indices/positions for the variables for which to compute the autocovariance.
- *varargin* : Additional options specified as key-value pairs.

Returns:

- *V* : [cell array] : Cell array of regime-specific covariances. When autocov_aggregate is false, each element of V is a 3-dimensional array, where the 3rd dimension represents the order of autocovariance. In other words, V{regime=i}(:,:,1) is the variance, and V{regime=i}(:,:,k+1) is the autocovariance of order k.
- *retcode* : [scalar] : Return code. 0 indicates success.

Note:

- The successful computation of the autocovariances depends on the solution of the discrete Lyapunov or Stein equation. Different Lyapunov algorithms can be set through the option "lyapunov_algo". The available algorithms are:
  - 'doubling' OR @lyap_solvers.doubling (the default)
  - 'bicgstab' OR @lyap_solvers.bicgstab
  - 'bicgstabl' OR @lyap_solvers.bicgstabl
  - 'cgs' OR @lyap_solvers.cgs
  - 'fevdcov' OR @lyap_solvers.fevdcov
  - 'direct' OR @lyap_solvers.direct
  - 'fix_point' OR @lyap_solvers.fix_point
  - 'robust' OR @lyap_solvers.robust
  - 'sandwich' OR @lyap_solvers.sandwich
  - 'schur' OR @lyap_solvers.schur
  - 'bartels_stewart' OR @lyap_solvers.bartels_stewart
- In the presence of a nonstationary model, it is not a good idea to use the doubling algorithm as it won't converge. A different algorithm, such as "bartels_stewart," may work better.
- Options autocov_aggregate, autocov_aggregate, and autocov_ar can be passed as pairs in varargin. In the case of autocov_ar, passing it directly overrides the version passed in varargin.

See also:

- *SOLVE*, *LYAPUNOV_EQUATION*

3.17.2.70 translate

translate – Translates RISE codes into comprehensible atoms

```
outList=translate(obj,inList)
outList=translate(obj,inList,order)
```

Args:

- obj (rise | dsge): scalar model object.
- inList (char | cellstr): List of atoms to translate e.g. v_3_1, v(3,1), c(4), c_4
- order (integer|0|{1}): If order>1 the returned list of atoms is a list of kroneckers in which the number of elements in each items is the order of the kronecker. This is useful for instance to understand what combination of variables makes up a column in a differentiation. If order=0, the translation is with respect to the static, rather than the dynamic model

Returns:

- **outList** [cellstr]: List of translated atoms

Examples:

```
translate(m,{'v(1,1)','v_1_1','c(4)','c_4'})

list=obj.routines.symbolic_probs_times_dynamic{2}
outList=translate(obj,list)
outList=translate(obj,list,3)
```

Help for rise/translate is inherited from superclass DSGE

3.17.2.71 unlink_parameters

UNLINK_PARAMETERS remove the selected links

m = UNLINK_PARAMETERS(m, expressions)

This function is responsible for unbinding dynamically bound parameters

- *m*: Scalar or vector of model objects.
- *targets*: Character or cell array of strings representing the parameters to unlink. e.g. targets = {'alpha','beta','gamma'} will remove the definitions of the specified parameters. It could also be '*', in which case all links are deleted.

Returns: - *m*: Updated model object with the remaining dynamically bound parameters.

Example:
m = UNLINK_PARAMETERS(m, {'alpha','beta','gamma'})

See also:

rise.view_linked_parameters, rise.link_parameters

Help for rise/unlink_parameters is inherited from superclass DSGE

3.17.2.72 update_file

update_file update files for the latest syntax

```
dsge.update_file(filnavn)

dsge.update_file(filnavn,nyfilnanv)

rise.update_file(filnavn)

rise.update_file(filnavn,nyfilnanv)
```

Args:

filnavn (char): name of the rise/dsge file (with or without extension)
to update for the new syntax

nyfilnanv (char): name of the new rise/dsge file with default being the name of the original file

Returns:
:

none

See also

Help for rise.update_file is inherited from superclass DSGE

3.17.2.73 variance_decomposition

Computes the variance decomposition for a DSGE model

```
[Vardec,obj]=variance_decomposition(obj)

[Vardec,obj]=variance_decomposition(obj,varargin)
```

Args:

- **obj** (dsge | rise): model object
- **varargin** : Pairwise list of extra arguments
  - **vardec_shocks** [empty|char|cellstr]: list of shocks
  - **vardec_periods** [vector|{[1 4 8 16 40 100 200 400]}]: periods to consider for the decomposition
  - **vardec_theoretical** [false|{true}]: if true, the theoretical variance decomposition is computed to the extent that this is possible. If false, the decomposition is based on simulated data.
  - **vardec_ergodic** [{false}|true]: if true, compute the ergodic variance decomposition for a regime-switching model. If false, the computations are regime specific

Returns:

- **Vardec** [struct]: structure containing the variance decomposition both for the
  - infinite horizon (**infinity**)
  - for each period (**conditional**)
- **retcode** [numeric]: = 0 if there is no problem

N.B: The calculation of the variance is done by solving a Lyapunov equation. There are different Lyapunov algorithms which can be set through the option "lyapunov_algo". The available algorithms are:

- 'doubling' OR @lyap_solvers.doubling (the default)
- 'bicgstab' OR @lyap_solvers.bicgstab
- 'bicgstabl' OR @lyap_solvers.bicgstabl
- 'cgs' OR @lyap_solvers.cgs
- 'fevdcov' OR @lyap_solvers.fevdcov
- 'direct' OR @lyap_solvers.direct
- 'fix_point' OR @lyap_solvers.fix_point
- 'robust' OR @lyap_solvers.robust
- 'sandwich' OR @lyap_solvers.sandwich
- 'schur' OR @lyap_solvers.schur
- 'bartels_stewart' OR @lyap_solvers.bartels_stewart

Notes:

- In the presence of a nonstationary model, it is not a good idea to use the doubling algorithm : it won't converge. A different algorithm such as "bartels_stewart" may work better.
- The algorithms listed above are invoked either as char/string or directly as function handles prefixed with "lyap_solvers.". e.g. 'fevdcov' or @lyap_solvers.fevdcov
- In the presence of a nonstationary model, it is not a good idea to use the doubling algorithm : it won't converge. A different algorithm such as "bartels_stewart" may work better.

3.17.2.74 view_linked_parameters

VIEW_LINKED_PARAMETERS displays the dynamically-bound expressions

VIEW_LINKED_PARAMETERS(m)

This function is responsible for unbinding dynamically bound parameters

- *m*: Scalar or vector of model objects.

Example:
VIEW_LINKED_PARAMETERS(m)

See also:

rise.link_parameters, rise.unlink_parameters

Help for rise/view_linked_parameters is inherited from superclass DSGE

CHAPTER

FOUR

DSGE-VAR MODELING

4.1 Description

The DSGE-VAR or elswhere known as BVAR-DSGE is a methodology that combines a BVAR and a DSGE model following the methodology in [Del Negro and Schorfheide, 2004] and [Del Negro *et al.*, 2007].

There are two possible interpretations. One is that the DSGE is used as a prior for the BVAR model and the other is that the BVAR serves to relax the tight restrictions in the DSGE model.

In the end we have four sub-models in one object:

- The VAR model
- The VAR approximation of the DSGE model
- The DSGE model
- The BVAR model i.e. the VAR model with the (VAR approximation of the) DSGE as prior

In RISE, the DSGE model can be a model with a simple instrument rule (e.g. Taylor rule) or an optimal policy under commitment or under discretion.

The DSGE model can be stationary or nonstationary.

4.2 A quick-start example

4.2.1 A simple New Keynesian DSGE model

```
dsgemodel={
'model:New Keynesian model'
'@endogenous X "Output gap" R "interest rate" P "Inflation" G U'
'@exogenous EG "Demand shock" EU "Monetary Policy shock"'
'@parameters beta "discount factor" kappa "Phillips curve slope" sigu sigg rhou rhog psi'
'@observables  P R'
'@model'
'       P = beta*P{+1}+kappa*X;'
'       X = X{+1}-(R-P{+1}-G);'
'       R = psi*P+U;'
'       U = rhou*U{-1} + sigu*EU;'
'       G = rhog*G{-1} + sigg*EG;'
    };
```

4.2.2 Setting up the BVAR-DSGE model

```
nlags=4;

constant=false;

mdl=bvar_dsge(dsgemodel,nlags,constant);
```

4.2.3 Fixed parameters

```
mdl=set(mdl,'parameters',{'beta',0.96});
```

4.2.4 Setting up and visualing the priors

```
priors=struct();

% ___priors on the DSGE parameters___
priors.kappa={0.2,0.5,0.5,'gamma'};
priors.psi={1.5,2,.5,'gamma'};
priors.rhou={0.75,0.75,0.1,'beta'};
priors.rhog={0.75,0.75,0.1,'beta'};
priors.sigu={0.01,0.01,4,'sichisq'};
priors.sigg={0.01,0.01,4,'sichisq'};

% ___prior on the dsge model___
priors.dsge_prior_weight={3,3,1,'gamma'};

plotOpts=struct();
plotOpts.prior_trunc=2e-3;
rdist.plot(priors,plotOpts)
```

4.2.5 Collecting and transforming the data

```
d=fetch_fred({'CPALTT01USQ661S','BOGZ1FL072052006Q'});

db=struct();

db.P=log(d(1).series/d(1).series{-1});

db.R=d(2).series/100;
```

4.2.6 Maximizing the posterior

```
mdlest=estimate(mdl,'priors',priors,'data_demean',true,'data',db,...
    'estim_start_date','1960Q2','estim_end_date','2022Q3');
```

4.2.7 IRFs of the BVAR-DSGE at the maximized posterior

```
myirfs_bvar_dsge=irf(mdlest);
```

4.2.8 IRFs of the DSGE model at the maximized posterior

```
myirfs_dsge=irf(mdlest.dsge);
```

4.2.9 IRF comparison

```
myirfs=ts.concatenator(myirfs_bvar_dsge,myirfs_dsge)

quick_irfs(mdlest.dsge,myirfs,{'P','R'})
```

4.3 Technical documentation for dsge_var objects

4.3.1 The properties

4.3.1.1 nlags

bvar_dsge/nlags is a property.

4.3.1.2 constant

bvar_dsge/constant is a property.

4.3.1.3 endogenous

bvar_dsge/endogenous is a property.

4.3.1.4 options

bvar_dsge/options is a property.

4.3.1.5 shocks

bvar_dsge/shocks is a property.

4.3.1.6 parameters

information on parameters (names, number, types, etc.)

Help for bvar_dsge/parameters is inherited from superclass ESTIMABLE

4.3.1.7 markov_chains

information on markov chains, regimes and related items

Help for bvar_dsge/markov_chains is inherited from superclass ESTIMABLE

4.3.1.8 estimation

information on estimation: posterior maximization and simulation

Help for bvar_dsge/estimation is inherited from superclass ESTIMABLE

4.3.2 The methods

4.3.2.1 autocov

Compute the autocovariances (and the auto-correlation) of endogenous variables given the parameter values

```
[C,R,retcode] = autocov(self);
[C,R,retcode] = autocov(self, params);
[C,R,retcode] = autocov(self, params, max_periods);
```

Args:

self (var object): var object params (cell of struct): struct containing var model related parameters (default: []) max_periods (integer): maximum number of period to calculate auto-covariance (default: 5)

Returns:

:

- **C** [4-dimensional array]: auto-covariance where dimensions correspond to
  - 1,2: covariance
  - 3: time lags
  - 4: Number of parameters
- **R** [4-dimensional array]: auto-correlation with same dimensions

4.3.2.2 bvar_dsge

bvar_dsge Construct an instance of the bvar_dsge class

obj = bvar_dsge(dsgemodel)

obj = bvar_dsge(dsgemodel,nlags)

obj = bvar_dsge(dsgemodel,nlags,const)

obj = bvar_dsge(dsgemodel,nlags,const,shock_list)

INPUTS

- dsgemodel [dsge|rise|char|cell array]

  * dsge or rise : well understood * char : rise file name * cell array

  a. multiple rise files {'file1','file2',...,'filen'}

  b. one file and arguments {'file1','arg1',argval1,...,'argn',argvaln}

  c. multiple rise files and arguments {{'file1','file2',...,'filen'},'arg1',argval1,...,'argn',argvaln}

- nlags [numeric|{4}] : number of lags
- const [false|{true}] : constant in the VAR
- shock_list [empty|cell array] : list of selected shocks in the DSGE in case the DSGE model has more shocks than the number of observables.

OUTPUTS

- obj [bvar_dsge]

TODO (if there is a demand for it)

- add exogenous variables as in a normal VAR
- Turn on the possibility of drawing from a Normal-inverse-Wishart prior when running impulse responses. At the moment the feature is turned of because for every parameter draw, only one single IRF is computed and not a distribution.

Documentation for bvar_dsge/bvar_dsge
doc bvar_dsge

4.3.2.3 draw_parameter

Random parameter draws for RISE model objects.

```
[draw,obj] = draw_parameter(obj,simulation_folder);
```

Args:

- obj (rise | dsge | rfvar | svar): RISE model object
- simulation_folder (char | struct):
  - char(1): simulation folder : the stored elements should be structures with fields:
    - **x** : parameter vectors
    - **f** : value of minus(log posterior kernel)
  - char(2): ['mode'|'prior']: draw from the prior distribution or from a multivariate normal distribution around the mode.

Returns:

- **draw** [cell]: the first entry is the names of the estimated parameters and the second is a vector of drawn parameters. The whole cell can be pushed in to a model as obj=set(obj,'parameters',draw).
- **obj** [rise|dsge|rfvar|svar]: RISE model object in which the drawn parameter has been pushed.

Help for bvar_dsge/draw_parameter is inherited from superclass ESTIMABLE

4.3.2.4 estimate

Estimate the parameters of a RISE model

```
obj=estimate(obj)
obj=estimate(obj,varargin)
[obj,filtration]=estimate(...)
```

Args:

- obj (bvar_dsge): model object
- varargin : additional optional inputs among which the most relevant for estimation. See ESTIMABLE/ESTIMATE

4.3.2.5 fisher

INTERNAL FUNCTION

Help for bvar_dsge/fisher is inherited from superclass ESTIMABLE

4.3.2.6 forecast

bvar_dsge/forecast is a function.
[fkst, retcode] = forecast(self, histdb, date_start, params, nsteps, shock_uncertainty, Rfunc, conditions)

4.3.2.7 get

GET Get object properties.

V = GET(H,'PropertyName') returns the value of the specified property for the graphics object with handle H. If H is a vector of handles, then get will return an M-by-1 cell array of values where M is equal to length(H). If 'PropertyName' is replaced by a 1-by-N or N-by-1 cell array of strings containing property names, then GET will return an M-by-N cell array of values.

GET(H) displays the names and current values of all user-gettable properties for the graphics object with handle H.

V = GET(H) where H is a scalar, returns a structure where each field name is the name of a user-gettable property of H and each field contains the value of that property.

V = GET(0, 'Factory') V = GET(0, 'Factory<ObjectType>') V = GET(0, 'Factory<ObjectType><PropertyName>') returns for all object types the factory values of all properties which have user-settable default values.

V = GET(H, 'Default') V = GET(H, 'Default<ObjectType>') V = GET(H, 'Default<ObjectType><PropertyName>') returns information about default property values (H must be scalar). 'Default' returns a list of all default property values currently set on H.

'Default<ObjectType>' returns only the defaults for properties of <ObjectType> set on H. 'Default<ObjectType><PropertyName>' returns the default value for the specific property, by searching the defaults set on H and its ancestors, until that default is found. If no default value for this property has been set on H or any ancestor of H up through the root, then the factory value for that property is returned.

Defaults can not be queried on a descendant of the object, or on the object itself - for example, a value for 'DefaultAxesColor' can not be queried on an axes or an axes child, but can be queried on a figure or on the root.

When using the 'Factory' or 'Default' GET, if PropertyName is omitted then the return value will take the form of a structure in which each field name is a property name and the corresponding value is the value of that property. If PropertyName is specified then a matrix or string value will be returned.

See also SET, RESET, DELETE, GCF, GCA, FIGURE, AXES.

4.3.2.8 hessian

Computes the hessian of the model at a specific point

```
[obj,H,issue] = hessian(obj)

[obj,H,issue] = hessian(obj,x)

[obj,H,issue] = hessian(obj,x,varargin)
```

Args:

- obj (rise | dsge | rfvar | svar): model object
- x ([] | vector): vector at which one wants to compute the hessian
- varargin: additional optional inputs among which the most relevant for estimation is:
  - **hessian_type** [{'fd'}|'opg']: The hessian is either computed by finite differences (fd) or by outer-product-gradient (opg)

Returns:

:

- **obj** [rise|dsge|rfvar|svar]: model object containing the new hessian in case the model was previously estimated.
- **H** [d x d matrix]: hessian matrix
- **issue** [char|'']: any issue encountered during the computation of the hessian

Help for bvar_dsge/hessian is inherited from superclass ESTIMABLE

4.3.2.9 historical_decomposition

varargin={shock_names}

4.3.2.10 irf

Compute impulse response function from the given parameter values

```
myirfs = irf(self)

myirfs = irf(self, shock_names)

myirfs = irf(self, shock_names, irf_periods)

myirfs = irf(self, shock_names, irf_periods, params)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc,girf_setup)
```

Args:

- **self** (var object): var object
- **shock_names** (cellstr): shocks for which to compute IRFs (has to be consistent with the identification scheme *identification*)
- **irf_periods** (int): number of periods to compute IRFs (default: 40)
- **params** (vector): parameter values of the model. If empty MLE/posterior-mode values are used.
- **Rfunc** (function handle): identification function. This is an output of `identification`. (default: choleski identification)
- **girf_setup** (struct|{empty}): structure containing information relevant for the computation of generalized impulse response functions. If empty, simple regime-specific impulse responses are computed, else girfs are computed. In that case the relevant information to provide in girf_setup is:
  - **nsims** : (default=300) number of simulations for the integration. Note that even setting girf_setup=struct() will trigger the computation of girfs. But in that case only the default options will apply.

Returns:

- **myirfs** : (struct) structure containing the IRFs

Note: Only successful IRFs are returned. If the structure does not return some IRFs make sure that IRFs properly identified.

4.3.2.11 load_parameters

load_parameters loads the parameters. This allows the user to quickly load the parameters from a file, which may be the output of estimation, and get going with irfs, simulations, etc.

```
model=load_parameters(model,the_mode_file)
```

Args:

- **model** (estimable object): model object
- **the_mode_file** (m-file): file containing the parameters and their values

Returns:

- **model** (estimable object): reparameterized model object

Help for bvar_dsge/load_parameters is inherited from superclass ESTIMABLE

4.3.2.12 log_posterior_kernel

Computes the log posterior of the dsge model

```
[log_post,log_lik,log_prior,Incr,retcode,obj]=log_posterior_kernel(obj, param)
```

Args:

- obj (estimable object): model object
- param (column vector): parameter values

Returns:

- **log_post** (double): log posterior
- **log_lik** (double): log likelihood
- **log_prior** (double): log prior
- **Incr** (double):
- **retcode** : return code
- obj (estimable object): model passed through

See also:

- *log_prior_density*

Note: In effort to make RISE modular, this function is available so that one can use a different sampler if needed, but most likely, one should just use available stock samplers.

Help for bvar_dsge/log_posterior_kernel is inherited from superclass ESTIMABLE

4.3.2.13 log_prior_density

Computes the probability density function of the prior corresponding to the parameter values

```
[lnprior, retcode] = log_prior_density(model)
[lnprior, retcode] = log_prior_density(model, param)
[lnprior, retcode] = log_prior_density(model, param,filtration)
```

Args:

- **model** (estimable object): model object
- **param** (column vector): parameter values
- **filtration** (empty|struct): results from model filtration that can potentially be used for endogenizing priors.

Returns:

- **lnprior** (double): log of prior density function
- **retcode**: return code

See also:

- *log_posterior_kernel*

Note: In effort to make RISE modular, this function is available so that one can use a different sampler if needed, but most likely, one should just use available stock samplers.

Help for bvar_dsge/log_prior_density is inherited from superclass ESTIMABLE

4.3.2.14 mode_curvature

Checks the curvature at the posterior mode

```
db = mode_curvature(obj)

db = mode_curvature(obj,varlist)

db = mode_curvature(obj,varlist,N)

db = mode_curvature(obj,varlist,N,type)
```

Args:

- **obj** (rise | dsge | rfvar | svar): model object
- **varlist** (char | cellstr | empty): list of parameters for which we want to check the curvature
- **N** ({20} | integer): Number of grid points
- **type** ({'max'} | 'min' | 'range'): normalization of the log-posterior and the log-likelihood.
- **dbin** (struct|empty): structure containing the information to plot the curvature. Each field is the name of a particular parameter. This is to avoid a costly recomputation of db

Returns:

- **db** [struct|cell array|vector]: structure containing the information to plot the curvature. Each field is the name of a particular parameter. Alternatively, when dbin is not empty, db is a handle to the plots.

Note:

- when no output is requested, plots are made but not saved.
- one way to plot the curvatures from the output is to use the function utils.plot.curvature

See also:

- utils.plot.curvature

Help for bvar_dsge/mode_curvature is inherited from superclass ESTIMABLE

4.3.2.15 posterior_sample

Computes a sample of any quantity of interest using parameter draws from a population e.g. a posterior simulation

```
[result]=posterior_sample(m,pop,dowhat)

[result]=posterior_sample(m,pop,dowhat,howmany)

[result]=posterior_sample(m,pop,dowhat,howmany,ouf)

[result]=posterior_sample(m,pop,dowhat,howmany,ouf,varargin)

[result,is_failed,time_it_took]=posterior_sample(...)
```

Args:

- **m** [rise|dsge|svar|rfvar|valid rise object]: model object
- **pop** [m x n struct]: parameter draws, with "m" the number of chains and "n" the number of draws in each chain. Each element of "pop" is a structure with fields "f" (not used), the value of the posterior and "x" the parameter vector
- **dowhat** [fhandle]: function (handle) to apply to each parameterized model object. e.g. dowhat=@irf, dowhat=@simulate, dowhat=@forecast, etc. "dowhat" need not be a method of "m": it represents the quantity of interest.
- **howmany** [integer|{m x n}]: number of draws to use in the calculation
- **ouf** [fhandle|{[]}]: output update function. Function that updates the output before storing it. e.g. if dowhat=@filter, one may be interested in the filters only and in that case ouf=@(x)x.filtering.
- **varargin** [pairwise args]: valid pairwise arguments for the model object

Returns:

- **result** [1 x howmany cell]: container of the various applications of the "dowhat" handle
- **time_it_took** [numeric]: number of seconds needed to run all the simulations.

See also:

draw_parameter

Note:

- the function will exploit parallel computation if there are workers idle.

- Because the solving of the model is sometimes iterative, a change of solver or of the settings of the solver can result in the model not being solved or more generally simulations failures. The algorithm will loop until the requested number of simulations is obtained. But it will not point to the parameter vectors that fail.

Help for bvar_dsge/posterior_sample is inherited from superclass ESTIMABLE

4.3.2.16 print_estimation_results

PRINT_ESTIMATION_RESULTS: Display the results of estimation.

retcode = print_estimation_results(obj) retcode = print_estimation_results(obj, detail)

Args:

- *obj* (estimable): Model object.
- *detail* (true|{false}): If true, the description of parameters is given alongside the code name. If they are the same, i.e., the description has not been provided, then the code name is given only once.

Returns:

- *retcode* (numeric): 0 if there is no problem.

Notes:

- If there are multiple objects in the array, each object's results will be displayed.
- The function displays information such as log-posterior, log-likelihood, log-prior, etc.
- The displayed results include information about the estimation sample, solution algorithm, estimation algorithm, number of estimated parameters, number of function evaluations, and the time taken for estimation.
- If there are any issues, a list of issues is displayed.

Example:
print_estimation_results(obj) print_estimation_results(obj, true)

See also: *TABLE_DISPLAYER*

Help for bvar_dsge/print_estimation_results is inherited from superclass ESTIMABLE

4.3.2.17 pull_objective

Pulls the objective function to optimize

```
[ff,lb,ub]=pull_objective(obj)
[ff,lb,ub]=pull_objective(obj,varargin)
[ff,lb,ub,x0]=pull_objective(obj,varargin)
[ff,lb,ub,x0,vcov]=pull_objective(obj,varargin)
[ff,lb,ub,x0,vcov,obj]=pull_objective(obj,varargin)
```

Args:

- obj (rise | dsge | svar | rfvar): initial model object
- varargin (pairwise addional inputs): usual RISE arguments

Returns:

- **ff** [function handle]: for computing "minus log posterior kernel"
- **lb** [d x 1 vector]: lower bound of the parameters to optimize
- **ub** [d x 1 vector]: upper bound of the parameters to optimize
- **x0** [d x 1 vector]: posterior mode if available
- **vcov** [d x d matrix]: covariance matrix at the posterior mode if available.
- **obj** [rise|dsge|svar|rfvar]: updated model object

Note:

- The function can be used for :
  - optimization,
  - gradient computation,
  - hessian computation,
  - posterior simulation
- The updated object should be used for doing various exercises (irfs, simulations, etc.) if the posterior mode is not computed.
- Using this function is potentially costly, one could alternatively simply use log_posterior_kernel. However, if there are restrictions, they will not be enforced. Nevertheless it is an interesting proposition that should be investigated further.

Help for bvar_dsge/pull_objective is inherited from superclass ESTIMABLE

4.3.2.18 residuals

Computes the residuals given the parameters

4.3.2.19 set

METHOD1 Summary of this method goes here
Detailed explanation goes here

4.3.2.20 solve

INTERNAL FUNCTION: intermediary file for computing key elements for dsge-var

```
[sol,retcode,self] = solve(self)
[sol,retcode,self] = solve(self,params)
[sol,retcode,self] = solve(self,params,whichVar)
```

Args:

- **self** (bvar_dsge): model object
- **params**
- **whichVar** ['var'|'var_approx'|{'var_dsge'}]: choose which VAR you want to solve. Where :
  - **var** : is the classical VAR without priors
  - **var_approx** : is the VAR approximation of the DSGE

  - **var_dsge** : is the Bayesian VAR with the (VAR approximation of the) DSGE as prior

Returns:

- **self** [bvar_dsge]: model object

4.3.2.21 variance_decomposition

Compute the variance decompsition of the VAR

```
[vd,retcode] = variance_decomposition(self)

[vd,retcode] = variance_decomposition(self,params)

[vd,retcode] = variance_decomposition(self,params,Rfunc)

[vd,retcode] = variance_decomposition(self,params,Rfunc,nperiods)
```

Args:

- self (var object):
- params :(optional) parameter values
- Rfunc (function handle): (optional) transform parameters into dynamics
- nperiods :(optional) periods for decomposition

Returns:

- **vd** (struct): a struct containing the variance decomposition:
  - **infinity** (struct):
  - **conditional** (struct):
- **retcode** (integer): This passes through the retcode given by Rfunc. Currently (2018/07/19), this output always returns 0.

CHAPTER

FIVE

REDUCED-FORM VAR MODELING

5.1 Description

$$y_t = C(r_t)\, x_t + B_1(r_t)\, y_{t-1} + ... + B_p(r_t)\, y_{t-p} + u_t$$

With $r_t = 1, 2, ..., h$ and transition probabilities $p_{r_t, r_{t+1}}(I_t)$

5.2 Quick-start examples

5.2.1 A constant-parameter VAR

5.2.1.1 Collecting and transforming data

```
% CLVMNACSCAB1GQNO : GDP Norway
% IR3TIB01NOQ156N : 3-month Interbank interest rate
% NORCPGRLE01IXOBQ : CPI excluding food and energy
% CCUSSP01NOQ650N : Spot Exchange rate : 1NOK = x USD
% POILWTIUSDQ : Global price of WTI Crude
xrange='1990Q1:2022Q3';
rawdb=fetch_fred({'NORCPGRLE01IXOBQ','IR3TIB01NOQ156N','CLVMNACSCAB1GQNO',...
    'CCUSSP01NOQ650N','POILWTIUSDQ'});
rawdb1.P=rawdb(1).series(xrange);
rawdb1.INTRATE=rawdb(2).series(xrange);
rawdb1.Y=rawdb(3).series(xrange);
rawdb1.EXRATE=rawdb(4).series(xrange);
rawdb1.POIL=rawdb(5).series(xrange);
rawdb=rawdb1; clear rawdb1

db=struct();
db.PAI=rawdb.P/rawdb.P{-1};
db.R=1+rawdb.INTRATE/100;
db.GROWTH=rawdb.Y/rawdb.Y{-1};
db.EXRATE=1/rawdb.EXRATE; % so that increase = depreciation of NOK
db.PAIOIL=rawdb.POIL/rawdb.POIL{-1};
```

5.2.1.2 Setting up the Reduced.form VAR

```
endog={'PAIOIL','GROWTH','PAI','R','EXRATE'};

exog={};

nlags=4;

const=true;

mdl = rfvar(endog, exog, nlags, const);
```

5.2.1.3 Estimating the VAR using classical techniques

```
data_range={db.GROWTH.start,db.GROWTH.finish};

mdlest=estimate(mdl,db,data_range);
```

5.2.1.4 Restrictions on the VAR : Domestic variables do not affect oil prices

```
linres={
    'b1(PAIOIL,PAI)=0'
    'b1(PAIOIL,GROWTH)=0'
    'b1(PAIOIL,R)=0'
    'b1(PAIOIL,EXRATE)=0'
    'b2(PAIOIL,PAI)=0'
    'b2(PAIOIL,GROWTH)=0'
    'b2(PAIOIL,R)=0'
    'b2(PAIOIL,EXRATE)=0'
    };

% or programmatically
% linres=cell(0,1);
% for ilag=1:nlags
%     for iv=2:numel(endog)
%         y=endog{iv};
%         linres{end+1,1}=sprintf('b%0.0f(PAIOIL,%s)=0',ilag,y); %#ok<SAGROW>
%     end
% end
```

5.2.1.5 Estimate the restricted VAR

```
prior=[];

mdlest_restr=estimate(mdl,db,data_range,prior,linres);
```

5.2.1.6 Identification

```
shock_names={'oilp','demand','costpush','mp','forex'};

ident_restr1={
    % normalization with sign restrictions
    'PAIOIL{0}@oilp','+'
    'GROWTH{0}@demand','+'
    'PAI{0}@costpush','+'
    'R{0}@mp','+'
    'EXRATE{0}@forex','+'
    % first set
    'PAIOIL{0}@demand',0
    'PAIOIL{0}@costpush',0
    'PAIOIL{0}@mp',0
    'PAIOIL{0}@forex',0
    % second set
    'GROWTH{0}@costpush',0
    'GROWTH{0}@mp',0
    'GROWTH{0}@forex',0
    % third set
    'PAI{0}@mp',0
    'PAI{0}@forex',0
    % fourth set
    'R{0}@forex',0
    };

agnostic=true;

max_trials=6000;

Rfunc=struct(); ident=struct();

[Rfunc.unrestr,ident.unrestr]=identification(mdlest,ident_restr1,shock_names,...
    agnostic,max_trials);

[Rfunc.restr,ident.restr]=identification(mdlest_restr,ident_restr1,shock_names,...
    agnostic,max_trials);

disp(ident.restr)

disp(ident.unrestr)
```

5.2.1.7 Compare structural shocks

```
params=[];
sshocks=struct();
sshocks.unrestr=structural_shocks(mdlest,...
    params,Rfunc.unrestr,shock_names);
sshocks.restr=structural_shocks(mdlest_restr,...
    params,Rfunc.restr,shock_names);

% plots
%-------

titel='Structural shocks';

figure('name',titel);

for ii=1:numel(shock_names)

    thisname=shock_names{ii};

    subplot(3,2,ii)

    d=[sshocks.unrestr.(thisname),sshocks.restr.(thisname)];

    plot(d,'linewidth',2)

    xtickangle(45)

    title(thisname)

    if ii==1

        legend({'unrestr','restr'})

    end

end

% xrotate(45)

[~,h]=sup_label(titel,'t');

set(h,'fontsize',12)
```

5.2.1.8 Compare CHOLESKI irfs

```
cholShocks=[];

myirfs=irf([mdlest,mdlest_restr],cholShocks,40);

cholShocks=fieldnames(myirfs);

tex=struct();
tex.PAIOIL='Oil price inflation';
tex.GROWTH='GDP growth';
tex.PAI='Inflation';
tex.R = 'Interest rate';
tex.EXRATE='exchange rate';

for ishock=1:numel(cholShocks)

    shock=cholShocks{ishock};

    titel=['Impulse responses to a ',shock,' shock'];

    figure('name',titel);

    for ii=1:numel(endog)

        subplot(3,2,ii)

        plot(myirfs.(shock).(endog{ii}),'linewidth',2)

        title(tex.(endog{ii}))

        if ii==1

            legend({'unrestr','restr'})

        end

    end

    [~,h]=sup_label(titel,'t');

    set(h,'fontsize',12)

end
```

5.2.1.9 Compare irfs based on the identification scheme

```
params=[];

% Note the two models have the same identification !!!
% This is why we can run them together
myirfs=irf([mdlest,mdlest_restr],shock_names,40,params,Rfunc.unrestr);

for ishock=1:numel(shock_names)

    shock=shock_names{ishock};

    titel=['Impulse responses to a ',shock,' shock'];

    figure('name',titel);

    for ii=1:numel(endog)

        subplot(3,2,ii)

        plot(myirfs.(shock).(endog{ii}),'linewidth',2)

        title(tex.(endog{ii}))

        if ii==1

            legend({'unrestr','restr'})

        end

    end

    [~,h]=sup_label(titel,'t');

    set(h,'fontsize',12)

end
```

5.2.1.10 Variance decomposition

```
params=[];

vd=variance_decomposition(mdlest_restr,params,Rfunc.restr);

% plot decompositions
%---------------------

range='0:50'; % pick a range for the plots

figure('name','Variance Decomposition');
```

(continues on next page)

(continued from previous page)

```
for iv=1:numel(endog)

    d=vd.conditional.(endog{iv});

    subplot(3,2,iv)

    plot_decomp(range,d)

    if iv==1

        legend(shock_names,'location','SE',...
            'Orientation','horizontal')

    end

    title(tex.(endog{iv}))

end
```

5.2.1.11 Historical decomposition

```
params=[];

hd=historical_decomposition(mdlest_restr,params,Rfunc.restr);

% plot decompositions
%----------------------

shock_only=true;

titel='Model with block exogeneity: Historical Decomposition';

figure('name',titel);

for iv=1:numel(endog)

    d=hd.(endog{iv});

    subplot(3,2,iv)

    if shock_only

        shock_tex=shock_names;

        d=d(shock_names);

    else

        shock_tex=d.varnames;
```

(continues on next page)

(continued from previous page)

```
    end

    plot_decomp(d)

    if iv==1

        legend(shock_tex,'location','SE','Orientation','horizontal')

    end

    title(tex.(endog{iv}))

end

[~,h]=sup_label(titel,'t');

set(h,'fontsize',12)
```

5.2.1.12 Bootstrap

```
n=1000;

params=bootstrap(mdlest_restr,n);
```

5.2.1.13 Variance decomposition distribution

```
ci=[30,50,68,90];

vd=variance_decomposition(mdlest_restr,params,Rfunc.restr);

shock_tex=shock_names;

myrange='1:50';

for iv=1:numel(endog)

    vname=tex.(endog{iv});

    titel=['Variance Decomposition (in %) of ',vname];

    figure('name',titel);

    d=vd.conditional.(endog{iv});

    d.varnames=shock_names;

    d=pages2struct(d);

    contributors=fieldnames(d); % = shock_names
```

(continues on next page)

(continued from previous page)

```
    for ii=1:numel(contributors)

        subplot(3,2,ii)

        % note we are multiplying by 100, this just by pure convenience
        %-----------------------------------------------------------------
        out=fanchart(100*d.(contributors{ii})(myrange),ci);

        plot_fanchart(out)

        title(contributors{ii})

        axis tight

    end

    [~,h]=sup_label(titel,'t');

    set(h,'fontsize',12)

end
```

5.2.1.14 Historical decomposition distribution

```
ci=[30,50,68,90];

% compute decompositions

    hd=historical_decomposition(mdlest_restr,...
        params,Rfunc.restr);

% plot decompositions

for iv=1:numel(endog)

    vname=tex.(endog{iv});

    titel=['Model with restrictions : Historical Decomposition of ',vname];

    figure('name',titel);

    d=pages2struct(hd.(endog{iv}));

    contributors=fieldnames(d);

    for ii=1:numel(contributors)

        subplot(5,2,ii)
```

(continues on next page)

(continued from previous page)

```
        out=fanchart(d.(contributors{ii}),ci);

        plot_fanchart(out)

        xtickangle(45)

        title(contributors{ii})

        axis tight

    end

    [~,h]=sup_label(titel,'t');

    set(h,'fontsize',12)

end
```

5.2.1.15 IRF distribution

```
    myirfs=irf(mdlest_restr,shock_names,...
        40,params,Rfunc.restr);

% IRFs plots
%--------------

for ishock=1:numel(shock_names)

    shock=shock_names{ishock};

    titel=['Model with restrictions : IRFs to a ',shock,' shock'];

    figure('name',titel);

    for ii=1:numel(endog)

        subplot(3,2,ii)

        d=myirfs.(shock).(endog{ii});

        out=fanchart(d,ci);

        plot_fanchart(out)

        title(tex.(endog{ii}))

        axis tight

    end
```

(continues on next page)

(continued from previous page)

```
    [~,h]=sup_label(titel,'t');

    set(h,'fontsize',12)

end
```

5.2.1.16 Bayesian estimation

```
v=rfvar(endog,exog,nlags,const);

%set prior

var_prior=rfvar.prior_template();
% modify as needed

var_prior.type='sz';

prior=struct('var',var_prior);
% prior.nonvar
% prior.endogenous

% unrestricted model
%--------------------

ve=estimate(v,db,data_range,prior);%,restrictions

% restricted model
%------------------

ve_lr=estimate(v,db,data_range,prior,linres);
```

5.2.1.17 Posterior sampling of parameters

```
params=struct();
params.ve=ve.estim_.sampler(1000);
params.ve_lr=ve_lr.estim_.sampler(1000);
```

5.2.1.18 Bayesian forecasting

```
myfkst=struct();
date_start='2003Q1';
myfkst.ve=forecast(ve,db,date_start,params.ve);

myfkst.ve_lr=forecast(ve_lr,db,date_start,params.ve_lr);

% set environment
```

(continues on next page)

(continued from previous page)

```
ci=[30,50,68,90];

modelnames={'ve','ve_lr'};

% Forecast plots

for jj=1:numel(modelnames)

    modname=modelnames{jj};

    figure('name',['model (',modname,') Forecasts of Norwegian Data']);

    for ii=1:numel(endog)

        subplot(3,2,ii)

        d=myfkst.(modname).(endog{ii});

        out=fanchart(d,ci);

        plot_fanchart(out,[244, 122, 66]/244)

        title(tex.(endog{ii}))

        axis tight

    end

    xrotate(45)

end
```

5.2.1.19 Conditional forecasting

```
myfkst=struct();
% date_start=[];
date_start='2003Q1';
nsteps=12;
shock_uncertainty=false;
Rfunc=[]; % No need for identification

conditions=struct();
conditions.R={'2003Q1','2004Q4'}; % range over which we want to condition

myfkst.ve=forecast(ve,db,date_start,params.ve,nsteps,...
    shock_uncertainty,Rfunc,conditions);

myfkst.ve_lr=forecast(ve_lr,db,date_start,params.ve_lr,nsteps,...
    shock_uncertainty,Rfunc,conditions);
```

(continues on next page)

(continued from previous page)

```
% set environment

ci=[30,50,68,90];

% Forecast plots

for jj=1:numel(modelnames)

    modname=modelnames{jj};

    figure('name',['model (',modname,') Forecasts of Norwegian Data']);

    for ii=1:numel(endog)

        subplot(3,2,ii)

        d=myfkst.(modname).(endog{ii});

        out=fanchart(d,ci);

        plot_fanchart(out,[244, 122, 66]/255)

        title(tex.(endog{ii}))

        axis tight

    end

    xrotate(45)

end
```

5.2.2 Adding regime switching with constant transition probabilities

```
%% adding constant transition probabilities

mc=struct();
mc.name='policy';
mc.number_of_states=2;
mc.controlled_parameters={'b(R,:)'};
mc.endogenous_probabilities=[];
mc.probability_parameters=[];

mdl = rfvar(endog, exog, nlags, const,mc);
```

```
%% Set priors and Estimate the VAR

switch_prior=struct();
switch_prior.policy_tp_1_2={0.5,0.1,0.3,'beta'};
switch_prior.policy_tp_2_1={0.5,0.1,0.3,'beta'};
```

(continues on next page)

(continued from previous page)

```
prior=struct();
prior.nonvar=switch_prior;

% ___Optional prior on the VAR__
% var_prior=svar.prior_template();
% var_prior.type='sz';
% prior.var=var_prior;

data_range={db.GROWTH.start,db.GROWTH.finish};

mdlest=estimate(mdl,db,data_range,prior);
```

5.2.3 Adding regime switching with time~varying transition probabilities

```
%% adding time-varying transition probabilities

mc=struct();
mc.name='policy';
mc.number_of_states=2;
mc.controlled_parameters={'b(R,:)'};
mc.endogenous_probabilities={
    'policy_tp_1_2=1/(1+exp(a12*(PAI-c12)))'
    'policy_tp_2_1=1/(1+exp(a21*(PAI-c21)))'
    };
mc.probability_parameters={'a12','a21','c12','c21'};

mdl = rfvar(endog, exog, nlags, const,mc);
```

```
%% Set priors and estimate the RFVAR

switch_prior=struct();
switch_prior.a12={0,0,1,'normal'};
switch_prior.a21={0,0,1,'normal'};
switch_prior.c12={1.05^.25,1.05^.25,0.05,'gamma'};
switch_prior.c21={1.05^.25,1.05^.25,0.05,'gamma'};

prior=struct();
prior.nonvar=switch_prior;

% ___Optional prior on the VAR__
% var_prior=svar.prior_template();
% var_prior.type='sz';
% prior.var=var_prior;

data_range={db.GROWTH.start,db.GROWTH.finish};

mdlest=estimate(mdl,db,data_range,prior);
```

5.3 Technical documentation for rfvar objects

5.3.1 The properties

5.3.1.1 endogenous

list of endogenous variables

Help for rfvar/endogenous is inherited from superclass ABSTVAR

5.3.1.2 exogenous

list of exogenous variables

Help for rfvar/exogenous is inherited from superclass ABSTVAR

5.3.1.3 parameters

list of parameters including var and non-var parameters

Help for rfvar/parameters is inherited from superclass ABSTVAR

5.3.1.4 nonvar_parameters

list of non-var parameters e.g. auxiliary params, transition probabilities, etc.

Help for rfvar/nonvar_parameters is inherited from superclass ABSTVAR

5.3.1.5 constant

boolean to specify whether the model has a constant term or not

Help for rfvar/constant is inherited from superclass ABSTVAR

5.3.1.6 debug

for debugging purposes

Help for rfvar/debug is inherited from superclass ABSTVAR

5.3.1.7 markov_chains

structure of markov chains

Help for rfvar/markov_chains is inherited from superclass ABSTVAR

5.3.1.8 is_time_varying_trans_prob

boolean to specify whether we are in presence of a time-varying transition probability model

Help for rfvar/is_time_varying_trans_prob is inherited from superclass ABSTVAR

5.3.1.9 is_switching

boolean specifying whether the model is switching or not

Help for rfvar/is_switching is inherited from superclass ABSTVAR

5.3.1.10 nlags

number of lags in the VAR

Help for rfvar/nlags is inherited from superclass ABSTVAR

5.3.1.11 nx

number of exogenous variables including the constant term

Help for rfvar/nx is inherited from superclass ABSTVAR

5.3.1.12 nvars

number of endogenous variables

Help for rfvar/nvars is inherited from superclass ABSTVAR

5.3.1.13 nparams

number of parameters

Help for rfvar/nparams is inherited from superclass ABSTVAR

5.3.1.14 nregs

number of regimes

Help for rfvar/nregs is inherited from superclass ABSTVAR

5.3.2 The methods

5.3.2.1 autocov

Compute the autocovariances (and the auto-correlation) of endogenous variables given the parameter values

```
[C,R,retcode] = autocov(self);
[C,R,retcode] = autocov(self, params);
[C,R,retcode] = autocov(self, params, max_periods);
```

Args:
self (var object): var object params (cell of struct): struct containing var model related parameters (default: []) max_periods (integer): maximum number of period to calculate auto-covariance (default: 5)

Returns:
:

- **C** [4-dimensional array]: auto-covariance where dimensions correspond to
  - 1,2: covariance
  - 3: time lags
  - 4: Number of parameters
- **R** [4-dimensional array]: auto-correlation with same dimensions

5.3.2.2 bootstrap

Bootstrap the parameter values resampling ε_t terms from the data sample

```
RepsRun = bootstrap(rfvar, boot)
```

Args:
rfvar (rfvar object): rfvar object boot (integer): number of bootstrap simulations (default: 1000)

Returns:
:

- **RepsRun** : bootstrapped values of the estimated parameters
  - dimension 1: parameter levels
  - dimension 2: different runs of bootstrap samples

Example:
For example, one can compute variance of the estimate of the parameters by computing:

```
RepsRun = bootstrap(rfvar, 10000);
var(RepsRun, [], 2)
```

5.3.2.3 codify

codify: transforms expressions such as a3(v1,v2,... into a3(2,3,... This can be used to facilitate the setting of priors on individual parameters of the VAR to estimate.

```
str=codify(self,str)

str=codify(self,str,doCode)
```

Args:

- **self** (var object): var object
- **str** (cellstr|string): string to transform. The string has one of the following forms:
  - a3(v1,v2,...: svar and proxy_svar parameters
  - b3(v1,v2,...: rfvar and prfvar parameters

  - c(v1,v2,... : deterministic terms
  - s(v1,v2,... : variance and covariance terms
- **doCode** (true|{false}):
  - if false, return expression with parentheses e.g. a3(2,3)
  - if true, return expression with underscores e.g. a3_2_3

Returns:

str : transformed string

Example:

```
str=codify(psv,'a2(FFR,ygap)=0')

str=codify(psv,'c(FFR,ygap,mpcoef,2)=0',true)

str=codify(psv,'s(FFR,ygap,mpcoef,2)=0',true)
```

Note:

- The function does not check whether the resulting parameter is actually a parameter in the model
- The function also codifies expressions and not just individual parameters

Help for rfvar/codify is inherited from superclass ABSTVAR

5.3.2.4 estimate

Estimates VAR parameters using frequentist/Bayesian interpretation

```
var = estimate(var);
var = estimate(var, data);
var = estimate(var, data, data_range);
var = estimate(var, data, data_range, prior);
var = estimate(var, data, data_range, prior, restrictions);
var = estimate(var, data, data_range, prior, restrictions,optimizer);
var = estimate(var, data, data_range, prior, restrictions,optimizer,is_fixed_
↪regime);
```

Args:

- **var** (var object): var object
- **data** (struct or ts obect): data for estimation
- **data_range** (serial): (optional) date_range
- **prior** (struct): priors for parameters
  - If no prior is given, maximum likehood estimators (frequentist) are set for parameters
  - With priors, posterior mode (Bayesian) values are set for parameters. There are three types of priors that can be applied individually or simultaneously. The prior structure can have fields :
    - **var** (struct) : (possibly modified) content of *prior template*

    * **nonvar** (struct): priors on individual parameters, which could be the VAR parameters, the transition probabilities or the parameters entering endogenous transition probabilities.
    * **endogenous** (function handle) : priors on the behavior of the system. e.g. a prior on some nonlinear function of the parameters.

- **restrictions** : linear and nonlinear restrictions on parameters.
- **optimizer** (char|function_handle|cell|{fmincon}) : optimization procedure. Used with optimization is required. e.g. markov switching. This can be the name of a standard matlab optimizer or RISE optimization routine or a user-defined optimization procedure available of the matlab search path. If the optimzer is provided as a cell, then the first element of the cell is the name of the optimizer or its handle and the remaining entries in the cell are options to the user-defined optimization routine. A user-defined optimization function must have the following syntax

```
[xfinal,ffinal,exitflag,H]=optimizer(fh,x0,lb,ub,options);
```

  That is, it accepts as inputs:

  - **fh**: the function to optimize
  - **x0**: a vector column of initial values of the parameters
  - **lb**: a vector column of lower bounds
  - **ub**: a vector column of upper bounds
  - **options**: a structure of options whose fields will be similar to matlab's optimset

  That is, it provides as outputs:

  - **xfinal**: the vector of final values
  - **ffinal**: the value of **fh** at **xfinal**
  - **exitflag**: a flag similar to the ones provided by matlab's optimization functions.
  - **H**: an estimate of the Hessian

- **is_fixed_regime** : (true|{false}): if true, the regimes are known in advance. In that case the data should contain time series for a variable called "hist_regimes"

Returns:

- **var** (var object): var object with parameters estimated based on data

See also:

- *posterior_mode*
- *identification*
- *prior template*

Help for rfvar/estimate is inherited from superclass ABSTVAR

5.3.2.5 filter

filte: Compute the log likelihood of the VAR for the given parameters

```
[LogLik,Incr,retcode,filtering] = filter(self);

[LogLik,Incr,retcode,filtering] = filter(self, param);
```

Args:

self (var object): var object

param (cell of struct): (optional) parameter values

Returns:

LogLik : log likelihood values of the VAR

Incr : period-by-period contributions to the likelihood

retcode : flag, 0 if no problem

filtering : structure containing filtration

Note:

Almost everything is automated in RISE, so see `estimate()` or `identification()` .

Help for rfvar/filter is inherited from superclass ABSTVAR

5.3.2.6 forecast

rfvar/forecast is a function.
[fkst, retcode] = forecast(self, histdb, date_start, params, nsteps, shock_uncertainty, Rfunc, conditions)

5.3.2.7 get

get Access/Query time series property values.

VALUE = get(TS,'PropertyName') returns the value of the specified property of the time series object. An equivalent syntax is

VALUE = TS.PropertyName

get(TS) displays all properties of TS and their values.

5.3.2.8 historical_decomposition

varargin={shock_names}

5.3.2.9 identification

Parses identifying restrictions into RISE interpretable identification function

Note:
This function is a part of a general workflow. See XXXXXXXXXX to check the workflow.

```
Rfunc = identification(var, restr, shock_names, varargin);
```

Args:

- var (rfvar object): rfvar object
- restr : restrictions can take two forms
  - **'choleski'**: choleski identification. Supply the ordering as a cell array as varargin{1}
  - **cellstr: n x 2 cell array, with n the number of restrictions.**
    A typical restriction takes the form: - 'vbl@shk','sign' - 'vbl{lags}@shk','sign' where sign = +|- where shk is the name of the shock where vbl is the name of the endogenous variable affected by shock lags with defaut 0 is the list of lags at which the shock affects the variable with a 'sign' sign. lags is any valid expression that matlab can evaluate such as 0, 0:3, [1,4,5], [1,inf,2,5] or more generally a:b, [a:b,c,d], etc.

shock_names (cellstr): cell of structural shock names

Returns:

- **Rfunc** (function handle): a function that takes parameters and then returns structural shock matrix. This function function is not intended to be used used on its own, and supposed to be used as an argument in different functions.

See also:

- *autocov*
- *forecast*
- *historical_decomposition*
- *irf*
- *variance_decomposition*

5.3.2.10 irf

Compute impulse response function from the given parameter values

```
myirfs = irf(self)

myirfs = irf(self, shock_names)

myirfs = irf(self, shock_names, irf_periods)

myirfs = irf(self, shock_names, irf_periods, params)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc,girf_setup)
```

Args:

- **self** (var object): var object
- **shock_names** (cellstr): shocks for which to compute IRFs (has to be consistent with the identification scheme *identification*)
- **irf_periods** (int): number of periods to compute IRFs (default: 40)
- **params** (vector): parameter values of the model. If empty MLE/posterior-mode values are used.
- **Rfunc** (function handle): identification function. This is an output of `identification`. (default: choleski identification)
- **girf_setup** (struct|{empty}): structure containing information relevant for the computation of generalized impulse response functions. If empty, simple regime-specific impulse responses are computed, else girfs are computed. In that case the relevant information to provide in girf_setup is:
  - **nsims** : (default=300) number of simulations for the integration. Note that even setting girf_setup=struct() will trigger the computation of girfs. But in that case only the default options will apply.

Returns:

- **myirfs** : (struct) structure containing the IRFs

Note: Only successful IRFs are returned. If the structure does not return some IRFs make sure that IRFs properly identified.

5.3.2.11 posterior_mode

Return the posterior mode from the BVAR estimation

```
dd = posterior_mode(self)
```

Args:
self (var object): var object

Returns:
:

- **dd** (struct): posterior mode of the parameters

Help for rfvar/posterior_mode is inherited from superclass ABSTVAR

5.3.2.12 print_solution

rfvar/print_solution is a function.
print_solution(obj, varlist)

5.3.2.13 prior_template

prior_template : provides the default settings for priors in Bayesian VARs

```
prior=prior_template()
```

Args : None

Returns:

- prior (struct) : structure with fields :
  - **L1** (numeric|{0.1}) : Overall tightness
  - **L2** (numeric|{0.5}) : Cross-variable specific variance parameter
  - **L3** (numeric|{1}) : Speed at which lags greater than 1 converge to zero
  - **L4** (numeric|{100}) : tightness on deterministic/exogenous terms
  - **L5** (numeric|{3}) : covariance dummies(omega)
  - **L6** (numeric|{5}) : co-persistence (lambda)
  - **L7** (numeric|{2}) : Own-persistence (mu)
  - **coefprior** (numeric scalar|vector|{1}): mean of coefficients on own first lag. This allows you to set the prior mean to whatever value you believe in. e.g. one might believe that, say, the growth rate is stationary and so is less persistent than say, the interest rate or inflation.
  - **type** (string): type of prior. one of the following:
    - 'minnesota' : Minnesota prior
    - 'normal-wishart' or 'nw' : Normal Wishart prior
    - 'indep-normal-wishart' or 'inw' : Independent normal wishart prior
    - 'jeffrey' : Jeffrey prior
    - 'sims-zha' or 'sz': Sims and Zha prior
  - **normal_wishart_eta** (numeric|{0.01}) : Prior variance for the normal-wishart prior
  - **independent_normal_wishart_eta** (numeric|{0.01}) : prior variance for the independent-normal-wishart prior

> **Warning:** At the moment only the Sims-Zha and (possibly) the Jeffrey prior work. The other priors need some updating.

Help for rfvar.prior_template is inherited from superclass ABSTVAR

5.3.2.14 pull_objective

Pulls the objective function to optimize

```
[ff,lb,ub]=pull_objective(self)
[ff,lb,ub,x0]=pull_objective(self)
[ff,lb,ub,x0,vcov]=pull_objective(self)
[ff,lb,ub,x0,vcov,self]=pull_objective(self)
```

Args:

self (svar | rfvar): initial model object

Returns:

:

- **ff** [function handle]: for computing "minus log posterior kernel"
- **lb** [d x 1 vector]: lower bound of the parameters to optimize
- **ub** [d x 1 vector]: upper bound of the parameters to optimize
- **x0** [d x 1 vector]: posterior mode if available
- **vcov** [d x d matrix]: covariance matrix at the posterior mode if available.
- **self** [rise|dsge|svar|rfvar]: updated model object

Note:

- The function can be used for :
  - optimization,
  - gradient computation,
  - hessian computation,
  - posterior simulation
- The updated object should be used for doing various exercises (irfs, simulations, etc.) if the posterior mode is not computed.
- Using this function is potentially costly, one could alternatively simply use log_posterior_kernel. However, if there are restrictions, they will not be enforced. Nevertheless it is an interesting proposition that should be investigated further.

Help for rfvar/pull_objective is inherited from superclass ABSTVAR

5.3.2.15 residuals

Computes the residuals given the parameters

Help for rfvar/residuals is inherited from superclass ABSTVAR

5.3.2.16 rfvar

Constructor for rfvar object

```
mdl = rfvar(endog)

mdl = rfvar(endog, exog)

mdl = rfvar(endog, exog, nlags)

mdl = rfvar(endog, exog, nlags, const)

mdl = rfvar(endog, exog, nlags, const, markov_chains)
```

Args:

- **endog** (cellstring): cell of endogenous variable names
- **exog** (cellstring): cell of exogenous variable names (default: [])
- **nlags** (integer): number of VAR lags to include (default: 4)
- **const** (bool): whether to include constants (default: true)
- **markov_chains** (struct): [optional] Struct containing the following fields:
  - **'name'** (string): name of the markov process
  - **'number_of_states'** (integer): number of states
  - **'controlled_parameters'** (cellstr): list of controlled parameters
  - **'endogenous_probabilities'** (cellstr): definitions of the endogenous probabilities
  - **'probability_parameters'** (cellstr): parameters entering the time-varying transition probabilities

Returns:

- **mdl** : constructed rfvar object

Note: For the controlled parameters

- The dynamic parameters of a rfvar model are denoted by "b"
- The static parameters or coefficients on exogenous by "c"
- for some parameter bd(row,col), e.g. b2(2,3)
  - d (empty|integer) : when empty, all lags are swept through
  - row (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable
  - col (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable

Documentation for rfvar/rfvar
doc rfvar

5.3.2.17 set

Sets options for the VAR XXXXXXXXXXX need to check

```
var_obj = set(var_obj,varargin);
```

Args:

var_obj (var object): VAR object varargin: Options to set. Must come in pairs

- **'data'**: Data for the VAR. Can be either a struct or ts object
- **'prior'**: Prior to use if BVAR. Available priors are
  - minnesota
  - jeffrey
  - nw
  - normal-wishart
  - inw
  - sz
- **'linear_restrictions'**: Linear restrictions. For the format, see XXXXXXXXX

Returns:

:

- var_obj (var object): New VAR with the given inputs updated.

Help for rfvar/set is inherited from superclass ABSTVAR

5.3.2.18 solve

solve – Solves the VAR model.

This function is used to solve a VAR (Vector Autoregressive) model. It takes model parameters, data, and additional inputs, and returns the solution *sol*, a return code *retcode*, and a matrix *M*.

Syntax:

[sol, retcode, M] = solve(self, param, yt)

Inputs:

- *self* [VARModel]: The VAR model object.
- *param* [empty|struct|matrix]: Model parameters. If it is a matrix, then each column corresponds to a particular parameterization. If it is a structure, then it is the solution. If it is empty, the returned solution is computed used the parameter values stored inside the model object.
- *yt* [matrix]: Data needed for endogenous probabilities. If not provided, the first observation from the model's data is used.

Outputs:

- *sol* [struct]: The solution to the VAR model.
- *retcode* [int]: A return code indicating the success of the solution.
- *M* [matrix]: Parameters for each regime

Example: - Solve the VAR model using provided parameters *param* and data *yt*:

`matlab [sol, retcode, M] = solve(varModel, param, yt); `

See also:

vartools.solve.

Help for rfvar/solve is inherited from superclass ABSTVAR

5.3.2.19 structural_shocks

rfvar/structural_shocks is a function.
[s, retcode] = structural_shocks(self, params, Rfunc, shock_names)

5.3.2.20 transform_var_into_dsge

transform_var_into_dsge : transforms a parameterized VAR into a DSGE model.

SYNTAX

m=transform_var_into_dsge(v)

m=transform_var_into_dsge(v,exogenousValues)

INPUTS

- **v** [rfvar|svar|proxy_var|prfvar]: parameterized VAR object
- **exogenousValues** [empty|struct]: structure containing the values for the exogenous variables. This is because the DSGE does not contain exogenous variables. Therefore the values of the exogenous variables has to be fixed.

OUTPUTS

- **m** [rise]: parameterized DSGE model

NB : At the moment, the routine does not support regime switching models. The feature will be implemented if there is a demand for it.

Help for rfvar/transform_var_into_dsge is inherited from superclass ABSTVAR

5.3.2.21 variance_decomposition

Compute the variance decompsition of the VAR

```
[vd,retcode] = variance_decomposition(self)

[vd,retcode] = variance_decomposition(self,params)

[vd,retcode] = variance_decomposition(self,params,Rfunc)

[vd,retcode] = variance_decomposition(self,params,Rfunc,nperiods)
```

Args:

- self (var object):
- params :(optional) parameter values
- Rfunc (function handle): (optional) transform parameters into dynamics
- nperiods :(optional) periods for decomposition

Returns:

- **vd** (struct): a struct containing the variance decomposition:
  - **infinity** (struct):
  - **conditional** (struct):
- **retcode** (integer): This passes through the retcode given by Rfunc. Currently (2018/07/19), this output always returns 0.

CHAPTER

SIX

PANEL REDUCED-FORM VAR MODELING

6.1 Description

$$\begin{bmatrix} y_{1t} \\ y_{2t} \\ \vdots \\ y_{nt} \end{bmatrix} = C\left(r_t\right) \begin{bmatrix} x_{1t} \\ x_{2t} \\ \vdots \\ x_{nt} \end{bmatrix} + B_1\left(r_t\right) \begin{bmatrix} y_{1t-1} \\ y_{2t-1} \\ \vdots \\ y_{nt-1} \end{bmatrix} + ... + B_p\left(r_t\right) \begin{bmatrix} y_{1t-p} \\ y_{2t-p} \\ \vdots \\ y_{nt-p} \end{bmatrix} + u_t$$

With $r_t = 1, 2, ..., h$ and transition probabilities $p_{r_t, r_{t+1}}\left(I_t\right)$

6.2 Quick-start examples

6.2.1 constant

6.2.2 constant transition probabilities

6.2.3 time~varying transition probabilities

6.3 Estimation

6.4 Simulation

6.5 Forecasting and conditional forecasting

6.6 Technical documentation for pvar objects

6.6.1 The properties

6.6.1.1 members

members of the panel

6.6.1.2 homogeneity

type of homogeneity of the panel (see list)

6.6.1.3 endogenous

list of endogenous variables

Help for prfvar/endogenous is inherited from superclass ABSTVAR

6.6.1.4 exogenous

list of exogenous variables

Help for prfvar/exogenous is inherited from superclass ABSTVAR

6.6.1.5 parameters

list of parameters including var and non-var parameters

Help for prfvar/parameters is inherited from superclass ABSTVAR

6.6.1.6 nonvar_parameters

list of non-var parameters e.g. auxiliary params, transition probabilities, etc.

Help for prfvar/nonvar_parameters is inherited from superclass ABSTVAR

6.6.1.7 constant

boolean to specify whether the model has a constant term or not

Help for prfvar/constant is inherited from superclass ABSTVAR

6.6.1.8 debug

for debugging purposes

Help for prfvar/debug is inherited from superclass ABSTVAR

6.6.1.9 markov_chains

structure of markov chains

Help for prfvar/markov_chains is inherited from superclass ABSTVAR

6.6.1.10 is_time_varying_trans_prob

boolean to specify whether we are in presence of a time-varying transition probability model

Help for prfvar/is_time_varying_trans_prob is inherited from superclass ABSTVAR

6.6.1.11 is_switching

boolean specifying whether the model is switching or not

Help for prfvar/is_switching is inherited from superclass ABSTVAR

6.6.1.12 nlags

number of lags in the VAR

Help for prfvar/nlags is inherited from superclass ABSTVAR

6.6.1.13 nx

number of exogenous variables including the constant term

Help for prfvar/nx is inherited from superclass ABSTVAR

6.6.1.14 nvars

number of endogenous variables

Help for prfvar/nvars is inherited from superclass ABSTVAR

6.6.1.15 nparams

number of parameters

Help for prfvar/nparams is inherited from superclass ABSTVAR

6.6.1.16 nregs

number of regimes

Help for prfvar/nregs is inherited from superclass ABSTVAR

6.6.2 The methods

6.6.2.1 autocov

Compute the autocovariances (and the auto-correlation) of endogenous variables given the parameter values

```
[C,R,retcode] = autocov(self);
[C,R,retcode] = autocov(self, params);
[C,R,retcode] = autocov(self, params, max_periods);
```

Args:
self (var object): var object params (cell of struct): struct containing var model related parameters (default: []) max_periods (integer): maximum number of period to calculate auto-covariance (default: 5)

Returns:
:

- **C** [4-dimensional array]: auto-covariance where dimensions correspond to
  - 1,2: covariance
  - 3: time lags
  - 4: Number of parameters
- **R** [4-dimensional array]: auto-correlation with same dimensions

6.6.2.2 bootstrap

Bootstrap the parameter values resampling ε_t terms from the data sample

```
RepsRun = bootstrap(rfvar, boot)
```

Args:
rfvar (rfvar object): rfvar object boot (integer): number of bootstrap simulations (default: 1000)

Returns:
:

- **RepsRun** : bootstrapped values of the estimated parameters
  - dimension 1: parameter levels
  - dimension 2: different runs of bootstrap samples

Example:
For example, one can compute variance of the estimate of the parameters by computing:

```
RepsRun = bootstrap(rfvar, 10000);
var(RepsRun, [], 2)
```

Help for prfvar/bootstrap is inherited from superclass RFVAR

6.6.2.3 codify

codify: transforms expressions such as a3(v1,v2,... into a3(2,3,... This can be used to facilitate the setting of priors on individual parameters of the VAR to estimate.

```
str=codify(self,str)

str=codify(self,str,doCode)
```

Args:

- **self** (var object): var object
- **str** (cellstr|string): string to transform. The string has one of the following forms:
  - a3(v1,v2,...: svar and proxy_svar parameters

  - b3(v1,v2,...: rfvar and prfvar parameters
  - c(v1,v2,... : deterministic terms
  - s(v1,v2,... : variance and covariance terms
- **doCode** (true|{false}):
  - if false, return expression with parentheses e.g. a3(2,3)
  - if true, return expression with underscores e.g. a3_2_3

Returns:

str : transformed string

Example:

```
str=codify(psv,'a2(FFR,ygap)=0')

str=codify(psv,'c(FFR,ygap,mpcoef,2)=0',true)

str=codify(psv,'s(FFR,ygap,mpcoef,2)=0',true)
```

Note:

- The function does not check whether the resulting parameter is actually a parameter in the model
- The function also codifies expressions and not just individual parameters

Help for prfvar/codify is inherited from superclass ABSTVAR

6.6.2.4 estimate

Estimates VAR parameters using frequentist/Bayesian interpretation

```
var = estimate(var);
var = estimate(var, data);
var = estimate(var, data, data_range);
var = estimate(var, data, data_range, prior);
var = estimate(var, data, data_range, prior, restrictions);
var = estimate(var, data, data_range, prior, restrictions,optimizer);
var = estimate(var, data, data_range, prior, restrictions,optimizer,is_fixed_
↪regime);
```

Args:

- **var** (var object): var object
- **data** (struct or ts obect): data for estimation
- **data_range** (serial): (optional) date_range
- **prior** (struct): priors for parameters
  - If no prior is given, maximum likehood estimators (frequentist) are set for parameters
  - With priors, posterior mode (Bayesian) values are set for parameters. There are three types of priors that can be applied individually or simultaneously. The prior structure can have fields :
    - **var** (struct) : (possibly modified) content of *prior template*

    * **nonvar** (struct): priors on individual parameters, which could be the VAR parameters, the transition probabilities or the parameters entering endogenous transition probabilities.
    * **endogenous** (function handle) : priors on the behavior of the system. e.g. a prior on some nonlinear function of the parameters.

- **restrictions** : linear and nonlinear restrictions on parameters.
- **optimizer** (char|function_handle|cell|{fmincon}) : optimization procedure. Used with optimization is required. e.g. markov switching. This can be the name of a standard matlab optimizer or RISE optimization routine or a user-defined optimization procedure available of the matlab search path. If the optimzer is provided as a cell, then the first element of the cell is the name of the optimizer or its handle and the remaining entries in the cell are options to the user-defined optimization routine. A user-defined optimization function must have the following syntax

```
[xfinal,ffinal,exitflag,H]=optimizer(fh,x0,lb,ub,options);
```

  That is, it accepts as inputs:

  - **fh**: the function to optimize
  - **x0**: a vector column of initial values of the parameters
  - **lb**: a vector column of lower bounds
  - **ub**: a vector column of upper bounds
  - **options**: a structure of options whose fields will be similar to matlab's optimset

  That is, it provides as outputs:

  - **xfinal**: the vector of final values
  - **ffinal**: the value of **fh** at **xfinal**
  - **exitflag**: a flag similar to the ones provided by matlab's optimization functions.
  - **H**: an estimate of the Hessian

- **is_fixed_regime** : (true|{false}): if true, the regimes are known in advance. In that case the data should contain time series for a variable called "hist_regimes"

Returns:

- **var** (var object): var object with parameters estimated based on data

See also:

- *posterior_mode*
- *identification*
- *prior template*

Help for prfvar/estimate is inherited from superclass ABSTVAR

6.6.2.5 filter

filte: Compute the log likelihood of the VAR for the given parameters

```
[LogLik,Incr,retcode,filtering] = filter(self);

[LogLik,Incr,retcode,filtering] = filter(self, param);
```

Args:

self (var object): var object

param (cell of struct): (optional) parameter values

Returns:

LogLik : log likelihood values of the VAR

Incr : period-by-period contributions to the likelihood

retcode : flag, 0 if no problem

filtering : structure containing filtration

Note:

Almost everything is automated in RISE, so see `estimate()` or `identification()` .

Help for prfvar/filter is inherited from superclass ABSTVAR

6.6.2.6 forecast

prfvar/forecast is a function.
fkst = forecast(self, histdb, varargin)

6.6.2.7 get

get Access/Query time series property values.

VALUE = get(TS,'PropertyName') returns the value of the specified property of the time series object. An equivalent syntax is

VALUE = TS.PropertyName

get(TS) displays all properties of TS and their values.

6.6.2.8 historical_decomposition

varargin={shock_names}

6.6.2.9 identification

Parses panel VAR identifying restrictions into RISE interpretable identification function

Note:
This function is a part of a general workflow. See XXXXXXXXXX to check the workflow.

```
Rfunc = identification(var, restr, shock_names, varargin);
```

Args:

- var (rfvar object): rfvar object
- restr : restrictions can take two forms
  - **'choleski'**: choleski identification. Supply the ordering as a cell array as varargin{1}
  - **cellstr**: n x 2 cell array, with n the number of restrictions. A typical restriction takes the form:
    - 'vbl@shk','sign'
    - 'vbl{lags}@shk','sign'

    where sign = +|- where shk is the name of the shock where vbl is the name of the endogenous variable affected by shock lags with defaut 0 is the list of lags at which the shock affects the variable with a 'sign' sign. lags is any valid expression that matlab can evaluate such as 0, 0:3, [1,4,5], [1,inf,2,5] or more generally a:b, [a:b,c,d], etc.

shock_names (cellstr): cell of structural shock names

Returns:
:

- **Rfunc** (function handle): a function that takes parameters and then returns structural shock matrix. This function function is not intended to be used used on its own, and supposed to be used as an argument in different functions.

See also:

- `autocov`
- `forecast`
- `historical_decomposition`
- `irf`
- `variance_decomposition`

6.6.2.10 irf

Compute impulse response function from the given parameter values

```
myirfs = irf(self)

myirfs = irf(self, shock_names)

myirfs = irf(self, shock_names, irf_periods)
```

(continues on next page)

(continued from previous page)

```
myirfs = irf(self, shock_names, irf_periods, params)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc)

myirfs = irf(self, shock_names, irf_periods, params, Rfunc,girf_setup)
```

Args:

- **self** (var object): var object
- **shock_names** (cellstr): shocks for which to compute IRFs (has to be consistent with the identification scheme *identification*)
- **irf_periods** (int): number of periods to compute IRFs (default: 40)
- **params** (vector): parameter values of the model. If empty MLE/posterior-mode values are used.
- **Rfunc** (function handle): identification function. This is an output of `identification`. (default: choleski identification)
- **girf_setup** (struct|{empty}): structure containing information relevant for the computation of generalized impulse response functions. If empty, simple regime-specific impulse responses are computed, else girfs are computed. In that case the relevant information to provide in girf_setup is:
  - **nsims** : (default=300) number of simulations for the integration. Note that even setting girf_setup=struct() will trigger the computation of girfs. But in that case only the default options will apply.

Returns:

- **myirfs** : (struct) structure containing the IRFs

Note: Only successful IRFs are returned. If the structure does not return some IRFs make sure that IRFs properly identified.

6.6.2.11 posterior_mode

Return the posterior mode from the BVAR estimation

```
dd = posterior_mode(self)
```

Args:
self (var object): var object

Returns:
:

- **dd** (struct): posterior mode of the parameters

Help for prfvar/posterior_mode is inherited from superclass ABSTVAR

6.6.2.12 prfvar

Constructor for rfvar object

```
mdl = prfvar(panel,endog);

mdl = prfvar(panel,endog, exog);

mdl = prfvar(panel,endog, exog, nlags);

mdl = prfvar(panel,endog, exog, nlags, const);

mdl = prfvar(panel,endog, exog, nlags, const, markov_chains);
```

Args:

- **panel** (struct): [optional] a struct of panel information including:
  - **members** (cellstring): cell of country names
  - **homogeneity** (string): assumptions made on the panel dimension. Available homogeneities are:
    - **'pooled'**: all countries are assumed to have the same coefficients
    - **'meanGroup'**: average across groups
    - **'independent'**: nothing in common
    - **'static'**: deterministic coefficients common
    - **'dynamic'**: lag coefficients common
- **endog** (cellstring): cell of endogenous variable names
- **exog** (cellstring): cell of exogenous variable names (default: [])
- **nlags** (integer): number of VAR lags to include (default: 4)
- **const** (bool): whether to include constants (default: true)
- **markov_chains** (struct): [optional] Struct containing the following fields:
  - **'name'**: name of the markov process
  - **'number_of_states'**: number of states
  - **'controlled_parameters'**: list of controlled parameters
  - **'endogenous_probabilities'**: definitions of the endogenous probabilities
  - **'probability_parameters'**: parameters entering the time-varying transition probabilities

Returns:

- **mdl** : constructed prfvar object

Note: For the controlled parameters

- The dynamic parameters of a rfvar model are denoted by "b"
- The static parameters or coefficients on exogenous by "c"
- for some parameter bd(row,col), e.g. b2(2,3)
  - d (empty|integer) : when empty, all lags are swept through

- row (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable
- col (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable

Documentation for prfvar/prfvar
doc prfvar

6.6.2.13 print_solution

prfvar/print_solution is a function.
print_solution(obj, varlist)

6.6.2.14 prior_template

prior_template : provides the default settings for priors in Bayesian VARs

```
prior=prior_template()
```

Args : None

Returns:

- prior (struct) : structure with fields :
  - **L1** (numeric|{0.1}) : Overall tightness
  - **L2** (numeric|{0.5}) : Cross-variable specific variance parameter
  - **L3** (numeric|{1}) : Speed at which lags greater than 1 converge to zero
  - **L4** (numeric|{100}) : tightness on deterministic/exogenous terms
  - **L5** (numeric|{3}) : covariance dummies(omega)
  - **L6** (numeric|{5}) : co-persistence (lambda)
  - **L7** (numeric|{2}) : Own-persistence (mu)
  - **coefprior** (numeric scalar|vector|{1}): mean of coefficients on own first lag. This allows you to set the prior mean to whatever value you believe in. e.g. one might believe that, say, the growth rate is stationary and so is less persistent than say, the interest rate or inflation.
  - **type** (string): type of prior. one of the following:
    - ‘minnesota’ : Minnesota prior
    - ‘normal-wishart’ or ‘nw’ : Normal Wishart prior
    - ‘indep-normal-wishart’ or ‘inw’ : Independent normal wishart prior
    - ‘jeffrey’ : Jeffrey prior
    - ‘sims-zha’ or ‘sz’: Sims and Zha prior
  - **normal_wishart_eta** (numeric|{0.01}) : Prior variance for the normal-wishart prior
  - **independent_normal_wishart_eta** (numeric|{0.01}) : prior variance for the independent-normal-wishart prior

> **Warning:** At the moment only the Sims-Zha and (possibly) the Jeffrey prior work. The other priors need some updating.

Help for prfvar.prior_template is inherited from superclass ABSTVAR

6.6.2.15 pull_objective

Pulls the objective function to optimize

```
[ff,lb,ub]=pull_objective(self)
[ff,lb,ub,x0]=pull_objective(self)
[ff,lb,ub,x0,vcov]=pull_objective(self)
[ff,lb,ub,x0,vcov,self]=pull_objective(self)
```

Args:

self (svar | rfvar): initial model object

Returns:

:

- **ff** [function handle]: for computing "minus log posterior kernel"
- **lb** [d x 1 vector]: lower bound of the parameters to optimize
- **ub** [d x 1 vector]: upper bound of the parameters to optimize
- **x0** [d x 1 vector]: posterior mode if available
- **vcov** [d x d matrix]: covariance matrix at the posterior mode if available.
- **self** [rise|dsge|svar|rfvar]: updated model object

Note:

- The function can be used for :
  - optimization,
  - gradient computation,
  - hessian computation,
  - posterior simulation
- The updated object should be used for doing various exercises (irfs, simulations, etc.) if the posterior mode is not computed.
- Using this function is potentially costly, one could alternatively simply use log_posterior_kernel. However, if there are restrictions, they will not be enforced. Nevertheless it is an interesting proposition that should be investigated further.

Help for prfvar/pull_objective is inherited from superclass ABSTVAR

6.6.2.16 residuals

Computes the residuals given the parameters

6.6.2.17 set

Sets options for the VAR XXXXXXXXXXX need to check

```
var_obj = set(var_obj,varargin);
```

Args:
var_obj (var object): VAR object varargin: Options to set. Must come in pairs

- **'data'**: Data for the VAR. Can be either a struct or ts object
- **'prior'**: Prior to use if BVAR. Available priors are
  - minnesota
  - jeffrey
  - nw
  - normal-wishart
  - inw
  - sz
- **'linear_restrictions'**: Linear restrictions. For the format, see XXXXXXXXX

Returns:
:

- var_obj (var object): New VAR with the given inputs updated.

Help for prfvar/set is inherited from superclass ABSTVAR

6.6.2.18 solve

INTERNAL FUNCTION: Solves for the consistent parameters for the VAR

6.6.2.19 structural_shocks

prfvar/structural_shocks is a function.
[s, retcode] = structural_shocks(self, params, Rfunc, shock_names)

6.6.2.20 transform_var_into_dsge

transform_var_into_dsge : transforms a parameterized VAR into a DSGE model.

SYNTAX

m=transform_var_into_dsge(v)

m=transform_var_into_dsge(v,exogenousValues)

INPUTS

- **v** [rfvar|svar|proxy_var|prfvar]: parameterized VAR object

- **exogenousValues** [empty|struct]: structure containing the values for the exogenous variables. This is because the DSGE does not contain exogenous variables. Therefore the values of the exogenous variables has to be fixed.

OUTPUTS

- **m** [rise]: parameterized DSGE model

NB : At the moment, the routine does not support regime switching models. The feature will be implemented if there is a demand for it.

Help for prfvar/transform_var_into_dsge is inherited from superclass ABSTVAR

6.6.2.21 variance_decomposition

Compute the variance decompsition of the VAR

```
[vd,retcode] = variance_decomposition(self)

[vd,retcode] = variance_decomposition(self,params)

[vd,retcode] = variance_decomposition(self,params,Rfunc)

[vd,retcode] = variance_decomposition(self,params,Rfunc,nperiods)
```

Args:

- self (var object):
- params :(optional) parameter values
- Rfunc (function handle): (optional) transform parameters into dynamics
- nperiods :(optional) periods for decomposition

Returns:

- **vd** (struct): a struct containing the variance decomposition:
  - **infinity** (struct):
  - **conditional** (struct):
- **retcode** (integer): This passes through the retcode given by Rfunc. Currently (2018/07/19), this output always returns 0.

CHAPTER

SEVEN

STRUCTURAL VAR MODELING

7.1 Description

$$A_0(r_t)\,y_t = C(r_t)\,x_t + A_1(r_t)\,y_{t-1} + ... + A_p(r_t)\,y_{t-p} + \varepsilon_t$$

With $r_t = 1, 2, ..., h$ and transition probabilities $p_{r_t, r_{t+1}}(I_t)$

7.2 Quick-start examples

7.2.1 constant

7.2.2 constant transition probabilities

7.2.3 time-varying transition probabilities

7.3 Estimation

7.4 Simulation

7.5 Forecasting and conditional forecasting

7.6 Technical documentation for svar objects

7.6.1 The properties

7.6.1.1 endogenous

list of endogenous variables

Help for svar/endogenous is inherited from superclass ABSTVAR

7.6.1.2 exogenous

list of exogenous variables

Help for svar/exogenous is inherited from superclass ABSTVAR

7.6.1.3 parameters

list of parameters including var and non-var parameters

Help for svar/parameters is inherited from superclass ABSTVAR

7.6.1.4 nonvar_parameters

list of non-var parameters e.g. auxiliary params, transition probabilities, etc.

Help for svar/nonvar_parameters is inherited from superclass ABSTVAR

7.6.1.5 constant

boolean to specify whether the model has a constant term or not

Help for svar/constant is inherited from superclass ABSTVAR

7.6.1.6 debug

for debugging purposes

Help for svar/debug is inherited from superclass ABSTVAR

7.6.1.7 markov_chains

structure of markov chains

Help for svar/markov_chains is inherited from superclass ABSTVAR

7.6.1.8 is_time_varying_trans_prob

boolean to specify whether we are in presence of a time-varying transition probability model

Help for svar/is_time_varying_trans_prob is inherited from superclass ABSTVAR

7.6.1.9 is_switching

boolean specifying whether the model is switching or not

Help for svar/is_switching is inherited from superclass ABSTVAR

7.6.1.10 nlags

number of lags in the VAR

Help for svar/nlags is inherited from superclass ABSTVAR

7.6.1.11 nx

number of exogenous variables including the constant term

Help for svar/nx is inherited from superclass ABSTVAR

7.6.1.12 nvars

number of endogenous variables

Help for svar/nvars is inherited from superclass ABSTVAR

7.6.1.13 nparams

number of parameters

Help for svar/nparams is inherited from superclass ABSTVAR

7.6.1.14 nregs

number of regimes

Help for svar/nregs is inherited from superclass ABSTVAR

7.6.2 The methods

7.6.2.1 autocov

Compute the autocovariances (and the auto-correlation) of endogenous variables given the parameter values

```
[C,R,retcode] = autocov(self);
[C,R,retcode] = autocov(self, params);
[C,R,retcode] = autocov(self, params, max_periods);
```

Args:

self (var object): var object params (cell of struct): struct containing var model related parameters (default: []) max_periods (integer): maximum number of period to calculate auto-covariance (default: 5)

Returns:

:

- **C** [4-dimensional array]: auto-covariance where dimensions correspond to
  - 1,2: covariance
  - 3: time lags

  - 4: Number of parameters
- **R** [4-dimensional array]: auto-correlation with same dimensions

7.6.2.2 codify

codify: transforms expressions such as a3(v1,v2,... into a3(2,3,... This can be used to facilitate the setting of priors on individual parameters of the VAR to estimate.

```
str=codify(self,str)

str=codify(self,str,doCode)
```

Args:

- **self** (var object): var object
- **str** (cellstr|string): string to transform. The string has one of the following forms:
  - a3(v1,v2,...: svar and proxy_svar parameters
  - b3(v1,v2,...: rfvar and prfvar parameters
  - c(v1,v2,... : deterministic terms
  - s(v1,v2,... : variance and covariance terms
- **doCode** (true|{false}):
  - if false, return expression with parentheses e.g. a3(2,3)
  - if true, return expression with underscores e.g. a3_2_3

Returns:

str : transformed string

Example:

```
str=codify(psv,'a2(FFR,ygap)=0')

str=codify(psv,'c(FFR,ygap,mpcoef,2)=0',true)

str=codify(psv,'s(FFR,ygap,mpcoef,2)=0',true)
```

Note:

- The function does not check whether the resulting parameter is actually a parameter in the model
- The function also codifies expressions and not just individual parameters

Help for svar/codify is inherited from superclass ABSTVAR

7.6.2.3 estimate

Estimates VAR parameters using frequentist/Bayesian interpretation

```
var = estimate(var);
var = estimate(var, data);
var = estimate(var, data, data_range);
var = estimate(var, data, data_range, prior);
var = estimate(var, data, data_range, prior, restrictions);
var = estimate(var, data, data_range, prior, restrictions,optimizer);
var = estimate(var, data, data_range, prior, restrictions,optimizer,is_fixed_
↪regime);
```

Args:

- **var** (var object): var object
- **data** (struct or ts obect): data for estimation
- **data_range** (serial): (optional) date_range
- **prior** (struct): priors for parameters
  - If no prior is given, maximum likehood estimators (frequentist) are set for parameters
  - With priors, posterior mode (Bayesian) values are set for parameters. There are three types of priors that can be applied individually or simultaneously. The prior structure can have fields :
    - **var** (struct) : (possibly modified) content of *prior template*
    - **nonvar** (struct): priors on individual parameters, which could be the VAR parameters, the transition probabilities or the parameters entering endogenous transition probabilities.
    - **endogenous** (function handle) : priors on the behavior of the system. e.g. a prior on some nonlinear function of the parameters.
- **restrictions** : linear and nonlinear restrictions on parameters.
- **optimizer** (char|function_handle|cell|{fmincon}) : optimization procedure. Used with optimization is required. e.g. markov switching. This can be the name of a standard matlab optimizer or RISE optimization routine or a user-defined optimization procedure available of the matlab search path. If the optimzer is provided as a cell, then the first element of the cell is the name of the optimizer or its handle and the remaining entries in the cell are options to the user-defined optimization routine. A user-defined optimization function must have the following syntax

```
[xfinal,ffinal,exitflag,H]=optimizer(fh,x0,lb,ub,options);
```

That is, it accepts as inputs:

- **fh**: the function to optimize
- **x0**: a vector column of initial values of the parameters
- **lb**: a vector column of lower bounds
- **ub**: a vector column of upper bounds
- **options**: a structure of options whose fields will be similar to matlab's optimset

That is, it provides as outputs:

- **xfinal**: the vector of final values
- **ffinal**: the value of **fh** at **xfinal**

  - **exitflag**: a flag similar to the ones provided by matlab's optimization functions.
  - **H**: an estimate of the Hessian
- **is_fixed_regime** : (true|{false}): if true, the regimes are known in advance. In that case the data should contain time series for a variable called "hist_regimes"

Returns:

- **var** (var object): var object with parameters estimated based on data

See also:

- *posterior_mode*
- *identification*
- *prior template*

Help for svar/estimate is inherited from superclass ABSTVAR

7.6.2.4 filter

filte: Compute the log likelihood of the VAR for the given parameters

```
[LogLik,Incr,retcode,filtering] = filter(self);

[LogLik,Incr,retcode,filtering] = filter(self, param);
```

Args:

self (var object): var object

param (cell of struct): (optional) parameter values

Returns:

LogLik : log likelihood values of the VAR

Incr : period-by-period contributions to the likelihood

retcode : flag, 0 if no problem

filtering : structure containing filtration

Note:

Almost everything is automated in RISE, so see `estimate()` or `identification()` .

Help for svar/filter is inherited from superclass ABSTVAR

7.6.2.5 forecast

svar/forecast is a function.

[fkst, retcode] = forecast(self, histdb, date_start, params, nsteps, shock_uncertainty, Rfunc, conditions)

7.6.2.6 get

get Access/Query time series property values.

VALUE = get(TS,'PropertyName') returns the value of the specified property of the time series object. An equivalent syntax is

VALUE = TS.PropertyName

get(TS) displays all properties of TS and their values.

7.6.2.7 historical_decomposition

varargin={shock_names}

7.6.2.8 identification

Parse identification

7.6.2.9 irf

Compute Impulse-Response Functions for a SVAR model

```
myirfs = irf(self)

myirfs = irf(self, shock_names)

myirfs = irf(self, shock_names, irf_periods)

myirfs = irf(self, shock_names, irf_periods, params)

myirfs = irf(self, shock_names, irf_periods, params,girf_setup)
```

Args:

- **self** (svar object): var object
- **shock_names** (empty|cellstr): List of shocks. Note that in each equation there is only one shock. If the list of shocks is not provided, RISE will create names **varname_shock** where "varname" is the name of an endogenous variable. The list of shocks will follow the order of declaration of the endogenous variables. If the number of shocks provided is less than the number of equations, those shocks will bevlisted first and RISE will create additional names to fill in the missing names.
- **irf_periods** (int|{40}): number of periods to compute IRFs
- **params** (empty|vector|matrix): parameter values of the model. If empty MLE/posterior-mode values are used. When not empty, the number of IRFs computed is equal to the number of columns of params.
- **girf_setup** (struct|{empty}): structure containing information relevant for the computation of generalized impulse response functions. If empty, simple regime-specific impulse responses are computed, else girfs are computed. In that case the relevant information to provide in girf_setup is:
  - **nsims** (integer|{300}) : number of simulations for the integration.

Note: Even setting girf_setup=struct() will trigger the computation of girfs. But in that case only the default options will apply.

Returns:

- **myirfs** : (struct) structure containing the IRFs

7.6.2.10 posterior_mode

Return the posterior mode from the BVAR estimation

```
dd = posterior_mode(self)
```

Args:
self (var object): var object

Returns:
:

- **dd** (struct): posterior mode of the parameters

Help for svar/posterior_mode is inherited from superclass ABSTVAR

7.6.2.11 print_solution

svar/print_solution is a function.
print_solution(obj, varlist)

7.6.2.12 print_structural_form

INTERNAL FUNCTION

7.6.2.13 prior_template

prior_template : provides the default settings for priors in Bayesian VARs

```
prior=prior_template()
```

Args : None

Returns:

- prior (struct) : structure with fields :
  - **L1** (numeric|{0.1}) : Overall tightness
  - **L2** (numeric|{0.5}) : Cross-variable specific variance parameter
  - **L3** (numeric|{1}) : Speed at which lags greater than 1 converge to zero
  - **L4** (numeric|{100}) : tightness on deterministic/exogenous terms
  - **L5** (numeric|{3}) : covariance dummies(omega)
  - **L6** (numeric|{5}) : co-persistence (lambda)
  - **L7** (numeric|{2}) : Own-persistence (mu)

- **coefprior** (numeric scalar|vector|{1}): mean of coefficients on own first lag. This allows you to set the prior mean to whatever value you believe in. e.g. one might believe that, say, the growth rate is stationary and so is less persistent than say, the interest rate or inflation.
- **type** (string): type of prior. one of the following:
  - 'minnesota' : Minnesota prior
  - 'normal-wishart' or 'nw' : Normal Wishart prior
  - 'indep-normal-wishart' or 'inw' : Independent normal wishart prior
  - 'jeffrey' : Jeffrey prior
  - 'sims-zha' or 'sz': Sims and Zha prior
- **normal_wishart_eta** (numeric|{0.01}) : Prior variance for the normal-wishart prior
- **independent_normal_wishart_eta** (numeric|{0.01}) : prior variance for the independent-normal-wishart prior

> **Warning:** At the moment only the Sims-Zha and (possibly) the Jeffrey prior work. The other priors need some updating.

Help for svar.prior_template is inherited from superclass ABSTVAR

7.6.2.14 pull_objective

Pulls the objective function to optimize

```
[ff,lb,ub]=pull_objective(self)
[ff,lb,ub,x0]=pull_objective(self)
[ff,lb,ub,x0,vcov]=pull_objective(self)
[ff,lb,ub,x0,vcov,self]=pull_objective(self)
```

Args:

self (svar | rfvar): initial model object

Returns:

:

- **ff** [function handle]: for computing "minus log posterior kernel"
- **lb** [d x 1 vector]: lower bound of the parameters to optimize
- **ub** [d x 1 vector]: upper bound of the parameters to optimize
- **x0** [d x 1 vector]: posterior mode if available
- **vcov** [d x d matrix]: covariance matrix at the posterior mode if available.
- **self** [rise|dsge|svar|rfvar]: updated model object

Note:

- The function can be used for :
  - optimization,
  - gradient computation,
  - hessian computation,

  - posterior simulation
- The updated object should be used for doing various exercises (irfs, simulations, etc.) if the posterior mode is not computed.
- Using this function is potentially costly, one could alternatively simply use log_posterior_kernel. However, if there are restrictions, they will not be enforced. Nevertheless it is an interesting proposition that should be investigated further.

Help for svar/pull_objective is inherited from superclass ABSTVAR

7.6.2.15 residuals

Computes the residuals given the parameters

Help for svar/residuals is inherited from superclass ABSTVAR

7.6.2.16 set

Sets options for the VAR XXXXXXXXXXX need to check

```
var_obj = set(var_obj,varargin);
```

Args:

var_obj (var object): VAR object varargin: Options to set. Must come in pairs

- **'data'**: Data for the VAR. Can be either a struct or ts object
- **'prior'**: Prior to use if BVAR. Available priors are
  - minnesota
  - jeffrey
  - nw
  - normal-wishart
  - inw
  - sz
- **'linear_restrictions'**: Linear restrictions. For the format, see XXXXXXXXX

Returns:

:

- var_obj (var object): New VAR with the given inputs updated.

Help for svar/set is inherited from superclass ABSTVAR

7.6.2.17 solve

solve – Solves the VAR model.

This function is used to solve a VAR (Vector Autoregressive) model. It takes model parameters, data, and additional inputs, and returns the solution *sol*, a return code *retcode*, and a matrix *M*.

Syntax:

[sol, retcode, M] = solve(self, param, yt)

Inputs:

- *self* [VARModel]: The VAR model object.
- *param* [empty|struct|matrix]: Model parameters. If it is a matrix, then each column corresponds to a particular parameterization. If it is a structure, then it is the solution. If it is empty, the returned solution is computed used the parameter values stored inside the model object.
- *yt* [matrix]: Data needed for endogenous probabilities. If not provided, the first observation from the model's data is used.

Outputs:

- *sol* [struct]: The solution to the VAR model.
- *retcode* [int]: A return code indicating the success of the solution.
- *M* [matrix]: Parameters for each regime

Example: - Solve the VAR model using provided parameters *param* and data *yt*:

`` `matlab [sol, retcode, M] = solve(varModel, param, yt); ` ``

See also:

vartools.solve.

Help for svar/solve is inherited from superclass ABSTVAR

7.6.2.18 svar

svar : constructor for structural VAR models

Syntax :

```
mdl=svar(varlist)

mdl=svar(varlist,exog)

mdl=svar(varlist,exog,nlags)

mdl=svar(varlist,exog,nlags,constant)

mdl=svar(varlist,exog,nlags,constant,markov_chains)
```

Args :

- **varlist** (cellstr): List of endogenous variables
- **exog** (empty|cellstr): List of exogenous variables excluding the constant term
- **nlags** (empty|integer|{4}): Number of lags in the VAR

- **constant** (empty|{true}|false): boolean for including a constant term in the VAR or not
- **markov_chains** (empty|struct): structure containing the following fields:
  - **'name'**: name of the markov process
  - **'number_of_states'**: number of states
  - **'controlled_parameters'**: list of controlled parameters
  - **'endogenous_probabilities'**: definitions of the endogenous probabilities
  - **'probability_parameters'**: parameters entering the time-varying transition probabilities

Returns :

- **mdl** (svar): Constructed SVAR model

Note: For the controlled parameters

- The dynamic parameters of a svar model are denoted by "a"
- The static parameters or coefficients on exogenous by "c"
- for some parameter bd(row,col), e.g. a2(2,3)
  - d (empty|integer) : when empty, all lags are swept through
  - row (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable
  - col (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable

Documentation for svar/svar
doc svar

7.6.2.19 transform_var_into_dsge

transform_var_into_dsge : transforms a parameterized VAR into a DSGE model.

SYNTAX

m=transform_var_into_dsge(v)

m=transform_var_into_dsge(v,exogenousValues)

INPUTS

- **v** [rfvar|svar|proxy_var|prfvar]: parameterized VAR object
- **exogenousValues** [empty|struct]: structure containing the values for the exogenous variables. This is because the DSGE does not contain exogenous variables. Therefore the values of the exogenous variables has to be fixed.

OUTPUTS

- **m** [rise]: parameterized DSGE model

NB : At the moment, the routine does not support regime switching models. The feature will be implemented if there is a demand for it.

Help for svar/transform_var_into_dsge is inherited from superclass ABSTVAR

7.6.2.20 variance_decomposition

Compute the variance decompsition of the VAR

```
[vd,retcode] = variance_decomposition(self)

[vd,retcode] = variance_decomposition(self,params)

[vd,retcode] = variance_decomposition(self,params,Rfunc)

[vd,retcode] = variance_decomposition(self,params,Rfunc,nperiods)
```

Args:

- self (var object):
- params :(optional) parameter values
- Rfunc (function handle): (optional) transform parameters into dynamics
- nperiods :(optional) periods for decomposition

Returns:

- **vd** (struct): a struct containing the variance decomposition:
  - **infinity** (struct):
  - **conditional** (struct):
- **retcode** (integer): This passes through the retcode given by Rfunc. Currently (2018/07/19), this output always returns 0.

CHAPTER

EIGHT

PROXY STRUCTURAL VAR MODELING

8.1 Description

$$A_0\left(r_t\right) y_t = C\left(r_t\right) x_t + A_1\left(r_t\right) y_{t-1} + ... + A_p\left(r_t\right) y_{t-p} + \varepsilon_t$$

The proxy process

$$m_t = \beta_m\left(r_t\right) \varepsilon_{y,t} + \sigma_m\left(r_t\right) \varepsilon_{m,t}$$

With $r_t = 1, 2, ..., h$ and transition probabilities $p_{r_t, r_{t+1}}\left(I_t\right)$

8.2 Quick-start examples

8.2.1 constant

8.2.2 constant transition probabilities

8.2.3 time~varying transition probabilities

8.3 Estimation

8.4 Simulation

8.5 Forecasting and conditional forecasting

8.6 Technical documentation for proxy_var objects

8.6.1 The properties

8.6.1.1 proxies

structure of proxies

8.6.1.2 endogenous

list of endogenous variables

Help for proxy_svar/endogenous is inherited from superclass ABSTVAR

8.6.1.3 exogenous

list of exogenous variables

Help for proxy_svar/exogenous is inherited from superclass ABSTVAR

8.6.1.4 parameters

list of parameters including var and non-var parameters

Help for proxy_svar/parameters is inherited from superclass ABSTVAR

8.6.1.5 nonvar_parameters

list of non-var parameters e.g. auxiliary params, transition probabilities, etc.

Help for proxy_svar/nonvar_parameters is inherited from superclass ABSTVAR

8.6.1.6 constant

boolean to specify whether the model has a constant term or not

Help for proxy_svar/constant is inherited from superclass ABSTVAR

8.6.1.7 debug

for debugging purposes

Help for proxy_svar/debug is inherited from superclass ABSTVAR

8.6.1.8 markov_chains

structure of markov chains

Help for proxy_svar/markov_chains is inherited from superclass ABSTVAR

8.6.1.9 is_time_varying_trans_prob

boolean to specify whether we are in presence of a time-varying transition probability model

Help for proxy_svar/is_time_varying_trans_prob is inherited from superclass ABSTVAR

8.6.1.10 is_switching

boolean specifying whether the model is switching or not

Help for proxy_svar/is_switching is inherited from superclass ABSTVAR

8.6.1.11 nlags

number of lags in the VAR

Help for proxy_svar/nlags is inherited from superclass ABSTVAR

8.6.1.12 nx

number of exogenous variables including the constant term

Help for proxy_svar/nx is inherited from superclass ABSTVAR

8.6.1.13 nvars

number of endogenous variables

Help for proxy_svar/nvars is inherited from superclass ABSTVAR

8.6.1.14 nparams

number of parameters

Help for proxy_svar/nparams is inherited from superclass ABSTVAR

8.6.1.15 nregs

number of regimes

Help for proxy_svar/nregs is inherited from superclass ABSTVAR

8.6.2 The methods

8.6.2.1 autocov

Compute the autocovariances (and the auto-correlation) of endogenous variables given the parameter values

```
[C,R,retcode] = autocov(self);
[C,R,retcode] = autocov(self, params);
[C,R,retcode] = autocov(self, params, max_periods);
```

Args:

self (var object): var object params (cell of struct): struct containing var model related parameters (default: []) max_periods (integer): maximum number of period to calculate auto-covariance (default: 5)

Returns:

:

- **C** [4-dimensional array]: auto-covariance where dimensions correspond to
  - 1,2: covariance
  - 3: time lags
  - 4: Number of parameters
- **R** [4-dimensional array]: auto-correlation with same dimensions

8.6.2.2 codify

codify: transforms expressions such as a3(v1,v2,... into a3(2,3,... This can be used to facilitate the setting of priors on individual parameters of the VAR to estimate.

```
str=codify(self,str)

str=codify(self,str,doCode)
```

Args:

- **self** (var object): var object
- **str** (cellstr|string): string to transform. The string has one of the following forms:
  - a3(v1,v2,...: svar and proxy_svar parameters
  - b3(v1,v2,...: rfvar and prfvar parameters
  - c(v1,v2,... : deterministic terms
  - s(v1,v2,... : variance and covariance terms
- **doCode** (true|{false}):
  - if false, return expression with parentheses e.g. a3(2,3)
  - if true, return expression with underscores e.g. a3_2_3

Returns:

str : transformed string

Example:

```
str=codify(psv,'a2(FFR,ygap)=0')

str=codify(psv,'c(FFR,ygap,mpcoef,2)=0',true)

str=codify(psv,'s(FFR,ygap,mpcoef,2)=0',true)
```

Note:

- The function does not check whether the resulting parameter is actually a parameter in the model
- The function also codifies expressions and not just individual parameters

Help for proxy_svar/codify is inherited from superclass ABSTVAR

8.6.2.3 estimate

Estimates VAR parameters using frequentist/Bayesian interpretation

```
var = estimate(var);
var = estimate(var, data);
var = estimate(var, data, data_range);
var = estimate(var, data, data_range, prior);
var = estimate(var, data, data_range, prior, restrictions);
var = estimate(var, data, data_range, prior, restrictions,optimizer);
var = estimate(var, data, data_range, prior, restrictions,optimizer,is_fixed_
↪regime);
```

Args:

- **var** (var object): var object
- **data** (struct or ts obect): data for estimation
- **data_range** (serial): (optional) date_range
- **prior** (struct): priors for parameters
  - If no prior is given, maximum likehood estimators (frequentist) are set for parameters
  - With priors, posterior mode (Bayesian) values are set for parameters. There are three types of priors that can be applied individually or simultaneously. The prior structure can have fields :
    - **var** (struct) : (possibly modified) content of *prior template*
    - **nonvar** (struct): priors on individual parameters, which could be the VAR parameters, the transition probabilities or the parameters entering endogenous transition probabilities.
    - **endogenous** (function handle) : priors on the behavior of the system. e.g. a prior on some nonlinear function of the parameters.
- **restrictions** : linear and nonlinear restrictions on parameters.
- **optimizer** (char|function_handle|cell|{fmincon}) : optimization procedure. Used with optimization is required. e.g. markov switching. This can be the name of a standard matlab optimizer or RISE optimization routine or a user-defined optimization procedure available of the matlab search path. If the optimzer is provided as a cell, then the first element of the cell is the name of the optimizer or its handle and the remaining entries in the cell are options to the user-defined optimization routine. A user-defined optimization function must have the following syntax

```
[xfinal,ffinal,exitflag,H]=optimizer(fh,x0,lb,ub,options);
```

That is, it accepts as inputs:

- **fh**: the function to optimize
- **x0**: a vector column of initial values of the parameters
- **lb**: a vector column of lower bounds
- **ub**: a vector column of upper bounds
- **options**: a structure of options whose fields will be similar to matlab's optimset

That is, it provides as outputs:

- **xfinal**: the vector of final values
- **ffinal**: the value of **fh** at **xfinal**

  - **exitflag**: a flag similar to the ones provided by matlab's optimization functions.
  - **H**: an estimate of the Hessian
- **is_fixed_regime** : (true|{false}): if true, the regimes are known in advance. In that case the data should contain time series for a variable called "hist_regimes"

Returns:

- **var** (var object): var object with parameters estimated based on data

See also:

- *posterior_mode*
- *identification*
- *prior template*

Help for proxy_svar/estimate is inherited from superclass ABSTVAR

8.6.2.4 filter

filte: Compute the log likelihood of the VAR for the given parameters

```
[LogLik,Incr,retcode,filtering] = filter(self);

[LogLik,Incr,retcode,filtering] = filter(self, param);
```

Args:

self (var object): var object

param (cell of struct): (optional) parameter values

Returns:

LogLik : log likelihood values of the VAR

Incr : period-by-period contributions to the likelihood

retcode : flag, 0 if no problem

filtering : structure containing filtration

Note:

Almost everything is automated in RISE, so see `estimate()` or `identification()` .

Help for proxy_svar/filter is inherited from superclass ABSTVAR

8.6.2.5 forecast

proxy_svar/forecast is a function.

[fkst, retcode] = forecast(self, histdb, date_start, params, nsteps, shock_uncertainty, Rfunc, conditions)

8.6.2.6 get

get Access/Query time series property values.

VALUE = get(TS,'PropertyName') returns the value of the specified property of the time series object. An equivalent syntax is

VALUE = TS.PropertyName

get(TS) displays all properties of TS and their values.

8.6.2.7 historical_decomposition

varargin={shock_names}

8.6.2.8 identification

Parse identification

Help for proxy_svar/identification is inherited from superclass SVAR

8.6.2.9 irf

irf Alias for srf.

See help on [*SVAR/srf*](SVAR/srf).

Help for proxy_svar/irf is inherited from superclass SVAR

8.6.2.10 posterior_mode

Return the posterior mode from the BVAR estimation

```
dd = posterior_mode(self)
```

Args:
self (var object): var object

Returns:
:

- **dd** (struct): posterior mode of the parameters

Help for proxy_svar/posterior_mode is inherited from superclass ABSTVAR

8.6.2.11 print_solution

proxy_svar/print_solution is a function.
print_solution(obj, varlist)

8.6.2.12 print_structural_form

INTERNAL FUNCTION

Help for proxy_svar/print_structural_form is inherited from superclass SVAR

8.6.2.13 prior_template

prior_template : provides the default settings for priors in Bayesian VARs

```
prior=prior_template()
```

Args : None

Returns:

- prior (struct) : structure with fields :
  - **L1** (numeric|{0.1}) : Overall tightness
  - **L2** (numeric|{0.5}) : Cross-variable specific variance parameter
  - **L3** (numeric|{1}) : Speed at which lags greater than 1 converge to zero
  - **L4** (numeric|{100}) : tightness on deterministic/exogenous terms
  - **L5** (numeric|{3}) : covariance dummies(omega)
  - **L6** (numeric|{5}) : co-persistence (lambda)
  - **L7** (numeric|{2}) : Own-persistence (mu)
  - **coefprior** (numeric scalar|vector|{1}): mean of coefficients on own first lag. This allows you to set the prior mean to whatever value you believe in. e.g. one might believe that, say, the growth rate is stationary and so is less persistent than say, the interest rate or inflation.
  - **type** (string): type of prior. one of the following:
    - 'minnesota' : Minnesota prior
    - 'normal-wishart' or 'nw' : Normal Wishart prior
    - 'indep-normal-wishart' or 'inw' : Independent normal wishart prior
    - 'jeffrey' : Jeffrey prior
    - 'sims-zha' or 'sz': Sims and Zha prior
  - **normal_wishart_eta** (numeric|{0.01}) : Prior variance for the normal-wishart prior
  - **independent_normal_wishart_eta** (numeric|{0.01}) : prior variance for the independent-normal-wishart prior

> **Warning:** At the moment only the Sims-Zha and (possibly) the Jeffrey prior work. The other priors need some updating.

Help for proxy_svar.prior_template is inherited from superclass ABSTVAR

8.6.2.14 proxy_svar

constructror for proxy_svar

mdl=proxy_svar(proxies,varlist)

mdl=proxy_svar(proxies,varlist,exog)

mdl=proxy_svar(proxies,varlist,exog,nlags)

mdl=proxy_svar(proxies,varlist,exog,nlags,constant)

mdl=proxy_svar(proxies,varlist,exog,nlags,constant,markov_chains)

Inputs:

- **proxies** [empty|struct] : when not empty it should be an array of structures with fields :
  - "var" : the value should be the name of an endogenous variable
  - **"eqtn"**
    [# for the proxy equation. It should correspond] to declaration order of "var"
  - "coef" : name of the proxy coefficient
  - **"shock_eqtn"**
    [Equation whose shock is to be related to] the proxy equation
- **varlist** [cellstr] : list of the endogenous variables. Should be ordered in a way that matches with the restrictions given in the proxies
- **exog** [empty|cellstr] : list of the exogenous variables.
- **nlags** [empty|{4}] : list of the exogenous variables.
- **constant** [empty|false|{true}] : include a constant term
- **markov_chains** [empty|struct] : array of structures describing markov processes

Outputs:

- **mdl** [proxy_svar] : proxy_svar model object

Note: For the controlled parameters

- The dynamic parameters of a svar model are denoted by "a"
- The static parameters or coefficients on exogenous by "c"
- for some parameter bd(row,col), e.g. a2(2,3)
  - d (empty|integer) : when empty, all lags are swept through
  - row (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable
  - col (integer|string) : denotes the row, which can be an integer of a string corresponding to the name of an endogenous variable

Documentation for proxy_svar/proxy_svar
doc proxy_svar

8.6.2.15 pull_objective

Pulls the objective function to optimize

```
[ff,lb,ub]=pull_objective(self)
[ff,lb,ub,x0]=pull_objective(self)
[ff,lb,ub,x0,vcov]=pull_objective(self)
[ff,lb,ub,x0,vcov,self]=pull_objective(self)
```

Args:

self (svar | rfvar): initial model object

Returns:

:

- **ff** [function handle]: for computing "minus log posterior kernel"
- **lb** [d x 1 vector]: lower bound of the parameters to optimize
- **ub** [d x 1 vector]: upper bound of the parameters to optimize
- **x0** [d x 1 vector]: posterior mode if available
- **vcov** [d x d matrix]: covariance matrix at the posterior mode if available.
- **self** [rise|dsge|svar|rfvar]: updated model object

Note:

- The function can be used for :
  - optimization,
  - gradient computation,
  - hessian computation,
  - posterior simulation
- The updated object should be used for doing various exercises (irfs, simulations, etc.) if the posterior mode is not computed.
- Using this function is potentially costly, one could alternatively simply use log_posterior_kernel. However, if there are restrictions, they will not be enforced. Nevertheless it is an interesting proposition that should be investigated further.

Help for proxy_svar/pull_objective is inherited from superclass ABSTVAR

8.6.2.16 residuals

Computes the residuals given the parameters

Help for proxy_svar/residuals is inherited from superclass ABSTVAR

8.6.2.17 set

Sets options for the VAR XXXXXXXXXXX need to check

```
var_obj = set(var_obj,varargin);
```

Args:

var_obj (var object): VAR object varargin: Options to set. Must come in pairs

- **'data'**: Data for the VAR. Can be either a struct or ts object
- **'prior'**: Prior to use if BVAR. Available priors are
  - minnesota
  - jeffrey
  - nw
  - normal-wishart
  - inw
  - sz
- **'linear_restrictions'**: Linear restrictions. For the format, see XXXXXXXXX

Returns:

:

- var_obj (var object): New VAR with the given inputs updated.

Help for proxy_svar/set is inherited from superclass ABSTVAR

8.6.2.18 solve

solve – Solves the VAR model.

This function is used to solve a VAR (Vector Autoregressive) model. It takes model parameters, data, and additional inputs, and returns the solution *sol*, a return code *retcode*, and a matrix *M*.

Syntax:

[sol, retcode, M] = solve(self, param, yt)

Inputs:

- *self* [VARModel]: The VAR model object.
- *param* [empty|struct|matrix]: Model parameters. If it is a matrix, then each column corresponds to a particular parameterization. If it is a structure, then it is the solution. If it is empty, the returned solution is computed used the parameter values stored inside the model object.
- *yt* [matrix]: Data needed for endogenous probabilities. If not provided, the first observation from the model's data is used.

Outputs:

- *sol* [struct]: The solution to the VAR model.
- *retcode* [int]: A return code indicating the success of the solution.
- *M* [matrix]: Parameters for each regime

Example: - Solve the VAR model using provided parameters *param* and data *yt*:

`matlab [sol, retcode, M] = solve(varModel, param, yt); `

See also:

vartools.solve.

Help for proxy_svar/solve is inherited from superclass ABSTVAR

8.6.2.19 transform_var_into_dsge

transform_var_into_dsge : transforms a parameterized VAR into a DSGE model.

SYNTAX

m=transform_var_into_dsge(v)

m=transform_var_into_dsge(v,exogenousValues)

INPUTS

- **v** [rfvar|svar|proxy_var|prfvar]: parameterized VAR object
- **exogenousValues** [empty|struct]: structure containing the values for the exogenous variables. This is because the DSGE does not contain exogenous variables. Therefore the values of the exogenous variables has to be fixed.

OUTPUTS

- **m** [rise]: parameterized DSGE model

NB : At the moment, the routine does not support regime switching models. The feature will be implemented if there is a demand for it.

Help for proxy_svar/transform_var_into_dsge is inherited from superclass ABSTVAR

8.6.2.20 variance_decomposition

Compute the variance decompsition of the VAR

```
[vd,retcode] = variance_decomposition(self)

[vd,retcode] = variance_decomposition(self,params)

[vd,retcode] = variance_decomposition(self,params,Rfunc)

[vd,retcode] = variance_decomposition(self,params,Rfunc,nperiods)
```

Args:

- self (var object):
- params :(optional) parameter values
- Rfunc (function handle): (optional) transform parameters into dynamics
- nperiods :(optional) periods for decomposition

Returns:

- **vd** (struct): a struct containing the variance decomposition:
  - **infinity** (struct):

  - **conditional** (struct):

- **retcode** (integer): This passes through the retcode given by Rfunc. Currently (2018/07/19), this output always returns 0.

CHAPTER

NINE

BAYESIAN ESTIMATION

9.1 Restrictions on parameters during estimation

Sometimes we need to set additional restrictions on parameters during estimation. For instance, in a regime switching model, one may need a normalization assumption in order to label the states. All kinds of functions of both estimated and non-estimated parameters are allowed as long as those functions are known to Matlab or on the Matlab path.

There is a way of setting these restriction in the model file of a DSGE model. However, setting these restrictions outside the model file as is done below should be preferred.

In this case, the restrictions are organized in a cell array

```
restrictions={
        'f1(theta)>=f2(theta)'
        ...
        'h1(theta)<=h2(theta)'
        };
```

and passed along to the model object using option **estim_nonlinear_restrictions**. See the section on estimation for more detail.

9.2 Prior distributions and implementation

9.2.1 Bayesian priors

N.B: In the description below :

- **start** denotes the initial value for estimation
- **mean** denotes the mean of the prior distribution
- **sd** denotes the standard deviation
- **hpp1** denotes the first hyperparameter
- **hpp2** denotes the second hyperparameter
- **hpp3** denotes the third hyperparameter
- **hpp4** denotes the fourth hyperparameter
- **lb** denotes the lower bound

- **ub** denotes the upper bound
- **lq** denotes the lower quantile
- **uq** denotes the upper quantile
- **p** is the structure containing the parameters to estimate
- **alpha** is the name of some parameter to be estimated

9.2.1.1 Setting Uniform Priors

```
p.alpha = {start,lb,ub};

N.B: This prior will be rewritten internally as
p.alpha = {start,lb,ub,'uniform(1)'};
```

If you set all your parameters to be estimated in this way, this is how you perform Maximum Likelihood (See e.g. [Ireland, 2004]) in RISE.

9.2.1.2 Setting priors using Means and Standard deviations

```
p.alpha = {start,mean,sd,'distribution'};

*adding truncation*
p.alpha = {start,mean,sd,'distribution',lb,ub};

*more hyperparameters e.g. generalized beta, t-student or truncated normal*
p.alpha = {start,mean,sd,hpp3,hpp4,'distribution'};

*adding truncation*
p.alpha = {start,mean,sd,hpp3,hpp4,'distribution',lb,ub};
```

9.2.1.3 Setting priors using quantiles

One cannot always characterize the distribution of parameters in terms of means and standard deviations :

- Not all distributions have means and standard deviations
- Some distributions are such that for some specific values of the hyperparameters, the first and second moments do not exist
- Oftentimes when we take our priors from the literature, the number of studies we take inspiration from are too few to rely on for the calculation of means and standard deviations.

Under such circumstances, one can instead use quantiles.

```
p.alpha = {start,lq,uq,'distribution(probability)'};

*adding truncation*
p.alpha = {start,lq,uq,'distribution(probability)',lb,ub};

*more hyperparameters e.g. generalized beta, t-student or truncated normal*
p.alpha = {start,lq,uq,hpp3,hpp4,'distribution(probability)'};
```

(continues on next page)

(continued from previous page)

```
*adding truncation*
p.alpha = {start,lq,uq,hpp3,hpp4,'distribution(probability)',lb,ub};
```

9.2.1.4 Setting priors using hyperparameters

In some cases, people prefer to characterize their distributions directly in terms of hyperparameters.

In fact all the previous approaches to setting priors will first determine the hyperparameters of the distributions.

RISE also allows you to set your priors directly in terms of hyperparameters if you choose to do so. Should you rely on this strategy, the recommendation is to plot the distributions and inspect them carefully before starting any estimation.

```
p.alpha = {start,hpp1,hpp2,'distribution(hpp)'};

*adding truncation*
p.alpha = {start,hpp1,hpp2,'distribution(hpp)',lb,ub};

*more hyperparameters e.g. generalized beta, t-student or truncated normal*
p.alpha = {start,hpp1,hpp2,hpp3,hpp4,'distribution(hpp)'};

*adding truncation*
p.alpha = {start,hpp1,hpp2,hpp3,hpp4,'distribution(hpp)',lb,ub};
```

9.2.1.5 About the t-student prior

The third hyperparameter of the t-student distribution is the DOF (degree of freedom). There is not 4th hyperparameter and so the field can be nan or empty e.g. p.alpha = {start,3,7,10,nan,'t(.9)'};

9.2.1.6 The dirichlet prior

consider a transition matrix of the form

$$M = \begin{bmatrix} pname(1,1) & pname(1,2) & ... & pname(1,n) \\ pname(2,1) & pname(2,2) & ... & pname(2,n) \\ ... & ... & ... & ... \\ pname(n,1) & pname(n,2) & ... & pname(n,n) \end{bmatrix}$$

Dirichlet priors can bet set as follows

```
p.dirichlet_1 = {stdev(1,1),pname(1,2),mean(1,2),pname(1,3),mean(1,3),...,pname(1,n),
↪mean(1,n)};

with mean(1,1)+mean(1,2)+mean(1,3)+...+mean(1,n)=1
and mean(1,1) the mean of the first diagonal element
and stdev(1,1) its standard deviation

p.dirichlet_2 = {stdev(2,2),pname(2,1),mean(2,1),pname(2,3),mean(2,3),...,pname(2,n),
↪mean(2,n)};
```

(continues on next page)

(continued from previous page)

```
...
p.dirichlet_n = {stdev(n,n),pname(n,1),mean(n,1),pname(n,2),mean(n,2),...,pname(n,n-1),
↪mean(n,n-1)};

with mean(n,1)+mean(n,2)+mean(n,3)+...+mean(n,n)=1
and mean(n,n) the mean of the n-th diagonal element
and stdev(n,n) its standard deviation
```

N.B: The order of the names dirichlet_1, dirichlet_2, ..., dirichlet_n is irrelevant and there need not be n such names. The key here is the syntax dirichlet_number

9.2.2 Endogenous Priors

Rather than setting priors on parameters, one can also set priors on the behavior or properties of the model. This is known in the literature under various appelations :

- **Endogenous priors** : See e.g. [Del Negro and Schorfheide, 2008] or [Christiano *et al.*, 2011]
- **System priors** : See e.g. [Andrle and Benes, 2013], [Andrle and Plasil, 2017] or [Andrle and Plašil, 2018]
- **property priors** : as suggested by Bob Tetlow
- **behavioral priors** :

9.2.3 User-defined Priors

It could be the case that you want to implement a prior that is not supported by RISE. In that case you should provide the PDF (not the log PDF!) of the prior as a function handle, the lower and upper bounds of the distribution. Internally, RISE will (empirically) construct routines for the CDF, the inverse CDF and everything needed. The syntax is as follows:

```
p.alpha = {start,LB,UB,function_handle};
```

9.2.4 Visualizing priors and posteriors

With the structure of priors in place, one can use the function rdist.plot to visualize priors and/or potentially posteriors, provided the simulation of the posterior distribution has been conducted.

The syntax is

```
rdist.plot(priors)

rdist.plot(priors,plotOpts)

rdist.plot(priors,plotOpts,this)

rdist.plot(priors,[],[])

rdist.plot(priors,[],this)
```

(continues on next page)

(continued from previous page)

```
rdist.plot(priors,plotOpts,this,'linewidth',2)

rdist.plot(priors,plotOpts,this,varargin)
```

Args:

- priors (struct): prior information
- plotOpts (empty|struct): options for the plots. The fields are
  - 'r0c0',[vector|{[3,3]}] number of rows (first element) and number of columns (second element) in a figure
  - 'prior_trunc',[scalar|{1e-3}] truncation for the plots of the priors
  - 'scale_IG_trunc',[scalar|{25}] correction of prior_trunc for inverse gamma family i.e. {'igamma','igamma1','igamma2',...}
  - 'npoints',[scalar|{1000}] number of points for the computation of the prior and the posterior densities
  - 'do',[struct] fields are "priors" and "posteriors" and their default values are true and true. This gives the flexibility to choose what one wants to plot : priors, posteriors or both
  - 'par_list',[empty|cell array] list of parameters to plot
- this (empty|mcmc object): this is where the information for plotting the posterior is obtained from
- varargin (empty|pairwise): pairwise additional arguments entering the plot function. e.g. linewidth

Example

```
priors=struct();
priors.beta={.5,95,5,'beta(hpp)'};
priors.delta={.5, 40,860,'beta(hpp)'};
priors.alpha={.5,360,590,'gamma(hpp)'};
priors.rhoz={.5,80,20,'beta(hpp)'};
priors.sdez={0.1, 0.01,0.01,'igamma(hpp)'};

plotOpts=struct();
plotOpts.prior_trunc=2.6e-3;

rdist.plot(priors,plotOpts,[],struct('linewidth',2))
```

9.2.5 List of prior distributions

1. arcsine: θ in $[0,1]$
2. beta: θ in $[0,1]$
3. dirichlet : θ_i in $[0,1]$, with $\sum_i \theta_i = 1$
4. betaprime : θ in $[0,\infty)$
5. cauchy : θ in $(-\infty,\infty)$
6. exponential: θ in $[0,\infty)$
7. folded_normal : θ in $[0,\infty)$
8. gamma : θ in $[0,\infty)$

9. gammatau (generalized gamma-tau): θ
10. gumbel : θ in $(-\infty, \infty)$
11. half_normal : θ in $[0, \infty)$
12. igamma (igamma2) : θ in $[0, \infty)$
13. igamma1 : θ in $[0, \infty)$: Dynare's inv_gamma
14. igamma2 : θ in $[0, \infty)$
15. inverse_gaussian : θ in $(0, \infty)$
16. kumaraswamy : θ in $(0, 1)$
17. laplace : θ in $(-\infty, \infty)$
18. left_triang (skewed to the left) : θ in $[a, b]$
19. levy : θ in $[\mu, \infty)$
20. logistic : θ in $(-\infty, \infty)$
21. lognormal : θ in $(0, \infty)$
22. normal : θ in $(-\infty, \infty)$
23. pareto : θ in $[x_m, \infty)$, $x_m > 0$
24. raised_cosine : θ in $[\mu - s, \mu + s]$, $s > 0$
25. sichisq (scaled inverse χ^2) : θ in $(0, \infty)$
26. t (t-student) : θ in $(-\infty, \infty)$
27. truncated_normal: θ in $[\underline{\theta}, \overline{\theta}]$
28. uniform : θ in $[\underline{\theta}, \overline{\theta}]$
29. weibull : θ in $[0, \infty)$

Note: The following distributions can be used for standard deviations: igamma, igamma1, inverse_gaussian sichisq

For parameters in the $[0, 1]$ interval, one case use beta, kumaraswamy

9.3 Posterior maximization

9.3.1 Optimizers

9.3.2 User-defined Optimizers

9.4 Posterior sampling

9.4.1 The sampling function

rsamplers.rsampler.sample is the function that runs the sampling for different chains on an rsamplers.rsampler object to get a sample.

The syntax is

```
results = sample(obj,varargin)
```

Args:

- obj is a member of one of the following classes:
  - rsamplers.rwmh : Random-Walk Metropolis-Hastings
  - rsamplers.imh : Independent Metropolis-Hastings
  - rsamplers.apt : Adaptive parallel tempering
  - rsamplers.slice : slice sampler
- *varargin* are class-specific extra arguments

Hint: After sampling, RISE has functions for thinning, burnin, etc. through subsetting. See e.g. help on **mdd** (constructor for marginal data density objects).

Warning: please check the sign of the target function and make sure the algorithm used conforms with that sign. Of course, changing the sign of the target function is easy

```
newtarget=@(varargin)-target(varargin{:})
```

Hint: For your convenience, some shortcuts are available so that you do not have to first create an object and then do the sampling. Moreover **you do not have to create a new target if the problem is already a minimization problem**. Those shortcuts are : **sampler_rwmh, sampler_imh, sampler_apt, sampler_slice**. They can be called using one of the following syntaxes (for the details for the inputs, see the base functions):

- results = sampler_XXXX(target,x0,lb,ub)
- results = sampler_XXXX(target,x0,lb,ub,opts)

9.4.2 Algorithm : Random-walk Metropolis Hastings

rsamplers.rwmh is the constructor for the Random-walk Metropolis Hastings algorithm

The syntax is

```
obj = rsamplers.rwmh(target,x0,lb,ub,opts)
```

INPUTS :

- *target* : objective to MAXIMIZE
- *x0* : vector of initial conditions
- *lb* : lower bound
- *ub* : upper bound
- *opts* : options for the class. These include the properties from the superclass rsamplers.rsampler as well as the specific properties of the rwmh algorithm that can be found in the properties list below.

OUTPUT:

- *obj* : object of class rsamplers.rwmh

OWN PROPERTIES AND DEFAULT VALUES:

- *c* = 1 : cov scaling parameter
- *tunedCov* : covariance matrix of the parameters
- *proposal* = 'normal' : proposal in {'normal','t-student'}

INHERITED PROPERTIES AND DEFAULT VALUES:

- *nchain* = 1 : number of chains
- *N* = 2000 : number of draws
- *thinning* =1 : number of thinning draws
- *burnin* = 0 : burnin sample
- *MaxTime* = inf : Max Time
- *MaxFunEvals* = inf :

See also:

rsamplers.apt, rsamplers.imh, rsamplers.rsampler

Documentation for rsamplers.rwmh/rwmh
doc rsamplers.rwmh

9.4.3 Algorithm : Independent Metropolis Hastings

rsamplers.imh is the constructor for the Independent Metropolis-Hastings algorithm

The syntax is

```
obj = rsamplers.imh(target,x0,lb,ub,opts)
```

INPUTS :

- *target* : objective to MAXIMIZE
- *x0* : vector of initial conditions
- *lb* : lower bound
- *ub* : upper bound
- *opts* : options for the class. These include the properties from the superclass rsamplers.rwmh as well as the specific properties of the imh algorithm that can be found in the properties list.

OUTPUT:

- *obj* : object of class rsamplers.imh

OWN PROPERTIES AND DEFAULT VALUES: None

INHERITED PROPERTIES : properties from rsamplers.rwmh

See also:

rsamplers.rwmh, rsamplers.apt, rsamplers.rsampler

Documentation for rsamplers.imh/imh
doc rsamplers.imh

9.4.4 Algorithm : Adaptive parallel tempering

rsamplers.apt is the constructor for the Adaptive Parallel tempering algorithm following [Miasojedow *et al.*, 2013]

The syntax is

```
obj = rsamplers.apt(target,x0,lb,ub,opts)
```

INPUTS :

- *target* : objective to MAXIMIZE
- *x0* : vector of initial conditions
- *lb* : lower bound
- *ub* : upper bound
- *opts* : options for the class. These include the properties from the superclass **rsamplers.rwmh** as well as the specific properties of the apt algorithm that can be found in the properties list below.

OUTPUT :

- obj : object of class rsamplers.apt

OWN PROPERTIES AND DEFAULT VALUES:

- *H* = 12 : Number of tempering stages
- *alpha* = .234 : target acceptance rate
- *alpha_swap* = .234 : target acceptance rate for swaps
- *rwm_fixed_p* = 0 : prob of drawing from a fixed initial proposal
- *rwm_exp* = 0.6 : Exponent of random-walk adaptation step size
- *sw_exp_rm* = 0.6 : Exp. of temperature adaptation step size
- *ram_adapt* = false : Use the robust AM adaptation
- *separate_shape_adaptation* = true :separate covariance for each temperature
- *fixed_temperatures* = false : fixed temperatures

INHERITED PROPERTIES : properties from rsamplers.rwmh

See also:

rsamplers.rwmh, rsamplers.imh, rsamplers.rsampler

Documentation for rsamplers.apt/apt
doc rsamplers.apt

9.4.5 Algorithm : Slice Sampler

rsamplers.slice is the constructor for the slice algorithm following [Planas *et al.*, 2015] and [Planas and Rossi, 2018]

The syntax is

```
obj = rsamplers.slice(target,x0,lb,ub,opts)
```

INPUTS:

- *target* : objective to MAXIMIZE
- *x0* : vector of initial conditions
- *lb* : lower bound
- *ub* : upper bound
- *opts* : options for the class. These include the properties from the superclass rsamplers.rsampler.

OUTPUT:

- *obj* : object of class rsamplers.rwmh

OWN PROPERTIES AND DEFAULT VALUES: None

INHERITED PROPERTIES AND DEFAULT VALUES (same as the inherited properties of rsamplers.rwmh):

- *nchain* = 1 : number of chains
- *N* = 2000 : number of draws
- *thinning* =1 : number of thinning draws
- *burnin* = 0 : burnin sample
- *MaxTime* = inf : Max Time
- *MaxFunEvals* = inf :

See also:

rsamplers.apt, rsamplers.imh, rsamplers.rsampler, rsamplers.rwmh

Documentation for rsamplers.slice/slice
doc rsamplers.slice

9.4.6 User-defined Algorithms

Due to the fact that the RISE toolbox is written in a modular manner, a user can apply his own sampling algorithm. However, if he wants RISE to process the draws, the output of the sampling should be organized in a way that RISE understands. Here is an example in which we write a wrapping function **wrapped_dramrun** around the delayed-rejection adpative metropolis (DRAM) sampler available here .

```
function r=wrapped_dramrun(m,x0,SIG,N)
% wrapped_dramrun : RISE wrapper for the DRAM code

[~,lb,ub,x0_,vcov]=pull_objective(m);

[model,data,param,opts]=dramrun_inputs();
```

(continues on next page)

(continued from previous page)

```
    % call to the DRAM code
    [results, chain] = dramrun(model, data, param, opts);

    r=results2rise();

        function [model,data,param,opts]=dramrun_inputs()

            if isempty(SIG),SIG=vcov; end

            if isempty(SIG),x0=x0_; end

            % RISE assumes column vector, DRAM assumes row vector
            param = struct(); param.par0 = x0.';  param.bounds=[lb,ub].';

            opts=struct(); opts.qcov = SIG; opts.nsimu=N;% Number of samples

            data = {};

            model=struct();
            model.ssfun = @(p)m.routines.likelihood(p.',m);
            model.priorfun = @(p)log_prior_density(m,p.');

        end

        function r=results2rise()

            draws=chain.';

            r=struct();

            for id=1:N
                r.pop(id)=struct('x',draws(:,id),'f',nan);
            end

            r.stats.accept_ratio=results.accepted;
            r.stats.funevals=nan; r.stats.time_elapsed=nan;
            r.SIG=results.qcov; % covariance matrix now hidden
            r.c=results.adascale^2;

        end

    end
```

As the code shows, there are three main articulations of the program :

1. Obtain from RISE and from the user the inputs that DRAM requires
2. Run DRAM
3. Transform the output of DRAM so that it can be further processed by RISE

9.4.6.1 Obtaining the DRAM inputs

Some samplers can be run directly using the log-posterior kernel, which can be obtain in RISE through the function *pull_objective*.

The DRAM code on the other hand, requires one function to evaluate log-prior and a separate function to evaluate the log-likelihood.

In RISE, we can get a handle to the function that evaluates the prior by using *log_prior_density*. We need to remember though that RISE stores the parameters vertically while DRAM stores them horizontally. Therefore, before evaluating the prior, we need to transpose the vector of parameters.

```
prior = @(theta) log_prior_density(model, theta.');
```

Where "model" is the dsge model object.

Likewise we can obtain the function computing the likelihood as follows:

```
likelihood = @(theta) - model.routines.likelihood(theta', model);
```

The other elements required by the DRAM code are easier to collect. These include : the starting vector (x0), the number of draws (n), the covariance matrix (qcov), additional inputs to the likelihood function if any (data).

9.4.6.2 Running DRAM

With the inputs collected, we can then go ahead and run the DRAM sampler.

```
[results, chain, s2chain] = DRAM code(model, data, param, options);
```

9.4.6.3 Transforming the DRAM output

If we want to process the output of DRAM through RISE, we need to transform the DRAM output in a form that RISE expects.

RISE expects each vector to be stored in a structure alongside the value of the posterior kernel. Unfortunately DRAM does not return the value of the posterior kernel. This is why we just set to "nan" all elements that cannot be obtained from the DRAM code.

9.5 Processing Posterior draws : The mcmc class

9.5.1 The properties

9.5.1.1 pnames

mcmc/pnames is a property.

9.5.1.2 nchains

mcmc/nchains is a property.

9.5.1.3 chain_names

mcmc/chain_names is a property.

9.5.1.4 npop

mcmc/npop is a property.

9.5.1.5 nparams

mcmc/nparams is a property.

9.5.1.6 draws

mcmc/draws is a property.

9.5.1.7 psrf

mcmc/psrf is a property.

9.5.1.8 best

mcmc/best is a property.

9.5.1.9 log_post

mcmc/log_post is a property.

9.5.2 The methods

9.5.2.1 autocorrplot

Plots autocorrelations of a given parameter

```
hdl=autocorrplot(obj,pname)
hdl=autocorrplot(obj,pname,chain_id)
hdl=autocorrplot(obj,pname,chain_id,order)
```

Args:

obj (mcmc object): mcmc object

pname (string): parameter name

chain_id (integer | {[]}): choice of chain for which to plot the autocorrelation. If empty, all chains are used.

order (integer | {40}): maximum order of the autocorrelation

Returns:

:

- **hdl** [integer]: handle to the plot

9.5.2.2 brooks_gelman

brooks_gelman computes the [Brooks and Gelman, 1998] convergence diagnostics, both the parameteric and the non-parameteric versions

INPUTS:

- this : mcmc object
- do_plot : [true|{false}|empty]

OUTPUTS:

- myoutput [struct] : Contains
  - UDIAG [by 6] double
    - 1st column: length of total sequence interval
    - 2nd column: sum of length of within sequence intervals; used to compute mean length of within sequence intervals
    - 3nd column: within sequence variance
    - 4nd column: sum of within sequence variances; used to compute mean within sequence variances
    - 5nd column: within sequence kurtosis
    - 6nd column: sum of within sequence kurtoses; used to compute mean within sequence kurtoses
  - MDIAG [by 6] double
    - 1st column: length of total sequence interval
    - 2nd column: sum of length of within sequence intervals; used to compute mean length of within sequence intervals
    - 3nd column: within sequence variance
    - 4nd column: sum of within sequence variances; used to compute mean within sequence variances
    - 5nd column: within sequence kurtosis
    - 6nd column: sum of within sequence kurtoses; used to compute mean within sequence kurtoses
  - h [numeric|empty] : handle to the figures for the multivariate and univariate plots

9.5.2.3 densplot

Plots marginal density of a given parameter

```
hdl=densplot(obj,pname)
hdl=densplot(obj,pname,chain_id)
hdl=densplot(obj,pname,chain_id,N)
```

Args:

obj (mcmc object): mcmc object

pname (string): parameter name

chain_id (integer | {[]}): choice of chain for which to plot the density. If empty, all chains are used.

N (integer | {250}): Number of point in the kernel density

Returns:

:

- **hdl** [integer]: handle to the plot

9.5.2.4 geweke

geweke computes Geweke convergence diagnostics

```
convergence=geweke(this)
convergence=geweke(this,Opts)
convergence=geweke(this,Opts,chain_id)
```

Args:

this (mcmc object): mcmc object

pname (string): parameter name

chain_id (integer | {1}): choice of chain for which to do the convergence analysis

Returns:

:

- **convergence** [struct]: convergence analysis for each parameter

9.5.2.5 inefficiency_factors

Computes inefficiency_factors of a given parameter

```
Ifac=inefficiency_factors(obj)
Ifac=inefficiency_factors(obj,pname)
Ifac=inefficiency_factors(obj,pname,order)
```

Args:

obj (mcmc object): mcmc object

pname (string|cell array): parameter name

Returns:
:

- **Ifac** [scalar|vector,struct]: inefficiency factors

9.5.2.6 mcmc

Constructor for mcmc object

```
mcmc_helper = mcmc(draws)
mcmc_helper = mcmc(draws, pnames)
mcmc_helper = mcmc(draws, pnames, subset)
mcmc_helper = mcmc(draws, pnames, subset, ilinres)
```

Args:

- **draws** (struct): output from samplers in 3 possible forms
  - The draws are stored to disk in which case "draws" is a string representing the location of the draws
  - a structure, output of the samplers when draws are not stored to disk. In this case there is only one chain
  - a cell array of structures, with each structure as above. Each entry then represents one chain
  - a two-element cell array {draws,-logposterior} such that the first entry is a np x ndraws x nchain array of parameter values and the second entry is a 1 x ndraws x nchain array of -logPosterior values
- **pnames** (cellstr): cell of parameter names
- **subset** (cell array|{empty}): When not empty, subset is a 1 x 2 cell array in which the first cell contains a vector selecting the columns to retain in each chain and the second column contains the chains retained. Any or both of those cell arrays can be empty. Whenever an entry is empty, all the information available is selected. E.g. subsetting with dropping and trimming mysubs={a:b:c,[1,3,5]}. In this example, the first element selected is the one in position "a" and thereafter every "b" element is selected until we reach element in position "c". At the same time, we select markov chains 1,3 and 5.
- **ilinres** (function handle|{empty}): function handle that untransforms the parameters in the presence of linear restrictions.

Returns:

- **mcmc_helper** : mcmc object

Note:

- It is the responsibility of the user to provide the names of the parameters as this routine aims to be independent from any estimation procedure or class. If the priors are set in separate structure, their names can easily be obtained via

```
pnames = fieldnames(priors);
```

- Alternatively, if using a RISE object, parameter names can be obtained via:

```
pnames = fieldnames(model.estimation.priors);
```

Documentation for mcmc/mcmc
doc mcmc

9.5.2.7 meanplot

Make a plot of cumulative means from the mcmc chain

```
hdl = meanplot(mcobj,pname);
hdl = meanplot(mcobj,pname,chain_id);
```

Args:

mcobj (mcmc object): mcmc object

pname (str): parameter to make the mean plot

chain_id (vector of int): id of the chain to use

Returns:

:

- **hdl** (handle object): handle to plot object

9.5.2.8 plot_brooks_gelman

Make a plot of cumulative means from the mcmc chain

```
h=plot_brooks_gelman(this,myoutput)
h=plot_brooks_gelman(this,myoutput,nparamsPerFigure)
```

Args:

this (mcmc object): mcmc object

myoutput (struct): output of brooks_gelman

nparamsPerFigure [numeric|{3}]: number of univariate parameters per figure

Returns:

:

- **h** (handle object): handle to all the plots (unvariates and multivariate)

9.5.2.9 posterior_parameter_statistics

posterior_parameter_statistics computes posterior parameter statistics

```
s = posterior_parameter_statistics(this)
s = posterior_parameter_statistics(this,Opts)
```

Args:

- this [mcmc object]: mcmc object
- Opts [empty|struct]: options with the following fields
  - **percnt** [vector|{[2.5,25,50,75,97.5]}] : vector used in the computation of percentiles

  - mh_conf_sig** [scalar|{0.9}]**
  [used in the computation of Highest] Probability Density Intervals.

  - **npoints** [integer|{512}] : number of points used in the computation of the density for each parameter

- **kernel** [char|{'normal'}] : kernel used in the computation of the density. Choices are 'epanechnikov'|'triangular'|'triweight',... 'uniform'|'cosine'
- **do** [struct] : structure whose fields are the same as the ones for the output. Each item should be true or false, with true triggering the computation of the relevant item.

Returns:

s (struct): output with fields

- posterior_mean [struct] : posterior mean
- posterior_var [struct] : posterior variance
- posterior_std [struct] : posterior standard deviation
- posterior_percentiles [struct] : posterior posterior_percentiles as defined by the user
- posterior_deciles [struct] : posterior deciles 0.1:0.1:0.9
- posterior_hpd_interval [struct] : Highest probability density intervals
- posterior_density [struct] : kernel density estimate

9.5.2.10 process_draws

mcmc.process_draws is a function.

[d, fd, summary] = mcmc.process_draws(draws, subset)

9.5.2.11 psrf_plot

Makes a plot of the "posterior scale reduction factor" i.e. the [Gelman and Rubin, 1992] diagnotics from the chains.

```
hdl = psrf_plot(obj, pname)
```

Args:

obj (mcmc object): mcmc object

pname (char): parameter name. N.B: One of the parameter's name is "multivariate_" and it represents the aggregated statistics.

start (numeric|{1}|function handle): iteration at which to start the
plot of the PSRF. If a function handle is used then it should take as input the total number of observations and return the point at which to start. e.g. @(x)round(0.5*x)

Returns:

- **hdl** (handle object): handle to plot object

Warning:

- This function requires multiple chains of MCMC samples. See **nchain** option of samplers.

9.5.2.12 raftery_lewis

raftery_lewis computes [Raftery and Lewis, 1992] convergence diagnostics

```
convergence=raftery_lewis(this)
convergence=raftery_lewis(this,Opts)
convergence=raftery_lewis(this,Opts,chain_id)
```

Args:

this (mcmc object): mcmc object

pname (string): parameter name

chain_id (integer | {1}): choice of chain for which to do the convergence analysis

Returns:

:

- **convergence** [struct]: convergence analysis for each parameter

9.5.2.13 reload_draws

subset is a 1 x 2 cell array in which - the first cell contains the columns to retain: can be empty, defaults to all - the second column contains the chains to retain: can be empty, defaults to all

9.5.2.14 scatterplot

Make a scatter plot from the mcmc chains

```
hdl = scatterplot(mcobj,pname1,pname2);
hdl = scatterplot(mcobj,pname1,pname2,chain_id);
hdl = scatterplot(mcobj,pname1,pname2,chain_id,varargin);
```

Args:

mcobj (mcmc object): mcmc object pname1 (str): x-axis parameter of scatter plot pname2 (str): y-axis parameter of scatter plot chain_id (vector of int): id of the chain to use varargin (varargin): options fed into scatter function of matlab

Returns:

:

- **hdl** (handle object): handle to plot object

9.5.2.15 statistics

statistics : statistics from the mcmc chains

```
s=statistics(obj);

s=statistics(obj,opts);
```

Args:

- obj (mcmc object): mcmc object

- opts (struct): structure with options of which statistics to compute. for more information see help utils.mcmc.statistics

Returns:

:

- **s** (struct): structure with statistics of interest
- **statTab** (table): table representation of some of the statistics

9.5.2.16 summary

Print summary information about the mcmc draws

```
[myMeanStdev, MyQuantiles] = summary(mcobj);
[myMeanStdev, MyQuantiles] = summary(mcobj, varargin);
```

Args:

mcobj (mcmc object): mcmc object

varargin (options): options need to come in pairs:

- 'percnt': Quantile points (default: [2.5, 25, 50, 75, 97.5])
- 'batch_size': sampling size of for the computation of thinned standard deviation (SD(batch)), every 'batch_size'th points are used to compute SD(batch).

Returns:

:

- **myMeanStdev** [cell]: mean and standard deviations
- **myQuantiles** [cell]: quantile values

9.5.2.17 traceplot

Make a trace plot from the mcmc chains

```
hdl = traceplot(mcobj,pname);
hdl = traceplot(mcobj,pname,chain_id);
hdl = traceplot(mcobj,pname,chain_id,ma_window);
```

Args:

mcobj (mcmc object): mcmc object

pname (str): parameter to make the trace plot

chain_id (vector of int): id of the chain to use

ma_window (int): window size if using moving average smoothing.

Returns:

:

- **hdl** (handle object): handle to plot object

9.5.3 Visualizing posteriors (and priors)

See *Visualizing priors and posteriors*.

9.6 Marginal Data Density computation : the mdd class

9.6.1 The properties

9.6.1.1 log_post_kern

mdd/log_post_kern is a property.

9.6.1.2 theta_draws

mdd/theta_draws is a property.

9.6.1.3 lb

mdd/lb is a property.

9.6.1.4 ub

mdd/ub is a property.

9.6.1.5 maximization

mdd/maximization is a property.

9.6.1.6 H

> Hessian used in the calculation of the laplace approximation. If not provided, finite differences will be used

9.6.1.7 subset

mdd/subset is a property.

9.6.1.8 thecoef

mdd/thecoef is a property.

9.6.2 The methods

9.6.2.1 bridge

BRIDGE Computes the log marginal data density using the "bridge" approximation by [Meng and Wong, 1996].

```
log_mdd=BRIDGE(obj)

log_mdd=BRIDGE(obj,fix_point)

log_mdd=BRIDGE(obj,fix_point,opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **fix_point** [{true}|false|empty]: if true, optimization is done using a fix point algorithm. Else, an iterative strategy is used.
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

:

- **log_mdd** [numeric]: log marginal data density
- **extras** [struct]: history of log MDD and convergences at each iteration

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.2 cj

CJ Computes the log marginal data density using the [Chib and Jeliazkov, 2001] approximation

```
log_mdd=CJ(obj)

log_mdd=CJ(obj,[],opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.3 global_options

GLOBAL_OPTIONS : generic options for the control of computations for Marginal Data Density calculations. Key options include

- **center_at_mean** [{false}|true]: if true, in the calculation of moments, the covariance is centered at the mean, otherwise it is centered at the mode.
- **L** [{500}|empty|integer]: number of extra draws
- **debug** [{false}|true]: displays certain messages for debugging purposes (algorithm specific)
- **draws_are_iid** [{false}|true]:
- and the options in fix_point_iterator()

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz

9.6.2.4 is

IS Computes the log marginal data density using the "importance sampling" approximation

```
log_mdd=IS(obj)

log_mdd=IS(obj,[],opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.mhm, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.5 laplace

laplace Computes the log marginal data density

```
log_mdd=laplace(obj)
```

Args:

- **obj** [mdd]: Marginal Data Density object

Returns:

:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz

9.6.2.6 laplace_mcmc

laplace_mcmc Computes the log marginal data density using the laplace approximation and a covariance matrix computed from the draws of posterior sampling

```
log_mdd=laplace_mcmc(obj)
```

Args:

- **obj** [mdd]: Marginal Data Density object

Returns:

:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.swz

9.6.2.7 laplace_mdd

Computes the log marginal data density using the laplace approximation

```
log_mdd=laplace_mdd(log_post,Hinv)
```

Args:

- **log_post** [numeric]: log of posterior kernel evaluated at the mode
- **Hinv** [matrix]: inverse Hessian (negative of the second derivatives of the log posterior kernel)

Returns:

:

- **log_mdd** [numeric]: log marginal data density

9.6.2.8 mdd

mdd : Constructor for marginal data density objects

Syntax:

```
obj=mdd(theta_draws,log_post_kern,lb,ub)

obj=mdd(theta_draws,log_post_kern,lb,ub,subset)

obj=mdd(theta_draws,log_post_kern,lb,ub,subset,H)

obj=mdd(theta_draws,log_post_kern,lb,ub,subset,H,maximization)
```

Inputs

- **theta_draws** [char|struct]: sampling drawss => In case of a char, this is the location of the folder containing the draws organized as described below => In case of a structure, the fields are "f" and "x". Each parameter vector is defined as a structure, which means that theta_draws is a vector of structures. "x" is the parameter vector and "f" is the NEGATIVE of the log posterior kernel evaluated at "x". In case "f" is instead the log posterior kernel itself, option **maximization** below has to be set to "true".

- **lb** [empty|vector]: lower bound of the search space. Necessary only for the swz algorithm. Conveniently replaced with the lower bounds of theta_draws if empty.
- **ub** [empty|vector]: upper bound of the search space. Necessary only for the swz algorithm. Conveniently replaced with the upper bounds of theta_draws if empty.
- **subset** (cell array|{empty}): When not empty, subset is a 1 x 2 cell array in which the first cell contains a vector selecting the columns to retain in each chain and the second column contains the chains retained. Any or both of those cell array containts can be empty. Whenever an entry is empty, all the information available is selected. E.g. subsetting with dropping and trimming mysubs={a:b:c,[1,3,5]}. In this example, the first element selected is the one in position "a" and thereafter every "b" element is selected until we reach element in position "c". At the same time, we select markov chains 1,3 and 5.
- **H** [empty|matrix]: Hessian matrix for the parameters Necessary only for the laplace algorithm. If left empty, finite differences are used.
- **maximization** [{false}|true|empty]: Informs the procedure about whether we have a maximization or a minimization problem.

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz

Documentation for mdd/mdd
doc mdd

9.6.2.9 mhm

mhm Computes the log marginal data density using the "modified harmonic mean" approximation by [Geweke, 1999].

```
log_mdd=mhm(obj)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **mhm_tau** [{(.1:.1:.9)}|vector|scalar|empty]: truncation probabilities
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.10 mueller

MUELLER Computes the log marginal data density using the Ulrich Mueller's approximation

```
log_mdd=MUELLER(obj)

log_mdd=MUELLER(obj,[],opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.11 normal_weighting

mdd.normal_weighting is a function.
[loglik, v_iF_v, lik] = mdd.normal_weighting(v, det_Sigma, Sigma_i)

9.6.2.12 ris

RIS Computes the log marginal data density using the "Reciprocal Importance Sampling" approximation as in [Frühwirth-Schnatter, 2006]

```
log_mdd=RIS(obj)

log_mdd=RIS(obj,[],opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

- **log_mdd** [numeric]: log marginal data density

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.swz, mdd.global_options

9.6.2.13 swz

SWZ Computes the log marginal data density using the [Sims *et al.*, 2008] approximation

```
log_mdd=SWZ(obj)

log_mdd=SWZ(obj,swz_pvalue)

log_mdd=SWZ(obj,swz_pvalue,opts)
```

Args:

- **obj** [mdd]: Marginal Data Density object
- **swz_pvalue** [{90}|empty|scalar]: scalar for the computation of the lower bound. Must be greater than 0 and less than 100
- **opts** [empty|struct]: options see help for mdd.global_options

Returns:

:

- **log_mdd** [numeric]: log marginal data density
- **extras** [struct]: lower bound and corresponding quantile

See also:

mdd.laplace, mdd.mhm, mdd.is, mdd.ris, mdd.cj, mdd.mueller, mdd.bridge, mdd.laplace_mcmc, mdd.global_options

9.7 Indirect Inference

A good reference in the DSGE literature is [Ruge-Murcia, 2012]

9.7.1 The principle of Indirect inference

9.7.2 Generalized Method of Moments

9.7.3 Simulated Method of Moments

9.7.4 Impulse Response Matching

CHAPTER

TEN

TIME SERIES AND DATA MANAGEMENT

10.1 Dates

In the RISE toolbox, dates play a crucial role in representing time series data. Dates can be either explicit, where each observation is associated with a specific date, or undated, where dates are simply represented as a sequence of integers.

To handle dates efficiently, RISE provides an abstract class called `rise_dates.dates`, from which various derived classes inherit. These derived classes include:

- `rise_dates.DailyDate` (shortcut: `rd`)
- `rise_dates.WeeklyDate` (shortcut: `rw`)
- `rise_dates.MonthlyDate` (shortcut: `rm`)
- `rise_dates.QuarterlyDate` (shortcut: `rq`)
- `rise_dates.HalfYearlyDate` (shortcut: `rh`)
- `rise_dates.YearlyDate` (shortcuts: `ry` or `ra`)

Each derived class represents a specific frequency of dates, such as daily, weekly, monthly, etc.

10.1.1 Base Class Methods

The base class `rise_dates.dates` provides several methods that can be used with any derived class. These methods allow you to manipulate and analyze date objects effectively. Here are the commonly used methods:

- `char()`: Converts dates to character representations.
- `convert()`: Converts dates to another frequency.
- `datestr()`: Returns a string representation of dates.
- `eq()`: Checks if dates are equal.
- `gt()`: Checks if dates are greater than a given date.
- `ismember()`: Checks if dates are members of a given set of dates.
- `lt()`: Checks if dates are less than a given date.
- `min()`: Returns the minimum date.
- `ne()`: Checks if dates are not equal.
- `unique()`: Returns unique dates in a sequence.
- `colon()`: Creates a sequence of dates.

- `double()`: Converts dates to double format.
- `ge()`: Checks if dates are greater than or equal to a given date.
- `intersect()`: Returns the intersection of two sets of dates.
- `le()`: Checks if dates are less than or equal to a given date.
- `max()`: Returns the maximum date.
- `minus()`: Computes the difference between two dates.
- `plus()`: Adds a given number of periods to a date.
- `ymd()`: Extracts the year, month, and day components of dates.

These methods provide powerful functionality for manipulating, comparing, and performing operations on date objects. For more detailed information on each method, refer to the documentation.

10.1.2 Help Text for Derived Classes

To make it easier for users to understand and use the derived classes, we have provided detailed help text for each of them (`rd`, `rw`, `rm`, `rq`, `rh`, `ry`, and `ra`). The help text contains information about the specific properties and methods available for each class.

10.1.2.1 Daily dates : rd

RD DailyDate constructor.

Usage:

obj = rd(year, month, dayOfTheMonth) creates a
rd object for the specified year, month, and day.

obj = rd(year, dayOfTheYear) creates a rd
object for the specified year and day of the year.

obj = rd(dateString) creates a rd object from
a string in the format 'mm/dd/yyyy' or 'yyyyDddd' or 'yyyy-mm-dd'.

obj = rd(dateNumber) creates a rd object from
a serial date number.

Inputs:

- year: Year of the date.
- month: Month of the date.
- dayOfTheMonth: Day of the month.
- dayOfTheYear: Day of the year.
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyyDddd'.
- dateNumber: A serial date number representing the date.

Example:

dateObj = rd(2023, 7, 15);

dateObj = rd(2023, 165);

dateObj = rd('07/15/2023');

dateObj = rd('2023D165');

dateObj = rd(737999);

See also rd, rw, rm, rq, rh, ra, ry

10.1.2.2 Weely dates : rw

RW constructor.

Usage:

obj = rw(year, fullWeeks)

obj = rw(dateString)

obj = rw(datenumber)

Inputs:

- year: Year of the date.
- fullWeeks: Number of full weeks.
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyyWp' or 'yyyy-mm-dd'.

Example:

dateObj = rw(2023, 10);

dateObj = rw('07/01/2023');

dateObj = rw('1990W1');

dateObj = rw(737999);

See also rd, rm, rq, rh, ra, ry

10.1.2.3 Monthly dates : rm

RM constructor.

Usage:

obj = rm(year, month)

obj = rm(dateString)

obj = rm(datenumber)

Inputs:

- year: Year of the date.
- month: Month of the date (1 to 12).
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyy-mm-dd'.
- datenumber: regular date number

Example:

dateObj = rm(2023, 7);

dateObj = rm('07/01/2023');

dateObj = rm(737999);

See also rd, rw, rq, rh, ra, ry

10.1.2.4 Quarterly dates : rq

RQ constructor.

Usage:

- obj = rq(year, quarter)
- obj = rq(dateString)
- obj = rq(datenumber)

Inputs:

- year: Year of the date.
- quarter: Quarter of the date (1, 2, 3, or 4).
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyyQp' where p=1,2,3,4 or 'yyyy-mm-dd'

Example:

```
dateObj = rq(2023, 2);
dateObj = rq('2023Q2');
dateObj = rq('04/01/2023');
dateObj = rq(737999);
```

See also rd, rw, rm, rh, ra, ry

10.1.2.5 Half-yearly dates : rh

RH constructor.

Usage:

```
obj = rh(year, semi)
obj = rh(dateString)
obj = rh(datenumber)
```

Inputs:

- year: The year of the date.
- semi: The semi-annual period (1 or 2).
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyyHp' where p=1,2 or 'yyyy-mm-dd'.
- datenumber: regular date number

Example:

```
dateObj = rh(2023, 1);
dateObj = rh('2023H1');
dateObj = rh('01/01/2023');
dateObj = rh(737999);
```

See also rd, rw, rm, rq, ra, ry

10.1.2.6 Yearly dates : ry

RY constructor for YearlyDate.

Usage:

obj = ry(year)

obj = ry(dateString)

dateObj = ry(datenumber);

Inputs:

- year: Year of the date in double (e.g., 2023)
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyy' or 'yyyy-mm-dd'
- datenumber: A regular matlab datenumber. Note that the function will treat the input as a date number if it has more than 4 digits. A warning will be issued as well

Example:

dateObj = ry(2023);

dateObj = ry('2023');

dateObj = ry('01/01/2023');

dateObj = ry(737999);

See also rd, rw, rm, rq, rh, ra.

10.1.2.7 Annual dates : ra

RA constructor for YearlyDate.

Usage:

obj = ra(year)

obj = ra(dateString)

dateObj = ra(datenumber);

Inputs:

- year: Year of the date in double (e.g., 2023)
- dateString: A string in the format 'mm/dd/yyyy' or 'yyyy' or 'yyyy-mm-dd'
- datenumber: A regular matlab datenumber. Note that the function will treat the input as a date number if it has more than 4 digits. A warning will be issued as well

Example:

dateObj = ra(2023);

dateObj = ra('2023');

dateObj = ra('01/01/2023');

dateObj = ra(737999);

See also rd, rw, rm, rq, rh, ry

Once you are familiar with the concepts and usage of date objects in the RISE toolbox, you can proceed to explore the handling of time series data, which builds upon the foundation of dates.

10.1.3 Technical documentation for rise_dates.dates objects

10.1.3.1 The properties

Frequency

Frequency - The frequency of the dates

Incr

Incr - Place holder for the increment value for the dates

date_time

date_time - The date and time values represented as a datetime object

10.1.3.2 The methods

char

CHAR Convert date objects to formatted date strings.

s = char(obj) converts the date objects in the input array obj to formatted date strings using the default format based on the frequency of the dates. The formatted date strings are returned as a character array.

s = char(obj, format) converts the date objects to formatted date strings using the specified format. The format can be a character vector or a string.

Inputs: - obj: Array of date objects. - format: Format for the date strings (optional). If not provided, the default format based on the frequency of the dates is used.

Output: - s: Character array of formatted date strings.

Examples: - daily dates: d = rd(1990,1,25):rd(1990,6,25); disp(char(d)) - weekly dates: w = rw(1990,1):rw(1990,53); disp(char(w)) - monthly dates: m = rm(1990,1):rm(1990,12); disp(char(m)) - quarterly dates: q = rq(1990,1):rq(1990,4); disp(char(q)) - half-yearly dates: h = rh(1990,1):rh(1995,2); disp(char(h)) - yearly dates: y = ry(1990):ry(1995); disp(char(y)) - annual dates: a = ra(1990):ra(1995); disp(char(a))

colon

colon Creates an array of dates using the colon operator.

Usage:
dateArray = colon(obj1, obj2)

Inputs:

- obj1: The starting date object.
- obj2: The ending date object.

Outputs:

- dateArray: An array of dates spanning from obj1 to obj2.

Example:
dates = obj1 : obj2;

See also rise_dates.dates.

convert

convert Converts the dates to a different frequency.

Usage:

d = convert(d, freq)

Inputs:

- d: The date object.
- **freq: The target frequency to convert the dates to. Valid values are**
  'D' (daily), 'W' (weekly), 'M' (monthly), 'Q' (quarterly), 'H' (half-yearly), and 'Y' (yearly).

Outputs:

- d: The date object with dates converted to the specified frequency.

Example:

newDateObj = convert(dateObj, 'M');

See also rise_dates.dates.

dates

dates Constructor method to create a dates object.
obj = dates(year, month, day, frequency, incr) creates a dates object with the specified year, month, day, frequency, and increment.

Arguments: - year: The year value of the dates. - month: The month value of the dates. - day: The day value of the dates. - frequency: The frequency of the dates (e.g., 'D' for daily, 'M' for monthly). - incr: The increment value for the dates.

Documentation for rise_dates.dates/dates
doc rise_dates.dates

datestr

DATESTR Convert date objects to formatted date strings.

s = datestr(obj) converts the date objects in the input array obj to formatted date strings using the default format based on the frequency of the dates. The formatted date strings are returned as a character array.

s = datestr(obj, format) converts the date objects to formatted date strings using the specified format. The format can be a character vector or a string.

Inputs:

- obj: Array of date objects.
- format: Format for the date strings (optional). If not provided, the default format based on the frequency of the dates is used.

Output:

- s: Character array of formatted date strings.

Examples:

daily dates

d=rd(1990,1,25):rd(1990,6,25); disp(datestr(d))

weekly dates

w=rw(1990,1):rw(1990,53); disp(datestr(w))

monthly dates

m=rm(1990,1):rm(1990,12); disp(datestr(m))

quarterly dates

q=rq(1990,1):rq(1990,4); disp(datestr(q))

half-yearly dates

h=rh(1990,1):rh(1995,2); disp(datestr(h))

yearly dates

y=ry(1990):ry(1995); disp(datestr(y))

annual dates

a=ra(1990):ra(1995); disp(datestr(a))

double

DOUBLE returns the serial number representation of the date object.

Returns:

- serial: The serial number representation of the date object.

Example:

dateObj = daily(2023, 5, 17); serialNum = double(dateObj); disp(serialNum) % 737988

See also datenum.

intersect

intersect returns the intersection of two date objects.

Usage:

[C, IA, IB] = intersect(A, B)

Inputs:

- A: First date object of class 'rise_dates.dates'.
- B: Second date object of class 'rise_dates.dates'.

Outputs:

- C: Date object representing the intersection of A and B.
- IA: Indices of the common elements in A.

- IB: Indices of the common elements in B.

Example:

A = DailyDate(2023, 1, 1:10); B = DailyDate(2023, 5, 1:10); [C, IA, IB] = intersect(A, B);

See also rise_dates.dates

ismember

ISMEMBER Determine which elements of a dates object are in another dates object.

[LIA, LOCB] = ismember(A, B) returns logical arrays LIA and LOCB, where LIA contains true for the elements in A that are also in B, and LOCB contains the indices of the corresponding elements in B. The input A can be either a character vector representing a date or a rise_dates.dates object. B must be a rise_dates.dates object.

Inputs:

- A: The input dates to check for membership. It can be either a character vector representing a date or a rise_dates.dates object.
- B: The dates object to search for membership.

Outputs:

- LIA: Logical array of the same size as A, containing true for the elements in A that are also in B, and false otherwise.
- LOCB: Array of the same size as A, containing the indices of the corresponding elements in B. If an element in A is not found in B, the corresponding element in LOCB is set to 0.

Example:

datesA = rd('2023-06-18'):rd('2023-06-20'); % Daily dates datesB = rd('2023-06-19'):rd('2023-06-22'); % Daily dates [LIA, LOCB] = datesA.ismember(datesB);

Notes:

- If A is a character vector representing a date, it is converted to a rise_dates.dates object using the date2serial function.
- The function checks for membership based on the frequency of the dates objects. If the frequencies do not match, the function returns empty arrays.
- The tolerance (Tol) for comparing dates is set to 1e-3. If the frequencies of the dates objects are 'M', 'Q', 'H', or 'Y', the tolerance is increased by 1.
- The function compares the elements of A with B and returns true for elements that are within the tolerance of B. LOCB contains the indices of the corresponding elements in B. If an element in A is not found in B, the corresponding element in LOCB is set to 0.

See also:

rise_dates.dates, date2serial

max

MAX Find the maximum date among two dates or a vector of dates.

MAX(DATE1, DATE2) returns the maximum date between DATE1 and DATE2.

MAX(DATES) returns the maximum date from the vector of dates DATES.

Input arguments:

- DATE1, DATE2: Date objects to compare.
- DATES: Vector of date objects.

Output argument:

- MAXDATE: Maximum date among the inputs.

min

MIN Find the minimum date among two dates or a vector of dates.

MIN(DATE1, DATE2) returns the minimum date between DATE1 and DATE2.

MIN(DATES) returns the minimum date from the vector of dates DATES.

Input arguments:

- DATE1, DATE2: Date objects to compare.
- DATES: Vector of date objects.

Output argument:

- MINDATE: Minimum date among the inputs.

minus

MINUS - Subtracts a specified number of periods to the date object.

Syntax:

newObj = obj-decrement

Input Arguments:

- obj: The date object of the rise_dates.dates class.
- decrement: The number of periods to subtract to the date object. It can be a numeric value (integer) or another rise_dates.dates object.

Output Argument:

- newObj: The new date object after subtracting the specified number of periods. If *decrement* is a rise_dates.dates object, the output is the difference of the date numbers.

Example:

newDateObj = obj+3;

See also:

rise_dates.dates

plus

plus - Adds a specified number of periods to the date object.

Syntax:

newObj = plus(obj, increment)

Input Arguments:

- obj: The date object of the rise_dates.dates class.
- increment: The number of periods to add to the date object. It can be a numeric value (integer) or another rise_dates.dates object.

Output Argument:

- newObj: The new date object after adding the specified number of periods. If *increment* is a rise_dates.dates object, the output is the sum of the date numbers.

Example:

newDateObj = obj+3;

See also:

rise_dates.dates

unique

UNIQUE Find unique elements in an array of dates.

[C, IA, IC] = unique(obj) returns the unique elements of the input date array obj. The output C contains the sorted unique values of obj. The output IA is an index vector such that C = obj(IA). The output IC is an index vector that specifies the index of each value of obj in the sorted list C. If obj is an array of date objects, the uniqueness is determined based on the date values stored in the objects. If obj is a numeric or character array, the uniqueness is determined based on the values in obj itself.

[C, IA, IC] = unique(obj, varargin) allows additional optional arguments to be passed to the unique function, See the unique function for "double".

Inputs:

- obj: Array of dates or numeric/character array.
- varargin (optional): Additional arguments for the unique function.

Outputs:

- C: Sorted array of unique values from obj.
- IA: Index vector such that C = obj(IA).
- IC: Index vector that specifies the index of each value of obj in the sorted list C.

Examples: - Unique dates from a date array: d1 = rm(1990,1):rm(1995,12); d2 = rm(1992,1):rm(1998,12); [C, IA, IC] = unique([d1,d2]) assert(isequal(C,rm(1990,1):rm(1998,12)))

xline

xline overloads xline for the rise_dates.dates class. It creates vertical lines at specified x-values. It is typically used to mark important dates or events on plots.

Syntax:

xline(VALUES) xline(VALUES, LINESPEC) xline(VALUES, LINESPEC, LABELS) xline(...Name,Value) XL = xline(...)

Arguments:

- values: A scalar or a vector of date values where the vertical lines will be placed.
- LINESPEC: line style, color, or both. For example, ':' creates a dotted line, 'r' creates a red line, and 'r:' creates a dotted, red line.
- LABELS: When VALUES contains multiple elements, you can display different labels for each line by specifying LABELS as either cell array of character vectors or a string array with the same number of elements as VALUES. Alternatively, specify LABELS as a character vector or a string scalar to display the same label on all the lines.

Example 1:

```
xstart=rq(1990,1);
db=ts(xstart,log((1:100).'));
plot(db)
xline([rq(1996,3),rq(2009,4)])
```

Example 2:

```
xline([rq(1996,3),rq(2009,4)], 'r--', {'Holiday', 'Event'})
```

Note : for more explanations, see Matlab's xline.

ymd

ymd - Extracts the year, month, and day components from the dates.

Syntax:
[Y, M, D] = ymd(this)

Input Arguments:

- this: An object of the rise_dates.dates class or its derived classes representing a collection of dates.

Output Arguments:

- Y: Numeric array of years.
- M: Numeric array of months.
- D: Numeric array of days.

Example:
[Y, M, D] = ymd(dateObj)

See also:

rise_dates.dates

10.2 Time Series

This section provides information about time series and their usage in the context of the RISE toolbox. It covers topics such as:

- Introduction to time series analysis
- Working with time series data in RISE
- Time series transformations and manipulations
- Statistical analysis techniques for time series

Time series data in essence is just a matrix, so in theory, one can just use matrices to represent the data. However, all practitioners will share their frustration in trying to remember the index location of the GDP and the annoyance in trying to find out what *data{:,5}* written by a co-worker represents. The RISE toolbox addresses this problem by providing a time series object that uses natural indices, improving readability and reducing errors.

To use the time series object, you can initialize it with the following code:

```
db = ts('1990q1', randn(10,4), {'gdp', 'r', 'wage', 'capital'})
```

The first argument specifies the starting date of the time series and its frequency. In this example, the time series starts from the first quarter of 1990. The second argument is the data matrix, where each row represents a time period and each column represents a variable. Here, the time series consists of 4 variables spanning 10 quarters. Lastly, the third argument provides the natural names of the variables for intuitive indexing.

With the time series object, you can access the data using natural indexing. For example, to retrieve the wage in the second quarter of 1991, you can use the following code:

```
print(db('1991q2', 'wage'))
```

Alternatively, you could access the data directly from the matrix representation using numerical indexing:

```
print(db{7, 3})
```

Note that numerical indexing is 1-based, which can lead to off-by-one errors.

You can also select a range of dates and multiple variables simultaneously:

```
print(db('1990q2:1991q2', {'gdp', 'wage'}))
```

Furthermore, if you want to provide descriptions or comments for each variable, you can construct the time series object with optional descriptions:

```
db = ts('1990q1', randn(10,2), {'gdp', 'pi'}, {'real gdp', 'inflation rate'})
```

You can access the descriptions/comments by calling:

```
print(db)
```

Alternatively, you can set variable names and descriptions using the *set* function:

```
set(db, 'varnames', {'gdp', 'pi'}, 'descriptions', {'real gdp', 'inflation rate'})
```

The time series object provides convenient methods and intuitive indexing, making it easy to work with time series data. However, if there are functions not supported within RISE, you can directly access the underlying data matrix using the *values* function:

```
values(db)
```

The time series object seamlessly integrates with other parts of the RISE toolbox, enabling you to perform VAR and DSGE analyses effortlessly. The automation provided by the time series object simplifies various data management tasks. The following are some commonly used functions and their functionalities:

10.2.1 Data Cleaning

- *reset_data*: Resets the data in the time series object.
- *reset_start_date*: Resets the start date of the time series object.
- *interpolate*: Fills in missing data by interpolating.
- *aggregate*: Aggregates data to a coarser frequency (e.g., monthly data to quarterly).
- *dummy*: Creates a dummy variable.
- *step_dummy*: Creates a step dummy variable.
- *cat*: Joins multiple time series objects.

10.2.2 Time Series Transformations

- *hpfilter*: Applies the Hodrick-Prescott (HP) filter to the time series data.
- *ma_filter*: Applies a moving-average filter to the time series data.
- *pdecomp*: Performs a parametric trend-seasonal-residual decomposition.
- *npdecomp*: Performs a non-parametric trend-seasonal-residual decomposition.

10.2.3 Statistical Analysis

- *regress*: Performs OLS regression on the time series data.
- *chowlin*: Conducts the Chow-Lin test.
- *jbtest*: Performs the Jarque-Bera (JB) test.

10.2.4 Plotting tools for time series

- *bar*: Creates a bar plot.
- *fanchart*: Generates a fan chart.
- *plot*: Plots the time series data.
- *plot_real_time*: Plots the time series data in real time conditioning.
- *plotyy*: Creates a plot with two separate axes.

10.2.5 Other Automations

- *transform*: Provides a variety of transformations for the time series data.

For a comprehensive understanding of all the automation and functionalities available in the time series object, refer to the *ts-technicals* documentation.

10.2.6 Technical documentation for ts objects

10.2.6.1 The properties

varnames

names of the variables in the database

data

time series

description

comments on the time series

NumberOfObservations

number of observations in the time series

NumberOfPages

number of pages (third dimension) of the time series

NumberOfVariables

number of variables in the time series

finish

end time of the time series

frequency

frequency of the time series

start

start time of the time series

10.2.6.2 The methods

abs

Computes the absolute value of a ts object

```
db=abs(db)
```

Args:

db (ts): RISE time series object

Returns:

:

- **db** : [ts]: a time series

acos

Overloaded acos function for ts object

acosh

Overloaded acosh function for ts object

acot

Overloaded acot for ts object

acoth

Overloaded acoth for ts object

allmean

Compute all different type of means for the time series variables

```
m = allmean(db);
```

Args:
db (ts object): the times series object to compute the mean of

Returns:
: - m (cell): Cell containing

- 1st column: strings denoting the type of mean (Harmonic, Geometric, Arithmetic, Quadratic)
- 1st row: variable names
- otherwise: mean value corresponding to the variable and mean concept.

and

INTERNAL FUNCTION: Combines two databases into a combined database

Note:

- It is assumed that the frequency and length of time series are the same between the two databases.

apply

APPLY - Applies a unary function to each element of a time series database

Syntax:
db = apply(db, fhandle)

Inputs:
db - Time series database (ts object) fhandle - Unary function handle that accepts a scalar input and returns a scalar output

Outputs:
db - Resulting time series database after applying the unary function to each element

Description:
The 'apply' function applies a unary function to each element of a time series database. It performs the operation element-wise and returns a new time series database with the transformed values.

Examples:
% Create a time series database dates = rd('2000-01-01'); data = rand(5, 3); varnames = {'Var1', 'Var2', 'Var3'}; db = ts(dates, data, varnames);

% Define a unary function fhandle = @(x) log(x);

% Apply the function to each element of the time series database transformed_db = apply(db, fhandle);

% Display the resulting time series database disp(transformed_db);

See also:

ts, unary_operation

asin

Overloaded asin for ts object

asinh

Overloaded asinh for ts object

atan

Overloaded atan for ts object

atanh

Overloaded atanh for ts object

bar

Make bar graph of the time series

```
varargout = bar(X,db,varargin);
```

Args:

X (dates): Date description

- string of dates, e.g., X='1990:2000'
- serial dates, e.g., X=date2serial('1990Q1'):date2serial('2000Q1')

db (ts object): Time series object with data options (options): Options need to come in pairs

- 'figsize' ([value, value]): the figure size (multiple plots) with default [3,3]
- 'figtitle' (string): the figure title (multiple plots) with default ''
- the number of tick marks (integer): 'nticks', with default 8
- 'date_format': the date format (see matlab's datestr) with default ''
- 'logy' (bool): the log scale with default false
- 'secondary_y': the list of variables going to the secondary y axis
- 'subplots' (bool): the flag for multiple plots with default false

Note:

- bar(db) uses the dates within db. The colors are set by the colormap.
- bar(X,db,WIDTH) or bar(db,WIDTH) specifies the width of the bars. Values of WIDTH > 1, produce overlapped bars. The default value is WIDTH=0.8
- bar(...,'grouped') produces the default vertical grouped bar chart. bar(...,'stacked') produces a vertical stacked bar chart. bar(...,LINESPEC) uses the line color specified (one of 'rgbymckw').
- H = bar(...) returns a vector of handles to barseries objects.
- Use SHADING FACETED to put edges on the bars. Use SHADING FLAT to turn them off.

Example:

```
bar('1994m7:1997m1',db(:,:,1),...
    'figsize',[2,2],...
    'figtitle','no title',...
    'nticks',10,...
    'legend',{'v1','v2'},...
    'legend_loc','BO',...
    'logy',true,...
    'secondary_y',{'v1','v4'},...
    'subplots',true,...
    'linewidth',2,...
    'date_format',17);
xrotate(90)
```

See also:

- `hist`
- `plot`
- `barh`
- `bar3`
- `bar3h`

barh

Make bar graph of the time series

```
varargout = bar(X,db,varargin);
```

Args:

X (string | serial): Date description

- string of dates, e.g., X='1990:2000'
- serial dates, e.g., X=date2serial('1990Q1'):date2serial('2000Q1')

db (ts object): Time series object with data options (pairs): Options need to come in pairs

- 'figsize' ([value, value]): the figure size (multiple plots) with default [3,3]
- 'figtitle' (string): the figure title (multiple plots) with default ''
- the number of tick marks (integer): 'nticks', with default 8
- 'date_format': the date format (see matlab's datestr) with default ''
- 'logy' (bool): the log scale with default false
- 'secondary_y': the list of variables going to the secondary y axis
- 'subplots' (bool): the flag for multiple plots with default false

Note:

- bar(db) uses the dates within db. The colors are set by the colormap.
- bar(X,db,WIDTH) or bar(db,WIDTH) specifies the width of the bars. Values of WIDTH > 1, produce overlapped bars. The default value is WIDTH=0.8

- bar(…,'grouped') produces the default vertical grouped bar chart. bar(…,'stacked') produces a vertical stacked bar chart. bar(…,LINESPEC) uses the line color specified (one of 'rgbymckw').
- H = bar(…) returns a vector of handles to barseries objects.
- Use SHADING FACETED to put edges on the bars. Use SHADING FLAT to turn them off.

Example:

```
bar('1994m7:1997m1',db(:,:,1),...
    'figsize',[2,2],...
    'figtitle','no title',...
    'nticks',10,...
    'legend',{'v1','v2'},...
    'legend_loc','BO',...
    'logy',true,...
    'secondary_y',{'v1','v4'},...
    'subplots',true,...
    'linewidth',2,...
    'date_format',17);
xrotate(90)
```

See also:

- `hist`
- `plot`
- `barh`
- `bar3`
- `bar3h`

boxplot

Make box plots of multiple time series in a frame. Overloads Matlab's boxplot function for ts objects

```
h = boxplot(db);
h = boxplot(xrange, db);
h = boxplot(..., varargin);
```

Args:

db (ts object): time series object

xrange (char | cellstr | serial date): Range of the data to plot

varargin (pairs): additional matlab (See BOXPLOT) and RISE (PARSE_PLOT_ARGS) options coming in pairs

See also:

parse_plot_args

bsxfun

Overloaded bsxfun for ts object

cat

Concatenates time series along the specified dimension

```
db=cat(1,db1,db2,...,dbn)
db=cat(2,db1,db2,...,dbn)
db=cat(3,db1,db2,...,dbn)
```

Args:

dim (1 | 2 | 3): dimension along which concatenation is done

dbi (ts object): time series object

Returns:

- **db** [ts] : time series with concatenated series

Note:

- all times series must be of the same frequency (see `aggregate`)
- Concatenation along the second dimension requires that variables have the same number of columns if no names are specified
- if names are specified in the first time series, then names should be specified in all of the others as well.
- empty time series are discarded but there should be at least one non-empty time series

chebyshev_box

Constructs chebyshev boxes for multivariate-multiperiods densities

```
mvcb=chebyshev_box(this,gam)
```

Args:

this (ts | rts)
[time series with many pages (number of simulations)] and potentially many columns (number of variables)

gam (scalar | vector) : percentile(s)

Returns:
:

- **mvcb** [struct] : structure with time series in its fields. Each field represents a particular variable
- **gam_** [scalar|vector] : sorted percentile(s)
- **my** [ts] : mean across simulations

chowlin

Temporal disaggregation using the Chow-Lin method

```
[yh,res]=chowlin(y0,Xh0)
[yh,res]=chowlin(y0,Xh0,aggreg_type)
[yh,res]=chowlin(y0,Xh0,aggreg_type,estim_method)
[yh,res]=chowlin(y0,Xh0,aggreg_type,estim_method,ngrid)
```

Args:

y0 (ts object): low-frequency left-hand-side variable

Xh0 (ts | struct): high-frequency right-hand-side explanatory variables

aggreg_type ('flow' | {'average'} | 'index' | 'last' | 'first'): type of aggregation:

- 'flow' (or 1) is the sum,
- 'average','index' (or 2) is the average,
- 'last' (or 3) is the last element,
- 'first' (or 4) is the first element,

estim_method ({0} | 1 | 2) : estimation method

- 0 : Generalized/Weighted Least squares (with grid)
- 1 : Maximum Likelihood (with grid),
- 2 : Maximum Likelihood (without grid) using fmincon as optimizer

ngrid (integer | {250}) : number of grid points

Returns:

:

- **yh** [ts] : (disaggregated) high-frequency left-hand-side variable
- **res** [struct] : structure with further details on the computations

REFERENCES:

- Chow, G. and Lin, A.L. (1971) "Best linear unbiased distribution and extrapolation of economic time series by related series", Review of Economic and Statistics, vol. 53, n. 4, p. 372-375.
- Bournay, J. y Laroque, G. (1979) "Reflexions sur la methode d'elaboration des comptes trimestriels", Annales de l'INSEE, n. 36, p. 3-30.

collect

Brings together several time series objects into a one time series

Syntax:

this=ts.collect(v1,v2,...,vn)

Args:

Vi (cell | ts | struct): time series in ts format:

- cell: When Vi is a cell, then its format should be {vname,ts} i.e. the first element is the name of the variable and the second is the data for the variable. In this case, the data must be a single time series
- ts:
- struct: the fields of the structure should be of the ts format.

Returns:

- **this** [ts]: a time series with many columns and potentially many pages

concatenator

Concatenate outputs from multiple time series or structures of time series

```
myirfs=concatenator(db1,db2,...,dbn)
```

Args:

dbi (struct | ts): time series or structure of time series

Returns:

- **myirfs** [struct] : structure containing the concatenated time series

convert

CONVERT Converts data in different frequencies.

Usage:

newdb = convert(db, newFrequency)

newdb = convert(db, newFrequency, squash)

Inputs

- db: Time series data structure (ts class) with the following fields:
  - db.dates: Dates of the time series data
  - db.data: Data values corresponding to the dates
- newFrequency: Desired frequency for the converted time series. Supported frequencies are:
  - 'D': Daily
  - 'W': Weekly
  - 'M': Monthly
  - 'Q': Quarterly
  - 'H': Half-yearly
  - 'Y': Yearly
- squash (optional): Aggregation function used to combine data points within each time period. It can be either a function handle or a character array representing a function name. If not provided, the default aggregation function is 'mean'. NOTE : The squash function will be applied ONLY when going from higher to lower frequency (aggregation). Otherwise, the higher frequency is written as is in the dates corresponding to the observations, and nan elsewhere.

Outputs:

- newdb: Converted time series data structure (ts class) with the following fields: - newdb.dates: Dates of the converted time series - newdb.data: Data values corresponding to the dates

  Example: dates = rd(2022, 1, 1); data = randn(365, 1); db = ts(dates, data); newdb = convert(db, 'M', @sum);

  See also MEAN, SUM.

Notes:

- The function converts the data in the time series db into the specified newFrequency by aggregating the data using the provided squash function.
- If the newFrequency is the same as the original frequency of the time series, the function returns the input db unchanged.
- The function supports different aggregation functions specified by the squash argument. The default aggregation function is 'mean'.
- The input db must be a time series represented by the ts class, with dates and data values provided.
- The function works with daily, weekly, monthly, quarterly, half-yearly and yearly frequencies.
- For frequencies higher than daily (e.g., weekly, monthly), the function aggregates the data within each time period based on the specified squash function.

corr

Computes correlation for time series: It is an interface to MATLAB implementation of corr

```
varargout = corr(db,varargin);
```

Args:

db (ts object): times series object

varargin: varargin for corr function in MATLAB

Returns:

:

varargout: output from corr function

corrcoef

Computes correlation for time series: It is an interface to MATLAB implementation of corrcoef

```
varargout = corrcoef(db,varargin);
```

Args:

db (ts object): times series object varargin: varargin for corrcoef function in MATLAB

Returns:

:

varargout: output from corrcoef function

cos

Overloaded cos function for ts object

cosh

Overloaded cosh function for ts object

cot

Overloaded cot function for ts object

coth

Overloaded coth function for ts object

cov

Computes covariances for time series: It is an interface to MATLAB implementation of cov adjusted to handle nan properly

```
varargout = cov(db,varargin);
```

Args:
db (ts object): times series object varargin: varargin for cov function in MATLAB

Returns:
:

varargout: output from cov function

cumprod

Overloaded cumprod function for ts object

cumsum

Overloaded cumsum function for ts object

describe

Print the description of the time series object

Args:

db (ts object): time series to describe

Note:

The function prints

- mean
- standard deviation
- min
- 25th percentile
- 50th percentile
- 75th percentile
- max
- variable names
- variable descriptions

disp

Overloaded disp function for ts object

double

DOUBLE: Returns the underlying data of the time series

```
data = double(db);
```

Args:

db (ts object): time series object

Returns:

: - data (numeric): vector/matrix/tensor form of the data underlying the time series

drop

Drops the variable from the time series

```
db=drop(db,'var_name');
```

Args:

db (ts object): time series object varargin (string): names of the variables to drop

Returns:

:

- db (ts object): time series object with corresponding variables dropped

dummy

Creates a dummy observation times series

```
db=dummy(start_date,end_date,dummy_date)
```

Args:

- **start_date** (char | serial date): start date of the time series
- **end_date** (char | serial date): end date of the time series
- **dummy_date** (char | serial date): date(s) at which to time series is 1 and not 0

Returns:

- **db** [ts]: scalar time series of dummy observations

dust_up

Sets insignificant digits to zero

```
this=dust_up(this)
this=dust_up(this,crit)
```

Args:

this (rts | ts): time-series object

crit (numeric | {sqrt(eps)}): cutoff point

Returns:

:

- **this** [rts|ts]: time-series object

end

END Overloads end to time series (ts) class.

ind = END(self, k, n)

This function overloads the end function for the time series (ts) class. It computes the index at the end of a given dimension.

- *self*: Time series object.
- *k*: Dimension requested.
- *n*: Number of dimensions of self

Returns: - *ind*: Index at the end of the specified dimension.

Example: - ind = END(self, k, n)

See also:

ts

exp

Overloaded exp function for ts object

expanding

Applies a function to an expanding window of the time series

```
h = expanding(db,func);
h = expanding(db,func,varargin);
```

Args:

db (ts object): time series object to get data

func: function that will apply to the expanding window of the time series. The function is recursively applied to the data 1:t for t=1,2,3,…,T, where T is the number of observations

varargin: additional arguments to func

fanchart

FANCHART Creates data for plotting a fan chart.

out = fanchart(this, ci) generates the data required to plot a fan chart. The input "this" is a time series object with either multiple pages or multiple columns, but not both. The input "ci" is a vector specifying the confidence levels in the range [0, 1] or [0, 100] to be used in calculating the width of the fans.

The output "out" is a structure with the following fields:

- ci: Vector of confidence levels (same as input ci)
- median: Vector of medians of the data
- variance: Vector of variances of the data
- quantiles: Matrix of quantiles defined by ci
- prob_index: Vector of locations of the cutoffs in the data
- probs: Vector of probabilities for the quantiles (function of ci)
- dates: Vector of serial dates for reconstructing the time series

Example:

- Generate data for a fan chart and plot it:

this = ts('1990Q2', rand(100, 1000));

out = fanchart(this, [30, 50, 70, 90]);

plot_fanchart(out, 'r', 10);

Note:

- If the input time series contains several variables, the output is a structure with the names of the different variables in the first level of the fields.

See also PLOT_FANCHART.

fold

Folds the first page of possibly several databases and adds further pages using the information from a pivot date.

Usage:

this = fold(last_hist_date, obj)

this = fold(last_hist_date, obj1, obj2, . . . , objn)

Inputs:

- last_hist_date: A character string or serial date representing the pivot date. The rows after this date will be folded behind the corresponding row.
- obj: A time series object (ts).
- obj1, obj2, . . . , objn: Additional time series objects (ts).

Output:

- this: A folded time series object (ts) with the first page of data from the input time series objects, followed by folded pages.

See also:

- `unfold`

get

get Access/Query time series property values.

VALUE = get(TS,'PropertyName') returns the value of the specified property of the time series object. An equivalent syntax is

VALUE = TS.PropertyName

get(TS) displays all properties of TS and their values.

group

Groups contributions

```
g=group(this,{group1,{v11,v12,...}},...,{groupn,{vn1,vn2,...}})
```

Args:

- **this** (ts): time series of multiple variables
- **groups** : [structure|cell array |{empty}] grouping of shocks in the decomposition. By default, the shocks are not grouped. The syntax is of the form {group1,{v11,v12,. . . },. . . ,groupn,{vn1,vn2,. . . }}. The shocks that are not listed are put in a special group called "others". The "others" group does not include the effect of initial conditions. e.g.:

```
p=struct();
p.demand={'Ey','Er'};
p.supply={'Ep'};
```

e.g. p={ 'demand',{ 'Ey','Er' },'supply',{ 'Ep' }};

Returns:

- **g** [ts]: new time series with grouped contributions

head

Returns the first few sample dates of the time series

Syntax:

db = head(db);

db = head(db,n);

Args:

- db (ts object): time series object
- n (integer): number of time steps to show

Returns:

- db (ts object): time series with the first n-time steps

Note:

- This is similar to the head function in stata.

hist

Make histograms from time series

```
h = hist(db);
h = hist(xrange, db);
h = hist(xrange, db, M);
h = hist(xrange, db, X);
```

Args:

db (ts object): time series object

xrange: date range. Check XXXXX for correct format

M (integer): number of bins (default 10)

X (vector): histogram with bin centers given by X

Returns:
: h (figure handle): handle to the plotted histogram

Example:

```
hist('1994m7:1997m1',db(:,:,1),...
    'figsize',[2,2],...
    'figtitle','no title',...
    'logy',true,...
    'subplots',true)
```

Note:
In addition to those matlab properties, RISE adds further properties, which allow to control for. See parse_plot_args

horzcat

Combines/concatenate databases into a combined database

Note:

- It is assumed that the frequency and length of time series are the same between the two databases.

hpfilter

HP filters a collection of time series.

Args:

db (ts object): time series object

lambda (double): scalar, lambda parameter. Refer to wikipedia

Returns:

:

- hptrend (double): T*n ts object, trend component of db.
- hpcycle (double): T*n ts object, cycle component of db.

imag

imag : overloads imag for time series

Syntax

```
this=imag(this);
```

Args:

this (ts object): time series object

Returns:

- **this** (ts object): time series object

interpolate

Interpolate based on the time series to fill in the missing dates

```
db = interpolate(db);
db = interpolate(db, method, varargin);
```

Args:

db (ts object): time series object to interpolate from

method (string): interpolate method (default: spline). Same as the option used in MATLAB, i.e.,

- 'nearest': nearest neighbor interpolation
- 'linear': linear interpolation
- 'spline': piecewise cubic spline interpolation (SPLINE)

- 'pchip': shape-preserving piecewise cubic interpolation
- 'cubic': same as 'pchip'
- 'v5cubic': the cubic interpolation from MATLAB 5, which does not extrapolate and uses 'spline' if X is not equally spaced.

varargin{1}: optional condition for extrapolation (default: no extrapolation)

- 'extrap': extrapolate out for dates outside the dates with observations
- extrapval (double): use extrapval for dates outside the dates with observations

isfinite

Returns whether the corresponding data is finite

```
flag = isfinite(db);
```

Args:

db (ts object): time series object

Returns:

:

- flag (bool): whether the corresponding data is finite or not

Note:

- Since the time series object supports logical indexing, one can directly use the resulting flag with the time series object.

isinf

Returns whether the corresponding data is infinite

```
flag = isinfinite(db);
```

Args:

db (ts object): time series object

Returns:

:

- flag (bool): whether the corresponding data is infinite or not

Note:

- Since the time series object supports logical indexing, one can directly use the resulting flag with the time series object.

isnan

Returns whether the corresponding data is nan

```
flag = isnan(db);
```

Args:

db (ts object): time series object

Returns:

:

- flag (bool): whether the corresponding data is nan or not

Note:

- Since the time series object supports logical indexing, one can directly use the resulting flag with the time series object.

jbtest

Overloads Matlab's jbtest for ts objects. performs the Jarque-Bera goodness-of-fit test of composite normality

```
[h,p,jbstat,critval]=jbtest(this)
[h,p,jbstat,critval]=jbtest(this,alpha)
[h,p,jbstat,critval]=jbtest(this,alpha,mctol)
```

Args:

- **this** (ts): time series object
- **alpha** (numeric | {0.05}): significance level
- **mctol** (numeric | {[]}): significance level when Monte Carlo is used (instead of interpolation) in the computation of **p** (see below)

Returns:

- **h** (0|1): result of the test. H=0 indicates that the null hypothesis ("the data are normally distributed") cannot be rejected at the 5% significance level. H=1 indicates that the null hypothesis can be rejected at the 5% level.
- **p**: p-value computed using inverse interpolation into the look-up table of critical values. Small values of p cast doubt on the validity of the null hypothesis
- **jbstat**: test statistic
- **critval**: critical value for the test

See also:

jbtest

kurtosis

Computes kurtosis for time series: It is an interface to MATLAB implementation of kurtosis

```
varargout = kurtosis(db,varargin);
```

Args:

db (ts object): times series object

varargin: varargin for kurtosis function in MATLAB

Returns:

:

- **varargout**: output from kurtosis function

log

Overloaded log function for ts object

ma_filter

Moving average filter

```
[trend,detrended]=ma_filter(y,q)
[trend,detrended]=ma_filter(y,q,extend)
```

Args:

y (ts object): time series object

q (integer | {0.5*frequency}): number of periods before or after the
current one to be considered in the moving average calculation. The total window length is 2q+1

extend (true | {'false'}): if true, replicated observations are
added both at the beginning and at the end of the original dataset in order to avoid losing some observations during the filtering process.

Returns:

- **trend** [ts] : (non-parametric) trend
- **detrended** [ts] : y-trend
- **q** (integer | {0.5*frequency}) : value used in the computations

max

Overloaded max function for ts object

mean

Computes the mean of the time series

```
m = mean(db);
m = mean(db,dim);
```

Args:

db (ts object): times series object to compute the mean

dim (integer): (optional; default 1) direction to compute the mean,
e.g., dim = 1 is time mean of each variables, and dim = 3 would be panel mean

Returns:
:

- m (double): mean values

median

Computes the median of the time series

```
m = median(db);
m = median(db,dim);
```

Args:
db (ts object): times series object to compute the median dim (integer): (optional; default 1) direction to compute the median, e.g., dim = 1 is time median of each variables, and dim = 3 would be panel median

Returns:
:

- m (double): median values

min

Overloaded min function for ts object

minus

Overloaded minus function for ts object

mode

Computes mode for the time series: It is an interface to MATLAB
implementation of mode

```
varargout = mode(db,varargin);
```

Args:
db (ts object): times series object varargin: varargin for mode function in MATLAB

Returns:
:

varargout: output from mode function

moments

Computes the empirical moments of a time series

```
oo_ = moments(db);
oo_ = moments(db,drange);
oo_ = moments(db,drange,ar);
oo_ = moments(db,drange,ar,lambda);
```

Args:

db (ts object): time series object to get data

drange (char | serial date | cellstr | {[]}): Range of the data to use

ar (integer | {1}): order of autocorrelation

lambda (numeric | {[]}): hyperparameter for hp-filtering the data
before computing the moments. If empty, the data are not hp-filtered.

Returns:

- **oo_** [struct]: structure with fields
  - vcov : variance covariance
  - skewness : skewness
  - kurtosis : kurtosis
  - variance : variance
  - stdev : standard deviation
  - corrcoef : correlation array

mpower

Overloaded mpower function for ts object

mrdivide

Overloaded mrdivide for ts object

Note:
It matrix multiplication does not make sense for a time series object, so automatically defaults to element-wise division

mtimes

Overloaded times function for ts object

Note:
Matrix product does not make sense for time series object, so automatically defaults to element-wise product

nan

Initializes a ts object with the given start data and data initialized to nan

```
db=ts.nan(start_date,varargin)
```

Args:

- **start_date** : [numeric|char]: a valid time series (ts) date
- varargin : [numeric]: arguments to matlab's **nan** function.

Returns:
:

- **db** : [ts]: a time series

Note:

- this is a static method and so it has to be called with the **ts.** prefix
- ts.nan does not allow more than 3 dimensions

Example:

```
db=ts.nan(1990,10,1)
db=ts.nan('1990',10,3)
db=ts.nan('1990Q3',10,5,100)
```

npdecomp

Non-parametric decomposition into trend, seasonal and irregular components

```
out=npdecomp(y)
out=npdecomp(y,doLog)
```

Args:

y (ts): time series to decompose

doLog (true | {false}): if log, do a multiplicative decomposition
otherwise the decomposition is additive

Returns:
:

- **out** [struct] :
  - **trend** [ts] : estimated trend
  - **sc** [ts] : estimated seasonal component
  - **sa** [ts] : seasonally adjusted data
  - **ic** [ts] : estimated irregular component

Note:

If there are many variables and the variables are named, the first level of the structure will be the names of the different variables.

See also:

- `pdecomp`

numel

Returns the number of data, i.e., (length of time)x(number of variables)x(number of panels)

Syntax:

n = numel(db);

Args:

db (ts object): time series object

Returns:

- n (integer): total number of data points, i.e., (length of time)x(number of variables)x(number of panels)

ones

Initializes a ts object with the given start data and data initialized to ones

```
db=ts.ones(start_date,varargin)
```

Args:

start_date (numeric | char): a valid time series (ts) date

varargin (numeric): arguments to matlab's **ones** function.

Returns:

:

- **db** : [ts]: a time series

Note:

- this is a static method and so it has to be called with the **ts.** prefix
- ts.ones does not allow more than 3 dimensions

Example:

```
db=ts.ones(1990,10,1)
db=ts.ones('1990',10,3)
db=ts.ones('1990Q3',10,5,100)
```

pages2struct

Turns multivariable ts object into a struct with time series object

```
output = pages2struct(input)
```

Args:

input (ts object): ts object to turn into struct form of data

Returns:

:

- **output** (struct): a struct with
  - fieldname: variable names of input
  - value: ts object corresponding to the variable (with data and description)

pdecomp

Parametric decomposition into trend, seasonal and irregular components

```
out=pdecomp(y)
out=pdecomp(y,doLog)
out=pdecomp(y,doLog,dorder)
```

Args:

- **y** (ts): time series to decompose

- **doLog** (true | {false}): if log, do a multiplicative decomposition otherwise the decomposition is additive
- **dorder** (integer | {2}): detrending order

Returns:

:

- **out** [struct] :
  - **trend** [ts] : estimated trend
  - **sc** [ts] : estimated seasonal component
  - **sa** [ts] : seasonally adjusted data
  - **ic** [ts] : estimated irregular component

Note: If there are many variables and the variables are named, the first level of the structure will be the names of the different variables.

See also:

- `npdecomp`

permute

Overloaded permute function for ts object

plot

Plots a rise time series

```
plot(db);
plot(daterange,db);
plot(daterange,db, varargin);
```

Args:

db (ts object): time series object daterange: date range. See ts for different formats varargin: need to come in pairs (except for string format)

- s (string): line type, symbol spec, and color spec (same as MATLAB's plot function)
- 'nticks': integer (default 8) number of xticks
- 'date_format': date format option used in MATLAB.
- **vline** [char|cellstr|serial dates|{''}] : vertical line(s) e.g. 'vline' = '2000Q1'= '2000Q1,2003Q2' must be in the same frequency as the database to be plotted
- **hline** [integer|{''}] : horizontal line(s) 'hline' =1, =[1 5.5 2]
- **logy** [true|{false}] : log the database or not

Returns:

:

- **varargout** [scalar|vector] : handle to the lines of plot

Example:

```
plot('1994m7:1997m1',db(:,:,1),...
    'figsize',[2,2],...
    'figtitle','no title',...
    'nticks',10,...
    'legend',{'v1','v2'},...
    'legend_loc','BO',...
    'logy',true,...
    'secondary_y',{'v1','v4'},...
    'subplots',true,...
    'linewidth',2,...
    'date_format',17);
xrotate(90)
```

Note:

In addition to those matlab properties, RISE adds further properties, which allow to control for. See parse_plot_args

plot_real_time

plot_real_time - hairy plot

```
plot_handle=plot_real_time(rts)
plot_handle=plot_real_time(xrange,rts)
plot_handle=plot_real_time(rts,varargin)
plot_handle=plot_real_time(xrange,rts,varargin)
```

Args:

- **rts** (ts | rts): valid time series object with possibly several columns and exactly one page
- **xrange** : range over which to restrict the plots
- **varargin** : valid matlab arguments for plot, coming in pairs

Returns:

- **plot_handle** : handle to the plot

plotyy

Plot figure with y tick labels on the left and right

```
[ax, h1, h2] = plotyy(y1,y2);
[ax, h1, h2] = plotyy(xrange, y1, y2);
[ax, h1, h2] = plotyy(xrange, y1, y2, func);
[ax, h1, h2] = plotyy(xrange, y1, y2, func1, func2);
```

Args:

y1 (ts object): time series to plot with left axis tick

y2 (ts object): time series to plot with right axis tick

xrange: date range to plot. See ts for formats

func (string/func-handle): use the function to make the plots instead

func1 (string/func-handle): use the function to make the plot for y1

func2 (string/func-handle): use the function to make the plot for y2

Returns:

:

- ax: axis handle
- h1: handle to the graphics object corresponding to y1
- h2: handle to the graphics object corresponding to y2

Example:

```
[ax,h1,h2]=plotyy('1994m7:1997m1',...
    db(:,'v1',1),...
    db(:,'v2',1),...
    'nticks',10,...
    'date_format',17)
    xrotate(90)
```

Note:

In addition to those matlab properties, RISE adds further properties, which allow to control for. See parse_plot_args

plus

Overloaded + function for ts object

power

Overloaded power function for ts object

prctile

Compute the percentiles of a time series (ts)

Syntax

db=prctile(db,p)

Args:

- db (ts): time series with many pages (third dimension). The time series may have one or several variables.
- p (scalar | vector): scalar or a vector of percent values

Returns:

- **db** [ts] : time series with as many pages as the length of **p**.

Example:

test=ts(1990,rand(100,3,200),{'a','b','c'}); tmp=prctile(test,[10,50,90]) plot(tmp('a'))

quantile

Computes quantile for time series: It is an interface to MATLAB implementation of quantile

```
varargout = quantile(db,varargin);
```

Args:

db (ts object): times series object

varargin: varargin for quantile function in MATLAB

Returns:

:

- **varargout**: output from quantile function

rand

Initializes a ts object with the given start data and data initialized to
uniform random numbers

```
db=ts.rand(start_date,varargin)
```

Args:

start_date (numeric | char): a valid time series (ts) date

varargin (numeric): arguments to matlab's **rand** function.

Returns:

:

- **db** : [ts]: a time series

Note:

- this is a static method and so it has to be called with the **ts.** prefix
- ts.rand does not allow more than 3 dimensions

Example:

```
db=ts.rand(1990,10,1)
db=ts.rand('1990',10,3)
db=ts.rand('1990Q3',10,5,100)
```

randn

Initializes a ts object with the given start data and data initialized to normal random number

```
db=ts.randn(start_date,varargin)
```

Args:

start_date (numeric | char): a valid time series (ts) date varargin (numeric): arguments to matlab's **randn** function.

Returns:

:

- **db** : [ts]: a time series

Note:

- this is a static method and so it has to be called with the **ts.** prefix
- ts.randn does not allow more than 3 dimensions

Example:

```
db=ts.randn(1990,10,1)
db=ts.randn('1990',10,3)
db=ts.randn('1990Q3',10,5,100)
```

range

Overloads Matlab's range for ts objects. returns the range of the
values in the time series

```
Y = range(this,varargin);
```

Args:

this (ts object): time series object varargin : additional matlab arguments for the range function

Returns:
:

- **Y** [numeric]: Difference between maximum and minimum values

See also:

- range

rdivide

Overloaded rdivide function for ts object

real

real : overloads real for time series

Syntax

```
this=real(this);
```

Args:

this (ts object): time series object

Returns:

- **this** (ts object): time series object

regress

Linear regression

```
[B,BINT,R,RINT,STATS] = regress(this,this2)
[B,BINT,R,RINT,STATS] = regress(this,this2,varargin)
```

Args:

- **this** (ts): left-hand-side variable
- **this2** (ts): right-hand-side variables
- **varargin** (comma separated): additional inputs for Matlab's REGRESS function

Returns:

- **B**: vector of regression coefficients in the linear model Y = X*B.
- **BINT**: of 95% confidence intervals for B
- **R**: vector of residuals
- **RINT**: matrix of intervals that can be used to diagnose outliers. If RINT(i,:) does not contain zero, then the i-th residual is larger than would be expected, at the 5% significance level. This is evidence that the I-th observation is an outlier.
- **STATS**: vector containing, in the following order, the R-square statistic, the F statistic and p value for the full model, and an estimate of the error variance.

Example:

```
y = ts(1990,rand(100,1)); % random series
X = y(-1)&y(-2)&y(-3); % columns of lags
X = ones(X); % add a column of ones
[B,BINT,R,RINT,STATS] = regress(y,X)
```

rmse

Compute the Root Mean Square Error

```
[Rmse,Pe] = rmse(rawdata)
```

Args:

rawdata (T x (h+1) x nsim ts): Time series with

- T observations
- h+1 columns, where the first column represents the actual data and the remaining h columns are forecasts
- nsim number of simulations

Returns:

:

- **Rmse** : [h x nsim]. Matrix of root mean square errors
- **Pe** : [T x h x nsim ts]. Time series of prediction errors

rolling

Applies a function to a rolling window of the time series

```
h = rolling(db,func,window);
h = rolling(db,func,window,varargin);
```

Args:

db (ts object): time series object to get data

func: function that will apply to the rolling window of the time series. The function is recursively applied to the data t+(1:window) for t=0,1,2,3,...,T-window, where T is the number of observations

window: number of observations in the rolling window

varargin: additional arguments to func

set

SET Set properties of time series object.

```
SET(THIS,'PropertyName',VALUE) sets the property 'PropertyName'
of the time series THIS to the value VALUE.  An equivalent syntax
is
    THIS.PropertyName = VALUE

SET(THIS,'Property1',Value1,'Property2',Value2,...) sets multiple
time series property values with a single statement.

SET(THIS,'Property') displays values for the specified property in THIS.

SET(THIS) displays all properties of THIS and their values.
```

Args:

this (ts object): time series object

varargin: valid pairwise arguments (see above)

See also TSGET.

sin

Overloaded sin function for ts object

sinh

Overloaded sinh function for ts object

size

Computes the size of the data stored in a ts object. See Matlab's size for more detail.

skewness

Computes skewness for time series: It is an interface to MATLAB implementation of skewness

```
varargout = skewness(db,varargin);
```

Args:

db (ts object): times series object

varargin: varargin for skewness function in MATLAB

Returns:

:

- **varargout**: output from skewness function

spectrum

Computes the spectral density of the data

```
[sw,jj,T]=spectrum(this)
```

Args:

- **this** (ts object): time series object

Returns:

- **sw** (matrix|vector): spectrum of potentially multiple time series in columns
- **jj** (vector): range of the spectrum (x-axis)
- **T** (integer): number of observations

std

Computes standard deviation for time series: It is an interface to MATLAB implementation of std, but adjusted to handle nan properly

```
varargout = std(db,varargin);
```

Args:

db (ts object): times series object

varargin: varargin for std function in MATLAB

Returns:
:

- **varargout**: output from std function

step_dummy

Implements a step dummy in the time series

Args:

start_date: start date of the time series

end_date: end date of the time series

dummy_start_date: start date of the step dummy

Returns:
:

- **db** (ts object): a time series with zeros from the first observation to the date before the start of the dummy and ones from then on.

subsasgn

SUBSASGN - Subscripted assignment for time series objects

Syntax:
obj = subsasgn(obj, s, b)

Inputs:
obj - Time series object (ts) s - Subscript structure defining the assignment b - Values to be assigned

Outputs:
obj - Updated time series object after the assignment

Description:
The 'subsasgn' function enables subscripted assignment for time series objects. It allows assigning values to specific elements, submatrices, or subarrays of the time series object. The assignment can be performed based on variables, dates, time shifts, or a combination of these.

Supported subscript forms: - One index: variables, dates, or time shift - Two indexes: dates (first dimension) and variables (second dimension) - Three indexes: dates (first dimension), variables (second dimension), and pages (third dimension)

For more details on the specification of dates, variables, pages, and time shift, refer to the "More info" section.

Examples:
% Create a time series object dates = rd('2000-01-01'); data = rand(5, 3); varnames = {'Var1', 'Var2', 'Var3'}; obj = ts(dates, data, varnames);

% Assign new values to specific elements obj(2, 'Var2') = 0.5;

% Assign new values to a submatrix obj(3:5, [1, 3]) = [1, 2; 3, 4; 5, 6];

% Assign new values to specific pages obj(:, :, 2) = 10;

% Display the updated time series object disp(obj);

More info:

- Specification of dates: - List: '1991Q3', '1990Q1:1990Q3,1991,1992Q2', etc. - Serial numbers: date2serial('2011Q1'):date2serial('2014Q1') - Boolean indexing [deprecated?]: e.g., obj{obj{'Var1'} > 0}
- Specification of variables: - List: 'Var1', 'Var1,Var4', {'Var1','Var4'} - Numbers: 1, [1,4], etc. (only when dates have been specified)
- Specification of pages: - Numbers: 1, 2:4, [1,3,5], etc. - Function handles: e.g., @isreal (only pages where all elements are real)
- Specification of time shift: Not supported

See also ts.subsref

subsref

subsref Subscripted reference for time series objects

```
this = subsref(obj, s)
```

Args:

- **obj** [ts]: Time series object to reference.
- **s** [struct]: Subscript structure specifying the elements to reference.

Returns:

- **this** [ts]: Resulting time series object.

Notes:

- *db(I)* returns a time series formed from the elements of *db* specified by the subscript vector *I*.
- *db(I, J)* returns a time series formed from the elements of the rectangular submatrix of *db* specified by the subscript vectors *I* and *J*.
- *db(I, J, K)* returns a time series formed from the elements of the cubic array of *db* specified by the subscript vectors *I*, *J*, and *K*.
- The subscript structure *s* can have one, two, or three levels depending on the desired reference.
- **One index:**
  - Variables, dates, or time shift can be specified.
- **Two indexes:**
  - Dates (first dimension) and variables (second dimension) can be specified.
- **Three indexes:**
  - Dates (first dimension), variables (second dimension), and pages (third dimension) can be specified.
- **Dates can be specified using the following formats:**
  - List: '1991Q3', '1990Q1:1990Q3,1991,1992Q2', etc.
  - Serial numbers: date2serial('2011Q1'):date2serial('2014Q1')
  - Boolean indexing [deprecated?]: e.g. *db{db{'v1'}>0}*
- **Variables can be specified using the following formats:**
  - List: 'v1', 'v1,v4', {'v1','v4'}

  - Numbers: 1, [1,4], etc. (only when dates have been specified)
- **Pages can be specified using the following formats:**
  - Numbers: 1, 2:4, [1,3,5], etc.
  - Function handles: e.g. *@isreal* (only pages where all elements are real)
- **Time shift can be specified using the following formats:**
  - db{-1}, db{+3}, etc.

Examples:

db = ts('1990q1', randn(10, 4, 3), {'v1','v2','v3','v4'});

% Pick a variable db{'v1'}

% Pick variables db{'v1,v4'} db(:, 1)

% Pick a year db{'1991'}

% Pick a date db{'1992Q1'}

% Pick dates db{'1990Q1:1990Q3,1991,1992Q2'}

% Pick date and variables db{'1991Q1','v3'} db{'1991Q1',{'v1','v2'}} db{'1991:1992',{'v1','v2'}} db{'1991Q1:1992Q2',{'v1','v2'}} db{:,{'v1','v2'}} db{'1991Q1:1992Q2',:} db{:,:} db{date2serial('2011Q1'):date2serial('2014Q1'),{'v1','v2'}}

% Pick date, variables, and pages db{'1991Q1',{'v1','v2'},3:4} db{'1991Q1',{'v1','v2'},:} db{:,:,:} db{:,:,:} db(:,:,@isreal)

% Shifting/lagging: lag or lead series db{-1} db{+3}

% Boolean indexing db{db{'v1'}>0}

See also ts.subsasgn

sum

Overloaded sum function for ts object

tail

Returns the last few sample dates of the time series

Syntax:

db = tail(db);

db = tail(db,n);

Args:

- db (ts object): time series object
- n (integer): number of time steps to show

Returns:

- **db** (ts object): time series with the last n-time steps

Note:

- This is similar to the tail function in stata.

times

Overloaded times function for ts object

transform

Apply different transformations following the definitions of Haver

Usage:

db = transform(db, type);

Args:

db (ts object): time series object

type: the type of the transformation. Available transformations are

- 1 or 'level': Untransformed data - level (default)
- 2 or 'pct_ch_ar': Percentage change at compound annual rate (% chg)
- 3 or 'pct_ch': Period to period percentage change (% change)
- 4 or 'yoy_pct_ch': Year over year percentage change (yr/yr % chg)
- 5 or 'diff': Period to period difference change
- 6 or 'yoy_diff': Year over Year difference change
- 7 or 'log_ch_ar': %Log Change - compound annual rate (ann l-chg)
- 8 or 'log_ch': %Log Change - period to period (log-change)
- 9 or 'yoy_log_ch': Year to year log change (yr-yr l-chg)

Returns:

- **db** (ts object): Transformed time series

ts

Constructor for the time series (ts) object

self=ts(); % construct a time series with no observations self=ts(start_date,data); self=ts(start_date,data,varnames); self=ts(start_date,data,varnames,description); self=ts(start_date,data,varnames,description,trailnans);

Args:

start_date (integer | char | serial date): start date of
the time series. The following are admitted:

- **annual data :**
  - '1990'|ry('1990')|ra('1990')
  - ry(1990)|ra(1990)
  - ry(datenum)|ra(datenum)
  - ry('mm/dd/yyyy')|ra('mm/dd/yyyy')
  - ry('yyyy-mm-dd')|ra('yyyy-mm-dd')

- **bi-annual data :**
  - '1990H1'|rh('1990H1')|rh(1990,1)
  - rh(datenum)
  - rh('mm/dd/yyyy')|rh('yyyy-mm-dd')
- **Quarterly data :**
  - '1990Q1'|rq('1990Q1')|rq(1990,1)
  - rq(datenum)
  - rq('mm/dd/yyyy')|rq('yyyy-mm-dd')
- **monthly data :**
  - '1990M1'|rm('1990M1')|rm(1990,1)
  - rm(datenum)
  - rm('mm/dd/yyyy')|rm('yyyy-mm-dd')
- **Weekly data :**
  - '1990W1'|rw('1990W1')|rw(1990,1)
  - rw(datenum)
  - rw('mm/dd/yyyy')|rw('yyyy-mm-dd')
- **Daily data :**
  - '1990D1'|rd('1990D1')|rd(1990,12,31)
  - rd(datenum)
  - rd('mm/dd/yyyy')|rd('yyyy-mm-dd')

data (numeric): the format is nobs x nvars x npages,
where:

- **nobs** is the number of observations
- **nvars** is the number of variables
- **npages** is the number of pages (3rd dimension)

varnames (char | cellstr): names of the variables in the
database

description (char | cellstr | {''}): comments on each
variable in the database

trailnans (true|{false}): keep or remove nans (missing observations)

Returns:

- **self** [ts] : time series

Documentation for ts/ts
doc ts

uminus

Overloaded uminus function for ts object

unfold

Takes a folded times series and turns it into a one-page time series

```
this=unfold(last_hist_date,obj)
this=unfold(last_hist_date,obj1,obj2,...,objn)
```

Args:

last_hist_date (char | serial date): obj (ts): time series

Returns:

:

- **this** [ts]: unfolded time series

values

Returns the underlying data of the time series

```
data = value(db);
```

Args:

- **db** (ts object): time series object

Returns:

- **d** (double): vector/matrix/tensor form of the data underlying the time series

var

Computes variance for time series: It is an interface to MATLAB implementation of var adjusted to handle nan properly

```
varargout = var(db,varargin);
```

Args:

db (ts object): times series object varargin: varargin for var function in MATLAB

Returns:

:

- **varargout**: output from var function

zeros

Initializes a ts object with the given start data and data initialized to zeros

```
db=ts.zeros(start_date,varargin)
```

Args:

start_date (numeric | char): a valid time series (ts) date varargin (numeric): arguments to matlab's **zeros** function.

Returns:

:

- **db** : [ts]: a time series

Note:

- this is a static method and so it has to be called with the **ts.** prefix
- ts.zeros does not allow more than 3 dimensions

Example:

```
db=ts.zeros(1990,10,1)
db=ts.zeros('1990',10,3)
db=ts.zeros('1990Q3',10,5,100)
```

CHAPTER

ELEVEN

GLOBAL SENSITIVITY ANALYSIS AND UNCERTAINTY QUANTIFICATION

11.1 Monte Carlo Filtering

11.1.1 Description

Monte Carlo filtering is a method for analyzing the sensitivity of a model's characteristics to its parameters. The approach involves generating draws of parameters from their distribution (e.g. prior, posterior, support) and examining the model's behavior for each draw.

By comparing the empirical marginal distributions of the draws that lead to desired behavior versus those that do not, key parameters that drive the model's behavior can be identified.

Monte Carlo filtering is useful for locating the most important parameters of a model that contribute to its behavior.

A key reference in Global sensitivity analysis is [Ratto, 2008].

11.1.2 Quick start examples

11.1.2.1 A very simple example

```
x=randn(2^13,8);

y=x(:,1)+x(:,2).^2+x(:,1).*x(:,2);

check_behavior=y<0;

obj=mcf(check_behavior(:).',x.')
%%
cdf_plot(obj)
%%
scatter(obj)
%%
correlation_patterns_plot(obj,[],'behave')
```

In this small example, we have a function y=y(x1,x2) and we interested in finding the combinations of parameters/covariates x1, x2, x3, x4, x5, x6, x7 and x8 for which y<0. In this controlled example, clearly, we know a priori that the only covariates that matter for the result are x1 and x2. But in general we may not know that in advance.

We first sample values for all the covariates and for each combination (x1(i), x2(i), x3(i), x4(i), x5(i), x6(i), x7(i), x8(i)), for i=1,2,…,2^13, we compute y(i) and set a flag telling us whether y(i) is negative for combination i.

Then we proceed to investigate whether there are some patterns among the different covariates both in situations where y<0 and in situations where y>=0.

11.1.2.2 A simple DSGE example

```
%% A DSGE model

simpleModel={
    'model:someModel'
    '%% model without parameters and shocks'
    '@endogenous X1, X2, X3'
    '@exogenous E1, E2, E3'
    '@parameters a1, a2, a3, vol_tp_1_2, vol_tp_2_1'
    '@parameters(vol,2) s1, s2, s3'
    '@model'
    'X1=a1*X1{-1}+s1*E1;'
    'X2=a2*X2{-1}+s2*E2;'
    'X3=a3*X3{-1}+s3*E3;'
    };

%% The parameters to vary and their ranges

plist={'a1', 'a2', 'a3', ...
    'vol_tp_1_2', 'vol_tp_2_1',...
    's1_vol_1', 's2_vol_1', 's3_vol_1',...
    's1_vol_2', 's2_vol_2', 's3_vol_2'
    };

bounds=[0.5,0.9;0.5,0.9;0.5,0.9;0,1;0,1;0,0.5;0,0.5;0,0.5;0.5,2;0.5,2;0.5,2];

%% rise the model and set the behavior to investigate in a function handle

m=rise(simpleModel);

% To do : write a simple function handle check that the model solves or that some␣
↪correlations match
% Behavior = @...

%% run the behavior through the mcf

obj=mcf(Behavior,nsim_or_draws,bounds(:,1),bounds(:,2),plist,'latin_hypercubes');


%% smirnov test for equality of distributions
close all
% hfig=utils.plot.multiple(@(x)cdf_plot(obj,x),parameter_names,...
%     'Smirnov test of equality of distributions',3,3)
%
cdf_plot(obj)

pause

```

(continues on next page)

(continued from previous page)

```
%% correlation patterns in the behavior sample
close all

correlation_patterns_plot(obj,[],'')

pause

%% correlation patterns in the behavior sample
close all

correlation_patterns_plot(obj,[],'behave')

pause

%% correlation patterns in the behavior sample
close all

correlation_patterns_plot(obj,[],'non-behave')

pause

%% scatter plot of the significant correlations
close all

scatter(obj,[],'',1/100)

pause
%% scatter plot of the significant correlations
close all

scatter(obj,[],'non-behave')
```

Todo: Add further comments to this example

11.1.3 Details on the mcf class

11.1.3.1 The properties

11.1.3.2 The methods

cdf

Cumulative distribution function

```
d=cdf(obj,pname)
```

Args:

- **obj** [mcf object]:
- **pname** [char]: name of the parameter which one wants the cdf for.

Returns:

- **d** [struct]: with fields lb, ub, x, f_behave, f_non_behave, x_good, x_bad

cdf_plot

plot of the cumulative distribution functions

```
hdl=cdf_plot(obj)
hdl=cdf_plot(obj,pnames)
hdl=cdf_plot(obj,pnames,title)
```

Args:

- **obj** [mcf object]:
- **pnames** [empty|char|cellstr]: names of the parameters of interest
- **titel** [empty|char]: title of the figures

Returns:

- **hdl** [vector]: handles to the plotted figures

correlation_patterns_plot

Plot of correlation patterns

```
hdl=correlation_patterns_plot(obj)
hdl=correlation_patterns_plot(obj,names)
hdl=correlation_patterns_plot(obj,names,type)
hdl=correlation_patterns_plot(obj,names,type,pval_cutoff)
```

Args:

- **obj** [mcf object]:
- **names** [empty|char|cellstr]: names of the parameters of interest
- **type** [empty|'behave'|'non-behave']: if empty, all the sample is considered. If 'behave' only the behavior sample is considered. If 'non-behave', only the non-behavior sample is considered.
- **pval_cutoff** [numeric|{0.05}]: cutoff for significant correlations

Returns:

- **hdl** [vector]: handles to the plotted figures

kolmogorov_smirnov_test

tests the equality of two distributions using their CDFs

```
[pValue,Dn,xn,largest]=kolmogorov_smirnov_test(d)
```

Args:
d : mcf object

Returns:

Note:
Documentation to be completed

mcf

Constructor for Monte Carlo filtering

```
obj=mcf(check_behavior,nsim_or_draws)

obj=mcf(check_behavior,nsim_or_draws,lb)

obj=mcf(check_behavior,nsim_or_draws,lb,ub)

obj=mcf(check_behavior,nsim_or_draws,lb,ub,names)

obj=mcf(check_behavior,nsim_or_draws,lb,ub,names,procedure_)
```

Args:

- **check_behavior** (function_handle | cell | vector):
  (i) when it is a function handle, the function takes as input a vector of parameters and returns in its first output a boolean that is true if the paramter satisfies the behavior and false otherwise. The procedure is going to check the number of output arguments that the function returns and collect all those extra output arguments while sampling.
  (ii) when it is a cell, it should be a two-element cell such that the first element is the function handle and the second is the number of outputs desired by the user.
  (iii) if it is a vector, all elements are either 0 or 1 or boolean, describing whether each parameter vector checks the behavior or not.
- **nsim_or_draws** (integer | matrix): When it is an integer, nsim_or_draws is the number of draws to sample. When it is a matrix, it is the draws. No further sampling will be performed.
- **lb** (empty | vector): lower bound of the search space
- **ub** (empty | vector): upper bound of the search space
- **names** (empty|char|cellstr]: names of the parameters. If empty, the names are created as p_i, where "i" is the order of the parameter in the list.
- **procedure_** ({'uniform'} | 'latin_hypercube' | 'sobol' | 'halton' | user-defined): when it is user-defined, it should be a function handle, and should take 3 inputs(lb, ub, nsim) and return a parameter draw.

Returns:

:

- **obj** [mcf object]: containing, among other things, the samples, a flag determining their behavior.

Documentation for mcf/mcf
doc mcf

scatter

Scatter plot of the data

```
hdl=scatter(obj)
hdl=scatter(obj,names)
hdl=scatter(obj,names,type)
hdl=scatter(obj,names,type,pval_cutoff)
hdl=scatter(obj,names,type,pval_cutoff,title)
```

Args:

- **obj** [mcf object]:
- **names** [empty|char|cellstr]: names of the parameters of interest
- **type** [empty|'behave'|'non-behave']: if empty, all the sample is considered. If 'behave' only the behavior sample is considered. If 'non-behave', only the non-behavior sample is considered.
- **pval_cutoff** [numeric|{0.05}]: cutoff for significant correlations
- **title** [empty|char]: title of the figures

Returns:

- **hdl** [vector]: handles to the plotted figures

11.1.4 Support facility for filtration : the mcfprop class

11.1.4.1 The properties

11.1.4.2 The methods

add

ADD : adds behaviors to check to and mcfprop object See the constructor for more details on the list of behaviors admissible.

create_handle

create_handle : creates a function handle that can be passed to the mcf function

mcfprop

mcfprop : constructor for Monte Carlo Filtration Property class

Syntax:

```
obj = mcfprop(model, paramList)

obj = mcfprop(model, paramList,prop1, prop2, ...,propn)
```

Inputs

- **model** [dsge|rise]: dsge model object
- **paramList** [cellstr]: List of parameters to be sampled
- **prop1** [char|1x2 cell]: Behavior to investigate. Two possibilities

  When prop is a char, it is one of the following :

  - 'solve' : behavior is where the model has a solution
  - 'pb_sstate' : behavior is where the steady state does not solve
  - 'pb_trans_matrix' : behavior is where there are problems in the transition matrix
  - 'pb_maxiter' : behavior is where the maximum number of iterations is reached
  - 'pb_multi_sol' : behavior is where there are multiple solutions
  - 'pb_nan_sol' : behavior is where there are nans in the solution
  - 'pb_explosive_sol' : behavior is where there are only explosive solutions
  - 'pb_unstable' : behavior is where the system is unstable

  When prop is a cell array and the first entry is a function handle, it expected

  - the function handle accepts 2 inputs : m (the dsge model object) and filtr (the output of filtering)
  - the function handle returns a scalar boolean
  - the second entry of the cell array is either 0 (no filtering), 1 (predictions only), 2 (1+updates), 3 (2+smoothed)
  - the underlying dsge model already contains all the information (e.g. data, options, etc) needed to perform the tasks required for mcf.

  When prop is a cell array and the first entry is a statistic to compute, the second entry must be a function handle to check the statistic. The recognized patterns for the first entry of the cell are:

  Correlations

  - corr{x,y} : horizon =1, regime = 1
  - corr{x,y,horizon}: regime = 1
  - corr{x,y,horizon,regime}

  Impulse responses

  - irf{x,y} : horizon =1, regime = 1
  - irf{x,y,horizon}: regime = 1
  - irf{x,y,horizon,regime}

  First-order model solution

- Tz{x,y}: regime = 1
- Tz{x,y,regime}

where y = [y,y{-1},@sig]

Outputs

- **obj** [mcfprop]: constructed mcfprop object

The steps in the entire process

obj = mcfprop(model, paramList,varargin)

obj = add(obj,prop1, prop2, . . .)

propfun = create_handle(obj)

filtObj=mcf(propfun,nsim,lb,ub,pnames,procedure)

Documentation for mcfprop/mcfprop

doc mcfprop

11.1.4.3 An example

```
    %% construct the support object

    m=rise(simpleModel);

    self = mcfprop(m, plist);

    %% Add a few properties
    self = self.add('solve'); % the model solves
    self = self.add({'corr{X2,X2,3:8,2}',@(x)x>0.01}); % correlation or order 3 in regime 2
    self = self.add({'corr{X3,X3,0,2}',@(x)abs(x-1)<1e-9}); % correlation or order 0 in regime 2
    self = self.add({'irf{X1,E1}',@(x)x>0.01}); % irf in period 1
    self = self.add({'irf{X1,E1,3}',@(x)x>0.01}); % irf in period 3
    self = self.add({'irf{X2,E2,3:5,2}',@(x)all(x>1e-3)}); % irf in period 3 to 5 in regime 2
    self = self.add({'Tz{X1,E1}', @(x) x > 0}); % regime 1
    self = self.add({'Tz{X1,@sig,2}', @(x) abs(x)<1e-8}); % regime 1

    %% Create a function handle for the behavior

    n=size(bounds,1);

    propfun = create_handle(self);

    %% Evaluate one parameter vector

    x = bounds(:,1)+ rand(n,1).*(bounds(:,2)-bounds(:,1));

    eval_ = propfun(x);
```

Todo: Comment on this example.

Hint: With the handle/behavior in hand we can go ahead and pass it to the mcf class as above

11.2 High Dimensional Model Representation

11.2.1 Description

High-dimensional model representation (HDMR) is a method of representing complex systems in high-dimensional spaces. It is a technique to decompose a high-dimensional function into a sum of lower-dimensional functions. These lower-dimensional functions are called "HDMR components".

$$f(\mathbf{x}) = \sum_{i=1}^{p} f_i(x_i) + \sum_{1 \leq i < j \leq p} f_{i,j}(x_i, x_j) + ... + f_{1,2,...,p}(x_1, x_2, ..., x_p)$$

$$f_0 = \int_{\mathbb{R}^p} f(\mathbf{x})\, d\mathbf{x}$$

$$f_i(x_i) = \int_{\mathbb{R}^p} f(\mathbf{x})\, d\mathbf{x}_{\sim i} - f_0$$

$$f_{i,j}(x_i, x_j) = \int_{\mathbb{R}^p} f(\mathbf{x})\, d\mathbf{x}_{\sim\{i,j\}} - f_0 - f_i(x_i) - f_j(x_j)$$

It is particularly useful in cases where the number of variables or parameters is large, making it difficult to understand the underlying relationships between the variables.

HDMR is commonly used in various fields such as engineering, physics, and chemistry, to model complex systems and predict their behavior. Here are an example/application to fix the ideas:

- *A Likelihood function*: Conditional on a dataset, the likelihood function is a function of the parameters of the model. But the exact relationship between the likelihood and the parameters is unknown. The reason we may be interested in such a problem is that the likelihood might be very costly to evaluate. HDMR could be used in this case to decompose the likelihood into lower-dimensional components. The resulting metamodel could be far less costly to evaluate.

The HDMR components are chosen such that they capture the most important features of the system, while still being computationally tractable. HDMR can also be used to estimate the sensitivity of a system to changes in its inputs, which can be useful for optimization and design.

Variances

$$D = \int_{\mathbb{R}^p} f^2(\mathbf{x})\, d\mathbf{x} - f_0^2$$

$$D_i = \int_{\mathbb{R}^p} f_i^2(x_i)\, dx_i$$

$$D_{i,j} = \int_{\mathbb{R}^p} \int_{\mathbb{R}^p} f_{i,j}^2(x_i, x_j)\, dx_i dx_j$$

Sensitivities

$$S_i = \frac{D_i}{D}$$

$$S_{i,j} = \frac{D_{i,j}}{D}$$

HDMR is a powerful tool for understanding and predicting the behavior of complex systems, but it also requires large computational resources and can be difficult to implement in practice.

11.2.2 Quick-start examples

11.2.2.1 A known Polynomial function

```
% Define the polynomial function
f = @(x) x.^3 + 2*x.^2 - 5*x + 3;

% Define the lower-dimensional components of the polynomial
f1 = @(x) x.^3;
f2 = @(x) 2*x.^2;
f3 = @(x) -5*x + 3;

% Plot the polynomial and its lower-dimensional components
x = linspace(-2, 2, 100);
y = f(x);
y1 = f1(x);
y2 = f2(x);
y3 = f3(x);
plot(x, y, x, y1, '--', x, y2, ':', x, y3, '-.','linewidth',2);
legend('f(x)', 'f1(x)', 'f2(x)', 'f3(x)');
```

11.2.3 Details

11.2.3.1 The properties

N

hdmr/N is a property.

Nobs

Number of observations

numInputs

number of factors or inputs

output_nbr

hdmr/output_nbr is a property.

theta

draws of the inputs

theta_low

lower bound of the inputs

theta_high

upper bound of the inputs

g

objective/blackbox function

x

hdmr/x is a property.

expansion_order

hdmr/expansion_order is a property.

pol_max_order

hdmr/pol_max_order is a property.

poly_coefs

hdmr/poly_coefs is a property.

Indices

hdmr/Indices is a property.

coefficients

hdmr/coefficients is a property.

aggregate

hdmr/aggregate is a property.

f0

hdmr/f0 is a property.

D

hdmr/D is a property.

sample_percentage

hdmr/sample_percentage is a property.

optimal

hdmr/optimal is a property.

param_names

hdmr/param_names is a property.

11.2.3.2 The methods

estimate

hdmr/estimate is a function.

obj = estimate(obj, debug)

first_order_effect

hdmr/first_order_effect is a function.
first_order_effect(obj, theta, index, output_index)

hdmr

objective is either : f and theta or a function that will help calculate f and theta

Documentation for hdmr/hdmr
doc hdmr

metamodel

hdmr/metamodel is a function.
fit = metamodel(obj, theta)

plot_fit

hdmr/plot_fit is a function.
plot_fit(obj, theta)

polynomial_evaluation

later on, the function that normalizes could come in here so that the normalization is done according to the hdmr_type of polynomial chosen.

polynomial_integration

polynomial is of the form a0+a1*x+…+ar*x^r the integral is then a0*x+a1/2*x^2+…+ar/(r+1)*x^(r+1)

polynomial_multiplication

each polynomial is of the form a0+a1*x+…+ar*x^r

11.3 Uncertainty Quantification

11.3.1 Description

11.3.2 Quick start examples

11.3.3 Details

CHAPTER

TWELVE

CONDITIONAL FORECASTING USING RELATIVE ENTROPY

This is how you do conditional forecasting using the ideas of Ellis and his friends

CHAPTER

THIRTEEN

RISE'S REPORTING SYSTEM

13.1 Description

13.2 Quick start examples

Let's create a quick report with or without a title page, with a figure, a table and sections

To initialize a simple report with a title an author name, we simply do the following:

```
xrep=rprt('Title of the report','Ola Normann');
```

xrep is a handle to which we will be able to add further items we want to include in our report.

Alternatively, we can have a more elaborate title page. In that case we simply use more options from the report function.

```
    authors=struct();
    authors(1).Name='first author';authors(1).Email='first@author.no';authors(1).
↪Institution='University of Nowhere';
    authors(2).Name='second author';authors(s).Email='second@author.no';authors(2).
↪Institution='University of Right here';
    authors(3).Name='third author';authors(3).Email='third@author.no';authors(3).
↪Institution='University of Somewhere';
xrep=rprt('This will be the title of the report',...
    authors,...
    'orientation','landscape');
```

More options such as the documentclass, the points, the papersize and packages can be added.

Adding a section is straightforward

```
xrep.section('title for the brand new section');
```

An analogous syntax applies subsections and subsubsections

Next, let's create and add a figure to the report. There are various ways to do that. The one we show below creates a figure handle and passes it to the reporting object

```
fighand=figure('name','just say whatever');
x=1:100;
plot(x,log(x),'linewidth',2)
xrep.figure(fighand,'caption','optional title for the figure');
```

To clearly separate sections, we can add a new page. One way to achieve this is to do the following:

```
xrep.newpage()
```

It is also possible to use pagebreak()

Now let's create a simple table. First we create a cell array and then we pass it on to the reporting object. The syntax is as follows

```
table_data=rand(4);
headers = {'Column 1', 'Column 2', 'Column 3', 'Column 4'}; % Optional
RowNames = {'Row 1', 'Row 2', 'Row 3', 'Row 4'}; Optional
xrep.tabletable_data,'headers',headers,'RowNames',RowNames,...
'caption', 'Optional title to the table')
```

To make sure tables and figures do not appear in subsequent pages, we can use clearpage or cleardoublepage.

```
xrep.clearpage()

xrep.cleardoublepage()
```

It is straightforward to add text, verbatim text, bullet points and enumeration lists as shown below

```
xrep.text('This is a text line')

xrep.verbatim('you are supposed to render things just as they are'})

xrep.itemize({'one thing','leads to another','and back'});

xrep.enumerate({'one thing','leads to another','and back'});
```

Finally we can publish the report to a PDF file simply by doing

```
publish(xrep)
```

This will save the report under the default name "report.html". Alternatively you can specify the name under which to save your report with or with extension.

```
publish(xrep,saveFileNamePossiblyWithExtension)
```

If the extension is missing, html will be added. Alternatively you can choose pdf, which requires that you have MikTex installed on your machine.

13.3 Details

13.3.1 The properties

13.3.1.1 Title

Title - Title for the report

13.3.1.2 Author

Author - Author(s) of the report

13.3.1.3 Date

Date - Date of the report

13.3.2 The methods

13.3.2.1 cleardoublepage

CLEARDOUBLEPAGE Adds a new page to the report.

Usage:

```
obj.cleardoublepage()
```

Inputs:

- obj: The report object.

Options: none

Example:

```
rprt.cleardoublepage();
```

13.3.2.2 clearpage

CLEARPAGE Adds a new page to the report.

Usage:

```
obj.clearpage()
```

Inputs:

- obj: The report object.

Options: none

Example:

```
rprt.clearpage();
```

13.3.2.3 code_formatter

CODE_FORMATTER - Returns a function handle that generates code for formatting text in different formats.

> CODEFUNC = CODE_FORMATTER(ITEM, FRMT) returns a function handle CODEFUNC that can be used to generate code for formatting text in different formats. ITEM specifies the type of formatting, such as 'bold', 'italics', 'underline', or a color name like 'red', 'blue', etc. FRMT specifies the target format, which can be 'pdf', 'latex', or 'html'.
>
> Example:
>
> codeFunc = code_formatter('bold', 'latex');

latexCode = codeFunc('This text is bold'); % Generates LaTeX code: 'textbf{This text is bold}'

Supported formats:

- 'pdf' or 'latex': Generates code in LaTeX format using LaTeX commands (textbf{}, textit{}, underline{}, textcolor{}).
- 'html': Generates code in HTML format using HTML tags (<b>, <i>, <u>,).

Note: The function supports a predefined set of colors for both LaTeX and HTML formats. Additional colors can be added to the 'colors' cell array as needed.

See also:

sprintf

13.3.2.4 create_figure

CREATE_FIGURE - Create a figure with multiple subplots.

Syntax:

obj.create_figure(Subplots)

obj.create_figure(Subplots,optName1,optValue1,...)

Description:

This method creates a figure with multiple subplots based on the given inputs. The subplots can contain different plots with optional titles, legends, plotting functions, data, highlighted dates, and style descriptions.

Inputs:

- obj : The report object.
- varargin : Optional name-value pairs to customize the figure and subplots.
  - 'DateFormat' : Optional. Date format for the x-axis labels.
  - 'range' : Optional. Range of values to plot on the x-axis.
  - 'subplots' : Required. cell array of subplots to add to the figure.

    Each subplot is defined as a cell within the cell array. The first element of the subplot cell is invariably the data to plot. The remaining elements are optional arguments coming in pairs and defined as follows:

    * 'caption' : Optional. Title of the subplot.
    * 'legend' : Optional. legend for the subplot.
    * 'plotfunc' : Optional. Plotting function to use (default: @plot).
    * **'highlighted_dates'**
      [Optional. Cell array of 1-by-2 vectors containing] the dates to be highlighted on the plot.
    * 'style' : Optional. Description of the plot style (e.g., linewidth, color, etc.).
    * 'yline' : Optional. horizontal line on the y axis
    * 'xline' : Optional. vertical line(s) on the x-axis
  - other options of rprt.figure : i.e. 'caption', 'width', 'placement', 'height', 'angle', 'scale', 'numbering'

Example:

```
  report = rprt('Sample Report', 'John Doe', 'portrait');
subplots={
    {cumsum(randn(100,1)),'caption','the first plot','highlighted_dates',{[20,30]}}
    {cumsum(randn(100,1)),'caption','the second plot','highlighted_dates',{[20,30]},
↪'style',{'linewidth',2}}
    };
  report.createFigure(subplots,'caption', 'My Figure', 'DateFormat', 'dd-mm-yyyy', ...
      'range', [1, 10]);
```

13.3.2.5 description

DESCRIPTION Adds a description list to the report.

Usage:
obj.description(Items)

Inputs:

- obj: The report object.
- Items: A cell array of structs representing the items in the list. Each struct should have 'term' and 'description' fields.

Options:
None

Example:

```
d=struct('term',{},'description',{});
d(1).term='DSGE';
d(1).description='Dynamic Stochastic General Equilibrium';
d(2).term='RBC';
d(2).description='Real Business Cycle';
d(3).term='RISE';
d(3).description='Rationality In Switching Environments';
  rprt.description(d);
```

13.3.2.6 enumerate

ENUMERATE Adds an enumerated list to the report.

Usage:
obj.enumerate(Items)

Inputs:

- obj: The report object.
- Items: A cell array of strings representing the items in the list.

Options: none

Example:

rprt.enumerate({'Apple', 'Banana', 'Orange'})

13.3.2.7 equation

EQUATION Adds an equation to the report.

Usage:
obj.equation('Expression') obj.equation('Expression', 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Expression: The mathematical expression to display as an equation.
- OptionName/OptionValue: Optional. Additional options for the equation.

Options:

- 'numbering': Boolean indicating whether the equation should be numbered (default: true).

Example:
rprt.equation('E = mc^2', 'numbering', false);

13.3.2.8 escape

rprt.escape is a function.
str = rprt.escape(str, type)

13.3.2.9 figure

FIGURE Adds a figure to the report.

Usage:
obj.figure('ImageFile') obj.figure('ImageFile', 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- ImageFile: The path or filename of the image file to include in the report.
- OptionName/OptionValue: Optional. Additional options for the figure.

Options:

- 'caption': The caption for the figure (default: '').
- 'width': The width of the figure in LaTeX units (default: 1, i.e. '1.0textwidth').
- 'placement': The placement specifier for the figure (default: 'ht').

Example:
rprt.figure('myimage.png', 'caption', 'This is my image', 'width', 0.8);

13.3.2.10 footnote

FOOTNOTE Adds a footnote to the report.

Usage:
obj.footnote('Text')

Inputs:

- obj: The report object.
- text: The text of the footnote.

Example:
rprt.footnote('This is a footnote.');

13.3.2.11 itemize

ITEMIZE Adds an itemized list to the report.

Usage:
obj.itemize(Items)

Inputs:

- obj: The report object.
- Items: A cell array of strings representing the items in the list.

Options:
None

Example:
rprt.itemize({'Item 1', 'Item 2', 'Item 3'});

13.3.2.12 newpage

NEWPAGE Adds a new page to the report.

Usage:

obj.newpage()

Inputs:

- obj: The report object.

Options: none

Example:

rprt.newpage();

13.3.2.13 pagebreak

PAGEBREAK Adds a new page to the report.

Usage:

obj.pagebreak()

Inputs:

- obj: The report object.

Options: none

Example:

rprt.pagebreak();

13.3.2.14 paragraph

PARAGRAPH Adds a paragraph to the report.

Usage:
obj.paragraph(Title) obj.paragraph(Title, 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Title: The title of the paragraph.
- OptionName/OptionValue: Optional. Additional options for the paragraph.

Options:

- 'numbering': Boolean indicating whether the paragraph should be numbered (default: true).

Example:
rprt.paragraph('Overview'); rprt.paragraph('Methodology');

13.3.2.15 plain

PLAIN Adds plain content to the report.

Usage:
obj.plain(Content)

Inputs:

- obj: The report object.
- Content: The plain content to be added.

Options: none

Example:
rprt.plain('This is some plain text');

13.3.2.16 publish

PUBLISH - Save the report to a file.

Syntax:

obj.publish()

obj.publish(savefile)

obj.publish(savefile, optName1, optValue1, ...)

Description:

This function saves the report to a file in either HTML or PDF format. By default, the report is saved as 'report.html'. The user can specify the file name and additional options using the input arguments.

Inputs:

- obj : The report object.
- savefile : Optional. Name of the file to save the report (default: 'report.html'). The name can be given without extension, in which case the default is html. Only two extensions are allowed : html and pdf.
- varargin : Optional. Additional options to customize the report generation.

Options:

- **'clean_up'**
  [Optional. Option to delete useless files (default: 'all').] Valid values are 'all', 'none', or 'all_but_tex'. This option works only for pdf files. The 'all_but_tex' variant avoids the deletion of the tex file through which the pdf file is generated
- 'graphics_path' : Optional. Path to the graphics folder (default: empty).

Example:

report = rprt('Sample Report', 'John Doe', 'portrait'); % Add elements to the report... report.publish(); % Saves the report to 'report.html' report.publish('myreport.pdf'); % Saves the report to 'myreport.pdf'

Notes:

- If a 'graphics_path' is not provided, the object will create a folder based on the name of the 'savefile'.
- The saving format is determined by the extension of the 'savefile' name. If no extension is provided, the default format is HTML.

13.3.2.17 quotation

QUOTATION Adds a quotation block to the report.

Usage:
obj.quotation('Text') obj.quotation('Text', 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Text: The text to include in the quotation block.
- OptionName/OptionValue: Optional. Additional options for the quotation block.

Options:

- 'author': The author of the quotation (default: '').

Example:
rprt.quotation('Lorem ipsum dolor sit amet.', 'author', 'John Doe');

13.3.2.18 quote

QUOTE Adds a block quote to the report.

Usage:
obj.quote('Text') obj.quote('Text', 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Text: The text to include in the block quote.
- OptionName/OptionValue: Optional. Additional options for the block quote.

Options:

- 'author': The author of the quote (default: '').

Example:
rprt.quote('Lorem ipsum dolor sit amet.', 'author', 'John Doe');

13.3.2.19 rise_file

RISE_FILE Reads the content of a RISE model file and writes it to the report.

Usage:
rise_file(obj, filename) rise_file(obj, filename, options)

Inputs:

- obj: The report object.
- filename: The name of the file to process.
- options (optional): Additional options for processing the file.

Options:

- latexAlias: [true|false] Option to replace atoms with their LaTeX aliases. CURRENTLY NOT ACTIVE
- lineNumbers: [true|false] Add line numbers.
- lines: ['all'|empty|vector] Lines of code to report.
- syntax: [true|false] Highlight keywords
- footnote: [{''}|char] Add a footnote to the model.

Example:
rise_file(obj, 'model_file.rs', 'lineNumbers', true);

13.3.2.20 rise_model_parameterization

RISE_MODEL_PARAMETERIZATION adds the parameterization of a dsge model to the report.

Usage:
rise_model_parameterization(obj, m)

Inputs:

- obj: The report object.
- m: parameterized model

13.3.2.21 rprt

rprt - Constructor for the rprt class.

Syntax:

```
  report = rprt();
  report = rprt(title);
  report = rprt(title, author);
  report = rprt(title, author, 'orientation','landscape');


Inputs:

  - title           : Title for the report.

  - author          : Author(s) of the report.

                      - char : e.g. 'Ola Normann'
                      - cellstr : e.g. {'Ola Normann','John Doe'}
                      - struct with fields : Name, Email, Institution
                      - cell array of structures

  - orientation     : Page orientation of the report ('portrait' or 'landscape').

Example:
```

```
report = rprt('Sample Report', 'John Doe', 'orientation', 'portrait');


Documentation for rprt/rprt
   doc rprt
```

13.3.2.22 section

SECTION Adds a section to the report.

Usage:
obj.section(Title) obj.section(Title, 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Title: The title of the section.
- OptionName/OptionValue: Optional. Additional options for the section.

Options:

- 'numbered': Boolean indicating whether the section should be numbered (default: true).

Example:
rprt.section('Introduction', 'numbered', false); rprt.section('Data Analysis');

13.3.2.23 set_options

rprt.set_options is a function.
options = rprt.set_options(d, Type, varargin)

13.3.2.24 subparagraph

SUBPARAGRAPH Adds a subparagraph to the report.

Usage:
obj.subparagraph(Title) obj.subparagraph(Title, 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Title: The title of the subparagraph.
- OptionName/OptionValue: Optional. Additional options for the subparagraph.

Options:

- 'numbered': Boolean indicating whether the subparagraph should be numbered (default: true).

Example:
rprt.subparagraph('Overview');

13.3.2.25 subsection

SUBSECTION Adds a subsection to the report.

Usage:
obj.subsection(Title) obj.subsection(Title, 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Title: The title of the subsection.

- OptionName/OptionValue: Optional. Additional options for the subsection.

Options:

- 'numbered': Boolean indicating whether the subsection should be numbered (default: true).

Example:
rprt.subsection('Overview', 'numbered', false); rprt.subsection('Methodology');

13.3.2.26 subsubsection

SUBSUBSECTION Adds a subsubsection to the report.

Usage:
obj.subsubsection(Title) obj.subsubsection(Title, 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Title: The title of the subsubsection.
- OptionName/OptionValue: Optional. Additional options for the subsubsection.

Options:

- 'numbered': Boolean indicating whether the subsubsection should be numbered (default: true).

Example:
rprt.subsubsection('Overview', 'numbered', false); rprt.subsubsection('Methodology');

13.3.2.27 table

TABLE - Add a table to the report.

Syntax:

obj.table(data)

obj.table(data, 'OptionName', OptionValue, ...)

Inputs:

- obj : The report object.
- data : The table data.
- **OptionName**
  [Optional. Additional options for the table.]
  - 'caption' : string
  - 'Headers' : Cell array of column headers.
  - 'RowNames' : Cell array of row names.
  - **'maxRows'**
    [Maximum number of rows in a table. If] exceeds, longtable is activated (default=10)
  - 'numbering' : to number the tables or not (default=true)
  - 'precision' : number of decimal places (default=4)format_funcs

- 'format_funcs': structure or array of structures with fields "test" and "action". The first field is a function handle evaluating numbers and returning a logical. The second field must be a and element (or a cell array combination) of the set : bold, italics, underline, red, pink, green, blue, yellow. So that any number passing the test will be transformed according to action

Examples:

data = rand(4); headers = {'Column 1', 'Column 2', 'Column 3', 'Column 4'}; RowNames = {'Row 1', 'Row 2', 'Row 3', 'Row 4'}; transf = struct('test',@(x)x>0.5,'action','red');

rprt.table(data);

rprt.table(data, 'Headers', headers);

rprt.table(data, 'Headers', headers,'RowNames',RowNames);

rprt.table(data, 'Headers', headers,'RowNames',RowNames, 'caption', 'Sample Table');

rprt.table(data, 'Headers', headers,'RowNames',RowNames, 'caption', 'Sample Table','format_funcs',transf);

Note : You can format numbers in tables

- bold : e.g. textbf{This text is bold.} or {bfseries This text is also bold.}
- Italics : e.g. textit{This text is italicized.} or {itshape This text is also italicized.}
- underlined : e.g. uline{This text is underlined.}
- color : e.g. textcolor{red}{This text is in red.}

13.3.2.28 text

TEXT Adds text content to the report.

Usage:
obj.text(Content)

Inputs:

- obj: The report object.
- Content: The text content to be added.

Options: none

Example:
rprt.text('This is some text.');

13.3.2.29 url

URL Adds a URL or hyperlink to the report.

Usage:
obj.url('Text', 'Link') obj.url('Text', 'Link', 'OptionName', OptionValue, ...)

Inputs:

- obj: The report object.
- Text: The text of the URL.
- Link: The URL link.

- OptionName/OptionValue: Optional. Additional options for the URL.

Options:

- 'newtab': Boolean indicating whether the URL should open in a new tab/window (default: false).
- MaskedText: Optional. The masked text to display instead of the URL (default: []).

Example:
rprt.url('OpenAI', 'https://openai.com'); rprt.url('Click here', 'https://example.com', 'MaskedText','Website', 'newtab', true);

13.3.2.30 verbatim

VERBATIM Adds verbatim text to the report.

Usage:
obj.verbatim('Text')

Inputs:

- obj: The report object.
- Text: The verbatim text to display.

Options: None

Example:
rprt.verbatim('This is verbatim text');

CHAPTER

FOURTEEN

STOCHASTIC GLOBAL OPTIMIZATION

1. The bee algorithm
2. Ant Colony Optimization
3. Differential Evolution
4. Glowworm swarm Optimization
5. Fireflies
6. etc

CHAPTER

FIFTEEN

TROUBLESHOOTING

15.1 deciphering return codes

decipher(retcode)

15.2 debugging tools

- debug = true
- fix_point_verbose = true
- Increase the number of iterations?
- Adjust tolerance levels

15.3 The steady state does not solve

- **have all the parameters been assigned a value?**
  isnan(m)
- **Is the model nonstationary and did you forgot to instruct RISE about it?**
  m=set(m,'solve_bgp',true)
- **Do you have an analytical solution to the steady state?**
  If yes, then it is probably wrong
- **Are there any variables that can only be positive?**
  set their bounds accordingly or declare them as log variables

15.4 The model does not solve

- **have all the parameters been assigned a value?**
  isnan(m)
- **Is the model nonstationary and did you forgot to instruct RISE about it?**
  m=set(m,'solve_bgp',true)
- **Is it about the steady state or about the dynamics?**
  try solving the steady state first

15.5 Speed issues : known bottlenecks

Here is a non-exhaustive list of things that can affect performance :

1. Symbolic calculation of derivatives
2. Evaluation of symbolic derivatives
3. forecasting function in filtration
4. use of loops in filtration?
5. Calculation of automatic derivatives
6. Inaccuracies of numerical derivatives
7. no function written to disk

15.6 A sneak peek into the future

1. Something new is dropping this weekend.
2. You asked, we listened. The documentation is here.
3. Get 10% off when you pre-order The documentation today only!
4. The wait is over. The documentation is here.

CHAPTER

SIXTEEN

PLOTTING TOOLS

Advanced plots

Hairy plots

real-time plots

etc.

CHAPTER

SEVENTEEN

REFERENCES

CHAPTER

EIGHTEEN

THINGS TO DO

Todo: Add further comments to this example

(The original entry is located in C:\Users\JMA\OneDrive\Documents\GitHub\RISE_Tbx_Beta_User_Guide\Docs\source\GSAandUQ\m
line 138.)

Todo: Comment on this example.

(The original entry is located in C:\Users\JMA\OneDrive\Documents\GitHub\RISE_Tbx_Beta_User_Guide\Docs\source\GSAandUQ\m
line 193.)

CHAPTER

NINETEEN

INDICES AND TABLES

- genindex
- modindex
- search

BIBLIOGRAPHY

[ABJ+11] Stéphane Adjemian, Houtan Bastani, Michel Juillard, Fréderic Karamé, Junior Maih, Ferhat Mihoubi, George Perendia, Johannes Pfeifer, Marco Ratto, and Sébastien Villemot. Dynare: Reference Manual Version 4. Dynare Working Papers 1, CEPREMAP, April 2011. URL: https://ideas.repec.org/p/cpm/dynare/001.html.

[And08] Gary Anderson. Solving Linear Rational Expectations Models: A Horse Race. *Computational Economics*, 31(2):95–113, March 2008. URL: https://ideas.repec.org/a/kap/compec/v31y2008i2p95-113.html, doi:10.1007/s10614-007-9108-0.

[AFVRR13] Martin M. Andreasen, Jesus Fernandez-Villaverde, and Juan Rubio-Ramirez. The pruned state-space system for non-linear dsge models: theory and empirical applications. NBER Working Papers 18983, National Bureau of Economic Research, Inc, Apr 2013. URL: http://ideas.repec.org/p/nbr/nberwo/18983.html.

[AB13] Michal Andrle and Jaromir Benes. System Priors: Formulating Priors about DSGE Models' Properties. IMF Working Papers 2013/257, International Monetary Fund, Dec 2013. URL: https://ideas.repec.org/p/imf/imfwpa/2013-257.html, doi:.

[AP17] Michal Andrle and Miroslav Plasil. System Priors for Econometric Time Series. Working Papers 2017/01, Czech National Bank, May 2017. URL: https://ideas.repec.org/p/cnb/wpaper/2017-01.html, doi:.

[AP18] Michal Andrle and Miroslav Plašil. Econometrics with system priors. *Economics Letters*, 172:134–137, 2018. URL: https://www.sciencedirect.com/science/article/pii/S0165176518303598, doi:https://doi.org/10.1016/j.econlet.2018.08.038.

[BSLK01] Y. Bar-Shalom, X.R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software*. Electronic and electrical engineering. Wiley, 2001. ISBN 9780471416555. URL: https://books.google.no/books?id=j2dGkwEACAAJ.

[bsLK04] Yaakov bar-shalom, X.-Rong Li, and Thia Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. The Author, 01 2004. ISBN 047141655X. doi:10.1002/0471221279.ch11.

[BM15] Andrew Binning and Junior Maih. Sigma point filters for dynamic nonlinear regime switching models. Working Paper 2015/10, Norges Bank, 2015. URL: https://EconPapers.repec.org/RePEc:bno:worpap:2015_10.

[BG98] Stephen P. Brooks and Andrew Gelman. General methods for monitoring convergence of iterative simulations. *Journal of Computational and Graphical Statistics*, 7(4):434–455, 1998. doi:10.1080/10618600.1998.10474787.

[CK13] Adam Cagliarini and Mariano Kulish. Solving Linear Rational Expectations Models with Predictable Structural Changes. *The Review of Economics and Statistics*, 95(1):328–336, March 2013. URL: https://ideas.repec.org/a/tpr/restat/v95y2013i1p328-336.html.

[CMT21] Yoosoon Chang, Junior Maih, and Fei Tan. Origins of monetary policy shifts: A New approach to regime switching in DSGE models. *Journal of Economic Dynamics and Control*, 2021. URL: https://ideas.repec.org/a/eee/dyncon/v133y2021ics0165188921001706.html, doi:10.1016/j.jedc.2021.10423.

[CJ01] Siddhartha Chib and Ivan Jeliazkov. Marginal likelihood from the metropolis–hastings output. *Journal of the American Statistical Association*, 96(453):270–281, 2001.

[Cho14] Seonghoon Cho. Characterizing Markov-Switching Rational Expectation Models. Working Paper, Yonsei University, February 2014.

[CTW11] Lawrence J. Christiano, Mathias Trabandt, and Karl Walentin. Introducing financial frictions and unemployment into a small open economy model. *Journal of Economic Dynamics and Control*, 35(12):1999–2041, 2011. URL: https://ideas.repec.org/a/eee/dyncon/v35y2011i12p1999-2041.html, doi:10.1016/j.jedc.2011.09.00.

[CFM05] Oswaldo Luiz do Valle Costa, M. D. (Marcelo Dutra) Fragoso, and Ricardo Paulino Marques. *Discrete-time Markov jump linear systems*. Probability and its applications. Springer, London, 2005. ISBN 1-85233-761-3. URL: http://opac.inria.fr/record=b1120049.

[DJChuChunLin03] Piet De Jong and Singfat Chu-Chun-Lin. Smoothing With An Unknown Initial Condition. *Journal of Time Series Analysis*, 24(2):141–148, March 2003. URL: https://ideas.repec.org/a/bla/jtsera/v24y2003i2p141-148.html, doi:10.1111/1467-9892.00298.

[DNS04] Marco Del Negro and Frank Schorfheide. Priors from General Equilibrium Models for VARS. *International Economic Review*, 45(2):643–673, May 2004. URL: https://ideas.repec.org/a/ier/iecrev/v45y2004i2p643-673.html, doi:.

[DNS08] Marco Del Negro and Frank Schorfheide. Forming priors for dsge models (and how it affects the assessment of nominal rigidities). *Journal of Monetary Economics*, 55(7):1191–1208, 2008. URL: https://www.sciencedirect.com/science/article/pii/S0304393208001396, doi:https://doi.org/10.1016/j.jmoneco.2008.09.006.

[DNSSW07] Marco Del Negro, Frank Schorfheide, Frank Smets, and Rafael Wouters. On the Fit of New Keynesian Models. *Journal of Business & Economic Statistics*, 25:123–143, April 2007. URL: https://ideas.repec.org/a/bes/jnlbes/v25y2007p123-143.html.

[DK12] James Durbin and Siem Jan Koopman. *Time Series Analysis by State Space Methods*. Volume of OUP Catalogue. Oxford University Press, edition, November 2012. ISBN ARRAY(0x60272498). URL: https://ideas.repec.org/b/oxp/obooks/9780199641178.html, doi:.

[FWZ11] Roger E.A. Farmer, Daniel F. Waggoner, and Tao Zha. Minimal state variable solutions to Markov-switching rational expectations models. *Journal of Economic Dynamics and Control*, 35(12):2150–2166, 2011. URL: https://ideas.repec.org/a/eee/dyncon/v35y2011i12p2150-2166.html, doi:10.1016/j.jedc.2011.08.00.

[FRRWZ16] Andrew Foerster, Juan F. Rubio-Ramirez, Daniel F. Waggoner, and Tao Zha. Perturbation methods for Markov-switching dynamic stochastic general equilibrium models. *Quantitative Economics*, 7(2):637–669, 07 2016. URL: https://ideas.repec.org/a/wly/quante/v7y2016i2p637-669.html, doi:.

[FruhwirthS06] Sylvia Frühwirth-Schnatter. *Finite mixture and Markov switching models*. Springer, Berlin, 1st edition, 2006.

[GLM13] Paolo Gelain, Kevin J. Lansing, and Caterina Mendicino. House Prices, Credit Growth, and Excess Volatility: Implications for Monetary and Macroprudential Policy. *International Journal of Central Banking*, 9(2):219–276, June 2013. URL: https://ideas.repec.org/a/ijc/ijcjou/y2013q2a11.html.

[GCSR04] Andrew Gelman, John B. Carlin, Hal S. Stern, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman & Hall/CRC, 2 edition, 2004.

[GR92] Andrew Gelman and Donald B. Rubin. Inference from Iterative Simulation Using Multiple Sequences. *Statistical Science*, 7(4):457 – 472, 1992. URL: https://doi.org/10.1214/ss/1177011136, doi:10.1214/ss/1177011136.

[Gew99] John Geweke. Using simulation methods for bayesian econometric models: inference, development,and communication. *Econometric Reviews*, 18(1):1–73, 1999.

[GI15] Luca Guerrieri and Matteo Iacoviello. OccBin: A toolkit for solving dynamic models with occasionally binding constraints easily. *Journal of Monetary Economics*, 70(C):22–38, 2015. URL: https://ideas.repec.org/a/eee/moneco/v70y2015icp22-38.html, doi:10.1016/j.jmoneco.2014.08.

[GI17] Luca Guerrieri and Matteo Iacoviello. Collateral constraints and macroeconomic asymmetries. *Journal of Monetary Economics*, 90(C):28–49, 2017. URL: https://ideas.repec.org/a/eee/moneco/v90y2017icp28-49.html, doi:10.1016/j.jmoneco.2017.06.

[GMH03] V. Gupta, R.M. Murray, and B. Hassibi. On the control of jump linear markov systems with markov state estimation. In *American Control Conference, 2003. Proceedings of the 2003*, volume 4, 2893–2898 vol.4. June 2003. doi:10.1109/ACC.2003.1243762.

[Ire04] Peter N. Ireland. A method for taking models to the data. *Journal of Economic Dynamics and Control*, 28(6):1205–1226, 2004. URL: https://www.sciencedirect.com/science/article/pii/S0165188903000800, doi:https://doi.org/10.1016/S0165-1889(03)00080-0.

[JM10] Michel Juillard and Junior Maih. Estimating DSGE Models with Observed Real-Time Expectation Data. 2010. URL: https://www.kansascityfed.org/publicat/events/research/2010CenBankForecasting/Maih_paper.pdf.

[KN99] C.J. Kim and C.R. Nelson. *State-space Models with Regime Switching: Classical and Gibbs-sampling Approaches with Applications*. MIT Press, 1999. ISBN 9780262112383. URL: http://books.google.no/books?id=eQFsQgAACAAJ.

[KN01] Chang-Jin Kim and Charles Nelson. A bayesian approach to testing for markov-switching in univariate and dynamic factor models. *International Economic Review*, 42(4):989–1013, 2001. URL: http://EconPapers.repec.org/RePEc:ier:iecrev:v:42:y:2001:i:4:p:989-1013.

[Kle00] Paul Klein. Using the generalized schur form to solve a multivariate linear rational expectations model. *Journal of Economic Dynamics and Control*, 24(10):1405–1423, September 2000. URL: http://ideas.repec.org/a/eee/dyncon/v24y2000i10p1405-1423.html.

[KD00] S. J. Koopman and J. Durbin. Fast Filtering and Smoothing for Multivariate State Space Models. *Journal of Time Series Analysis*, 21(3):281–296, May 2000. URL: https://ideas.repec.org/a/bla/jtsera/v21y2000i3p281-296.html, doi:10.1111/1467-9892.00186.

[KD03] S. J. Koopman and J. Durbin. Filtering and smoothing of state vector for diffuse state-space models. *Journal of Time Series Analysis*, 24(1):85–98, January 2003. URL: https://ideas.repec.org/a/bla/jtsera/v24y2003i1p85-98.html, doi:10.1111/1467-9892.00294.

[Mai10] Junior Maih. Conditional forecasts in DSGE models. Working Paper 2010/07, Norges Bank, April 2010. URL: https://ideas.repec.org/p/bno/worpap/2010_07.html.

[Mai15] Junior Maih. Efficient perturbation methods for solving regime-switching DSGE models. Working Paper 2015/01, Norges Bank, January 2015. URL: https://ideas.repec.org/p/bno/worpap/2015_01.html, doi:.

[MW18] Junior Maih and Daniel Waggoner. Perturbation Methods for DSGE Models with Time Varying Coefficients and Transition Matrices. Mimeograph, Norges Bank, 2018.

[MW96] Xiao-Li Meng and Wing Hung Wong. Simulating ratios of normalizing constants via a simple identity: a theoretical exploration. *Statistica Sinica*, 6(4):831–860, 1996. URL: http://www.jstor.org/stable/24306045.

[MMV13] Btażej Miasojedow, Eric Moulines, and Matti Vihola. An adaptive parallel tempering algorithm. *Journal of Computational and Graphical Statistics*, 22(3):649–664, 2013.

[PRR15] Christophe Planas, Marco Ratto, and Alessandro Rossi. Slice sampling in bayesian estimation of dsge models. Working Papers, Joint Research Centre, European Commission, 2015.

[PR18] Christophe Planas and Alessandro Rossi. The slice sampler and centrally symmetric distributions. Working Papers 2018-11, Joint Research Centre, European Commission, November 2018. URL: https://ideas.repec.org/p/jrs/wpaper/201811.html, doi:.

[RL92] Adrian E. Raftery and Steven Lewis. How many iterations in the gibbs sampler? In A. P. Dawid J. M. Bernardo, J. Berger and A. F. M. Smith, editors, *Bayesian Statistics*, volume 4, 763–73. Oxford University Press, 1992.

[Rat08] Marco Ratto. Analysing dsge models with global sensitivity analysis. *Computational Economics*, 31:115–139, 02 2008. doi:10.1007/s10614-007-9110-6.

[RM12] Francisco Ruge-Murcia. Estimating nonlinear dsge models by the simulated method of moments: with an application to business cycles. *Journal of Economic Dynamics and Control*, 36(6):914–938, 2012. URL: https://www.sciencedirect.com/science/article/pii/S0165188912000231, doi:https://doi.org/10.1016/j.jedc.2012.01.008.

[Sim02] Christopher A Sims. Solving Linear Rational Expectations Models. *Computational Economics*, 20(1-2):1–20, October 2002. URL: https://ideas.repec.org/a/kap/compec/v20y2002i1-2p1-20.html.

[SWZ08] Christopher A. Sims, Daniel F. Waggoner, and Tao Zha. Methods for inference in large multiple-equation Markov-switching models. *Journal of Econometrics*, 146(2):255–274, October 2008. URL: https://ideas.repec.org/a/eee/econom/v146y2008i2p255-274.html, doi:.

[VanLoan00] Charles F. Van Loan. The ubiquitous Kronecker product. *Journal of Computational and Applied Mathematics*, 123(123):85–100, 2000.

INDEX

A

aim2rise, 68

B

bvar_dsge.autocov, 130
bvar_dsge.bvar_dsge, 130
bvar_dsge.constant, 129
bvar_dsge.draw_parameter, 131
bvar_dsge.endogenous, 129
bvar_dsge.estimate, 132
bvar_dsge.estimation, 130
bvar_dsge.fisher, 132
bvar_dsge.forecast, 132
bvar_dsge.get, 132
bvar_dsge.hessian, 133
bvar_dsge.historical_decomposition, 133
bvar_dsge.irf, 134
bvar_dsge.load_parameters, 134
bvar_dsge.log_posterior_kernel, 135
bvar_dsge.log_prior_density, 135
bvar_dsge.markov_chains, 130
bvar_dsge.mode_curvature, 136
bvar_dsge.nlags, 129
bvar_dsge.options, 129
bvar_dsge.parameters, 130
bvar_dsge.posterior_sample, 137
bvar_dsge.print_estimation_results, 138
bvar_dsge.pull_objective, 138
bvar_dsge.residuals, 139
bvar_dsge.set, 139
bvar_dsge.shocks, 130
bvar_dsge.solve, 139
bvar_dsge.variance_decomposition, 140

D

dirichlet prior, 213
dynare2rise, 66
dynare_sstate_file2rise, 67

E

endogenous priors, 73, 76, 214

F

filtering, 15, 28, 60, 61, 66, 79, 160, 175, 188, 202, 269, 272, 276, 277, 293, 297

G

gumbel, 215

H

hdmr.aggregate, 304
hdmr.coefficients, 304
hdmr.D, 304
hdmr.estimate, 304
hdmr.expansion_order, 303
hdmr.f0, 304
hdmr.first_order_effect, 304
hdmr.g, 303
hdmr.hdmr, 305
hdmr.Indices, 303
hdmr.metamodel, 305
hdmr.N, 302
hdmr.Nobs, 302
hdmr.numInputs, 302
hdmr.optimal, 304
hdmr.output_nbr, 302
hdmr.param_names, 304
hdmr.plot_fit, 305
hdmr.pol_max_order, 303
hdmr.poly_coefs, 303
hdmr.polynomial_evaluation, 305
hdmr.polynomial_integration, 305
hdmr.polynomial_multiplication, 305
hdmr.sample_percentage, 304
hdmr.theta, 303
hdmr.theta_high, 303
hdmr.theta_low, 303
hdmr.x, 303

K

kumaraswamy, 215

M

mcf.cdf, 295

mcf.cdf_plot, 296
mcf.correlation_patterns_plot, 296
mcf.kolmogorov_smirnov_test, 296
mcf.mcf, 297
mcf.scatter, 298
mcfprop.add, 298
mcfprop.create_handle, 298
mcfprop.mcfprop, 298
mcmc.autocorrplot, 223
mcmc.best, 223
mcmc.brooks_gelman, 224
mcmc.chain_names, 223
mcmc.densplot, 224
mcmc.draws, 223
mcmc.geweke, 225
mcmc.inefficiency_factors, 225
mcmc.log_post, 223
mcmc.mcmc, 226
mcmc.meanplot, 226
mcmc.nchains, 222
mcmc.nparams, 223
mcmc.npop, 223
mcmc.plot_brooks_gelman, 227
mcmc.pnames, 222
mcmc.posterior_parameter_statistics, 227
mcmc.process_draws, 228
mcmc.psrf, 223
mcmc.psrf_plot, 228
mcmc.raftery_lewis, 228
mcmc.reload_draws, 229
mcmc.scatterplot, 229
mcmc.statistics, 229
mcmc.summary, 230
mcmc.traceplot, 230
mdd.bridge, 232
mdd.cj, 232
mdd.global_options, 232
mdd.H, 231
mdd.is, 233
mdd.laplace, 233
mdd.laplace_mcmc, 233
mdd.laplace_mdd, 234
mdd.lb, 231
mdd.log_post_kern, 231
mdd.maximization, 231
mdd.mdd, 234
mdd.mhm, 235
mdd.mueller, 235
mdd.normal_weighting, 236
mdd.ris, 236
mdd.subset, 231
mdd.swz, 236
mdd.thecoef, 231
mdd.theta_draws, 231
mdd.ub, 231

P

pareto, 215
prfvar.autocov, 171
prfvar.bootstrap, 172
prfvar.codify, 172
prfvar.constant, 170
prfvar.debug, 170
prfvar.endogenous, 170
prfvar.estimate, 173
prfvar.exogenous, 170
prfvar.filter, 174
prfvar.forecast, 175
prfvar.get, 175
prfvar.historical_decomposition, 175
prfvar.homogeneity, 169
prfvar.identification, 175
prfvar.irf, 176
prfvar.is_switching, 171
prfvar.is_time_varying_trans_prob, 170
prfvar.markov_chains, 170
prfvar.members, 169
prfvar.nlags, 171
prfvar.nonvar_parameters, 170
prfvar.nparams, 171
prfvar.nregs, 171
prfvar.nvars, 171
prfvar.nx, 171
prfvar.parameters, 170
prfvar.posterior_mode, 177
prfvar.prfvar, 177
prfvar.print_solution, 179
prfvar.prior_template, 179
prfvar.pull_objective, 180
prfvar.residuals, 180
prfvar.set, 181
prfvar.solve, 181
prfvar.structural_shocks, 181
prfvar.transform_var_into_dsge, 181
prfvar.variance_decomposition, 182
prior, 163, 179, 190, 204, 211, 214, 215
proxy_svar.autocov, 199
proxy_svar.codify, 200
proxy_svar.constant, 198
proxy_svar.debug, 198
proxy_svar.endogenous, 197
proxy_svar.estimate, 200
proxy_svar.exogenous, 198
proxy_svar.filter, 202
proxy_svar.forecast, 202
proxy_svar.get, 202
proxy_svar.historical_decomposition, 203
proxy_svar.identification, 203

proxy_svar.irf, 203
proxy_svar.is_switching, 198
proxy_svar.is_time_varying_trans_prob, 198
proxy_svar.markov_chains, 198
proxy_svar.nlags, 199
proxy_svar.nonvar_parameters, 198
proxy_svar.nparams, 199
proxy_svar.nregs, 199
proxy_svar.nvars, 199
proxy_svar.nx, 199
proxy_svar.parameters, 198
proxy_svar.posterior_mode, 203
proxy_svar.print_solution, 203
proxy_svar.print_structural_form, 203
proxy_svar.prior_template, 204
proxy_svar.proxies, 197
proxy_svar.proxy_svar, 204
proxy_svar.pull_objective, 205
proxy_svar.residuals, 206
proxy_svar.set, 206
proxy_svar.solve, 207
proxy_svar.transform_var_into_dsge, 208
proxy_svar.variance_decomposition, 208

R

ra, 243
rd, 240
rfvar.autocov, 156
rfvar.bootstrap, 157
rfvar.codify, 157
rfvar.constant, 155
rfvar.debug, 155
rfvar.endogenous, 155
rfvar.estimate, 158
rfvar.exogenous, 155
rfvar.filter, 159
rfvar.forecast, 160
rfvar.get, 160
rfvar.historical_decomposition, 160
rfvar.identification, 160
rfvar.irf, 161
rfvar.is_switching, 156
rfvar.is_time_varying_trans_prob, 155
rfvar.markov_chains, 155
rfvar.nlags, 156
rfvar.nonvar_parameters, 155
rfvar.nparams, 156
rfvar.nregs, 156
rfvar.nvars, 156
rfvar.nx, 156
rfvar.parameters, 155
rfvar.posterior_mode, 162
rfvar.print_solution, 162
rfvar.prior_template, 162
rfvar.pull_objective, 163
rfvar.residuals, 164
rfvar.rfvar, 164
rfvar.set, 165
rfvar.solve, 166
rfvar.structural_shocks, 167
rfvar.transform_var_into_dsge, 167
rfvar.variance_decomposition, 167
rh, 242
rise.abcd, 70
rise.accuracy, 71
rise.calculate_loss, 72
rise.calibrate, 72
rise.check_derivatives, 73
rise.condition_draws_on_model, 73
rise.counterfactual, 73
rise.dbminusdb, 74
rise.definitions, 69
rise.doc, 4
rise.draw_parameter, 74
rise.drop, 75
rise.endogenous, 69
rise.equations, 69
rise.estimate, 75
rise.estimation, 70
rise.exit, 4
rise.exogenous, 70
rise.extract_first_order_structure, 78
rise.filename, 69
rise.filter, 79
rise.fisher, 81
rise.fold_solution, 81
rise.forecast, 81
rise.forecast_real_time, 82
rise.frontier, 83
rise.get, 84
rise.growth_database, 87
rise.hessian, 87
rise.historical_decomposition, 87
rise.historical_decomposition_switch, 88
rise.indirect_inference, 89
rise.initial_conditions, 90
rise.initial_sstate, 90
rise.irf, 91
rise.is_forward_guidance_puzzle, 92
rise.is_stable_system, 93
rise.is_stationary_system, 93
rise.isnan, 94
rise.itranslate, 94
rise.legend, 69
rise.link_parameters, 95
rise.load_parameters, 95
rise.loadObj, 95
rise.log_posterior_kernel, 95

rise.log_prior_density, 96
rise.markov_chains, 70
rise.max_discrepancy, 97
rise.mode_curvature, 97
rise.model_description, 69
rise.model_information, 98
rise.nsols, 69
rise.observables, 70
rise.observables_decomposition, 98
rise.optimal_simple_rule, 99
rise.options, 70
rise.parameters, 70
rise.parameters_to_file, 99
rise.perfect_foresight, 99
rise.plan2database, 101
rise.posterior_sample, 102
rise.predictive_analysis, 102
rise.print_estimation_results, 104
rise.print_solution, 104
rise.pull_objective, 105
rise.randsample, 105
rise.refresh, 106
rise.release, 5
rise.report, 106
rise.resid, 107
rise.resimulate, 107
rise.rise, 108
rise.saveObj, 109
rise.set, 110
rise.set_z_eplus_horizon, 111
rise.simulate, 111
rise.simulate_nonlinear, 112
rise.solve, 113
rise.solve_alternatives, 116
rise.srf, 117
rise.sstate, 118
rise.startup, 5
rise.state_var_list, 120
rise.stoch_simul, 121
rise.theoretical_autocorrelations, 121
rise.theoretical_autocovariances, 122
rise.toolbox, 5
rise.translate, 123
rise.unlink_parameters, 124
rise.update_file, 124
rise.user_data, 69
rise.variance_decomposition, 125
rise.version, 6
rise.view_linked_parameters, 126
rise.whatsup, 7
rise.why, 6
rise_dates.dates.char, 244
rise_dates.dates.colon, 244
rise_dates.dates.convert, 245
rise_dates.dates.date_time, 244
rise_dates.dates.dates, 245
rise_dates.dates.datestr, 245
rise_dates.dates.double, 246
rise_dates.dates.Frequency, 244
rise_dates.dates.Incr, 244
rise_dates.dates.intersect, 246
rise_dates.dates.ismember, 247
rise_dates.dates.max, 247
rise_dates.dates.min, 248
rise_dates.dates.minus, 248
rise_dates.dates.plus, 248
rise_dates.dates.unique, 249
rise_dates.dates.xline, 249
rise_dates.dates.ymd, 250
rm, 241
rprt.Author, 310
rprt.cleardoublepage, 311
rprt.clearpage, 311
rprt.code_formatter, 311
rprt.create_figure, 312
rprt.Date, 311
rprt.description, 313
rprt.enumerate, 313
rprt.equation, 313
rprt.escape, 314
rprt.figure, 314
rprt.footnote, 314
rprt.itemize, 315
rprt.newpage, 315
rprt.pagebreak, 315
rprt.paragraph, 316
rprt.plain, 316
rprt.publish, 316
rprt.quotation, 317
rprt.quote, 318
rprt.rise_file, 318
rprt.rise_model_parameterization, 318
rprt.rprt, 319
rprt.section, 319
rprt.set_options, 320
rprt.subparagraph, 320
rprt.subsection, 320
rprt.subsubsection, 321
rprt.table, 321
rprt.text, 322
rprt.Title, 310
rprt.url, 322
rprt.verbatim, 323
rq, 242
rsamplers.apt, 219
rsamplers.imh, 218
rsamplers.rsample.sample, 216
rsamplers.rwmh, 217

rsamplers.slice, 219
rw, 241
ry, 242

S

simplan.append, 55
simplan.Conditions, 54
simplan.end_date, 54
simplan.endo_list, 54
simplan.endval, 55
simplan.exo_list, 54
simplan.export, 56
simplan.histval, 57
simplan.InitialRegime, 54
simplan.initval, 58
simplan.simplan, 59
simplan.simul_periods, 55
simplan.start_date, 54
simplan.variable_list, 55
svar.autocov, 185
svar.codify, 186
svar.constant, 184
svar.debug, 184
svar.endogenous, 183
svar.estimate, 186
svar.exogenous, 183
svar.filter, 188
svar.forecast, 188
svar.get, 188
svar.historical_decomposition, 189
svar.identification, 189
svar.irf, 189
svar.is_switching, 184
svar.is_time_varying_trans_prob, 184
svar.markov_chains, 184
svar.nlags, 184
svar.nonvar_parameters, 184
svar.nparams, 185
svar.nregs, 185
svar.nvars, 185
svar.nx, 185
svar.parameters, 184
svar.posterior_mode, 190
svar.print_solution, 190
svar.print_structural_form, 190
svar.prior_template, 190
svar.pull_objective, 191
svar.residuals, 192
svar.set, 192
svar.solve, 192
svar.svar, 193
svar.transform_var_into_dsge, 194
svar.variance_decomposition, 194
switching_cubature_kalman_filter, 63
switching_divided_difference_filter, 62
switching_unscented_kalman_filter, 61

T

ts.abs, 254
ts.acos, 254
ts.acosh, 254
ts.acot, 254
ts.acoth, 254
ts.allmean, 254
ts.and, 255
ts.apply, 255
ts.asin, 255
ts.asinh, 256
ts.atan, 256
ts.atanh, 256
ts.bar, 256
ts.barh, 257
ts.boxplot, 258
ts.bsxfun, 258
ts.cat, 259
ts.chebyshev_box, 259
ts.chowlin, 259
ts.collect, 260
ts.concatenator, 261
ts.convert, 261
ts.corr, 262
ts.corrcoef, 262
ts.cos, 262
ts.cosh, 263
ts.cot, 263
ts.coth, 263
ts.cov, 263
ts.cumprod, 263
ts.cumsum, 263
ts.data, 253
ts.describe, 263
ts.description, 253
ts.disp, 264
ts.double, 264
ts.drop, 264
ts.dummy, 264
ts.dust_up, 265
ts.end, 265
ts.exp, 265
ts.expanding, 266
ts.fanchart, 266
ts.finish, 253
ts.fold, 266
ts.frequency, 253
ts.get, 267
ts.group, 267
ts.head, 268
ts.hist, 268

ts.horzcat, 268
ts.hpfilter, 269
ts.imag, 269
ts.interpolate, 269
ts.isfinite, 270
ts.isinf, 270
ts.isnan, 270
ts.jbtest, 271
ts.kurtosis, 271
ts.log, 272
ts.ma_filter, 272
ts.max, 272
ts.mean, 273
ts.median, 273
ts.min, 273
ts.minus, 273
ts.mode, 274
ts.moments, 274
ts.mpower, 274
ts.mrdivide, 275
ts.mtimes, 275
ts.nan, 275
ts.npdecomp, 275
ts.NumberOfObservations, 253
ts.NumberOfPages, 253
ts.NumberOfVariables, 253
ts.numel, 276
ts.ones, 276
ts.pages2struct, 277
ts.pdecomp, 277
ts.permute, 278
ts.plot, 278
ts.plot_real_time, 279
ts.plotyy, 279
ts.plus, 280
ts.power, 280
ts.prctile, 280
ts.quantile, 280
ts.rand, 281
ts.randn, 281
ts.range, 282
ts.rdivide, 282
ts.real, 282
ts.regress, 282
ts.rmse, 283
ts.rolling, 283
ts.set, 284
ts.sin, 284
ts.sinh, 284
ts.size, 285
ts.skewness, 285
ts.spectrum, 285
ts.start, 254
ts.std, 285
ts.step_dummy, 286
ts.subsasgn, 286
ts.subsref, 287
ts.sum, 288
ts.tail, 288
ts.times, 288
ts.transform, 289
ts.ts, 289
ts.uminus, 290
ts.unfold, 291
ts.values, 291
ts.var, 291
ts.varnames, 253
ts.zeros, 291

U

User-defined priors, 214

W

weibull, 215